jlens 项目 · Qwen3-32B 作者记忆 · 方法答辩用 · 2026-10-30

Stage 1–13 方法手册

本手册记录**实际做了什么**，不是计划。每个数字、参数和样本量后面都给了出处（脚本路径和行号或函数名、结果文件、作业号、git commit），出处标为 ⟶ 出处 示例。查不到的地方写成 **未知/需确认：向谁确认**，全文共 18 处，汇总表见附录。

状态标签：已完成 复核过 待复核 作废 计划未做 。每个实验节固定用 9 个小标题：1 问题与作用 · 2 材料 · 3 工具 · 4 步骤 · 5 公式 · 6 对照 · 7 结果 · 8 局限与质疑 · 9 状态。

路径约定：除非另写，`R10` = /scratch/zhang.zhiqi2/jlens/351_stage10，`R11` = …/361_stage11，`R12` = …/371_stage12，`R13` = …/381_stage13；代码在 ~/jlens_work/<NN_stage>/；git 仓库是 /scratch/zhang.zhiqi2/jlens-authorship-recall_update（GitHub ZhiQ1/jlens-authorship-recall）。

目录

1. 一 全局（上）：模型、数据、标签、提示词、打分
   G1 模型：Qwen3-32B 的版本、结构、加载方式与 chat 模板 · G2 数据来源：9,108 篇论文从哪来 · G3 标签流程：开放回答、姓名匹配、R0/R1/R2、多选题、分组 · G4 提示词与 token 位置 · G5 打分：log10(1/名次) 与各级聚合
2. 二 全局（下）：假标题、读出工具、统计、泄漏审计、预注册、算力
   B1. 假标题（false_0 … false_4）与删标题条件 · B2. 读出工具：logit lens 与三种 Jacobian lens（及 Stage 12 F1 作者提示 lens） · B3. 统计：引用档、分层权重、bootstrap、作者团队、多重比较 · B4. 切分与泄漏审计 · B5. 预注册（Stage 13） · B6. 计算资源与数值一致性检查
3. 三 Stage 1–9
   Stage 1–9 方法手册（小样本阶段） · Stage 1 · 队列与行为标签 · Stage 2 · 模型、提示词与打分协议 · Stage 3 · J-Lens 搭建与验证（Exp 1–7，含 provenance audit） · Stage 4 · 作者读出基准（Benchmark20） · Stage 5 · 交叉标题–作者对照 · Stage 6 · 完整作者名的条件 JMRR · Stage 7 · 排除替代解释（7.1–7.4） · Stage 8 · 200 篇新论文的独立验证 · Stage 9 · FullFit 拟合可靠性
4. 四 Stage 10–11
   Stage 10：全量逐层读出（9,108 篇论文、149,999 个名字 token、7 种标题条件） · Stage 11：干预实验（替换、阻断、标题位置读出与 probe、作者连写）
5. 五 Stage 12
   Stage 12 总览：单元构成、共用设定、GPU 型号守卫 · P1 单个位置替换（patch_pos） · P2 注意力 vs MLP 组件替换（patch_comp） · N1 多选选项先验（prior_mc） · N2 纯名字先验（prior_pure） · F1 作者提示 lens 拟合（alens_fit） · F2 作者提示 lens 读出（alens_read） · Stage 12 CPU 分析（stage12_cpu.py）：名字先验 A/B/C 与探针对照 X1
6. 六 Stage 13
   Stage 13：中间层装的如果不是名字，是什么（M1–M5，X1–X7）
7. 七 附录

附录 G1　术语表 · 附录 G2　导师可能追问的 20 个问题与标准答案 · 附录 G3　全部"未知 / 需确认"汇总 · 附录 G4　计划未做 / 作废 / 被复核取代 / 待复核 汇总 · G5 索引：脚本、结果文件、作业号、git commit

8. 八 自查
   附录 F · 自查：随机抽取 15 个数字，回到原始结果文件核对
9. 九 附表：未知项查证
   附录 I · "未知/需确认"自查结果（U08–U44 中可自查的条目）
10. 十 附表：Dreaming Up Authors 代码包核对
    附录 J　Dreaming Up Authors 代码包（dua v1.0.0）逐条核对

G1 模型：Qwen3-32B 的版本、结构、加载方式与 chat 模板

G1.1 模型身份与结构

项	值	出处
模型	`Qwen/Qwen3-32B`（`Qwen3ForCausalLM`）	⟶ 出处 10_stage10/351_stage10.py:27 `MODEL` · HF 缓存 config.json `architectures`
revision	`9216db5781bf21249d130ec9da846c4624c16137`（缓存里唯一的 snapshot，`refs/main` 也指向它）	⟶ 出处 10_stage10/351_stage10.py:28 `REV` · /scratch/zhang.zhiqi2/hf/hub/models--Qwen--Qwen3-32B/refs/main · snapshots/9216db57…/
权重	17 个 safetensors 分片；`torch_dtype: bfloat16`	⟶ 出处 snapshots/9216db57…/model.safetensors.index.json · config.json
运行精度	所有 GPU 实验 bf16（`dtype=torch.bfloat16`）；lens 读出时残差先 `.float()`，再按 lm_head 的 dtype（bf16）过 final norm + lm_head，logits 再转 float32	⟶ 出处 351_stage10.py:381 · :434–437 `fast_readout` · ~/projects/jacobian-lens/jlens/hf.py:166–174 `unembed`
层数	`num_hidden_layers = 64`（decoder block 0–63）。本项目的 "L*l*" = 第 *l* 个 block 的输出残差；L63 = 最后一个 block 的输出；readout `final` = unembed(残差$_{63}$) = 模型真实的下一 token logits	⟶ 出处 config.json · 351_stage10/config/protocol.json `residual`, `readouts.final` · 351_stage10.py:30 `FINAL_LAYER = 63`
隐藏维度	`hidden_size = 5120`；`intermediate_size = 25600`；注意力头 64，KV 头 8，`head_dim = 128`；RMSNorm eps 1e-6；`rope_theta = 1e6`；`max_position_embeddings = 40960`；无 sliding window；`tie_word_embeddings = false`	⟶ 出处 config.json
词表	lm_head 输出 **151,936** 行（config `vocab_size`）。tokenizer：`len(tok) = 151,669` = 151,643 普通 token + 26 个 added/special token（本次计算）。所以 lm_head 有 151,936 − 151,669 = 267 行没有对应的 tokenizer 条目（填充行）。	⟶ 出处 config.json `vocab_size` · 本次计算：AutoTokenizer(rev 9216db57)，`len(tok)`，`tok.vocab_size`，`len(tok.added_tokens_decoder)`
特殊 token	`bos_token = None`（tokenizer）；`eos_token_id = 151645`（`<|im_end|>`）；pad = `<|endoftext|>`（151643）；tokenizer_config `add_bos_token = False`	⟶ 出处 snapshots/9216db57…/tokenizer_config.json · config.json · 本次计算
generation_config 默认值	`do_sample: true, temperature 0.6, top_k 20, top_p 0.95`。我们自己的代码只有 X3 para_gen 调用 `generate`，并显式 `do_sample=False`（贪心），所以这些默认值不起作用；其余实验都只做 forward。	⟶ 出处 snapshots/9216db57…/generation_config.json · 13_stage13/381_stage13.py:281

软件版本	当前 env：transformers 5.19.0（本次 `import` 读出）。Stage 13 输出 meta 记录 `torch 2.11.0+cu128`。各 Stage 运行时的 transformers 版本没有逐个写进输出（Stage 10 meta 只存 torch 版本和 GPU 名）。	⟶ 出处 381_stage13/raw/para_gen/shard_000.npz `meta` · 351_stage10.py:543 `env`

G1.2 加载方式（各 Stage 不同）

早期（Stage 3 的部分验证、Stage 4–8）在 A100-40GB 上跑，用显式 device_map 把前 29 个 block 放 GPU、后 35 个 block 放 CPU 内存（embedding、final norm、rotary、lm_head 都在 GPU 0）。Stage 10 起整模型放在 GPU 上，禁止 CPU/磁盘 offload。

Stage / 实验	加载代码	放置	实际运行证据
Stage 1 标注（组长）	不在我们手里（`elm` 包）	"local bf16 weights … on an A100 80GB"	⟶ 出处 labels_20261015/…/README_first.md 第 3 行 **未知/需确认：向组长确认（加载方式、batch、GPU、时长；elm 两次运行的 run.json 不在标注包里，也不在集群上，U10）** 。Dreaming Up Authors 代码包里 dua 自己的 Qwen3 运行是 batch 1、bf16，不能代表 elm 运行 ⟶ 出处 附录 J 第 10 行
Stage 3 公开 lens smoke / fidelity / validate / 作者特异性	`device_map='cuda'`	整模型单卡	⟶ 出处 01_public_lens_validation/351_public_jlens_smoke.py:63 · 351_public_lens_fidelity.py:84 · 351_validate_public_lens.py:145 · 351_author_specificity.py:152
Stage 3 公开 lens 带 offload 的验证	`--cpu-offload --gpu-layers 29`（默认 29）	29 GPU + 35 CPU	⟶ 出处 351_validate_public_lens_offload_fixed.py:108–161 · 日志 351_all/351_offload_fixed_validation.log：`"BF16 explicit CPU offload; GPU blocks 29 CPU blocks 35"`；⟶ 出处 351_paper_prompt_fidelity_offload.py:3（`"on A100-40GB with CPU-layer offload"`）, :55, :88–91
Stage 3 self100 拟合	`device_map="cuda"`，dim_batch 8，max_seq_len 128	整模型单卡	⟶ 出处 jlens-authorship-recall_update/home_project/05_fit_jlens_prelim_100.py:116–118, :170–174, :210–212
Stage 3 self lens 验证、gold-vs-false	默认 `--gpu-layers 29`	29 GPU + 35 CPU	⟶ 出处 02_self_lens_validation/351_self_jlens_validation.py:105–107, :242 · 351_self_gold_false.py:127, :184–186；split-half 要求整模型 80GB 卡：⟶ 出处 351_self_jlens_split_half.py:55, :69
Stage 4 readout	`--gpu-layers` 默认 29	29 GPU + 35 CPU	⟶ 出处 04_stage4/351_stage4_readout.py:385, :438–441 · 日志 351_all/351_stage4_readout.log：`"Loading BF16 32B with 29 blocks GPU and 35 CPU", gpu=29.5GiB`

Stage 5	默认 29	29 + 35	⟶ 出处 05_stage5/351_stage5.py:239–241, :397 · 351_stage5_crossed_v2/stage5_run.log ("offloaded to the cpu")
Stage 6	默认 29	29 + 35	⟶ 出处 06_stage6/351_stage6.py:159–161, :381 · jlens_archive_stage1_8/experiment_files/351_stage6_run.log
Stage 7.1B	代码写死 29	29 + 35	⟶ 出处 07_1_stage7_1/351_stage7_1B_terminal.py:524–546 · 351_stage7_1B_terminal/runtime.json `gpu_layers: 29`
Stage 7.2 / 7.3 打分	默认 29	29 + 35	⟶ 出处 07_2_stage7_2/351_stage7_2_score.py:102–104 · 351_stage7_2/config/gpu_scoring_runtime.json · 07_3_stage7_3/351_stage7_3_score.py:127–129 · 351_stage7_3/config/stage73b_runtime.json（均 `gpu_layers: 29`）
Stage 7.4、8D、8D supplement	不加载模型（CPU 统计）	—	⟶ 出处 grep `from_pretrained` 在 07_4_stage7_4/、08_stage8/351_stage8D*.py 无结果
Stage 8C	默认 29	29 + 35	⟶ 出处 08_stage8/351_stage8C.py:338–342, :448 · 351_stage8/config/stage8C_runtime.json `gpu_layers: 29`
Stage 9 FullFit 拟合	`device_map='auto'`（无 max_memory），`torch_dtype=bfloat16`	由 accelerate 自动放置	⟶ 出处 09_stage9/351_stage9.py:201。GPU 型号（U12，2026-10-30 查实）：全部在 A100 上；Fit A 第一段（0→4，10-18 16:34–16:55，作业 11191976@d1026）是 A100-40GB（OOM 报错 "total capacity of 39.49 GiB"），其余所有 Fit A / Fit B 段都在 d1028/d1029 的 A100-80GB（报错 "total capacity of 79.25 GiB"）⟶ 出处 351_stage9_full_A.log:127, 223, 453, 481 · 351_stage9_full_B.log:57, 85 · sacct -u zhang.zhiqi2 -S 2026-10-15 · 附录 I U12 **复核过**
Stage 10	`load_model()`：`device_map='auto'`，每卡 `max_memory` = 总显存 − 6 GiB（首/末卡）或 − 3 GiB（中间卡），`'cpu': '0GiB'`；放置里出现 cpu/disk 就报错。GPU 作业实际经 `351_stage10_fix1.py` 的 `load_model` 运行（同样的 max_memory，外加权重预读和 transformers 5.x 下 `hf_device_map` 缺失的修复）	整模型在 GPU；1×A100-80GB 或 2×A100-40GB 都允许	⟶ 出处 351_stage10.py:371–387 · 351_stage10_fix1.py:38–53；抽查 300 / 23,688 个 raw npz 的 meta：全部 `['NVIDIA A100-SXM4-80GB']`（本次计算）
Stage 11	`Ctx.__init__` → `s10.load_model()`（fix1 版）	同 Stage 10	⟶ 出处 11_stage11/361_stage11.py:46–48, :183；抽查 300 / 1,033 个 raw npz：全部 1×A100-80GB（本次计算）。CPU 真权重检查：⟶ 出处 stage11_cpu_realcheck.py:22 `device_map='cpu'`, bf16

Stage 12	沿用 `s11.Ctx`	同上；F1 author lens 拟合只允许单张 A100-80GB（≥70 GB）	⟶ 出处 `12_stage12/371_stage12.py:296` · `STATUS.md §3 Stage 13 段 "only a single A100-80GB … takes the author-lens fit"`；抽查 300 / 408 个 raw npz：全部 1×A100-80GB
Stage 13（M3、X1、X3、X4、X6、X7、judge、x4tok）	`Ctx.__init__` → `s10.load_model()`	同上	⟶ 出处 `13_stage13/381_stage13.py:378–387`；全部 265 个 raw npz 的 meta：176 个 1×A100-80GB，89 个 2×A100-80GB（模型被 device_map 拆到两张卡）（本次计算）

G1.3 `jlens` 包装

- 来源：`~/projects/jacobian-lens`（当前 env 里 `import jlens` 解析到 `/home/zhang.zhiqi2/projects/jacobian-lens/jlens/__init__.py`）。git 只有一个提交 `581d398 Initial release`；工作区改动只有一处：`jlens/__init__.py` 多一行 `from jlens import _stage10_compat`，副本在 GitHub 镜像 `code/jlens_package_patch/__init__.py.diff`（commit `b4b76ed`）。⟶ 出处 `git -C ~/projects/jacobian-lens log/diff --stat · jlens/_stage10_compat.py 文件头`
- `_stage10_compat` 只在运行脚本名以 `351_stage10` 开头时生效：给没有 `hf_device_map` 的模型补一个只读属性（参数/缓冲区所在设备的集合），不改任何计算。⟶ 出处 `jlens/_stage10_compat.py:1–35`
- `jlens.from_hf(hf, tok)` 把 HF 模型包成 `HFLensModel`：`encode(text, max_length=512)` = tokenizer(text, truncation=True)；`unembed(h)` = `lm_head(final_norm(h.to(bf16)))`；`forward` 调用文本主干、`use_cache=False`。⟶ 出处 `jlens/hf.py:157–174, :177–210`
- `force_bos=True`（默认）只在 `tokenizer.bos_token_id` 非空时把 `add_bos_token` 设成 True（⟶ 出处 `jlens/hf.py:105–110`）。Qwen3 tokenizer 的 `bos_token` 是 None，所以**不加 BOS**。Stage 10 每篇论文运行前都核对 `model.encode(prompt)` 与冻结的 `prompt_ids` 一致，不一致就报错（⟶ 出处 `351_stage10.py:556–559`）；protocol 写明 "tokenizer default, no BOS for Qwen3"（⟶ 出处 `351_stage10/config/protocol.json` `context`）。
- 残差记录：`jlens.hooks.ActivationRecorder(model.layers, at=layers)` 取 block 输出（⟶ 出处 `351_stage10.py:425–439`）；J-Lens 读出 = `lens.transport(h, l)` 后 `unembed`；logit lens = 直接 `unembed(h)`（⟶ 出处 `protocol.json` `readouts`）。

G1.4 chat 模板与 thinking：哪些实验用、哪些是裸提示

结论：**除了组长的标注（开放回答 + 多选题）和 Stage 13 的 Qwen 判断器，所有实验都是裸文本提示（不套 chat 模板，不加 BOS）。**在 `~/jlens_work` 全部 .py 里 grep `apply_chat_template|enable_thinking`，起草时只命中 `13_stage13/stage13_gpu_followup.py:65`（及其旧版 `.v1.py`）；2026-10-30 新增的判断器 v2 `13_stage13/stage13_judge2.py:55` 同样套 chat 模板、`enable_thinking=False`。

实验	chat 模板	thinking	生成/读法	出处
标注：开放回答	是（system + user 两条消息）	关（按 Dreaming Up Authors 代码包）：开放题的 system / user 文字与代码包 Qwen3 探针 `stage1_generate.py` 逐字相同，该脚本对 qwen3 调 `apply_chat_template(..., add_generation_prompt=True, enable_thinking=False)` ⟶ 出处 dreaming-up-authors-code-v0.zip: src/dua/probing/stages/stage1_generate.py:164–171, 252–284 · 附录 J 第 8 行。旁证：9,108 条 `generated_text` 里 0 条含 "think"（本次计算）。注意：标签由 elm 生成，elm 源码与 run.json 不在集群上，"elm 也关闭 thinking"是按提示逐字相同与数据一致推断的 **复核过**	贪心生成	⟶ 出处 `labels_20261015/…/step1_README.md "Sample prompts (complete, as sent to the model inside its chat template)"` · `README_first.md` 第 3 行
标注：多选题	是（一条 user 消息）	**未知/需确认：向组长确认（thinking 设置；"answer position" 是否位于空 `<think></think>` 块之后；Dreaming Up Authors 代码包的多选只有 GPT-4o API 版、解析回答文字，没有对应代码，U03 多选部分）**	读答案位置下一 token 分布	⟶ 出处 `step1_README.md "Kind team, order 0 -- one user message"`
Stage 1–9 全部作者提示	否	不适用	forward + teacher forcing	⟶ 出处 `01_*–09_*` 无 `apply_chat_template`；模板见 G4
Stage 10、11、12（含 N1/N2、F1 拟合文本、F2）	否	不适用	forward	⟶ 出处 `351_stage10.py:29` · `371_stage12.py:38, :56`

Stage 13 M3 / X1 / X4 / X6 / X7 / x4tok	否	不适用	forward	⟶ 出处 381_stage13.py:48–50, :123–124
Stage 13 X3 para_gen（改写标题）	**否**：裸文本 `PARA_PROMPT`	不适用（没有模板，也就没有 thinking 开关）	`hf.generate(ids, max_new_tokens=60, do_sample=False, pad_token_id=eos)`；只取第一行，去首尾空白和引号	⟶ 出处 381_stage13.py:22（文档串 "raw-text greedy, no chat template"），:276–285；已跑的 para_gen 输出 meta 的 `script_sha256 = a7b01a8f…` = 当前文件 = commit `fb8be6f`，所以跑的就是这段代码。**注意：任务说明里说 para_gen 用了 chat 模板 + enable_thinking=False，这与代码不符；用 chat 模板的只有下面的判断器。**
Stage 13 Qwen 判断器 JUDGE（X3 语义核对）	是：`apply_chat_template([{'role':'user',…}], add_generation_prompt=True, enable_thinking=False, tokenize=False)`，再 `tok(txt, add_special_tokens=False)`	关（模板因此在 assistant 开头插入空的 `<think>\n\n</think>\n\n`，本次渲染确认）	一次 forward，取最后位置 logits，只在 {Yes=9454, No=2753} 两个 token 上 softmax：p_yes = P(Yes)/(P(Yes)+P(No))；两种顺序各问一次	⟶ 出处 13_stage13/stage13_gpu_followup.py:25–29, :56–70 · commit 8d1a5c0, 44d084f（磁盘上的文件比 git HEAD 新：sha256 前缀 5e064427 vs 66d16514）

判断器状态：第一版只跑了 `--sample`（`381_stage13/followup/judge_sample.csv`，20 条，03:25 写入），`--all` 未跑。之后写了判断器 v2（`13_stage13/stage13_judge2.py`，同样套 chat 模板、关闭 thinking，作业 11501072）：在 A 组 20 条上定阈值（一致率 0.90），在用户新标注的 B 组 20 条上检验一致率只有 0.55（去掉边界项 0.79），低于用户定的 0.80，**判断器 v2 不用**，改走备用方案（人工核对 + 有/无敏感性），见 s13-x3。⟶ 出处 381_stage13/followup/judge2_summary.json · stage13_judge2.py:1–11 · PROGRESS.md 2026-10-30 15:00 条 **作废**（判断器路线，按规则不用）

G2 数据来源：9,108 篇论文从哪来

G2.1 组长标注包

- 文件：`~/jlens_work/00_cohort/zhiqi_qwen3-32b_labels_2026-10-15.zip`，9,130,689 字节，sha256 `58c1b5bc2cc61e6e5f5d276960c9340c770a9cf3bfa614d94873251dee9ee619`（本次计算）。zip 内文件时间 2026-10-17 19:20（数据）/ 21:18（README_first.md）。解压位置 `/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/`（另有 macOS 打包残留 `__MACOSX/`）。git：`experiment_files/labels_20261015/`，commit `e2c7156`。
- 包内两次运行的名字：开放回答 + 多选 = `2026-10-15_qwen3-32b_v3-measure-a100`（"filter, measure path (plan v3 step 1)"，代码 `elm.stages.filter.measure.report.write_readme`）；打标签 = `2026-10-15_qwen3-32b_v3_pilot`（"plan v3 step 2"，代码 `elm.stages.label.report`）。⟶ 出处 `step1_README.md 第 1–3 行 · label_README.md 第 1–3 行`
- 两次运行的 `run.json`（设置、提示词文件 sha256、代码版本、torch/transformers 版本）以及 `elm` 代码、`open_results_noise.jsonl`、`paper_labels.csv`（不带论文元数据的版本）都**不在包里**。**未知/需确认：向组长要 run.json（两次）、elm 代码版本、open_results_noise.jsonl（U01；2026-10-30 在 ~ 与 /scratch 下 find 也没有 elm/、panel.py 或标注运行的 run.json）** ⟶ 出处 附录 J "最重要的一点"

包内文件	字节	sha256（本次计算）	内容
paper_labels_with_papers.csv	3,312,650	`b4ff5e1d045c6f9ec2f42226e537dbb32be32d7df9684545917bad04b09faf0e`（与 ⟶ 出处 `351_all/351_audit.json` `source_sha256` 和 ⟶ 出处 `351_stage10/config/manifest.json` `inputs.labels_sha256` 一致）	每篇一行：元数据 + 全部标签列
open_results.jsonl	4,385,522	`2df45b29ebaf78de629536b6bf57af8cda1a21451b55f8752c40e1597c2b0a04`	开放回答原文与打分，9,108 行
mc_results.jsonl	23,543,395	`f0a28062ae9bc055d550d604079253819fb2fcc4596ae976d9ca7698b525d9b0`	多选每个顺序一行：team 36,432 + team_b 2,000
mc_team.jsonl	5,588,525	`a37090d2bd9573d231a1ac42ec2ed130ea59600aac9a38445cf424e84ea233f5`	每篇的 4 个选项（第一次抽取）
mc_team_b.jsonl	395,372	`b9b50ab049e2a159fda4d7449e8c9d9ec87420c321b3ad090e3ef9c1b631f1ce`	500 篇的第二次抽取
rules.yaml	250	`674adabea2fd3e4f74c89e2afe2a1318cb5ed3c628edfbee40fa99004425aec3`	阈值

bin_table.csv	2,419	`b9238370ecaa1d3a340dded3957855a36a316db1c0e60fa7d3addc0256dd3819`	每个引用档的计数
README_first.md / label_README.md / step1_README.md	2,449 / 16,576 / 7,410	`f01d7e61…` / `53e56986…` / `7263aca1…`	说明

G2.2 原始数据源：Dreaming Up Authors 的评测样本

这 9,108 篇不是我们抽的，也不是组长为本项目新抽的，而是 *Dreaming Up Authors*（EMNLP 2026）研究的 9,108 篇分层评测样本。

代码包原件（2026-10-30 核对）：`~/archives/dreaming-up-authors-code-v0.zip`，13,678,036 字节，sha256 `88b75daae08911f530adee2aadd75c125bf74efc295414f1473414dfd724d12d`（本次 sha256sum 一致）；包名 `dua`，版本 1.0.0；发布清单 `data/release/manifest.json`：`generated_at_utc 2026-08-29T01:51:44`、`git_commit 3379af2`、`seed 42`、`rows 9108`。用户说明：这个包由 Hana Feng 于 2026-09-28 邮件发送。`~/projects/dreaming-up-authors-code-v0/` 只是不完整的解压（空目录骨架 + egg-info，没有源码和数据），以 zip 为准。⟶ 出处 附录 J 文件头与"包的身份" · zip 内 data/release/manifest.json · PROGRESS.md 2026-10-30 14:30、16:30 条 复核过

标签不是这个包生成的。组长的标注由另一套代码 `elm` 生成（⟶ 出处 `label_README.md:3` `elm.stages.label.report` · `step1_README.md:3` `elm.stages.filter.measure.report`）；**elm 源码和两次运行的 run.json 都不在集群上**。所以 dua 包能确定的是论文样本、元数据字段、引用档、开放题提示与生成设置、NameMatcher；panel.py 的剥离规则、多选干扰团队的抽法、et al. 截断只能从标注包数据推断（见 G3）。⟶ 出处 附录 J "最重要的一点"

与 GitHub 镜像副本的对应（本次计算）：

- GitHub 镜像里有该研究的代码副本 `home_project/`，其中 `data/release/eval_sample_9108.csv.gz`（sha256 `da5934eb…f0ab2e9`，与同目录 `checksums.sha256` 一致；commit `e2c7156`）。
- 标注包 9,108 个 `paper_id` 全部在该样本里；`year`、`field`、`citations`、`citation_bin`、`n_authors`、`authors`（样本里是 JSON 列表，标注包里用 ";" 连接）逐篇 100% 相同；`title` 9,091 篇相同，17 篇只差 Unicode 空白（U+2005、U+2009、U+00A0 被换成普通空格或去掉末尾的）。
- 标注包说元数据来自 `data/prepared/papers_9108.csv`（⟶ 出处 `label_README.md:23` · `step1_README.md:35`）。17 个不同的标题全部满足"标注包标题 = `' '.join(eval_sample 标题.split())`"（U+00A0 ×6、U+2009 ×3、U+2005 ×1、换行 ×7 被折成单个空格并去首尾），其余字段逐字相同，所以 papers_9108.csv 就是 eval_sample 做了空白规范化的版本；Stage 10–13 的 `papers.parquet` 标题与标注包 9,108/9,108 相同，用的是规范化后的标题。⟶

出处 附录 J 第 2 行 · 附录 I U08 复核过 （U08 已解决，数据层面。组长提示的原文日志不在包里，"提示里也是规范化标题"是由两份 README 都写取自 papers_9108.csv 推断的）

原研究的构建流程（出处：⟶ 出处 home_project/config/config.yaml:7–64 · home_project/src/dua/data/filters.py:1–8, :20–23, :116–142 · home_project/src/dua/sampling/stratified.py · home_project/README.md:88–95, :346 · home_project/DATA_LICENSE.md）：

1. 从 Microsoft Open Academic Graph v2 随机取 9,999,863 条记录（`dataset.oag_pool_size`）。
2. 用 Semantic Scholar Academic Graph API 补全（快照"as of September 2025"）。
3. 五步过滤（顺序有关，比例相对上一步）：文档类型只留 journal / conference（去掉 56.9%）；作者数 1–20（含 20，去掉 0.3%；`MAX_AUTHORS = 20`）；FastText 英文且置信度 ≥ 0.8（去掉 47.9%）；作者名格式（去掉 0.8%）；OAG 与 Semantic Scholar 的标题和第一作者一致（去掉 13.5%）。剩 1,921,209 篇（`curated_size`）。注意：已发布报告 §3.1 写"去掉作者 ≥ 20 位的记录"，**有误**：代码保留 1–20 位（`MAX_AUTHORS = 20`，只有 `n > 20` 才剔除）；样本里恰为 20 位作者的 10 篇，>20 的 0 篇。⟶ 出处 zip: src/dua/data/filters.py:20–23, 128–138 filter_author_count · 附录 J 第 1 行 复核过
4. 8 个领域 × 13 个引用档分层，每格最多 100 篇（`per_stratum: 100`，理论上限 10,400），无放回；高引用格不足 100 篇的全取，最后 9,108 篇（104 格全部有论文，81 格满 100）。**原始抽样在收集用的 notebook 里 `RANDOM_SEED = None`，不可复现**；包内 `stratified.py` 是 seed 42 的重抽版（`config.yaml:6`），只与发布样本比较重叠度、不要求相同，发布样本中有 81 篇会被当前过滤器剔除。⟶ 出处 zip: src/dua/sampling/stratified.py:1–9（RANDOM_SEED=None 的说明）, 93–129, 189–217 verify_against · config/config.yaml:6, 62–64 · 附录 J 第 1 行 复核过

原研究对数据再分发的许可尚未确认（⟶ 出处 home_project/DATA_LICENSE.md "Bibliographic metadata: unresolved"）。

G2.3 每个字段的来源

标注包列	来源	出处
`paper_id`	Semantic Scholar `paperId`（40 位十六进制）	⟶ 出处 home_project/src/dua/sampling/stratified.py:153
`title`	Semantic Scholar `title`，缺失时用 OAG 标题；标注包又把 17 个标题里的 Unicode 空白规范化了（见上）	⟶ 出处 stratified.py:155 · 本次比对
`year`	Semantic Scholar `year`，缺失时用 OAG（原研究说两者只在约 0.02% 的语料上不一致，9,108 篇里有 1 篇 S2=2020、OAG=2019，用 2020）	⟶ 出处 stratified.py:156–167
`authors`, `n_authors`	Semantic Scholar 作者记录的 `name`，按 S2 顺序；`n_authors = len(authors)`。标注包里用 "; " 连接；拼写保持原样（如 "S. V. Diest"、"Karin G. Coifman"）	⟶ 出处 stratified.py:150, :170–171

`citations`	只用 Semantic Scholar `citationCount`（不回退到 OAG `n_citation`），2025 年 9 月快照	⟶ 出处 `stratified.py:24–36` · README.md:93–95
`citation_bin`	13 个指数间隔档，`lo`、`hi` 两端都含，最后一档开放：0-2, 3-4, 5-8, 9-16, 17-32, 33-64, 65-128, 129-256, 257-512, 513-1024, 1025-2048, 2049-4096, 4097+。按这 13 个边界用标注包的 citations 重新分档，9,108/9,108 与 `citation_bin` 相同；manifest 记 `citation_bin_recomputed_mismatches 0` 复核过	⟶ 出处 zip: `config/config.yaml:44–59` · `src/dua/bins.py:12–20` · 附录 J 第 9 行
`field`	S2 `fieldsOfStudy` 列表里按 S2 顺序第一个属于 8 个领域之一的条目（不一定是列表第一项）	⟶ 出处 `stratified.py:55–75`（`paper_field`）
其余列（`recall_level`, `m`, `h`, `u`, `hr2`, `fa1`, `mc_team_*`, `group`, `group_tight` …）	组长的 Qwen3-32B 标注运行	⟶ 出处 `label_README.md` 列表；见 G3

以上字段规则在代码包原件 zip 中逐条核对过（`to_release_row`，stratified.py:143–176；field 按"第一个属于八个领域的条目"复现 100%，严格取第一项只复现 98.31%）⟶ 出处 附录 J 第 3 行 复核过。

样本概况（本次计算，与已发布报告 §3 一致）：医学 1,230、生物 1,179、计算机 1,174、心理学 1,154、化学 1,134、数学 1,088、物理 1,082、材料 1,067；年份 1817–2020（中位数 2006）；引用 0–63,901（中位数 50.5）；作者数 1–20（均值 3.71）。

G2.4 从 9,108 到 6,841 篇分析队列

脚本 `00_cohort/351_prepare_all.py`（commit `e2c7156`），命令 `--labels …/paper_labels_with_papers.csv --outdir /scratch/zhang.zhiqi2/jlens/351_all`。规则（第 15–21 行）：

```
group_tight == 'target'  且 recall_level == 'R0'  → A（认出 · R0）
group_tight == 'target'  且 recall_level == 'R1'  → C（认出 · R1）
group_tight == 'control' 且 recall_level == 'R0'  → B（未认出 · R0）
group_tight == 'control' 且 recall_level == 'R1'  → D（未认出 · R1）
其他（guard 214 + excluded 2,053）                  → 'excluded'
对 A–D 断言：recall_level ∈ {R0, R1}，hr2 > 0.5，(R0 ⇔ m == 0)
```

结果（⟶ 出处 `351_all/351_audit.json`）：A 4,110、B 2,186、C 511、D 34，合计 **6,841**（`four_group_total`）；`excluded` 2,267 = guard 214 + excluded 2,053。输出 `351_four_groups.csv/.jsonl`（6,841 行）、`351_all_papers.csv`（9,108 行，加 `group_351`、`recognition_351` 列）、`351_bin_counts.csv`。脚本不做抽样、配对或 GPU 计算（`notes` 字段）。

与后续 Stage 的关系：Stage 4–8 用 A/B/C/D（各 Stage 再从中抽小样本，见各自章节）。Stage 10 冻结时读入**全部 9,108 篇**（按 `paper_id` 排序，保留所有标签列），分析时按 `group_tight` 分组：target 4,621（= A+C）、control 2,220（= B+D），合起来正好是同一批 6,841 篇；guard 和 excluded 只

作描述。⟶ 出处 351_stage10.py:104–118 · 351_stage10/config/protocol.json `cohort` · config/analysis_plan.json `grouping`

G3 标签流程：开放回答、姓名匹配、R0/R1/R2、多选题、分组

标签由组长的 `elm` 流程生成，我们原样使用。**elm 是另一套代码，不是 Dreaming Up Authors 的 dua 包；elm 源码和两次运行的 run.json 都不在集群上**（⟶ 出处 附录 J "最重要的一点"）。下面每条规则都标明是"包内文档写明"、"dua 代码包写明"、"本次从数据反推/核对"还是"未知"。

G3.1 开放回答的生成

提示（逐字，⟶ 出处 step1_README.md "Open question -- a system message and a user message"），套 Qwen3 chat 模板：

```
[system] You are a knowledgeable assistant. Answer factually and concisely. When asked for authors, return a semicolon separated list of full names only, with no additional text.
[user]   Who are the authors of the paper titled '{title}' which was published in {year}? Return a semicolon separated list of author names only.
```

参数	值	依据
解码	贪心（greedy）	⟶ 出处 README_first.md 第 3 行 · step1_README.md `generated_text` 列说明
max_new_tokens	**96**：dua 代码包 `MAX_NEW_TOKENS = 96`（注释 "up to ~20 authors × ~4 tokens each"），`MAX_PROMPT_TOKENS = 512`；数据一致：743 条 `finish_reason = length` 的回答 `n_tokens` 全部恰好是 96，stop 的最大 95 **复核过**（U02 已解决）	⟶ 出处 zip: src/dua/probing/stages/stage1_generate.py:170–171 · 本次计算 open_results.jsonl · 附录 J 第 8 行
停止条件	`stop` = 模型写出 end-of-turn token；`length` = 到达 token 上限。9,108 条中 stop 8,365、length 743	⟶ 出处 step1_README.md `finish_reason` · 本次计算
thinking	关：dua 的 `build_prompt` 对 qwen3 用 `enable_thinking=False`；贪心 `do_sample=False, temperature=None, top_p=None`（elm 的实际设置未见源码，见 G1.4）	⟶ 出处 zip: stage1_generate.py:252–284, 697–760

batch	推断 batch size 16：570 个 `batch_id`，569 个各 16 篇 + 1 个 4 篇（569×16 + 4 = 9,108）。包内文档说：400 篇在 batch size 1 下重答，45.5% 文本不同（"batching noise"）	本次计算 · ⟶ 出处 `step1_README.md "Result in 3 sentences" 第 3 条`
解析	`parse_class`：clean_list 8,316、cut_off 743、refusal 43、preamble 6（本次计数）；`names` = 剥离后找到的名字（规则在 `metrics/panel.py`，不在包里）	⟶ 出处 `step1_README.md` `parse_class`，`names`。从数据推断的 panel 行为（dua 包里没有 panel.py）：8,187 条 clean_list 的 `names` 恰好等于按分号切开；627 条 cut_off 丢掉了末尾被 96 token 截断的半个名字（另 116 条恰好截在分隔处）；6 条 preamble 去掉引导语但保留名字末尾句号；43 条 refusal 一律 `names = []`；129 条段内还有逗号的被再按逗号切碎；9,108 篇上恒有 `h = n_names − m`、`u = 真作者数 − m` ⟶ 出处 附录 J 第 5 行 **未知/需确认：向组长要 panel.py（上述剥离规则是从数据推断的，需组长确认；U04）**

G3.2 姓名匹配与 M / H / U / HR2

- 定义（包内）：**M** = 匹配上的真作者数；**H** = 回答里编造的名字数（不匹配任何真作者）；**U** = 没被写出的真作者数；**HR2 = (H + U) / (M + H + U)**；`exact` = (H = 0 且 U = 0)；`fa1` = 写出的第一个名字是真第一作者；`first_is_any_true` = 第一个名字是任一真作者。⟶ 出处 `step1_README.md` `m, h, u, hr2` 行 本次核对：9,108 行全部满足 `hr2 == (h+u)/(m+h+u)`，无分母为 0 的行。
- 匹配规则：标注流程的匹配代码不在包里。已发布报告的说法是"模糊姓名匹配（考虑姓、名的首字母、顺序、缩写和特殊字符）"。原研究仓库的匹配器 `home_project/src/dua/evaluate/matcher.py` 是逐级规则：L1 原串相同 → L2 Unicode 折叠后相同 → L3 再去标点后相同 → L3.5 姓 + 完整名 → L4 姓 + 名首字母 → L5 只有姓；姓前缀（van、de、der…）可去掉再比；按真作者顺序贪心一对一。原研究只接受 L1–L4（`evaluation.match_level_threshold: 4`，即至少"姓 + 名首字母"），L5 记为一个 H 加一个 U。⟶ 出处 `matcher.py:1–18, :110–150 · config.yaml evaluation`。**行为核对（U05 已解决）**：用 dua 包内 `Evaluator` 对标注包 open_results 的 `names` 重新打分，阈值 L4 时 (m, h, u) 与标注包逐篇相同 **9,095/9,108（99.86%）**，m 相同 9,106/9,108；换成 L3.5 只有 8,470、L5 只有 8,762。剩下 13 篇来自 dua NameSplitter 对逗号连接字符串的二次切分，不是匹配规则不同。所以 elm 的匹配在行为上就是 NameMatcher + L4（是否直接 import dua 未见源码，不影响结论）。⟶ 出处 `zip: src/dua/evaluate/matcher.py:256–359, pipeline.py:64–98 · 附录 J 第 4 行` 复核过
- 独立旁证：用户在 R0/R1 分析里自己重建了逐作者匹配（姓 + 名首字母、两种姓名顺序、一对一），与标注包的 `m` 在 99.3% 的 R1 论文上一致。⟶ 出处 `STATUS.md 2026-10-28 段 · 10_stage10/stage10_extra_r0r1.py · 351_stage10/reports/r0_r1_analysis.md`

G3.3 R0 / R1 / R2

```
R2 : HR2 ≤ 0.5                    (recall_failure_threshold = 0.5)
R1 : HR2 > 0.5 且 M > 0           (写对了一部分，但编造/遗漏更多)
```

```
R0 : M = 0                                   （一个真作者都没写出；此时 HR2 = 1）
```

⟶ 出处 label_README.md `recall_level` · rules.yaml `recall_failure_threshold: 0.5`。本次核对：按这条规则从 `m, hr2` 重算，9,108 篇 100% 与 `recall_level` 一致。全样本 R0 8,298、R1 596、R2 214（⟶ 出处 bin_table.csv 行 all）。

举例：真作者 3 人，模型写 2 个名字、其中 1 个对 → M=1, H=1, U=2, HR2 = 3/4 = 0.75 → R1（见下文例 1）。模型写 7 个名字全错 → M=0, H=7, U=3, HR2 = 1 → R0（例 2）。

G3.4 多选题：选项构造、4 种顺序、概率读法

选项（`kind = team`，⟶ 出处 mc_team.jsonl · step1_README.md `kind`）：`options[0]` = 真作者名单，`options[1..3]` = 另外 3 篇论文的真实作者名单（`false_sources` 给出这 3 篇的 paper_id）。所有 9,108 条 `format = "initial_surname"`：名缩成首字母、中间名首字母去掉（如 "Karin G. Coifman" → "K. Coifman"，"K. L. Baker" → "K. Baker"）。包内文档说干扰团队要求"same field, similar team size, citation bin and year"（⟶ 出处 README_first.md "The groups"）。具体匹配算法、容差和"不与真作者重名"的排除规则都不在标注包里，**也不在 dua 代码包里**（dua 的多选是 GPT-4o 版、三种不同的干扰构造，没有"其他论文的真实团队"这种干扰，也没有这三个字段）⟶ 出处 附录 J 第 6 行。下面的含义是从数据推断的（**需组长确认**）：`bin_window` = 放宽到的引用档差窗口（0/1/2 三类题中假团队与真论文的最大档差恰为 0/1/2；取 3 的题档差 0–12 都有，即不再限制）；`over_cap` = 本题超过"每篇论文最多作 3 次干扰"上限的假团队数（9,031 篇恰被用 3 次，42 篇用了 4 次，各题 over_cap 之和 25×1 + 7×2 + 1×3 = 42）；`et_al_mismatch` = 本题中 et al. 状态与真名单不同的假选项数；年份没有硬容差（像是就近优先）；没有按姓排除共同作者（假选项与真名单的 initial_surname 字符串完全相同 0/27,324，但同姓 341 个）；随机种子无从得知。**未知/需确认：向组长确认干扰团队的抽取规则原文（上述字段含义、年份是否参与、是否排除共同作者、随机种子；U06）**

本次对 27,324 个干扰团队的实际核对（以标注包元数据为准）：

条件	结果
同领域	27,324 / 27,324（100%）
作者数相同	25,899（94.8%）；最大差 11
同引用档	25,447（93.1%）；差 1 档 1,362；最大差 12 档
年份差	中位数 3 年，95 分位 25 年，最大 192 年（所以"年份相近"不是硬约束）
`bin_window` 取值	0: 8,025 篇；1: 762；2: 143；3: 178（推断：放宽到的档差窗口，3 = 不再限制；需组长确认）
`over_cap`	0: 9,075；1: 25；2: 7；3: 1
`et_al_mismatch`	0: 9,107；3: 1

提示（逐字，⟶ 出处 step1_README.md "Kind team, order 0 -- one user message"；在 chat 模板里作为一条 user 消息；含两道固定示例题）：

```
You are an academic assistant. Your task is to identify the correct author list
for an academic paper. You will be given a paper title, its publication year, and
four candidate author lists labelled A, B, C, and D. Exactly one is correct.

- Respond with a single letter: A, B, C, or D.
- Do not add any explanation, notes, or extra content.

Here are some examples:

Question: Which of the following is the author list of the paper titled 'Why men
are at higher risk for hepatocellular carcinoma?' which was published in 2012?
A: V. Keng, D. Largaespada, A. Villanueva
B: M. Sherman, J. Bruix, A. Forner
C: J. Llovet, S. Ricci, V. Mazzaferro
D: H. El-Serag, K. Rudolph, A. Marrero
Answer: A

Question: Which of the following is the author list of the paper titled 'Research
as an influence on teaching' which was published in 1984?
A: J. Centra
B: K. R. Jolls
C: P. Ramsden
D: M. Prosser
Answer: B

Now, answer the following question:
Question: Which of the following is the author list of the paper titled '{title}'
which was published in {year}?
A: {option shown as A}
B: {option shown as B}
C: {option shown as C}
D: {option shown as D}
Answer:
```

选项显示：名字用 ", " 连接；包内示例里 11 人和 13 人的名单只显示前 8 人再加 ", et al."。dua 代码包不截断（全部名字用 ", " 连接、不转成"首字母. 姓"）；数据上真名单 ≤ 8 人时假团队人数完全相同、> 8 人时假团队也 > 8 人，与"超过 8 人显示前 8 人 + et al."一致（推断）。⟶ 出处 附录 J 第 7 行 · zip: src/dua/recognition/run.py:81–85 **未知/需确认：向组长确认截断规则（恰为 8 人是否也截断、et al. 的确切写法；U07）**。注意 Stage 12 N1 打分用的是 `mc_team.jsonl` 里的**完整**名单，不截断（⟶ 出处 12_stage12/371_stage12.py:183–206）。

4 种顺序：`orders = [[0,1,3,2], [1,2,0,3], [2,3,1,0], [3,0,2,1]]`，第 k 项 = 显示在 A、B、C、D 位置上的规范选项编号（0 = 真名单）。所以真名单依次出现在 A、C、D、B，每个字母各一次；文档把它写作 "ABDC, BCAD, CDBA, DACB over the canonical options"。所有论文用同一组顺序。⟶ 出处 mc_team.jsonl `orders` · step1_README.md `order_index`, `order`

概率（⟶ 出处 step1_README.md `probs, mass, chosen, correct`）：在答案位置取下一 token 的全词表分布，

```
P(X) = p("X") + p(" X")            X ∈ {A, B, C, D}，不重新归一化
mass = P(A) + P(B) + P(C) + P(D)
chosen = argmax_X P(X)；correct = [chosen == 真名单的字母]
```

每篇的汇总（⟶ 出处 label_README.md 列说明）：

```
mc_team_a      = #{顺序 : 真字母概率最高 且 mass ≥ letter_mass_min(0.5)}      (0–4；
最高概率并列 = 没有选择，不算赢)
mc_team_p_true = (1/4) Σ_顺序 P(真字母)          (原始概率，未归一化)
mc_team_p_min  = min_顺序 P(真字母)
mc_team_margin = (1/4) Σ_顺序 [P(真字母) − max P(假字母)]
```

全样本 mass 低于 0.5 的顺序占 0.0%（⟶ 出处 label_README.md "Result in 3 sentences" 第 3 条）；有并列最高的论文 904 篇（`mc_team_n_ties` ⩾ 1，本次计数）。

G3.5 阈值与分组

rules.yaml 键	值	作用
`letter_mass_min`	0.5	某顺序 mass < 0.5 时不算赢
`strong_orders` / `pass_min_orders` / `uncertain_orders`	4 / 3 / 2	普通规则：strong a=4，pass a⩾3，uncertain a=2，fail a⩽1
`tightened.min_orders` / `tightened.min_p_true`	4 / 0.5	收紧规则：pass 需 a=4 且 p_true ⩾ 0.5；a=2 或 3 为 uncertain
`recall_failure_threshold`	0.5	R2 的 HR2 上限
`floor_bin` / `floor_max_pass_rate`	0-2 / 0.1	最低引用档的通过率检查（**未通过**：普通 38.6%，收紧 25.5%，都 > 10%；两种标签仍都写出）
`known_positive_min_pass_rate`	0.9	R2 论文多选通过率（实际 98.1%，通过）
`low_mass_max_share`	0.1	低 mass 顺序占比上限（实际 0.0%，通过）

```
group_tight（Stage 10 起使用）：
  guard    : recall_level = R2
  target   : R0/R1 且 mc_team_a = 4 且 mc_team_p_true ≥ 0.5
  control  : R0/R1 且 mc_team_a ≤ 1
  excluded : 其余（R0/R1 且 a ∈ {2,3}，或 a = 4 但 p_true < 0.5，或顺序不全）
group（普通规则，Stage 4–8 不用，Stage 10 敏感性分析用）: target = R0/R1 且 a ≥ 3；
control = R0/R1 且 a ≤ 1；excluded = a = 2 或不全
```

⟶ 出处 label_README.md `group`，`mc_team_outcome_tight`，`group_tight` · rules.yaml。本次核对：按上式从 `recall_level, mc_team_a, mc_team_p_true` 重算 `group_tight`，9,108 篇 100% 一致；R0/R1 中 a=4 但 p_true < 0.5 的只有 1 篇（归 excluded）。交叉表：target 4,621 = R0 4,110 + R1 511；control 2,220 = R0 2,186 + R1 34；excluded 2,053 = R0 2,002 + R1 51；guard 214 = R2。

其他标签列：`team_b` = 500 篇第二次独立抽取干扰团队，通过与否一致率 83.6%、kappa 0.63、两次都 strong 48.4%（⟶ 出处 `label_README.md 第 3 句`）；`coauthor` 类多选在本包里为空（⟶ 出处 `bin_table.csv n_coauthor_items = 0 · 351_stage10/config/analysis_plan.json not_available`）；`margin_*`、`own_margin`、`n_false_lists` 来自"gap run"，本包里为空（"No gap run: the margins are empty"）。

G3.6 两篇论文的完整真实例子

以下原始数据全部直接取自标注包（`open_results.jsonl`、`mc_team.jsonl`、`mc_results.jsonl`、`paper_labels_with_papers.csv`）。提示原文是按 step1_README 的模板代入本篇标题/年份/选项得到的；**包里没有每篇实际发送的提示日志**，所以"实际发送的字节"不能核对（特别是 chat 模板里的 thinking 部分）。

例 1：target • R1 • strong（paper_id `28578a60d8116010f736203c6a48b0b7c52a0f0b`）

项	值
元数据	"Affect dynamics, bereavement and resilience to loss"，2007，psychology，citations 95（65-128 档），3 位作者：Karin G. Coifman; G. Bonanno; Eshkol Rafaeli
开放回答 user 消息	`Who are the authors of the paper titled 'Affect dynamics, bereavement and resilience to loss' which was published in 2007? Return a semicolon separated list of author names only.`
模型输出	`George A. Bonanno; Karen E. Kaltman`（finish_reason stop，12 tokens，clean_list，batch_id 328）
匹配结果	names = ["George A. Bonanno", "Karen E. Kaltman"]；M=1（Bonanno：姓 + 名首字母 G 相同；按 dua 匹配器为 L4），H=1（Kaltman），U=2（Coifman、Rafaeli）；HR2 = 3/4 = 0.75；fa1 = 0；first_is_any_true = 1；own_first_name "G. Bonanno"。逐个名字匹配到哪一级包里没有记录（只有计数）。
recall_level	R1（M > 0，HR2 > 0.5）
多选选项（规范顺序）	0（真）: K. Coifman, G. Bonanno, E. Rafaeli 1: H. Sagar, J. Schofield, H. Snyder（来源 `ef36f6c2…`，psychology，1983，3 人，65-128） 2: F. Mcglynn, W. Mealiea, M. Nawas（`6930cf3f…`，psychology，1969，3 人，33-64） 3: D. Bloch, E. Silber, S. Perry（`323e1c94…`，psychology，1956，3 人，129-256） `bin_window` = 1（两个干扰团队来自相邻引用档；年份差 24–51 年）

顺序 0 的问题部分	`Question: Which of the following is the author list of the paper titled 'Affect dynamics, bereavement and resilience to loss' which was published in 2007?`
	`A: K. Coifman, G. Bonanno, E. Rafaeli`
	`B: H. Sagar, J. Schofield, H. Snyder`
	`C: D. Bloch, E. Silber, S. Perry`
	`D: F. Mcglynn, W. Mealiea, M. Nawas`
	`Answer:`

<table>
<tr><td>4 个顺序的读数</td><td>
<table>
<tr><th>order_index</th><th>order</th><th>真字母</th><th>P(A)</th><th>P(B)</th><th>P(C)</th><th>P(D)</th><th>mass</th><th>chosen</th><th>correct</th></tr>
<tr><td>0</td><td>[0,1,3,2]</td><td>A</td><td>0.99980</td><td>1.58e-4</td><td>3.54e-5</td><td>3.65e-8</td><td>0.99999</td><td>A</td><td>1</td></tr>
<tr><td>1</td><td>[1,2,0,3]</td><td>C</td><td>4.54e-5</td><td>2.56e-6</td><td>0.99985</td><td>6.14e-6</td><td>0.99990</td><td>C</td><td>1</td></tr>
<tr><td>2</td><td>[2,3,1,0]</td><td>D</td><td>2.56e-6</td><td>3.54e-5</td><td>4.78e-6</td><td>0.99986</td><td>0.99990</td><td>D</td><td>1</td></tr>
<tr><td>3</td><td>[3,0,2,1]</td><td>B</td><td>5.04e-7</td><td>0.99983</td><td>7.89e-6</td><td>1.37e-6</td><td>0.99984</td><td>B</td><td>1</td></tr>
</table>
"A" 与 " A" 分开的概率：如顺序 0 中 p("A") = 0.999801，p(" A") = 5.6e-15（带空格的变体几乎为 0）。top_tokens 第 4 名常是 "Answer"（id 16141）。
</td></tr>
<tr><td>汇总</td><td>mc_team_a = 4；p_true = 0.99984；p_min = 0.99980；margin = 0.99977；mass_min 0.99984；n_ties 0 → mc_team_outcome = strong，mc_team_outcome_tight = strong</td></tr>
<tr><td>最终标签</td><td>group = target；group_tight = target；351_prepare_all → C（认出 · R1）；在 Stage 11 样本里（shard 20）</td></tr>
</table>

例 2：control · R0 · fail（paper_id `04c94c62eb7e8e84f076612db5fd62015a3e933b`）

项	值
元数据	"Personality and drugs of choice"，2008，psychology，citations 36（33-64 档），3 位作者：C. Hopwood; K. L. Baker; L. Morey
开放回答 user 消息	`Who are the authors of the paper titled 'Personality and drugs of choice' which was published in 2008? Return a semicolon separated list of author names only.`
模型输出	`David R. Jacobs; David A. Klenosky; David M. Almeida; Karen A. Matthews; Mostafa A. Hussein; Gregory A. Miller; Sharon L. Manuck`（stop，41 tokens，clean_list，batch_id 260）
匹配结果	7 个名字都不匹配：M=0，H=7，U=3，HR2 = 10/10 = 1.0；fa1 = 0；first_is_any_true = 0；own_first_name "D. Jacobs"
recall_level	R0
多选选项（规范顺序）	0（真）: C. Hopwood, K. Baker, L. Morey 1: M. Grosjean, J. Zwickel, W. Prinz（`043d8d70…`，2008，3 人，33-64） 2: P. Olszewski, H. Schiöth, A. Levine（`535a6198…`，2008，3 人，33-64） 3: A. Landis, K. Parker, S. Dunbar（`9ed292f6…`，2009，3 人，33-64） `bin_window` = 0

<table>
<tr><td>4 个顺序的读数</td><td>
<table>
<tr><th>order_index</th><th>order</th><th>真字母</th><th>P(A)</th><th>P(B)</th><th>P(C)</th><th>P(D)</th><th>mass</th><th>chosen</th><th>correct</th></tr>
<tr><td>0</td><td>[0,1,3,2]</td><td>A</td><td>0.6543</td><td>0.2407</td><td>0.0690</td><td>0.0326</td><td>0.9966</td><td>A</td><td>1</td></tr>
<tr><td>1</td><td>[1,2,0,3]</td><td>C</td><td>0.3237</td><td>0.3237</td><td>0.2521</td><td>0.0928</td><td>0.9924</td><td>A（A、B 并列最高）</td><td>0</td></tr>
<tr><td>2</td><td>[2,3,1,0]</td><td>D</td><td>0.8405</td><td>0.1137</td><td>0.0254</td><td>0.0198</td><td>0.9994</td><td>A</td><td>0</td></tr>
<tr><td>3</td><td>[3,0,2,1]</td><td>B</td><td>0.4226</td><td>0.1211</td><td>0.1211</td><td>0.3291</td><td>0.9940</td><td>A</td><td>0</td></tr>
</table>
模型在 4 个顺序里有 3 次选 A（位置偏好），只有真名单恰好在 A 时答对。
</td></tr>
<tr><td>汇总</td><td>mc_team_a = 1；p_true = (0.6543 + 0.2521 + 0.0198 + 0.1211)/4 = 0.2618；p_min = 0.0198；margin = −0.195；n_ties = 1（顺序 1）；consistent_wrong = False → outcome = fail（两种规则都是）</td></tr>
<tr><td>最终标签</td><td>group = control；group_tight = control；351_prepare_all → B（未认出 · R0）；在 Stage 11 样本里</td></tr>
</table>

G4 提示词与 token 位置

G4.1 全部提示模板（逐字）

除标注两项和判断器外，都是裸文本、无 BOS（G1.3–G1.4）。`\n` 表示换行符。

名称 / 用在哪里	模板原文	出处
作者问法 `PROMPT`：Stage 3 公开/自拟合 lens 的作者任务、Stage 4、5、6、7.1B、7.2、7.3、8C、10（true / false_k 条件）、11、12（P1、P2、N1-true、F2）	`Who are the authors of the paper titled "{title}" ({year})?` `Authors:`	⟶ 出处 10_stage10/351_stage10.py:29 · 04_stage4/351_stage4_readout.py:12 · 05_stage5/351_stage5.py:11 · 06_stage6/351_stage6.py:13 · 07_1/351_stage7_1B_terminal.py:14 · 07_2/351_stage7_2_score.py:11 · 07_3/351_stage7_3_score.py:11 · 01_public_lens_validation/351_validate_public_lens.py:17 等 · 351_all/351_stage4_artifacts/config/run_config.json `prompt_template`。最早出现在 Stage 3 的 01_* 脚本里。**选择过程未记录，未系统比较其他问法**（Stage 2 没有独立目录和记录；按用户 2026-10-30 的决定照此写明）⟶ 出处 PROGRESS.md 2026-10-30 14:00 条 U11 复核过
删标题 `DELETE_PROMPT`：Stage 10 delete 条件、Stage 12 N1-delete	`Who are the authors of the paper ({year})?` `Authors:`	⟶ 出处 351_stage10.py:244
假标题 false_0..4（Stage 10/11/12/13 X6）	同 `PROMPT`，`{title}` 换成供体论文的标题，`{year}` 保留本篇的年份	⟶ 出处 351_stage10.py:323–325 · conditions_manifest.json `false_title_year`
名字接续（Stage 10 起每位作者单独）	`PROMPT` 之后接 `" " + 作者名`（作者名取自标注包 `authors` 列，按 ";" 切开、strip，拼写不改）	⟶ 出处 351_stage10.py:118–122 · protocol.json `context`, `authors`
Stage 11 E3 作者连写 / Stage 12 N1 选项连写	`{PROMPT 或 DELETE_PROMPT} A1, A2, A3, …` 即 " A1" + "," + " A2" + "," + …；每个名字的 token 与单独打分时完全相同（`encode(" " + 名字)`），名字之间插入 "," 的 token（id 11，`assert self.comma == [11]`）	⟶ 出处 11_stage11/361_stage11.py:187–188, :286–293 · 12_stage12/371_stage12.py:190–206

Stage 12 F1 author lens 的拟合文本（40 篇，种子 12，不在 Stage 11 样本内的 target/control）	`Who are the authors of the paper titled "{title}" ({year})?`
	`Authors: {作者1}, {作者2}, …`
	
	（`PROMPT + ' ' + ', '.join(authors)`）
		⟶ 出处 371_stage12.py:50–57

Stage 12 N2 纯名字先验 `PURE_PROMPT`	`The following is the name of a researcher.`
	`Name:`
	
	之后接 `" " + 作者名`（11 个 prompt token）
		⟶ 出处 371_stage12.py:38, :213–221

Stage 13 M3 三种问法 `TASKS`（不含年份；标题前都是 9 个 token）	`authors: Who are the authors of the paper titled "{t}"?`
	`Authors:`
	`year:  In what year was the research paper titled "{t}" published?`
	`Year:`
	`field:  In what research field is the paper titled "{t}"?`
	`Field:`
		⟶ 出处 13_stage13/381_stage13.py:48–50。注意：同文件的文档串（第 4–6 行）还写着最初版本 "In which year was the paper titled …" / "Which research field does the paper titled … belong to?"；代码在 commit `43c8f0c` 改成现在的写法（为了三种问法标题前都是 9 个 token）。全部 72 个 task 输出的 `script_sha256 = a7b01a8f…` = 当前文件（commit `fb8be6f`），所以实际运行的是表中写法。**X1、X3 para_run、X4 的 "authors prompt" 也是这一条（不含年份），不是 Stage 10 的 `PROMPT`。**

Stage 13 X4 patchscope 目标提示 `PS_TARGETS`	`year:  Paper: X`
	`The paper was published in the year`
	`author: Paper: X`
	`The first author of the paper is`
	`field:  Paper: X`
	`Research field:`
	
	占位符是 " X" 这个 token（三条里都在位置 2）
		⟶ 出处 381_stage13.py:123–124, :243–247；本次 tokenizer 核对：`['Paper', ':', ' X', '\n', …]`

Stage 13 X3 改写 `PARA_PROMPT`（裸文本，贪心，max_new_tokens 60）	`Rewrite the title of a research paper using different words while keeping exactly the same meaning. Do not add any information.`
	`Original title: {t}`
	`Rewritten title:`
		⟶ 出处 381_stage13.py:125–126, :276–285

Stage 13 Qwen 判断器 `JUDGE`（chat 模板，thinking 关）	``` Do these two research paper titles have exactly the same meaning? Rules: any added, removed or changed specific information counts as a change. A technical term replaced by a near-synonym that does not keep its precise technical sense (for example "generation" of a group replaced by "creation") counts as a change. Only rewording that keeps every fact and every technical term's precise meaning counts as the same meaning. Title A: {a} Title B: {b} Answer with Yes or No. ``` 套模板后实际字符串： ``` <|im_start|>user {上面的 JUDGE}<|im_end|> <|im_start|>assistant <think> </think> ```	⟶ 出处 13_stage13/stage13_gpu_followup.py:25–29, :63–66 · 本次渲染核对（tokenizer.apply_chat_template, enable_thinking=False）
标注：开放回答（chat，system + user）	见 G3.1	⟶ 出处 step1_README.md
标注：多选题（chat，user）	见 G3.4	⟶ 出处 step1_README.md

G4.2 token 对齐方式与关键位置的定义

- **上下文与 teacher forcing**（Stage 10 起；文档串称"each author alone after "Authors:", as Stage 6"）：`input_ids = prompt_ids + name_ids[:-1]`；名字第 j 个 token 在位置 `len(prompt_ids) − 1 + j` 被预测（j = 0 在 ":" 处预测）。最后一个名字 token 只作目标，不作输入。⟶ 出处 351_stage10.py:563 · protocol.json `input_ids`

- **标题位置**（Stage 11 `title_positions`）：用 offset mapping 取与标题字符区间重叠的所有 token，区间 = `'titled "'` 之后到最后一个 `'" ('` 之前。**标题最后一个 token** = 其中最大者。⟶ 出处 `361_stage11.py:89–94`
- **Stage 13 的 `build()`**：区间 = `'titled "'` 之后的 `len(title)` 个字符；`it` = 最后一个重叠 token；`colon = len(ids) − 1`，并检查它解码后是 ":"。`title_span = range(PREFIX_TOKENS, it + 1)`，`PREFIX_TOKENS = 9`（`freeze()` 断言三种问法 `'… titled "'` 前缀都是 9 个 token）。⟶ 出处 `381_stage13.py:51, :80–91, :138–140, :149–151`
- **":"** = prompt 的最后一个 token（Stage 10–13 都是 `len(ids) − 1`）。**名字位置** = ":" 之后的名字输入位置（加上 ":" 本身就是全部"预测名字 token 的位置"）。
- **对齐的尾段**（Stage 11 E1、Stage 12 P1/P2）：true 与 false_0 prompt 的公共后缀，截到两边都在最后一个标题 token 之后，长度 `tail_prompt`，再加名字输入。Stage 11 样本 1,186 篇中 9 个 token 708 篇、8 个 token 478 篇（⟶ 出处 `361_stage11/config/sample.parquet` `tail_prompt` · `361_stage11.py:121–125`）。Stage 12 P1 从尾往前编号 p1 = ":"、p2 = "Authors"、p3 = ")?\n"、p4–p7 = 年份的 4 个数字（从个位往前）、p8 = " ("、p9 = 闭引号；尾段只有 8 个 token 的论文没有 p9（⟶ 出处 `371_stage12.py:115–120` · `MAX_TAIL = 9`）。

本次核对发现的边界情况（对 9,108 篇 Stage 10 prompt 逐篇 tokenize）：

- 标题第一个 token 在位置 9 的有 9,088 篇；**20 篇在位置 8**：标题以 `[ ( " ' # * ` ` 开头时，这个字符与前面的 `"` 合成一个 token（如 `"[`）。`title_positions` 按 offset 重叠会把位置 8 算进标题；Stage 13 的 `title_span = range(9, it+1)` 则从 9 开始，会漏掉位置 8。涉及 Stage 11 样本 1 篇（`4be6e776…`，"'Tiger tail' pattern …"），另有 6 篇作为 X1/X4 的 partner 或 random_c 出现（`381_stage13/config/pairs.parquet`）。
- **2,412 篇（26.5%）的最后一个标题 token 同时包含闭引号**：标题以 "." 或 ")" 等结尾时与 `"` 合成 `."`、`)"`。这些论文的"标题最后一个 token"也是闭引号所在的 token，标题之后只剩 8 个 token（其余 6,696 篇是 9 个）。这就是上面 tail 为 8 或 9 的来源。
- Stage 13 三种问法下，标题占的位置和 token id 在全部 9,108 篇上完全相同（0 篇不同），与 `PREFIX_TOKENS` 的设计一致；上述 20 篇 / 2,412 篇的情况在三种问法下同样存在。

G4.3 真实例子：Stage 10 作者提示 + 第一作者的 token 表

论文 = 例 1（`28578a60…`，"Affect dynamics, bereavement and resilience to loss"，2007），第一作者 "Karin G. Coifman"。用 `AutoTokenizer.from_pretrained('Qwen/Qwen3-32B', revision=REV)`（HF_HOME=/scratch/zhang.zhiqi2/hf，HF_HUB_OFFLINE=1，只加载 tokenizer）生成；prompt ids 与 `351_stage10/cohort/papers.parquet` 冻结的 `prompt_ids` 完全相同（29 个）；名字 token 与 `cohort/name_tokens.parquet` 的 row 25025–25031 完全相同。最后两列是 Stage 10 `final`（L63）读出的金标准 token 名次，取自 `351_stage10/raw/final/L63/shard_011.npz` 和 `raw/cond_false_0/final/L63/shard_011.npz`。

位置	token id	文本	角色	说明	作为目标时的名次（true / false_0）
0	15191	`Who`	prefix		
1	525	`␣are`	prefix		
2	279	`␣the`	prefix		
3	12014	`␣authors`	prefix		
4	315	`␣of`	prefix		
5	279	`␣the`	prefix		
6	5567	`␣paper`	prefix		
7	24849	`␣titled`	prefix	"titled" 本身	
8	330	`␣"`	prefix	"titled \"" 的开引号；前缀共 9 个 token（0–8）= `PREFIX_TOKENS`	
9	32	`A`	title	标题第一个 token	
10	1705	`ffect`	title		
11	29195	`␣dynamics`	title		
12	11	`,`	title		
13	26933	`␣bere`	title		
14	402	`av`	title		
15	1114	`ement`	title		
16	323	`␣and`	title		
17	54962	`␣resilience`	title		
18	311	`␣to`	title		
19	4709	`␣loss`	**title_last**	Stage 11 E1t / E4 / F2-title、Stage 13 M3 / X4 读的"标题最后一个 token"（`title_pos` = [9…19]）	
20	1	`"`	tail	闭引号，P1 的 p9；尾段起点（`tail_prompt` = 9）	
21	320	`␣(`	tail	p8	
22	17	`2`	tail	p7（年份数字逐个成 token）	
23	15	`0`	tail	p6	
24	15	`0`	tail	p5	
25	22	`7`	tail	p4	
26	86427	`)?\n`	tail	p3（")?" 和换行是一个 token）	

27	57890	`Authors`	tail	p2	
28	25	`:`	**colon**	p1；prompt 最后一个 token（`len(ids)-1`）；在这里预测名字第 0 个 token	
29	13225	`␣Kar`	name（given_first）	j=0，由位置 28 预测	83 / 300
30	258	`in`	name（given_cont）	j=1，由 29 预测	2 / 1
31	479	`␣G`	name（initial_first）	j=2	25 / 20
32	13	`.`	name（punct）	j=3	1 / 1
33	3539	`␣Co`	name（surname_first）	j=4	61 / 330
34	333	`if`	name（surname_cont）	j=5	1 / 115
35	1515	`man`	name（surname_cont）	j=6，**只作目标**，不进入 input_ids（输入长度 35 = 29 + 7 − 1）	1 / 1

（␣ = 前导空格。）同一篇在 Stage 13 authors 问法下是 24 个 token：0–8 前缀、9–19 标题（`it` = 19）、20 `"`、21 `?\n`、22 `Authors`、23 `:`（`colon` = 23）。删标题 prompt 是 15 个 token：`Who ␣are ␣the ␣authors ␣of ␣the ␣paper ␣( 2 0 0 7 )?\n Authors :`。（本次计算）

其他答案的切分（本次计算）：M3/X4 的年份答案 `encode(" 2007")` = `[␣, 2, 0, 0, 7]` 共 5 个 token（第一个是单独的空格 token，也被打分）；领域答案 `" psychology"` = 1 个 token，`" computer science"` = 2 个；M3 的作者答案只取第一作者的第一个 token（`au_ids = [first[0][2][0]]`），X4 作者答案是第一作者的全部名字 token。⟶ 出处 381_stage13.py:98–100, :236–239

G5 打分：log10(1/名次) 与各级聚合

G5.1 名次与 log10(1/名次)

```
对一个读出（readout r、层 l）在某位置给出的 logits z ∈ ℝ^V（V = 151,936，lm_head 全部输出行）和金标准 token g：
  rank   = 1 + #{ v : z_v > z_g }                    （严格大于；并列不往后排 = 对金标准有利）
  ties   = #{ v ≠ g : z_v = z_g }                     （另存，不进入 rank）
  nlr    = log10(1 / rank) = −log10(rank)             （rank 1 → 0；rank 10 → −1；最小 −log10(151,936) ≈ −5.18）
  logprob = log_softmax(z)_g                          （另存；不同读出之间不可比）
```

- 代码：`score()` ⟶ 出处 351_stage10.py:442–449；protocol `rank`, `ties`, `logprob` 三项写明这一定义；Stage 11–13 用同一函数（`s11.ranks` 调 `s10.score`，⟶ 出处 361_stage11.py:192–193）或相同表达式（⟶ 出处 381_stage13.py:200–205 `score_tf`；stage13_gpu_followup.py `_x4tok`）。
- **不去掉特殊 token，也不去掉 267 个填充行**：比较对象是 lm_head 的全部 151,936 个输出。logits 是 bf16 的 lm_head 输出转成 float（`model.unembed(h).float()`），所以并列较常见：例 1 第一作者的 " Kar" 在 true 条件下 rank 83、并列 6 个（G4.3 数据）。Stage 10 parity 修订：快路径与逐前缀参考路径最大 |Δlogit| 0.16–0.37（约 1–1.5 个 bf16 单位），容差在看到数据后从 0.25 放宽到 0.5（用户 2026-10-25 批准）。⟶ 出处 351_stage10/config/analysis_plan.json `amendments` · reports/parity.json `tolerance_amendment`
- 主指标的演变：Stage 10 冻结时主指标写的是"每篇 Σ 1/rank"；2026-10-23 在分析层修订为 log10(1/rank)，理由是 1/rank 只看前几名、而 log 名次能反映 50000 → 500 这类倍数变化；raw 文件存的是名次，修订不改任何原始输出。⟶ 出处 protocol.json `primary_metric` · analysis_plan.json `amends`, `primary_metric`, `reason` Stage 4–8 用的是 RR = 1/rank 和 JMRR（作者内 token 平均 RR 再对作者平均），见各 Stage 章节（⟶ 出处 06_stage6/351_stage6.py:120–123, :223–231）。

G5.2 名字怎么切 token、token 角色

- 作者列表：标注包 `authors` 列按 ";" 切开、strip、丢空串，拼写保持原样。每位作者：`tok(" " + 作者名, add_special_tokens=False, return_offsets_mapping=True)`。排除条件：名字 > 32 个 token（`MAX_NAME_TOKENS`）或上下文 > 512 个 token（`MAX_CONTEXT`）——实际被排除 0 位（`cohort/excluded_authors.csv` 为空）。同一篇里重名的作者标 `duplicate_in_paper`（237 个上下文，本次计数），照常打分。⟶ 出处 351_stage10.py:31–32, :118–131 · manifest.json `n_excluded_authors: 0`

- 规模：9,108 篇、33,811 个作者上下文、149,999 个名字 token、72 个 shard（每 shard 128 篇）。⟶ 出处 351_stage10/config/manifest.json
- 角色 `token_roles()`（⟶ 出处 351_stage10.py:80–101）：先把 `" " + 名字` 按空白切成词；**词角色**：最后一个词 = surname；在 `PARTICLES`（de, da, di, do, dos, das, del, della, der, den, van, von, le, la, du, ten, ter, bin, al, el, y）里 = particle；匹配 `^[^\W\d_]\.?$`（单个字母，可带点）= initial；其余 = given。**token 角色**：token 文本全是空白/非字母数字（`^[\s\W_]+$`）= `punct`；否则为"词角色_first"（该词的第一个非标点 token）或"词角色_cont"（同一词的后续 token）。

role 取值	含义	token 数（name_tokens.parquet，本次计数）
`surname_first`	姓（最后一个词）的第一个 token	33,810
`surname_cont`	姓的后续 token	39,995
`initial_first`	首字母词（如 "G."）的字母 token；其后的 "." 记为 punct	28,382
`given_first` / `given_cont`	完整名（非首字母、非最后一个词）的第一个 / 后续 token	9,431 / 9,357
`particle_first`	姓氏前缀词（van、de…）的第一个 token	323
`punct`	标点 token（"." 28,216 个，"-" 167，" - " 134 等）	28,701
`initial_cont`、`particle_cont`	代码允许但本 cohort 中 0 个	0

本次发现的小问题：Qwen 的字节级 BPE 会把部分非 ASCII 字符拆成两个字节 token，单独解码是 "\ufffd"（U+FFFD），会被 `PUNCT` 规则归为 `punct`：共 89 个 token、25 篇论文。极端例子 "J. Marcos do Ó"（context 5845）：姓 "Ó" 的两个字节 token 都成了 punct，所以这个上下文没有 `surname_first`（这就是 33,810 比 33,811 少 1 的原因）。这些 token 也进入了"标点阴性对照"（89 / 28,701 ≈ 0.3%）。

G5.3 单独打分、连写打分与聚合

- **单独打分（Stage 10 起的默认）**：每位作者单独接在 "Authors:" 之后，与其他作者无关；对每个名字 token 用 teacher forcing 取名次。⟶ 出处 351_stage10.py 文档串第 3–5 行 · protocol.json `context`
- **连写打分（Stage 11 E3、Stage 12 N1）**：`chain_ids()` 把同一篇的全部作者按顺序接成 `prompt + 名字1 + "," + 名字2 + …`；每个名字 token 在它前一个位置被预测并打分；"," 只作输入、不打分；输出保留 Stage 10 的 `row_id`，所以连写分数与单独分数逐 token 配对。连写里第一位作者的上下文与单独打分完全相同，smoke 检查要求两者名次一致率 ⩾ 0.75、平均 $|\Delta \log_{10}\text{rank}| \leqslant 0.05$（不达标记 WARN，不停 GPU）。⟶ 出处 361_stage11.py:286–305, :439–444, :470

- **一篇论文的分数**（`paper_scores()`，⟶ 出处 `351_stage10.py:651–667`）：

```
mean_nlr(p, r, l)         = 该论文所有被打分作者的所有名字 token 的 nlr 的平均
← 主指标（token 加权，punct 也在内）
author_mean_nlr(p, r, l) = (1/A_p) Σ_a [ 该作者各 token nlr 的平均 ]
← 次要指标（作者加权）
mean_nlr_<role>           = 只取某一角色的 token 再平均（surname_first 等）
sum_nlr, sum_rr           = 只用于同一篇论文内比较不同层/读出（token 集相同），不跨论
文比较
```

  所以主分析是"**对 token 平均**"，不是对作者平均；作者平均作为次要指标一并报告（⟶ 出处 `analysis_plan.json` `between_papers_and_groups`，`within_paper`）。组间与层间的结果是先得到每篇的值，再对论文求平均（`group_means`，:694–705）。

- **Stage 13 的 `score_tf`**：对给定答案 token 序列做 teacher forcing，在模型输出（最后一层）上取 `mean_j −log10(rank_j)`，即对该答案的 token 平均（⟶ 出处 `381_stage13.py:200–206`）。**Stage 12 N1**：每个选项算 token 平均、第一作者 token 平均、第一个 token 三个值（⟶ 出处 `371_stage12.py:203–206`）。

G5.4 标题特异信号与组间比较

```
对每篇论文 p、读出 r、层 l、指标 m（主指标 m = mean_nlr）：
  true_m   = m(真标题)
  false_m  = (1/5) Σ_{k=0..4} m(false_k)        K_FALSE = 5 个按元数据匹配的假标题，年
份保留本篇的
  delete_m = m(删标题)
  spec_m   = true_m − false_m                   ← 标题特异信号
  total_m  = true_m − delete_m
  topic_m  = false_m − delete_m
  spec_strict_m = true_m − mean{ m(false_k) : 供体属于 strict 层 }（没有 strict 供体
则为 NaN）
  tail_fraction = (1 + #{k : mean_nlr(false_k) ≥ mean_nlr(true)}) / 6       （参考
尾部比例，不是 p 值）
```

七个条件用的是**同一组名字 token**（同一篇、同样的作者和 token），只换 prompt，所以名字本身好不好猜在差值里抵消。⟶ 出处 `351_stage10.py:838–870`（`analyze_title`）· `:243–248`（`K_FALSE, DELETE_PROMPT, TIERS, NEAR_DUP_JACCARD`）· 结果 `351_stage10/summary_title/paper_title_contrasts.parquet`

假标题的匹配（`match_false_titles`，:255–307）：同领域；排除自身、与本篇共享任何姓氏的论文、标题词 Jaccard（≥4 字母的词，ASCII 化小写）> 0.6 的论文；代价 `|Δyear|/5 + |Δn_authors|/max(1, 0.25·n) + |Δtitle_tokens|/max(1, 0.2·len) + 1.5·|Δbin|`；按 strict → medium → loose → nearest 四层，每层反复做最小代价指派（`scipy.optimize.linear_sum_assignment`），每个供体最多用 5 次（nearest 层 7 次）；`false_0`

= 层级最好、代价最低的那个。具体层级容差见 ⟶ 出处 `config/conditions_manifest.json` `matching.tiers`，属 Stage 10 章节。

target − control 比较（`stratified_diff`，:670–691）：只在两组都 ≥ 30 篇的引用档内做差（`MIN_STRATUM = 30`，主分析为 0-2 … 513-1024 共 10 档），按 `w_b = n_t·n_c/(n_t+n_c)` 归一化加权；95% CI = 在"档 × 组"内对论文有放回重抽 2,000 次（`B_BOOT = 2000`）的 2.5/97.5 分位；标题分析的随机种子 3511（:875）。

G5.5 逐步算例（例 1，readout = final，L63）

原始名次取自 `351_stage10/raw/[cond_*/]final/L63/shard_011.npz`（row 25025–25040，GPU meta：A100-SXM4-80GB），3 位作者共 16 个 token。

条件	16 个 token 的名次（作者 1：7 个 | 作者 2：4 个 | 作者 3：5 个）	mean_nlr	author_mean_nlr
true	83, 2, 25, 1, 61, 1, 1 | 143, 1, 2, 1 | 92, 48, 1, 1, 1	−0.7190	−0.7050
false_0（"The conjunction fallacy and the many meanings of and"，strict，cost 0.655）	300, 1, 20, 1, 330, 115, 1 | 4, 1, 68, 8 | 67, 102, 7, 1, 1	−1.0234	−0.9881
false_1	855, 3, 18, 1, 304, 58, 1 | 67, 1, 215, 28 | 54, 82, 2, 1, 1	−1.1540	−1.1546
false_2	823, 3, 7, 2, 519, 154, 1 | 27, 1, 1378, 6 | 23, 86, 5, 26, 1	−1.2625	−1.2560
false_3	278, 1, 19, 1, 170, 230, 1 | 28, 1, 248, 15 | 44, 240, 2, 3, 1	−1.1334	−1.1342
false_4	664, 1, 26, 1, 158, 20, 1 | 35, 1, 550, 3 | 11, 98, 2, 1, 1	−0.9895	−0.9875
delete	185, 2, 17, 1, 113, 5, 1 | 11, 1, 163, 1 | 18, 10, 27, 4, 1	−0.8808	−0.8690

```
true mean_nlr  = (−1.919 −0.301 −1.398 −0 −1.785 −0 −0 −2.155 −0 −0.301 −0 −1.964
−1.681 −0 −0 −0) / 16 = −0.7190
false_m        = (−1.0234 −1.1540 −1.2625 −1.1334 −0.9895) / 5 = −1.1126
spec  = −0.7190 − (−1.1126) = +0.3935
total = −0.7190 − (−0.8808) = +0.1618
topic = −1.1126 − (−0.8808) = −0.2318
```

与 `351_stage10/summary_title/paper_title_contrasts.parquet` 中该篇 `readout = final, layer = 63` 的行一致：`true_mean_nlr −0.719048`、`false_mean_nlr −1.112564`、`delete_mean_nlr −0.880811`、`spec_mean_nlr 0.393516`、`total_mean_nlr 0.161763`、`topic_mean_nlr −0.231753`、`spec_author_mean_nlr 0.399071`（本次核对）。

B1. 假标题（false_0 … false_4）与删标题条件

一句话：每篇论文配 5 个"同领域、元数据相近、但不是这篇论文"的真实标题（来自 cohort 里的其他论文），年份保留目标论文自己的年份，只换标题；再加一个完全不给标题的 delete 条件。这样，名字本身好不好猜（name prior）在 true 与 false 之间抵消，差值只剩"这篇论文"的信息。 ⟶ 出处 ~/jlens_work/10_stage10/351_stage10.py:242-307（match_false_titles）、310-349（freeze_conditions） · /scratch/zhang.zhiqi2/jlens/351_stage10/config/conditions_manifest.json · freeze 作业 11333739（351_stage10/logs/freeze_conditions_11333739.log） · git b4b76ed

候选池与硬约束

项目	实际做法	出处
候选池	同一 `field`（8 个领域）内的全部其他论文；cohort 共 9,108 篇，每个领域 1,067–1,230 篇。donor 只能来自 cohort 本身（不是外部标题库）。	⟶ 出处 351_stage10.py:262 `for field, g in df.groupby('field')` · cohort/papers.parquet 列 field
硬排除 1：自己	对角线置 False。	⟶ 出处 351_stage10.py:267, 276
硬排除 2：任何共享姓氏	两篇论文的作者姓氏集合有交集就不能互为 donor。姓氏 = 每位作者按空格切分后的**最后一个词**，经 NFKD→ASCII→小写归一化。这一条同时排除了共享作者的论文（含同名不同人，保守）。	⟶ 出处 351_stage10.py:258, 268-275（注释 "shared surname (incl. shared author)"）
硬排除 3：近重复标题	标题词集合（归一化后长度 ⩾4 的字母词）Jaccard > 0.6 的排除（`NEAR_DUP_JACCARD = .6`）。	⟶ 出处 351_stage10.py:248, 269, 277-280
年份	假标题 prompt 用**目标论文**的年份（"title swapped only"）。因此 false prompt 中的年份与 donor 论文自身年份一般不同（Stage 13 审计：与 donor 相同的只有 17.5%）。	⟶ 出处 351_stage10.py:325 `PROMPT.format(title=d.donor_title, year=p.year)` · conditions_manifest.json `false_title_year` · PROGRESS.md 2026-10-28 ~08:00 条
delete 条件	`Who are the authors of the paper ({year})?\nAuthors:`	⟶ 出处 351_stage10.py:244 DELETE_PROMPT

代价函数（各项与权重）

```
cost(i, j) = |Δyear| / 5
           + |Δn_authors| / max(1, 0.25 · n_authors_i)
           + |Δtitle_tokens| / max(1, 0.2 · title_tokens_i)
           + 1.5 · |Δcitation_bin_index|
```

i = 目标论文，j = donor；title_tokens 用 Qwen3-32B tokenizer（`add_special_tokens=False`）计数；citation_bin_index 是 13 个引用档按下界排序后的序号（0-2 → 0 … 4097+ → 12）。权重 1/5、0.25、0.2、1.5 只在代码里写定，没有找到选取理由的书面记录。 ⟶ 出处 351_stage10.py:264-266, 281

· conditions_manifest.json matching.cost **这些权重是经验设定，没有做过敏感性分析**（用户 2026-10-30 确认，U13）。⟶ 出处 PROGRESS.md 2026-10-30 14:00 条 U13 复核过

注意文档不一致：STATUS.md §2 把代价写成 "|Δyear|/5 + |Δn_authors| + |Δtitle tokens| + 1.5 · |Δcitation bin|"，省掉了作者数和标题长度两项的相对归一化分母；代码与 conditions_manifest.json 是带分母的版本，以代码为准。⟶ 出处 /scratch/zhang.zhiqi2/jlens/STATUS.md §2 "Title conditions" · 351_stage10.py:281

分档（tier）与"严格档"定义

档	|Δyear| 上限	|Δn_authors| 上限	|Δtitle_tokens| 上限	|Δbin| 上限	同一 donor 最多被用几次
strict（严格档）	5	max(1, round(0 · n)) = **1**	max(1, 0.20 · len)	1	5
medium	5	max(1, round(0.25 · n))	max(1, 0.25 · len)	1	5
loose	8	max(1, round(0.34 · n))	max(1, 0.30 · len)	1	5
nearest	无元数据约束（只保留 3 条硬排除）				7

"严格档"= 同领域、年份差 ≤5、作者数差 ≤1（相对容差 0，但代码里的 `max(1, …)` 使下限为 1）、标题 token 数差 ≤ max(1, 20%)、引用档相差 ≤1 档。⟶ 出处 351_stage10.py:245-247 TIERS（元组顺序：tier, max |dyear|, n_authors 相对容差, 标题相对容差, max dbin, donor 使用上限）, 286 · conditions_manifest.json matching.tiers

分配算法与 false_0..false_4 的排序

1. 每个领域单独做。每篇论文需要 K = 5 个 donor（`K_FALSE = 5`）。
2. 按 strict → medium → loose → nearest 顺序逐档进行。在一档内反复做"最小代价指派"：行 = 还缺 donor 的论文，列 = 还没到使用上限的论文；不满足本档约束、或该 (i,j) 已配过的格子代价设为 BIG = 1e6；另外拼上每行一列 BIG/10 的哑列，允许某行本轮不配。用 `scipy.optimize.linear_sum_assignment` 求解。一轮中每个 donor 最多用一次（指派是一对一的）；一轮配不出任何有效对时进入下一档。
3. donor 使用次数 `uses` 跨档累计；strict/medium/loose 上限 5，nearest 上限 7。实测最大使用次数 7。
4. 所有档结束仍有论文不足 5 个 donor 则报错停止（实际没有发生）。
5. 每篇论文的 5 个 donor 按 (档序, cost, donor_paper_id) 排序，序号 k = 0..4 即 false_0..false_4。所以 **false_0 = 最好档里代价最小的那个**，不是随机的。

⟶ 出处 351_stage10.py:282-307（循环与 sort_values(['paper_id','tier_rank','cost','donor_paper_id'])） · conditions_manifest.json donor_use_max = 7

"79%" 是怎么算的

PROGRESS.md 2026-10-23 条写 "5 matched false titles (…; 79% from the strict tier)"。这是 **全部 45,540 个（论文, 假标题）配对里来自 strict 档的比例**：35,898 / 45,540 = 0.7883 ≈ 79%。不是"79% 的论文 5 个都是 strict"——后者是 6,056 / 9,108 = 66.5%。 ⟶ 出处 ~/jlens_work/PROGRESS.md:1272 · conditions_manifest.json tier_summary（strict n = 35,898；medium 1,638；loose 2,565；nearest 5,439） · 本次用 cohort/conditions/false_titles.parquet 复算：strict 比例 0.788274

k	strict	medium	loose	nearest
false_0	8,372	226	229	281
false_1	7,732	349	443	584
false_2	7,145	366	578	1,019
false_3	6,593	377	626	1,512
false_4	6,056	320	689	2,043

每篇论文拥有的 strict donor 数：0 个 736 篇，1 个 640，2 个 587，3 个 552，4 个 537，5 个 6,056。各档中位 |Δyear|：strict 2、medium 2、loose 6、nearest 4（nearest 的 p90 为 14 年）；各档 cost 均值：strict 1.96、medium 3.00、loose 3.49、nearest 3.82（最大 30.1）。false_0 共用到 6,247 个不同 donor，其中 2,373 个 donor 作为 false_0 被用了不止一次（这一点后来造成 M2 的泄漏，见 h-leak-a）。 ⟶ 出处 conditions_manifest.json papers_by_n_strict_donors、tier_summary · false_titles.parquet 本次复算（crosstab k×tier；groupby tier cost.describe；k==0 的 donor value_counts）

"严格档"在分析中的用途：`analyze_title` 另算 `spec_strict_*` = true − 仅 strict donor 的 false 均值（没有 strict donor 的 736 篇为 NaN）。 ⟶ 出处 351_stage10.py:859-870 · 351_stage10/summary_title/paper_title_contrasts.parquet 列 spec_strict_mean_nlr、n_strict_false

后续各 Stage 用的是哪几个假标题

用途	假标题	出处
Stage 10 title-specific 信号 spec = true − mean(false_0..4)	5 个全用（等权平均）	⟶ 出处 351_stage10.py:838-866
Stage 11 E1/E2/E4/E4b、Stage 13 M2（false_0 runs）、X7	只用 false_0	⟶ 出处 stage13_e4b_group.py:21 DONOR = ft[ft.k == 0] · stage13_cpu.py:152 · stage13_x7_analyze.py docstring
Stage 12 F2（作者提示 lens 读出）	true + false_0..4	⟶ 出处 371_stage12.py:41 TC
Stage 13 X6（E1 用 false_1..4 重做）	5 个分别	⟶ 出处 stage13_x6_analyze.py:14 TITLES

两个真实例子

例 1（5 个都来自 strict 档） paper_id 03095413eda1…，chemistry，2004，引用档 513-1024，3 位作者。

真标题：*Chain-growth polymerization for poly(3-hexylthiophene) with a defined molecular weight and a low polydispersity*

k	档	cost	Δyear	Δn_auth	Δtok	Δbin	donor paper_id（年, 引用档, 作者数）	假标题
0	strict	0.345	0	0	2	0	4c4a8b954edc…（2004, 513-1024, 3）	Ultrasensitive electrical biosensing of proteins and DNA: carbon-nanotube derived amplification of the recognition and transduction events.
1	strict	2.062	1	1	5	0	dc65c4fe8f83…（2003, 513-1024, 2）	Reactive oxygen species (ROS) and reactive nitrogen species (RNS) generation by silica in inflammation and fibrosis.
2	strict	2.272	3	0	1	1	1c12d4389668…（2007, 1025-2048, 3）	Palladium complexes of N-heterocyclic carbenes as catalysts for cross-coupling reactions--a synthetic chemist's perspective.
3	strict	2.500	0	1	0	1	9a4aaf9db8a4…（2004, 257-512, 2）	Congener-specific dechlorination of dissolved PCBs by microscale and nanoscale zerovalent iron in a water/methanol solution.
4	strict	3.300	4	1	0	1	cf3d351eaad5…（2000, 257-512, 2）	Recognition of oxytocin and oxytocin-related peptides in aqueous media using a molecularly imprinted polymer synthesized by the epitope approach.

例 2（跨档：strict、medium、loose 混合） paper_id 00a30d561d7b…，medicine，2012，引用档 129-256，13 位作者，标题 15 token。

真标题：*Glucose deprivation activates a metabolic and signaling amplification loop leading to cell death*

k	档	cost	Δyear	Δn_auth	Δtok	Δbin	donor paper_id（年, 引用档, 作者数）	假标题
0	strict	0.308	0	1	0	0	dbc7dac161f1…（2012, 129-256, 12）	Inflammatory breast cancer: what we know and what we need to learn.
1	strict	3.008	1	1	3	1	62c586f88f1b…（2011, 65-128, 12）	Roflumilast attenuates allergen-induced inflammation in mild asthmatic subjects
2	medium	2.956	1	3	1	1	7e773f9772dc…（2011, 65-128, 10）	A microRNA survival signature (MiSS) for advanced ovarian cancer.
3	medium	3.915	4	2	3	1	6cd4b17bb322…（2016, 65-128, 15）	Technical aspects of neurostimulation: Focus on equipment, electric field modeling, and stimulation protocols

4	loose	2.256	0	3	4	0	b26281050ffc…（2012, 129-256, 16）	The EPILEPSIAE database: An extensive electroencephalography database of epilepsy patients

例 2 说明：k 先按档、再按 cost 排，所以 loose 档 cost 2.256 的排在 medium 档 cost 3.915 之后；medium 档 cost 2.956 也排在 strict 档 cost 3.008 之后。 ⟶ 出处 本次读 `351_stage10/cohort/conditions/false_titles.parquet（列 k, tier, cost, d_year, d_n_authors, d_title_tokens, d_bin, donor_paper_id, donor_title）与 cohort/papers.parquet；例 1 = papers.parquet 第 100 行，例 2 = 第一个含 ≥3 个档的 paper_id`

局限与质疑

- false prompt 里的年份是目标论文的年份，与假标题不配套；"title + year"组合本身是不自然的。M2 在 false_0 上读 donor 的年份时，要记住 prompt 年份属于目标论文。⟶ 出处 `PROGRESS.md 2026-10-28 ~08:00 条`
- 同一 donor 最多可被 7 篇论文借用，false_0 donor 中 2,373 个被多次使用 → 任何按论文切分的 probe 都要按 donor 分组（见 h-leak-a）。
- 姓氏排除只看"最后一个词"，复姓/姓在前的写法可能漏掉或多排除；未单独审计。
- 17% 的配对来自 medium/loose/nearest 档，nearest 档没有元数据约束；为此另算了 spec_strict。

已完成 （2026-10-23 17:03 冻结，conditions_manifest.json 记录 false_titles.parquet sha256 f9b410ce…、prompts.parquet sha256 8c846307…）

B2. 读出工具：logit lens 与三种 Jacobian lens（及 Stage 12 F1 作者提示 lens）

读出的统一定义（是否经过 final norm）

readout	计算	出处
`logit`（logit lens / "Vanilla"）	`lm_head(final_norm(h_l))`：第 l 个 decoder block 的输出残差，**经过模型的 final RMSNorm** 再乘 lm_head。	⟶ 出处 351_stage10/config/protocol.json readouts · 351_stage10.py:425-439 fast_readout · ~/projects/jacobian-lens/jlens/hf.py:166-174 unembed = lm_head(final_norm(x)) · lens.py:135-144 transport = h @ J_l^T
`jlens_<lens>`	`lm_head(final_norm(J_l · h_l))`：先用 J_l 把 L_l 残差搬到 L63 残差基，再同样经过 final norm + lm_head。	
`final`	`lm_head(final_norm(h_63))` = 模型真实下一 token logits。	
残差位置	"output of decoder block l"（jlens ActivationRecorder 约定）；h 先转 float32 再做 transport，unembed 时转回 lm_head 的 bf16。	⟶ 出处 protocol.json residual · 351_stage10.py:433 · hf.py:167-170
rank	1 + 词表中严格大于 gold logit 的个数（并列对 gold 有利；并列数另存 ties）。	⟶ 出处 protocol.json rank/ties · 351_stage10.py:442-449

Jacobian 的估计与平均（三种 lens 共用的 jlens 库）

- 库：Anthropic jlens 公开代码（本地 `~/projects/jacobian-lens`，git `581d398 Initial release`）；本地只加了 `_stage10_compat.py`（transformers 5.x 下补 `hf_device_map` 属性，只在运行 351_stage10 时生效，不改数学）。仓库镜像 `code/jlens_package_patch/`（git b4b76ed）。⟶ 出处 ~/projects/jacobian-lens/jlens/_stage10_compat.py · git diff jlens/__init__.py
- 每条 prompt：一次前向（prompt 复制 dim_batch 份），然后 ceil(5120/dim_batch) 次反向。每次反向在目标层（L63）输出的**所有有效位置**同时放一个 one-hot cotangent（第 d 维），对各源层残差求梯度，再在**有效源位置上取平均**，得到 J_l 的第 d 行。所以每个位置 p 的贡献是 Σ_{p'≥p} ∂h_63[p']/∂h_l[p]（跨位置项也算在内），这是论文使用的估计。⟶ 出处 fitting.py:1-20 模块说明, 100-205 jacobian_for_prompt（198 行 .mean(dim=1)）
- 有效位置：`[skip_first, seq_len-1)`，默认 skip_first = 16（跳过注意力 sink 位置，最后一个位置无下一 token）。⟶ 出处 fitting.py:42, 45-70

- 跨 prompt：对每条 prompt 的 J_l 做**不加权平均**（running sum / n_done），与 prompt 长度无关；太短（seq_len ⩽ 17）的 prompt 抛 ValueError 被跳过。保存为 fp16。⟶ 出处 `fitting.py:340-366 · lens.py:52-65`
- 目标层：三种 Stage 10 lens 与 F1 都是 L63（最后一个 block 的输出），之后再走 final norm + lm_head。public lens 的 config 写 `target_layer: null`；按公开 jlens 代码 null 默认为最后一层，且其源层是 0–62，与"目标 63"一致，但 Neuronpedia 包装脚本本身无法核对。⟶ 出处 `fitting.py:73-98 _check_layer_indices · public_neuronpedia/…/config.yaml · 351_provenance_audit/summary/protocol_comparison.csv ( target_layer: UNKNOWN_PUBLIC )`

三种 Stage 10 lens 与 F1 作者 lens 的拟合参数

	public（Neuronpedia）	self100	fitA（Stage 9 Fit A）	F1 author lens（Stage 12）
文件	public_neuronpedia/qwen3-32b/jlens/Salesforce-wikitext/Qwen3-32B_jacobian_lens.pt	qwen32b_jlens_prelim_100.pt	351_stage9/runner_runs/full_A_repair_101_100valid/lens.pt	371_stage12/alens/lens.pt
sha256	0af5ccf2…	268aba15…	4881b830…	未在 protocol 中登记 lens 本身 sha（fit_prompts.parquet sha be0c3ab7…）
拟合文本	Salesforce/wikitext-103-raw-v1 train，text 字段，max_chars 2000；Neuronpedia 没有提供条目清单，但 2026-10-30 已按其 `fit_lens.py` 的确定性算法复原：流式读取、跳过空行和 "=" 开头的标题行、拼接后切成 2,000 字符一段、取前 557 段（共 1,113,787 字符）	本地冻结语料 fitting_corpus/wikitext103_qwen32b_1000.jsonl（WikiText-103 train，seed 42 抽 1,000 条恰 128 token 的片段，候选 970,664 段），用索引 0–99	WikiText-103 train 逐行（去 "= 标题 =" 行，按字符长度过滤，sha256 去重），random.Random(3519001) 洗牌后切出 fit_A 100 / fit_B 100 / holdout 100（互不重叠）	Stage 10 问句 + 空格 + 真作者列表（逗号分隔）；40 篇 target/control 论文，来自 Stage 11 样本之外，seed 12 抽取（target 31、control 9）；token 长度 27–104（中位 50），没有截断
条数	557（运营者在 557 处手动停止；原计划上限 1,000）	100	100 有效 / 101 候选（原运行跳过 1 条，追加 1 条 128 token 替补）	40
源层	L0–L62 全部	L24/32/40/48/56/62	L16/24/32/40/48/56/62	L24/32/40/44/48/52/56/62
目标层	config 写 null；lens 文件 provenance 字段记 `target_layer = 63`	63	63	63
max_seq_len / skip_first	128 / 16（provenance 字段；convergence.csv 中 seq_len 128 → n_valid 111 与之一致）	128 / 16	128 / 16	128 / 16

dim_batch	64	8	1	1
精度 / 硬件	bf16，1×B200，torch 2.11，transformers 5.17，jlens_commit 22f0412f，compile=true	bf16，A100-SXM4-80GB	bf16，A100-80GB（约 325–360 s/prompt）	bf16，单卡 A100-80GB（约 269 s/prompt）
停止规则	stop_at_delta 0.002 未达到；Δmean 在 5e-3–8e-3 平台，估计均值 J 的相对标准误 ~13%	固定 100	固定 100	固定 40
出处	⟶ 出处 …/Salesforce-wikitext/config.yaml（stop_reason.detail） · 351_stage10/config/lenses.json · 351_provenance_audit/summary/protocol_comparison.csv	⟶ 出处 qwen32b_jlens_prelim_100_config.json · fitting_corpus/manifest.json · qwen32b_jlens_prelim_100.log（2026-10-17 23:06 → 10-18 04:45，日志含 4 次续跑）	⟶ 出处 ~/jlens_work/09_stage9/351_stage9.py:10-11, 41-66 · 351_stage9/config/plan.json · runner_runs/full_A_repair_101_100valid/{repair_manifest.json, final_status.json}	⟶ 出处 12_stage12/371_stage12.py:40, 50-62, 233 · 371_stage12/config/protocol.json fit · alens/progress.json（done 40，2026-10-28 17:12）

- **public（U14，部分解决）**：557 条拟合提示已由 Neuronpedia 的 `fit_lens.py` 复原（skip_first = 16、target_layer = 63、n_prompts = 557、dim_batch 64、t_max 128、bf16、jlens_commit 22f0412f 都写在 lens 文件的 provenance 字段里）；用本地 wikitext parquet 重建出 557 段，9,108 个队列标题（≥25 字符）**无一逐字出现**在其中，即不含本研究任何标题。前 556 条是用 `--n_prompts 1000` 跑的，之后改为 557 收尾，前 557 段不变。⟶ 出处 ~/projects/neuronpedia-audit（HEAD 22f0412）utils/neuronpedia-utils/neuronpedia_utils/jlens/fit_lens.py:51–106, 465–470 · ~/projects/jacobian-lens/jlens/fitting.py:42, 67 · 351_provenance_audit 结果包 summary/audit_summary.json:116–134 · 351_stage10/reports/methods/research_scripts/recon557.py, public557_prompts_reconstructed.json · 附录 I U14 复核过。拟合时的模型与语料 revision 没有记录（`from_pretrained` 不带 revision，取的是拟合当时 2026-09-20/21 的 main；本地缓存 refs/main = 9216db57 是 10-15 下载的），复原的前提是 wikitext 在 HF 上没有改版。**未知/需确认：向 Neuronpedia（lens 发布者）确认拟合时的 Qwen3-32B 与 wikitext revision（U14）**
- **self100（U15，已解决）**：拟合脚本就是 GitHub 镜像里的 `home_project/05_prepare_jlens_corpus.py`（sha a975c2a2…，固定 Salesforce/wikitext wikitext-103-raw-v1 train、seed 42、128 token、N = 1,000、蓄水池抽样；manifest 记 candidate_chunks_seen 970,664、corpus_sha256 1af8dbff… 与语料文件一致）和 `home_project/05_fit_jlens_prelim_100.py`（sha 4dcfb96a…，取前 100 条，层 [24, 32, 40, 48, 56, 62]，target 63，dim_batch 8，max_seq_len 128，skip_first 16，revision 9216db57；生成的 config 与 qwen32b_jlens_prelim_100_config.json 逐字段相同），commit e2c7156；Stage 1–8 归档 `jlens_archive_stage1_8/home_project/` 有同 sha 副本。⟶ 出处 附录 I U15 · git -C /scratch/zhang.zhiqi2/jlens-authorship-recall_update log -- home_project/05_* 复核过

- **fitA（U16，已解决）**：Fit A 准备时 `--min-chars / --max-chars = 100 / 1200`（即脚本默认值）。按 351_stage9.py:40–58 的流程在本地 wikitext parquet 上重做，得到 665,604 条去重行，fit_A、fit_B、holdout 三份 text_sha256 列表逐条完全一致；100/1196、64/2000、100/2000 都不一致。⟶ 出处 351_stage9.py:40–58, 223–224 · 351_stage10/reports/methods/research_scripts/s9split.py · 附录 I U16 复核过

Stage 9 Fit A / Fit B / holdout 的状态

子项	状态	出处
9A Fit A	已完成 状态 `VERIFIED_100_VALID_OF_101_CANDIDATES`。原运行到 n_done 94/next_idx 95 时发现 jlens.fit 跳过了 1 条（太短），协议 A_APPEND_ONE_V1 追加 1 条经核验的 128 token WikiText-103 文本（"Senjo̅ no Valkyria 3 …"），估计方法不变。逐层 roundtrip 误差 2.0e-4–2.6e-4。	⟶ 出处 full_A_repair_101_100valid/repair_manifest.json, final_status.json · 351_stage9_A_repair.log（REPAIRED A COMPLETE, 2026-10-22 05:33）
9A Fit B	计划未做完 检查点 n_done = 66/100，最后写入 2026-10-21 19:12，之后没有续跑。停止原因：作业 11289356 到达 2 小时时限被杀，日志无错误，检查点完好可续跑（U33）。**用户 2026-10-30 决定答辩前不补做**；Stage 10 没有用 Fit B。⟶ 出处 sacct -j 11289356 · 附录 I U33 · PROGRESS.md 2026-10-30 14:00 条	⟶ 出处 351_stage9_full_B.log 末行 "prompt 66/100" · STAGES.md Stage 9 表 · PROGRESS.md:556, 1271
9B holdout 比较（Fit A vs Fit B）	计划未做 prompts/holdout.jsonl（100 条）已冻结，没有任何比较结果。	⟶ 出处 STAGES.md:170
9C	计划未做	⟶ 出处 PROGRESS.md:556

所以"换一批 WikiText 拟合，lens 结论还稳不稳"目前只有：self100 的 50+50 split-half（Stage 3.5.1，只比矩阵和少量 prompt）和 Stage 10 中三把 lens 的并列结果，没有正式的 Fit A vs Fit B 留出集比较。 ⟶ 出处 02_self_lens_validation/351_self_jlens_split_half.py:117 · 351_self100_split_half/

Stage 10 中怎么用这些 lens

- 层：主轮 L24/32/40/48/56/62（三把 lens 共有的层）+ final L63；扩展轮（ext）只用 public 与 logit，层 L50–55、57–61。⟶ 出处 351_stage10.py:43 DEFAULT_LAYERS · 351_stage10/logs/pool/job_s10p_ext_true_11363851.log 头行 readouts=jlens_public:logit layers=50:…:61
- 每把 lens 的 J_l 放在持有第 l 个 block 的 GPU 上（同 dtype），只保留需要的层。⟶ 出处 351_stage10.py:407-422 open_lenses
- F1 作者 lens 在 Stage 12 F2 用于读名字位置与最后一个标题 token。⟶ 出处 371_stage12.py:251-264 do_alens_read

局限与质疑

- 三把 Stage 10 lens 都是在通用文本（WikiText-103）上拟合的，不是在作者 prompt 上；F1 是唯一的作者提示 lens，只有 40 条。
- self100 与 fitA 都只有 100 条，public 也未收敛（相对标准误 ~13%）；dim_batch 不同（64/8/1）只影响显存，不改变估计量（审计说明），但数值实现差异未单独验证。⟶ 出处 `351_provenance_audit/reports/audit_report.md Audit 4`
- public 与自拟合 lens 拟合文本同源（WikiText-103）；public 的 557 条已按算法复原（见上表下 U14），逐字匹配未见队列标题，但这依赖 wikitext 数据集未改版。

B3. 统计：引用档、分层权重、bootstrap、作者团队、多重比较

引用档（citation_bin）

13 档，来自组长的标注包（列 `citation_bin`），按 2 的幂分档：0-2、3-4、5-8、9-16、17-32、33-64、65-128、129-256、257-512、513-1024、1025-2048、2049-4096、4097+。本次用 papers.parquet 的 citations 复核每档实际最小/最大值都落在标签范围内（例：513-1024 实际 513–1023；4097+ 实际 4,136–63,901）。前 10 档各约 800 篇（0-2 为 796），高档较少（665 / 326 / 122）。 ⟶ 出处 `labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/paper_labels_with_papers.csv` 列 `citation_bin` · `bin_table.csv` · 本次复算 `cohort/papers.parquet`

边界规则出自 Dreaming Up Authors 代码包 `config/config.yaml:44–59`：13 个指数间隔档，`lo`、`hi` 两端都含，最后一档开放；按此边界重算 9,108/9,108 与 citation_bin 相同（U09 已解决）。 ⟶ 出处 `~/archives/dreaming-up-authors-code-v0.zip: config/config.yaml:44–59, src/dua/bins.py:12–20` · 附录 J 第 9 行 **复核过**

各档 group_tight 人数（target / control）：0-2 200/352，3-4 207/355，5-8 237/332，9-16 288/275，17-32 327/234，33-64 401/185，65-128 441/165，129-256 497/126，257-512 533/105，513-1024 603/53，1025-2048 539/29，2049-4096 264/9，4097+ 84/0。所以"两组都 ≥30"只剩前 10 档（1025-2048 control 只有 29）。 ⟶ 出处 本次复算 `cohort/papers.parquet` crosstab · `351_stage10/summary/B_target_vs_control_within_bin.csv` 列 `n_strata = 10`, `n_a = 3,734`, `n_b = 2,182`

target − control 分层估计（档内比较）

```
est = Σ_b w_b · ( mean_target,b − mean_control,b ),
w_b ∝ n_t,b · n_c,b / (n_t,b + n_c,b),  Σ_b w_b = 1,
只用 n_t,b ≥ min_n 且 n_c,b ≥ min_n 的档。
```

实现	min_n	bootstrap 方式	出处
Stage 10 `stratified_diff`	30（主分析）；10（bin×field、guard-target、group_both 敏感性）	权重固定；在每个 档×组 内独立有放回重抽论文，B = 2,000，百分位 95%	⟶ 出处 `351_stage10.py:642-645, 670-691, 748-760`
Stage 11 `strat_diff`	30	同上（档×组内重抽论文），B = 2,000	⟶ 出处 `11_stage11/stage11_analyze.py:8-9, 32-41`
Stage 12 `strat`	30	同上	⟶ 出处 `12_stage12/stage12_analyze.py:7, 21-30`
Stage 13 `strat_cluster`（团队版）	30	点估计与上相同；区间改为按作者团队重抽（多项分布权重）	⟶ 出处 `13_stage13/stage13_cluster_ci.py:30-50`

Stage 13 X4 `strat_diff_team`	**10**	团队重抽	⟶ 出处 `13_stage13/stage13_x1x4_analyze.py:80-90`

Bootstrap：单位、次数、种子（逐脚本）

脚本	单位	B	种子
10_stage10/351_stage10.py analyze	论文（档×组内）	2,000	3510（全部表共用一个 rng，按表顺序消耗）
351_stage10.py analyze_title	论文	2,000	3511
351_stage10.py parity 抽样	（抽 12 个 context）	—	10
10_stage10/stage10_extra.py	论文（context 级按论文聚类）	2,000	3512
stage10_extra_r0.py	论文	2,000	3512
stage10_extra_cond_role.py / stage10_extra_r0r1.py	论文	2,000	3513（r0r1 内调用 Stage 11 时 3611；probe 折 11）
stage10_extra_format.py	论文	2,000	3514
11_stage11/stage11_analyze.py	论文（比值用配对重抽）	2,000	3611；E4b probe 折种子 11
12_stage12/stage12_cpu.py	论文	2,000	3712；probe 折 11
12_stage12/stage12_analyze.py	论文	2,000	3713
13_stage13/stage13_cpu.py（M1/M2）	论文	**1,000**	3813；折 13（paper）或 group_folds(seed 13)
stage13_cluster_ci.py	**作者团队**	2,000	函数默认 1/2/3，各表传入
stage13_x6_analyze.py	团队（5 个假标题共用同一组抽样）	2,000	13
stage13_x7_analyze.py	团队（T、F、S 共用）	2,000	17
stage13_x1x4_analyze.py	团队（pairs.parquet 列 team）	2,000	X1 101；X4 202；X4 recognition 303+层号
stage13_m3_analyze.py	团队	2,000	31
stage13_x4_textbase.py / textbase2.py	团队	2,000	41 / 43
stage13_x1_similarity.py	（置换）	—	5

⟶ 出处 以上均为 `grep -n "default_rng(\|B_BOOT\|NB = \|B = "` 于 ~/jlens_work/1[0-3]_*/*.py 的结果（例：351_stage10.py:645, 748, 875；stage13_cpu.py:26；stage13_cluster_ci.py:18；stage13_x6_analyze.py:13, 51；stage13_x7_analyze.py:15, 50；stage13_m3_analyze.py:19, 96）

区间一律是百分位法 95%。Stage 13 起（prereg addendum 2）团队版为主，但 Stage 10–12 已发表的表仍是论文级区间，团队版另见 cluster_ci.md。

作者团队（group_folds）定义

1. 作者全名归一化：NFKD → ASCII（丢弃非 ASCII）→ 小写 → 非字母替换为空格 → 按空格切词；**只有 ≥2 个词的名字**才参与（单词名字不连边）。
2. 用 union-find：同一 paper_id 合并；任意两篇共享一个归一化全名即合并（同名不同人也合并，保守）。
3. false_0 行（`donors` 参数）：每行再与其 donor 论文（及 donor 的团队）合并，保证同一 donor 标题不跨折。
4. 分折：团队按大小降序（同大小先用 seed 13 洗牌），依次放入当前最轻的折，k = 5。

⟶ 出处 13_stage13/stage13_cpu.py:80-109 group_folds · git cf7106d

复现性问题（2026-10-30 发现）：`stage13_cpu.group_folds()` 用 pandas `value_counts` 给团队按大小排序（第 106 行），对同样大小的组排序不稳定，所以不同 CPU 型号上折的分配可能不同：在 d0026（cascadelake）上折分配不同、点估计最多差 0.006；在 c0622（E5-2680 v4）上精确复现原结果。M2 团队区间用的折已固定保存在 `381_stage13/summary_cpu_teamci/folds_group.npz`；其他调用 group_folds 的脚本是否受影响尚未逐一检查。⟶ 出处 13_stage13/stage13_cpu.py:80–109（106 行 value_counts）· 381_stage13/reports/m2_teamci.md · PROGRESS.md 2026-10-30 17:30 条（tiny 作业 11501115 折不同、11503471 一致） 待复核 （其他脚本）

本次复算（9,108 篇）：7,247 个团队；6,503 个单篇团队；最大团队 228 篇（其次 76、42）；5 折大小 1,822/1,822/1,822/1,821/1,821。与 cluster_ci.md（7,247）和 PROGRESS（1,821–1,822）一致。⟶ 出处 本次运行 group_folds（CPU）· 381_stage13/reports/cluster_ci.md 第 3 行

多重比较

Stage 10–13 没有做任何多重比较校正（Bonferroni/Holm/FDR 在 10–13 的脚本里都搜不到）。采用的规则是"所有层都报告，不只挑一层"：prereg.json general_rules 第 1 条 "All layers and positions are reported; no result is reported for a hand-picked layer only."，以及 "A difference counts only if its interval excludes 0"。层数多（Stage 10 主轮 6 层 × 4 readout + final，ext 再加 11 层），逐层"区间不含 0"的判断没有族错误率控制。⟶ 出处 grep -rniE "bonferroni|holm|fdr|benjamini|multipletest" ~/jlens_work（10–13 无结果）· 381_stage13/config/prereg.json general_rules

较早阶段：Stage 8 的协议写了"7 个逐层比较 Holm 校正"，但 Stage 8 代码里只在 analysis_plan 字符串中出现 Holm，没有找到实现；Stage 8D supplement 明写 "unadjusted CIs; per-layer rows … not

Holm-adjusted"；Stage 7.2/7.4 报告写明未校正。 ⟶ 出处 08_stage8/351_stage8.py:125 · 351_stage8D_supplement.py:63 · 07_2_stage7_2/351_stage7_2_score.py:193 · 07_4_stage7_4/351_stage7_4.py:91 原始代码确实没有实现 Holm。**2026-10-30 已按冻结计划事后补算**（作业 11500770，CPU，不改任何冻结文件）：在"计划的 7 个逐层 A–B（J-Lens）"这一族里做 Holm，只有 L62 通过（p_Holm 0.029），L48 0.17、L56 0.19；冻结主终点（7 层积分 A–B）原始 p = 0.002。p 值公式（由百分位 bootstrap 反推的双侧 p，B = 100,000）和检验族的划分是我们的选择，冻结计划只写了 bootstrap 区间、没有写 p 值公式。详见 s19-s8d。⟶ 出处 ~/jlens_work/08_stage8/recompute_2026-10-30/stage8_frozen_plan_recompute.py 文档串（2）· 351_stage8/recompute_2026-10-30/s8_holm.csv · PROGRESS.md 2026-10-30 13:00 条 **复核过**（U17 已解决）

B4. 切分与泄漏审计

背景：Stage 11 E4b（标题 token probe）和 Stage 13 M2/M2b/M2c 原来都按**论文**随机分 5 折（E4b 种子 11，M2 种子 13，所有层/位置/目标共用一套折）。2026-10-28 上午的 probe 切分审读发现：按论文分折本身是一致的，但 donor 标题和作者团队会跨折。 ⟶ 出处 `PROGRESS.md 2026-10-28 11:45 条 ("probe-split audit")`

#	泄漏	发现（日期 / 哪次审读）	影响的结果	修复（脚本 · 作业）	修前 → 修后
(a)	**M2 false_0：借出标题同时在训练和测试。** false_0 共 6,247 个 donor，2,373 个被多篇论文借用；按论文随机分折后，**2,004 个 donor 标题**的行同时落在训练折与测试折，probe 等于在测试时见过同一标题。	2026-10-28（PROGRESS 记为 11:45 条）probe 切分审读。本次复算：Stage 11 titlepos/false_0 的 9,108 行按 default_rng(13).integers(0,5) 分折，donor 出现在 >1 折的有 2,004 个。	M2 false_0 的 year / citations / field 读数（"表征跟随所显示标题的论文"）；Stage 11 E4b 的"用假标题训练的 probe 更强"对照（同源泄漏，见 c）。	stage13_cpu.group_folds，false_0 按 donor 及其团队分组；SPLIT=group 重跑 M2 → M2b → M2c：作业 **11443403**（2026-10-28 12:43–13:56），输出 summary_cpu_group/；对比 stage13_split_compare.py 作业 11443607 → reports/split_compare.md。核验：0 个 donor 标题跨折。	L48 title 位置（R^2）：donor year 0.623 → 0.535；log citations 0.588 → 0.488；citations|year 0.570 → 0.471；field acc 0.788 → 0.716。L52：year 0.619 → 0.538，citations 0.580 → 0.484。结论：泄漏确认，但修后数值仍与 true run 可比（true L48 year 0.555）。paper-split 的 false_0 数字被 addendum 1 判为无效。⟶ 出处 `381_stage13/reports/split_compare.md 第 126–133 行（列 value_paper, value_group）· prereg_addendum1.json rules[3]`
(b)	**同作者团队跨折**：2,319 对论文共享 ⩾1 个全名（64 对共享 ⩾2 个），按论文分折时被分在不同折。也影响所有按论文的 bootstrap 区间（论文不独立）。	同 (a)，2026-10-28 probe 切分审读；区间问题在 report v5 审读（PROGRESS 2026-10-28 12:35 条 review item 3）。	M2/M2b/M2c、M3/X 系列的 tune/test、Stage 10–11 的全部论文级 CI。	① probe：group_folds，作业 11443403（同上）；X1/X3/X4 的 tune/test 在 pairs.parquet 按团队重切（0 个团队两边都有，GPU 数据产生前完成）。② 区间：stage13_cluster_ci.py 团队 bootstrap，作业 **11443686**、11443696 → reports/cluster_ci.md。	M2 true runs：paper vs team 差 ⩽0.005（L48 year 0.556 vs 0.555；recognised AUROC 0.670 vs 0.671）。M2b/M2c 变化 ⩽0.02。团队 CI：区间宽度平均放大 0–8%（最大 22%）；380 行中 4 行"区间不含 0"翻转，都在 0 附近：6.4 logit L32（0.005）、6.2 logit L56（+0.035 → [−0.001, 0.069]）、C−A logit L55、6.3 target public L48。⟶ 出处 `split_compare.md M2 true 行 · cluster_ci.md`

(c)	**标题一题两用（title reuse，E4b）**：probe 在训练时见过测试论文 prompt 里的标题——true 训练的 probe：测试论文的假标题的 donor 在训练折；false 训练的 probe：同一 donor 标题，或测试论文自己的标题借给了训练论文。	2026-10-28 report v5 审读（PROGRESS 12:35 条），写入 prereg addendum 2 E4b 条。	Stage 11 E4b（报告 6.11、6.12 的 probe 陈述）及其对照（随机 gold、同领域 gold、"用假标题训练的 probe"）。	13_stage13/stage13_e4b_group.py（只替换原函数的折行并注入 hook，per-paper/无排除配置必须复现已发表数字），作业 **11443587** → reports/e4b_split_compare.md。确认版 = 团队折 + 排除 title reuse。	受影响比例：约 79.5–82.8% 的测试论文被排除；保留 1,863 篇（target 896、control 499、excluded 433、guard 35）；false 训练对照保留 1,568（810/399）。target−control（pooled within bin）：paper/none L24 +0.024 [0.010, 0.038] → group/dropped +0.052 [0.017, 0.088]；L56 +0.039 → +0.075 [0.038, 0.115]。"用假标题训练的 probe"（全部论文 spec）：L24 0.068 → 0.050，L40 0.081 → 0.057，L56 0.094 → 0.074，修后不再高于标准 probe（0.050 / 0.066 / 0.080），"假标题训练更强"的说法撤回。复现检查：paper/none 与已发表 E4b max |diff| 0.0000，对照 ≤0.0080。⟶ 出处 `e4b_split_compare.md 三张表与 Sanity 行 · summary_cpu_group/perpaper_probe_spec.parquet 本次复算保留数`
(d)	**Stage 12 F2 评估集与 F1 拟合集同名作者**：40 篇拟合论文本来就不在 Stage 11 样本里（论文不重叠），但 3 篇样本论文与拟合论文共享一个全名："qiang xu"、"j schwartz"、"j braun"（可能是同名不同人）。	2026-10-28 审读（PROGRESS 12:00 条）。	Stage 12 F2（作者提示 lens 的 title-specific 信号）。	不重拟合（拟合当时 35/40，重拟合约 4 GPU-h），改为评估时排除这 3 篇：371_stage12/config/alens_eval_exclude.json（mtime 2026-10-28 10:58），stage12_analyze.py:136-139 读取并剔除。git 663ac1c。	**没有修前数字**：排除文件写于 F1 完成（17:12）和 F2 分析（报告 19:55）之前，F2 只按排除后的样本算过一次。⟶ 出处 `371_stage12/reports/stage12_report.md F2 表 · ls --full-time 371_stage12/config, alens, reports`
(e1)	**":" 位置的年份泄漏**：Stage 11 残差来自含年份的 prompt；冒号位置已看到年份，所以在 ":" 处 probe 年份（R^2 0.98）是泄漏；citations/recognition 也可能借年份。标题最后一个 token 还没看到年份（因果注意力），那里是干净的。	2026-10-28 ~08:00 数据来源审计（用户要求"年份问题不能用含年份的 prompt"）。	M2 的 ":" 位置读数。	":" 处年份只当阳性对照；citations/recognition 加 year-only 基线和"去年份残差"目标；新 GPU 实验（X 系列）改用不含年份、前缀等长的模板。作业 11439283（M2v2）；git 43c8f0c。	":" 处 citations（去年份）0.40 → 0.62；recognised 0.67 → 0.72（随层，见 PROGRESS）；year-only 基线 citations R^2 0.047、recognised AUROC 0.559。⟶ 出处 `PROGRESS.md 2026-10-28 09:35 条 · split_compare.md year-only baseline 行`
(e2)	false_0 prompt 带目标论文年份（与 donor 年份相同仅 17.5%）	同 e1。	M2 false_0 读 donor 年份的解释。	只在报告中注明，未改数据。	—

(e3)	lens 拟合文本与评估文本重叠	Stage 3 provenance 审计（Audit 3）。	self100、public 的 fidelity 评估。	self100：40 条评估 prompt 与拟合 prompt 0 条完全匹配、0 条 8 词重叠标记。public：拟合条目未提供，**无法核对**。	—⟶ 出处 351_provenance_audit/reports/audit_report.md
(e4)	BERT 基线（M2c）的预训练数据（Wikipedia）可能包含著名论文	report v5 审读 → addendum 2 M2c 条。	M2c "Qwen 超过 BERT"的解读。	规则改为单向：BERT 追上 Qwen 才算支持"风格"，Qwen 超过 BERT 只作参考。	—

局限：团队定义只靠全名字符串，单词名字不连边、同名不同人会被合并、拼写变体（如中间名缩写不同）不会合并，没有人工核验。⟶ 出处 stage13_cpu.py:87-89

复核过 （a）（b）（c）的修后数字已由 split_compare / e4b_split_compare / cluster_ci 输出，(a) 的 2,004 与 7,247 个团队本次已独立复算一致。

B5. 预注册（Stage 13）

文件、哈希与写定时间

目录 `/scratch/zhang.zhiqi2/jlens/381_stage13/config/` 下只有 prereg.json 与 addendum 1–4，**没有 addendum 5：补充 5 尚未写定**，草稿已发给用户、等用户确认（内容拟为 X3 的 jaccard2 修正、排除 27 条完全相同改写、37 条姓氏误拒维持拒绝、意义检查改走备用方案）；因为判断器 v2 在新 20 条上一致率只有 0.55 < 0.80，**判断器 v2 不可用**（见 s13-x3）。⟶ 出处 PROGRESS.md 2026-10-30 14:00、15:00 条 · 381_stage13/followup/judge2_summary.json五个 json 都是只读（-r--r--r--）。 ⟶ 出处 ls -la 381_stage13/config · PROGRESS.md:103, 124, 176

文件	sha256（.sha256 文件）	本次 sha256sum	文件内 written	文件 mtime（EDT）	git commit 时间
prereg.json	18409398945f… 32dce701	一致	2026-10-28T11:00-04:00	2026-10-28 10:39:12	355e64d 10:40:15
prereg_addendum 1.json	608045ca5946… 169ad60f	一致	2026-10-28T11:40-04:00	10:51:25	cf7106d 10:52:00
prereg_addendum 2.json	377b79630aa0… 2d34205f	一致	2026-10-28T12:30-04:00	11:05:49	0dde4de 11:06:51
prereg_addendum 3.json	e01aa6e3d965… 5241a74bf	一致	2026-10-28T14:00-04:00	11:42:05	6d667ab 11:42:31
prereg_addendum 4.json	ac77da9f1f0f… f512e6d193	一致	2026-10-28T14:40-04:00	11:55:18	54d8ac2 11:56:02

⟶ 出处 cat *.sha256; sha256sum prereg*.json（2026-10-30 本次运行）· ls --time-style=full-iso · git log -- experiment_files/381_stage13/config/*（仓库 /scratch/zhang.zhiqi2/jlens-authorship-recall_update）

written 字段是手填错误（U18 已解决）：这些 written 值是当时会话手写的，比真实写入时间晚 20 分钟到 2 小时 45 分钟（PROGRESS 的条目标题时间也用了较晚的时间）。**实际写入时间是 2026-10-28 10:39 / 10:50 / 11:05 / 11:41 / 11:55 EDT**（prereg、addendum 1–4），会话记录、文件 mtime 与 git 提交时间三者一致。⟶ 出处 PROGRESS.md 2026-10-30 12:30 条 "U18 resolved" · 上表 mtime 与 git 列 复核过 mtime、git 时间与 sacct 互相吻合：prereg 写于 10:39，紧接 M2c 作业 11443232 提交（10:38:42，"submitted, no output yet"）；addendum 1 写于 group 重跑 11443403 提交（10:49:37）之后；addendum 2 写于 E4b 11443587（10:59:40）和 clci 11443686（11:05:08）提交之后。因此应以 mtime/git 为准，written 字段不可当作写定时间引用。 ⟶ 出处 sacct -j 11443232,11443403,11443587,11443686 -o Submit 答辩时引用 git 提交时间。

写定时已有哪些数据（status_at_writing 与核对）

文件	status_at_writing 原文要点	核对
prereg	M1、M2、M2b 已跑；M2c 已提交无输出；M3、X1、X3、X4 无数据（0/144 GPU 单元）。M1/M2/M2b 标为探索性。	M2b 11441993 结束于 10:25:59 ✓；M2c 12:32 才结束 ✓；381_stage13/raw 下最早 GPU 文件是 xf_patch 2026-10-28 19:46，task 10-29 00:21，xpatch 00:54，pscope 02:44 ✓
addendum1	M3/X1/X3/X4 仍无数据；M2c 与 paper-split M2/M2b 已有；group 重跑 11443403 无输出。	11443403 开始于 12:43 ✓
addendum2	同上；E4b 重跑、团队 CI、grouped M2/M2b、M2c 已提交无输出。	E4b 12:45 开始、clci 14:17 开始 ✓
addendum3	X6、X7 无数据；Stage 12 P1/P2 在写之前做过 CPU 预览（促成了审读，但不决定规则）。	stage12_P1P2_preview.md mtime 11:23 < 11:42 ✓；xf_patch 最早 19:46、ko_false 10-29 04:18 ✓
addendum4	X3（para_gen/para_run）无数据；P1 按 token 角色的拆分是写之前的 CPU 预览，记为探索性。	para_gen 最早 10-29 03:09 ✓

内容摘要

文件	内容
prereg.json	通用规则：所有层/位置都报告；X1/X3/X4 用 pairs.parquet 的 tune/test（target/control 内 50/50），选择在 tune 上做、确认看 test；区间 = 论文 bootstrap 2,000 次，区间不含 0 才算；容差/parity 未达标记 WARN 不改分析；X2 运行前另写补充。 M2c：BERT 最佳层 ⩾ Qwen 最佳层 − 0.05 → "风格"；Qwen 超过 BERT ⩾0.10（R^2）/⩾0.05（AUROC）→ 超出风格。 M3：身份指数 I(l,pos)（同论文跨任务 cos − 不同论文跨任务 cos）；迁移比 T(l)（任务 a 训练、任务 b 测试 / b 内训练测试；year、log citations、field，按论文 5 折）；H2 预测 L8–L24 起 I>0 且 L16–L40 T⩾0.8；H1 预测 I≈0 或 T<0.5；lexical floor = L4 的同一量。 X1：恢复率 R(src,l)；在 tune 上选 R(authors)−R(random_c) 最大的层；test 上 R(year)、R(field) 都超过 R(random_c) 且在 R(authors) ±0.1 内 → 任务无关身份；random_c 任一层 >0.1 则全部相对它报告并标记。 X3：G(l) = cos(原,改写) − cos(原,partner)。 X4：D = self − partner；最早区间 >0 的层（tune 选、test 确认）；H2 顺序：year、field 的最早层 ⩽ first author；recognition：target 的 D_year、D_author 在档内大于 control（区间不含 0）。 X5：身份 → author-by-probe → name-by-lens 的层序。
addendum1	所有 Stage 13 probe 用 group folds；false_0 按 donor 分组；X1/X3/X4 tune/test 按团队；group-split 为确认版，paper-split 的 false_0 数无效；E4b 重跑后才能再引用。
addendum2	M3：I 只能看 I(l)−I(L4)；主判据改为 T，在 L16–L40 判断并相对 T(L4)。X1：取消选层，报告全部层的 R(a)−R(y)、R(a)−R(f) 曲线；⩽L16 只作文本替换阳性对照；L24 起：两差的区间都在 [−0.1, 0.1] 内且 R(a)−R(random_c)>0 → 任务无关；R(a)−R(y) 区间下界 >0.1 → 任务特异。M2c 改单向。X3 加词重叠协变量。E4b 四配置，确认版 = 团队折 + title-reuse 排除。区间：两种都算，团队版为主，列出所有翻转。
addendum3	X6：R(l) = mean(patched−false)/mean(true−false)；5 个假标题平均 R(L48)∈[0.70,0.95] 且 R(L52)⩾0.95，每个假标题 R(L48) 与 false_0 差 ⩽0.10 且 R(L52)⩾0.90 → Stage 11 结论成立。X7：S(l) = T(l) − F(l)；S(L52)⩽0.05 且 S(L0)⩾0.5 → 成立。P1 措辞由最终 Stage 12 分析决定。

addendum 4	X3 主比较换成 lexical neighbour D（G' = cos(原,改写) − cos(原,D)，需 G'>0 且 G'−G'(L4)>0）；所有名字上的 patching/probe 结果（E1/E1t/E2、P1/P2/F2、X6/X7）按 token 角色另报；措辞限定为 teacher forcing 下的偏好，不是生成过程。

实际分析与预注册的偏离（逐条核对）

实验	预注册规则	实际做法	判断	出处
M3 判据	addendum 2：T 在 L16–L40 判断，"also relative to T(L4)"；prereg 原文 H2 为 T(l) ≥ 0.8（点估计）	自动判定用**区间**：H2 要求所有层 lo ≥ 0.8，H1 要求所有层 hi < 0.5（比点估计更严）；T−T(L4) 和 I−I(L4) 写在表里，但**没有进入判定行**。报告 title_last 判为 H2（T 0.92–0.99），而 STATUS 的解读是"T 不高于 T(L4)"。	**偏离（判定方式）**	⟶ 出处 `stage13_m3_analyze.py:143-150`（147 行 h2/h1） · `reports/m3_report.md:83-90` · STATUS.md §6 0c
M3 lexical floor	L4 的同一量	按 L4 做差并给团队区间	一致	⟶ 出处 `stage13_m3_analyze.py:117-127`
M3 分折	prereg "5-fold by paper" → addendum 1 改 group folds	group_folds	一致（按补充）	⟶ 出处 `stage13_m3_analyze.py:93`
X1 选层	prereg 在 tune 上选层 → addendum 2 取消，报告全部层	全部层（L4–L48）×test/tune/all 都报告	一致（按补充）	⟶ 出处 `stage13_x1x4_analyze.py:40-78`
X1 random_c 零检验	prereg：random_c 任一层 >0.1，所有恢复率相对它报告并标记	只打 NULL FLAG（L4/L8/L16 约 −0.13～−0.15，L48 +0.12），判定仍用 R(authors)−R(random_c)>0；year/field 的恢复率**没有**相对 random_c 重算；L48 在 FLAG 下仍判"task-independent"。	**部分偏离**	⟶ 出处 `stage13_x1x4_analyze.py:40-78`（61 行 null_flag） · `reports/x1_report.md:18, 48`
X4 earliest / 顺序	tune 选最早层、test 确认；H2 顺序	按规则实现；field 在 title 位置 tune 上无显著层 → 报告写"order undetermined"。STATUS 写成"H2 order does not hold"，与报告措辞不一。	实现一致，解读措辞不一	⟶ 出处 `stage13_x1x4_analyze.py:105-114, 128-131` · `reports/x4_report.md:16-17` · STATUS.md §6 0c
X4 recognition	档内 target > control（区间不含 0）；Stage 10 估计量（Stage 10 min_n = 30）；通用规则要求在 tune 选、test 确认	`min_n=10`；在**全部论文**（tune+test 合并）上逐层报告，没有 tune/test 区分，也没有自动判定行	**偏离**	⟶ 出处 `stage13_x1x4_analyze.py:80`（`min_n=10`），115-122
区间（addendum 2）	论文版与团队版都给，团队为主	X1/X4/X6/X7/M3 只给团队版	**偏离（缺论文版）**	⟶ 出处 各分析脚本 docstring

X6	addendum 3 公式与阈值	按公式；比值为均值之比；5 个标题平均为 R 的简单平均；判定 target/control/all 三组都 HOLDS	一致	⟶ 出处 stage13_x6_analyze.py:55-92（90 行判定）· reports/x6_report.md 末段
X7	S(L52)≤0.05 且 S(L0)≥0.5	按公式；control S(L52) = 0.051 → DOES NOT HOLD（差 0.001）；target 0.018、all 0.029 HOLD	一致（结果如实报告）	⟶ 出处 stage13_x7_analyze.py:82-86（84 行判定）· reports/x7_report.md · STATUS.md §6 0b
token 角色（addendum 4）	X6/X7 等名字上的结果也按 token 角色报告	stage13_x6_analyze.py、stage13_x7_analyze.py 中没有任何 role 拆分；x6/x7 报告也没有。P1 的角色拆分已于 2026-10-30 从原始数据重算（s12-p1）；X4 的角色拆分在逐 token 全量（作业 11500437）后完成（s13-x4）	**部分偏离（X6/X7 未做）**	⟶ 出处 grep -n role stage13_x6_analyze.py stage13_x7_analyze.py（无结果）
X2、X5	X2 运行前另写补充；X5 层序	没有 X2 补充；**X2 暂停**（用户 2026-10-30 决定）；X5 计划在 M3 与 X4 全量完成后用已有数据在 CPU 上做，尚未做	**计划未做**	⟶ 出处 config 目录 · PROGRESS.md:477
X3	addendum 2/4 规则；para_ok 人工把关	para_gen 已完成；判断器 v2 检验一致率 0.55 < 0.80，不用，改走备用方案；para_run 等补充 5（待用户确认）与 para_ok，主分析未做	**计划未做完**	⟶ 出处 PROGRESS.md 2026-10-29/30 条

B6. 计算资源与数值一致性检查

硬件与总量

- 所有 Stage 10–13 GPU 工作的日志头都写 `NVIDIA A100-SXM4-80GB`（37 次单卡、2 次双卡），节点 d1028/d1029；模型 Qwen3-32B bf16 整模型放在 GPU 上，不允许 CPU offload。⟶ 出处 `grep "gpus \[" 351_stage10/logs 361_stage11/logs 371_stage12/logs 381_stage13/logs · 351_stage10.py:371-386`
- sacct（2026-10-15 起，用户全部作业）：申请到 GPU 的作业 135 个，合计约 **124.3 GPU-h**（含失败与等待放弃的时间）：交互式 `bash` 作业 69 个 89.4 GPU-h（2026-10-15 至 10-22，Stage 3–9），`s10p_*` 43 个 21.1，`s12w` 12 个 9.2，`s12fit` 3 个 3.5，parity 类 5 个 0.9，`s11w` 3 个 0.07，`s13_fugpu` 1 个 0.17。按节点：d1029 58.5、d1028 55.9、d1026 6.3（A100-40GB）、d3233 2.0、d1002 1.0、d4054 0.5、d3204 0.1 GPU-h。CPU 作业 97 个（含长时间等待的 gate，墙钟合计 833 h，不代表计算量）。⟶ 出处 `sacct -u zhang.zhiqi2 -S 2026-10-15 -X -o JobID,JobName,Partition,AllocTRES,ElapsedRaw,State,NodeList`（2026-10-30 本次汇总；GPU 卡数取 AllocTRES 的 gres/gpu）
- 每个作业只有 1 张卡，除 s12w 11428778（2×A100，50 min，d1028）和 11414108（2×，2.6 min 失败）。

其余节点的 GPU 型号（U19 已解决）：d1002 = V100-SXM2（gpu:v100-sxm2:4）；d3204 = A100（gpu:a100:8）；d3233 = A100（gpu:a100:3）；d4054 = H200（gpu:h200:8）。d3204/d3233 没有 a100@80g 标记，显存大小节点信息里没写。只有 2026-10-15/18 少量交互作业落在这些节点（self100 拟合最后一段 10-18 04:43 在 H200 上续跑，作业 11180979@d4054）。⟶ 出处 `scontrol show node d1002 d3204 d3233 d4054` · 附录 I U19、U15 **复核过**

各实验的 GPU、作业号与时长

Stage 10–13 的 GPU 作业是"池化"的：任何作业可以接手其他 Stage 的单元（hand-over/steal），所以按作业名统计会混在一起。下表的"计算时长"取自工作日志里每个 shard 的秒数之和（不含每个作业约 25–30 min 的权重加载），作业号取自 claims/owner 文件。

实验	GPU	shard 计算时长合计	主要作业号	日期	出处
Stage 3 self100 拟合（100 prompt，~150 s/prompt）	1×A100-80GB	约 5.7 h 墙钟（日志 23:06 → 04:45，4 次续跑）	交互式 srun bash（作业号无法逐一对应）	10-17/18	⟶ 出处 `qwen32b_jlens_prelim_100.log`

Stage 3.5.1 split-half A/B	1×A100-80GB	未逐条统计	交互式	10-18	⟶ 出处 351_split_A.log, 351_split_B.log
Stage 4–8 GPU 读出（6、7.2、7.3、8C 等）	交互式作业多在 d1028/d1029	未逐条统计（例：Stage 8C 4,200 条，约 6 s/条）	交互式	日志 mtime 10-18 至 10-20	⟶ 出处 351_stage6_run.log, 351_stage7_2_gpu.log, 351_stage7_3_gpu.log, 351_stage8C_full.log
Stage 9 Fit A（含修复）	第一段（0→4）1×A100-40GB（d1026），其余 1×A100-80GB（d1028/d1029）	~325 s/prompt × 101	交互式，按日志 budget_seconds 认出：0→4 11191976@d1026；4→20 11195756@d1029；23→42 11211146；42→59 11239485；47→56 11242682；56→100 11248534；恢复 80→100 11259149；修复 11291707、11295242/6、11296088/9（均 d1028）⟶ 出处 附录 I U12	10-18 → 10-22 05:33	⟶ 出处 351_stage9_full_A.log, 351_stage9_A_repair.log
Stage 9 Fit B（66/100 停止）	1×A100-80GB	~330–360 s/prompt × 66	交互式：0→16 11254812/4@d1028；16→34 11257769/71@d1028；34→51 11259150@d1029；51→66 11289356@d1028（2 h 时限到达被杀）⟶ 出处 附录 I U12、U33	10-20 05:57 → 10-21 19:12	⟶ 出处 351_stage9_full_B.log
Stage 10 parity	1×A100-80GB	4:07（ext）/ 33:59（main）/ 5:22（重复）	11338000（ext PASS）、11347585（main，0.25 下 FAIL）、11350174（重复，数值完全相同）、11334054（hf_device_map 崩溃）	10-24/25	⟶ 出处 351_stage10/logs/parity_*.log · reports/parity.json
Stage 10 主轮（7 条件 × 72 shard，5 readout × 6 层）	1×A100-80GB	6.31 h（每条件 0.87–0.91 h，~45 s/shard）	11363844–11363850（s10p_main_*）及 steal	10-25 → 10-27	⟶ 出处 351_stage10/logs/pool/job_s10p_*.log（"[cond] shard … Ns" 行合计，本次解析）
Stage 10 扩展轮（public + logit，L50–55、57–61）	1×A100-80GB	5.87 h	11363851–11363863（s10p_ext_*）；第一块 11363852	10-25 → 10-27	
Stage 10 早期失败	—	19 个 pool 作业拿到卡后数秒退出（parity 总状态 FAIL 时 run() 拒绝运行）	11338070–11338105、11347140/41、11350173、11358551–54	10-25	⟶ 出处 PROGRESS.md 2026-10-25 ~21:40 条

Stage 11（E1 patch、E1t、E2 knockout、E2n、E3 chain、E4 titlepos）	1×A100-80GB	4.66 h（chain 1.46、titlepos 1.80、patch 0.48、patch_title 0.44、knockout 0.25、knockout_name 0.25）	11363847–11363851（hand-over，claims owner）	10-27	⟶ 出处 361_stage11/logs/claims/*/owner · work 日志 shard 行
Stage 12（P1、P2、N1、N2、F2）	1×A100-80GB	4.42 h（patch_pos 1.52、prior_mc 1.24、prior_pure 0.61、alens_read 0.56、patch_comp 0.47）	11413886、11413591、11374283、11413586、11413584、11413569、11413572、11413557、11371622	10-27 → 10-28	⟶ 出处 371_stage12/logs/claims · work 日志
Stage 12 F1 作者 lens 拟合（40 prompt）	单卡 A100-80GB（规则：只有单卡 80GB 才接拟合）	~269 s/prompt × 40 ≈ 3.0 h	s12fit 11413557、11413569、11416869 与 s12w 11413572 等	10-27 15:28 → 10-28 17:12	⟶ 出处 371_stage12/alens/progress.json · logs/claims/alens_fit.done_* · STATUS.md §3 Stage 13 条
Stage 13（X6 xf_patch、M3 task、X1 xpatch、X4 pscope、X3 para_gen、X7 ko_false）	1×A100-80GB（1142877 8 为 2 卡）	4.41 h（xf_patch 1.87、para_gen 1.08、task 0.54、pscope 0.38、ko_false 0.28、xpatch 0.27）	11428778、11453127、11416869、11415844、11374284 等	10-28 19:46 → 10-29	⟶ 出处 381_stage13/logs/claims · work 日志
Stage 13 GPU follow-up（x4tok + Qwen judge 校准）	1×A100-80GB	10:28（x4tok 检查失败后按规则停止）	11485349	10-30	⟶ 出处 381_stage13/logs/fugpu_11485349.log · PROGRESS.md 2026-10-30 03:30 条

注：shard 行合计按日志中出现的行计（重跑的 shard 也计入；例如 ko_false 出现 27 行而单元数是 24）。

交互作业与实验的对应（U20，按时间对应）：按用户 2026-10-30 的决定，用 sacct 的起止时间与 /scratch/zhang.zhiqi2/jlens 下文件的 mtime、拟合日志的 budget_seconds 对应；69 个交互式 bash GPU 作业中大部分能对上（例：10-15 03:29–09:17 的 11095644 等 = Stage 3 早期验证与 fitting_corpus；10-17 21:50–10-18 04:58 的 11176076 等 = self100 拟合与 351_all；Stage 9 各段见上表），**约 15 个作业期间没有写出任何文件（失败或空等），用途无法追溯**，记为"无法追溯"。mtime 只

能证明文件在该时段写过，不排除同时段另一个作业。⟶ 出处 `sacct -X -o JobID,JobName,AllocTRES,Start,End,State,NodeList` · 附录 I U20 · PROGRESS.md 2026-10-30 14:00 条 U20 复核过 （部分无法追溯）

stage10_extra_r0.py（U35 已解决）：没有单独的作业号。它是 Claude 在会话里用 Bash 直接运行的（输出写于 10-27 18:57:08–18:58:47 EDT），运行在 Claude 所在的交互作业 11371284（bash@c0591，short，10-25 21:34 – 10-27 21:34）里，CPU。⟶ 出处 附录 I U35 · `351_stage10/summary_r0/` 与 `logs/r0_sensitivity.log` 的 mtime 复核过

未知/需确认：向 组长 确认（Stage 1 标注运行——开放回答与 4 选 1——使用的 GPU、时长与 run.json；U10）

bf16 一致性检查

检查	比较的是什么	结果	出处
Stage 10 parity（ext）	快路径（一次前向读所有位置）vs jlens `apply` 逐前缀单独前向；12 个 context、52 个 token 位置（rng 10）	22 对全部 ≤0.25，max 0.25（public L52/L58/L61、logit L61）；rank 不一致 2–7/52，全部在 rank ≥37	⟶ 出处 `351_stage10/reports/parity.json runs[0..2]` · `parity_before_tol05_amendment.json` · `351_stage10.py:464-508`
Stage 10 parity（main，两次）	同上，25 对（3 把 lens + logit × 6 层 + final）	10/25 对 >0.25，max 0.367 在 **final@L63**（无 lens 代码）；两次结果逐位相同（确定性）；rank ≤41 全部一致，最小不一致 rank 42，相对变化 ≤18%，log10(1/rank) 变化 ≤0.070	
Stage 10 crosscheck vs Stage 6	Pilot50 重叠的 972 个 token，jlens_public / logit / final 与 Stage 6 原始行	token id 100% 一致；rank 完全一致 77–95%；|Δlogprob| 中位 3e-6–7e-5，最大 0.43	⟶ 出处 `351_stage10/reports/crosscheck_stage6.csv` · `351_stage10.py:613-633` · 作业 11373362
Stage 11 GPU smoke	4+2 篇论文写入临时根目录，对比 Stage 10 rank 等	**WARN**：ko_baseline 与 Stage 10 一致 0.704（阈值 0.75），patch_title_L63_equals_false 0.704，chain 一致 0.833；其余通过。按规则继续。	⟶ 出处 `361_stage11/reports/smoke.json`（2026-10-27 05:17） · `361_stage11.py:401-475`（466 行阈值）

Stage 11 real-weight CPU check	真实权重在 CPU 上（无 GPU batch 差异）复算一篇论文	**REALCHECK PASS 13/13**（1,582 s）；E1t L63 在 CPU 上与 false **完全相等** → GPU smoke 的 0.70 是 batch 形状的 bf16 舍入，不是 bug	⟶ 出处 361_stage11/logs/realcheck_11396804.log · PROGRESS.md:705
Stage 12 非 A100 parity	非单卡 80GB 作业先重算 prior_mc delete shard 0，只记 WARN	只有 mock 测试文件 v100_smoke.mocktest_20261028.jsonl；实际所有工作都在 A100-80GB，未触发	⟶ 出处 371_stage12/logs/ · PROGRESS.md:505
Stage 13 X6 复现	xf_patch false_0 shard 0 vs Stage 11 patch shard 0	row_id、layers、rank_true/false/patch（516×15）**完全相同**	⟶ 出处 PROGRESS.md 2026-10-28 21:30 条
Stage 13 X7 自检	不屏蔽的 false 元素 vs Stage 11 false run	WARN：只有 59.6% 行完全相同（均 |Δlog10 rank| 0.0105，有符号 −0.0003）；T 与 Stage 11 E2 最大差 9.9e-17	⟶ 出处 381_stage13/reports/x7_report.md 第 6–8 行
Stage 13 x4tok（例外：致命）	batch 2（L8、L40）重算 vs 已存 batch 7 的 X4 分数	第一版 FAIL：max |diff| 0.068 > 0.05，作业 11485349 按设计停止；用户裁定这是正确性检查而非容差，改为致命。**2026-10-30 修法**：同时算 b7（与原 X4 同样一次注入 7 层）与 b2。样本 11500083：b7 与已存分数逐值相同（max |d| 0）。全量 11500437：b7 与已存 4,712 个值完全相同 → CHECK PASS；L8 平均 D（1,178 篇）b7 +0.0484、b2 +0.0490，差 0.0006 < 0.005，批大小不影响结论。旧标记已改名为 `followup/X4TOK_CHECK_FAILED.resolved_b7_reproduces_exactly_2026-10-30`（未删除）。⟶ 出处 381_stage13/logs/fugpu_11500437.log:30–34 · PROGRESS.md 2026-10-30 11:50、15:00 条 复核过	⟶ 出处 PROGRESS.md 2026-10-30 03:30 与 2026-10-30 "User-approved rewrite" 条 · 381_stage13/logs/fugpu_11485349.log "STOP: x4tok does not reproduce"

"容差 0.25 → 0.5"的经过

1. **2026-10-23** 冻结：parity 门槛 max |Δlogit| ⩽ 0.25（`--tol`），作为 run() 的放行条件。⟶ 出处 351_stage10/config/analysis_plan_before_tol05.json（无 amendments） · 351_stage10.py:497
2. **10-24 15:18** ext parity（11338000）PASS，max 正好 0.25。**10-24 23:48** main parity（11347585）FAIL：10/25 对超 0.25，max 0.367（final@L63）。**10-25 00:37** 重复 main parity（11350174）结果逐位相同。
3. **10-25 00:22–00:31、~07:10**：19 个 pool 作业拿到 A100 后数秒退出（run() 检查到总状态 FAIL）——这是"容差卡住 GPU"的直接代价。⟶ 出处 PROGRESS.md 2026-10-25 ~15:00、~21:40 条
4. 诊断：差异在模型自身前向（final@L63 无 lens 代码也 0.367；logit 0.16–0.23），来自一次前向 vs 逐前缀前向的 bf16 累加顺序；|logit| 32–64 时 bf16 最小间隔就是 0.25，0.367 ≈ 1–1.5 个 ulp。⟶ 出处 PROGRESS.md 2026-10-25 ~22:00 条 · STATUS.md §5

5. **10-25 11:00:31** 用户批准改为 0.5：在 parity.json 加 `tolerance_amendment`（old 0.25 → new 0.5，三次运行在 0.5 下均 PASS，各运行原状态保留），analysis_plan.json 加 amendments 条目；备份 parity_before_tol05_amendment.json 与 analysis_plan_before_tol05.json。只在 CPU 上用已存数字重判，没有重跑 GPU。⟶ 出处 351_stage10/reports/parity.json tolerance_amendment · config/analysis_plan.json amendments[0]
6. 已记录的问题：0.5 是在看到数据之后定的，偏离了冻结的 0.25（计划里已注明）；parity 也抓不到两条路径共有的 bug。⟶ 出处 PROGRESS.md:1115
7. **10-27 03:10** 用户把它升为长期规则：parity/容差/一致性未达标一律记 WARN、放宽或忽略后继续算，只有代码崩溃才能停队列；Stage 11 smoke 改为 PASS/WARN 都继续、ERROR 才 stop_queue。写入 CLAUDE.md、STATUS.md §5 与 memory。⟶ 出处 PROGRESS.md:730-741 · ~/jlens_work/CLAUDE.md 长期约定 · HANDOFF.md:78（之后的 parity 必须用 --tol 0.5，否则脚本会把总状态重算成 FAIL）
8. 例外：2026-10-30 x4tok 自检被用户明确定为"正确性检查"，失败即停（见上表）。

已完成 所有数字取自 parity.json、smoke.json、realcheck 日志与 sacct；GPU 型号对 Stage 10–13 已由日志核实。

Stage 1–9 方法手册（小样本阶段）

本部分逐个写出 Stage 1–9 每个子实验的实际做法。所有数字都回到了 `/scratch/zhang.zhiqi2/jlens/351_*` 下的结果文件（与归档 `/scratch/zhang.zhiqi2/jlens_archive_stage1_8/experiment_files/` 内容相同）核对过；只在已发布报告（`stage1_11_report.html` 中 `p1-` 开头的节）里出现、在原始文件里查不到的数字，标"未在原始文件中核实"。由本手册用 CPU 从原始 CSV 重算的数字，注明"本手册重算"。

共同约定（各 Stage 都用到）

项目	设定	出处
模型	Qwen/Qwen3-32B，revision `9216db5781bf21249d130ec9da846c4624c16137`，BF16，`.eval()`，不生成文本，只做前向（teacher forcing）	⟶ 出处 `05_stage5/351_stage5.py:9-10` · `04_stage4/351_stage4_readout.py:9-10`

读出提示词	`Who are the authors of the paper titled "{title}" ({year})?`
	`Authors:`
	
	不套 chat template；读出位置是最后一个 token（":" 之后），`positions=[-1]`
		⟶ 出处 `05_stage5/351_stage5.py:11, :203-204` · `04_stage4/351_stage4_readout.py:12`

名字切词	`encode(prompt)` 一次，再接 `tokenizer.encode(" " + 名字, add_special_tokens=False)` 的 token ID；不对"提示+名字"整体重新切词	⟶ 出处 `04_stage4/351_stage4_readout.py:49-61`（token_plan） · `351_all/351_stage4_artifacts/config/run_config.json "tokenization"`
名次	r = 1 + 词表中 logit **严格大于**目标 token 的个数（并列取最好名次）；倒数名次 RR = 1/r	⟶ 出处 `04_stage4/351_stage4_readout.py:93-98`（measure） · `351_stage6/config/protocol.json "ranking"`
读出方式	J-Lens（`lens.apply(..., use_jacobian=True)`）、Vanilla = logit lens（`use_jacobian=False`）、Final = 模型真实 L63 输出；同一前向同时得到三种	⟶ 出处 `05_stage5/351_stage5.py:191-209`
层	源层 L16/24/32/40/48/56/62（0 起算），目标层 L63；Stage 3 自拟合 lens 只有 L24–62 六层	⟶ 出处 `05_stage5/351_stage5.py:8`
实现保护	强制使用指定 token ID（替换 `model.encode`）并核对；J-Lens 与 Vanilla 两次前向的 final logits 最大差 > 0.1 即报错停止（实现保护，不是效应阈值）	⟶ 出处 `05_stage5/351_stage5.py:205-206` · `04_stage4/351_stage4_readout.py:86-89`

显存布局	29 个 transformer block 放 GPU，35 个放 CPU；embedding、final norm、rotary、lm_head 固定在 GPU。原因：`device_map='auto'` 时 jlens 直接调用 final norm/lm_head 会遇到 meta tensor 报错（`NotImplementedError: Cannot copy out of meta tensor`），不 offload 则在 40GB 卡上 OOM	⟶ 出处 01_public_lens_validation/351_validate_public_lens_offload_fixed.py:150-159 · 351_all/351_offload_validation.log 末行 · 351_all/351_public_task_validation.log 末行（CUDA OOM, 39.49 GiB）
Public-557 lens	Neuronpedia `qwen3-32b/jlens/Salesforce-wikitext/Qwen3-32B_jacobian_lens.pt`，sha256 `0af5ccf2e1c251bedc202607a7ca214f3b9f60980bab73d67a3d0b887db213d8`，n_prompts = 557，覆盖 L0–L62	⟶ 出处 351_provenance_audit/summary/audit_summary.json → checkpoint.neuronpedia · 351_stage5_crossed_v2/config/runtime.json
作业号	Stage 1–9 都是在 `gpu-interactive` / `short` 的交互式 `srun ... bash` 会话里手动运行的，`sacct -S 2026-10-15 -E 2026-10-23` 中作业名几乎全是 `bash` / `pty`，无法从作业名把作业号对应到实验。2026-10-30 按用户决定，用 sacct 起止时间与文件 mtime、日志时间戳逐段对应：大部分能对上（Stage 9 各段见 h-compute-table），约 15 个作业期间没有写出文件，记为"无法追溯"。⟶ 出处 附录 I U20 · PROGRESS.md 2026-10-30 14:00 条 U20 **复核过**	⟶ 出处 `sacct -u zhang.zhiqi2 -S 2026-10-15 -E 2026-10-23 -X`
Git	Stage 1–8 的代码和结果整体在 commit `e2c7156`（"Add Stage 1–8 experiment code, data, figures and archive records"）；Stage 7.1B 和 Stage 9 不在 git 里（仓库 README 写"Excluded … Stage 7.1B, Stage 9 (still running at archive time)"）	⟶ 出处 `git -C /scratch/zhang.zhiqi2/jlens-authorship-recall_update log --oneline -- code/scratch_scripts/351_stage5.py` · README.md:194

字母 A/B 有三种用法：行为分组 A/B（本手册默认）、Stage 3 split-half 的 A/B 两半、Stage 9 的 Fit A/Fit B，三者无关。

Stage 1 · 队列与行为标签

1.1 标注包与四组队列（6,841 篇）

1 问题与作用

定义研究对象：哪些论文、模型"认得但说不出"作者。后续所有 Stage 都从这里取论文和分组。

2 材料

- 组长的标注包 `zhiqi_qwen3-32b_labels_2026-10-15`（zip 在 `00_cohort/`，解压在 `/scratch/zhang.zhiqi2/jlens/labels_20261015/`）。主表 `paper_labels_with_papers.csv`，9,108 行，sha256 `b4ff5e1d045c6f9ec2f42226e537dbb32be32d7df9684545917bad04b09faf0e` ⟶ 出处 `351_all/351_audit.json`。
- 论文来自 *Dreaming Up Authors* 评测集（OAG v2 + Semantic Scholar，8 领域 × 13 引用档，每格最多 100 篇）⟶ 出处 `~/jlens_work/STAGES.md "Stage 1"`；标注包内 bin_table 每档 796–800 篇（高档较少：1025-2048 档 665、2049-4096 档 326、4097+ 档 122）⟶ 出处 `labels_20261015/.../label_README.md:78-90`。

3 工具

标注由组长的 `elm` 流程生成（`elm.stages.filter.measure`、`elm.stages.label.report`），模型 Qwen3-32B 本地 bf16、贪心解码、A100 80GB ⟶ 出处 `label_README.md:3 · README_first.md:3-4 · step1_README.md:1-3`。本项目只用 `00_cohort/351_prepare_all.py`（CPU）把标签映射成四组。

4 步骤

1. **开放回答**：system "You are a knowledgeable assistant. Answer factually and concisely. When asked for authors, return a semicolon separated list of full names only, with no additional text." + user "Who are the authors of the paper titled '{title}' which was published in {year}? Return a semicolon separated list of author names only."，贪心解码 ⟶ 出处 `step1_README.md:107-112`。用姓名匹配得到 m（对）、h（编造）、u（漏）和 HR2 = (h+u)/(m+h+u) ⟶ 出处 `step1_README.md:41`。
2. **recall_level**：HR2 > 0.5 且 m = 0 → R0；HR2 > 0.5 且 m > 0 → R1；HR2 ⩽ 0.5 → R2 ⟶ 出处 `label_README.md:24 · rules.yaml recall_failure_threshold: 0.5`。
3. **多选题**：真作者名单 vs 另 3 篇论文的真实作者团队（同领域、相近团队规模、引用档和年份），4 种选项顺序（ABDC、BCAD、CDBA、DACB）各问一次，取下一个 token 中字母 "A"+" A" 等概

率；`mc_team_a` = 真选项概率最高且字母总概率 ≥ 0.5 的顺序数（0–4）⟶ 出处 `label_README.md:27 · step1_README.md:53-58 · README_first.md:20-21`。提示词全文见 ⟶ 出处 `step1_README.md:74-103`（两个 few-shot 例子 + 题目）。

4. **group_tight**（本项目使用）：target = R0/R1 且 a = 4 且真选项平均概率 ≥ 0.5；control = R0/R1 且 a ≤ 1；guard = R2；其余 excluded ⟶ 出处 `label_README.md:37-38 · rules.yaml tightened: min_orders 4, min_p_true 0.5`。
5. **四组映射**（351_prepare_all.py）：target 且 R0 → A；target 且 R1 → C；control 且 R0 → B；control 且 R1 → D；其余 → excluded ⟶ 出处 `00_cohort/351_prepare_all.py:17`。纳入的行用 `assert level in ('R0','R1') and hr>0.5 and ((level=='R0')==(m==0))` 自检 ⟶ 出处 `351_prepare_all.py:19`——HR2 > 0.5 是自检条件，不是额外筛选（target/control 的定义本身已要求 R0/R1）。HR2 直接取自标注包，未重算。
6. 输出 `351_four_groups.csv`（6,841 行）、`351_bin_counts.csv`、`351_audit.json`。

5 公式

HR2 = (h + u) / (m + h + u)；识别通过（tight）⇔ a = 4 ∧ mean p(真字母) ≥ 0.5；失败 ⇔ a ≤ 1。

6 对照

标注包自带三项检查：低引用"地板"档 0-2 的通过率应 ≤ 10%；guard（R2）的通过率应 ≥ 90%；字母总概率 < 0.5 的顺序比例应 ≤ 10% ⟶ 出处 `rules.yaml · label_README.md:7-9`。另有 500 篇用第二套干扰团队（team_b）重抽。

7 结果

组	识别 / 开放回答	篇数
A	target / R0	4,110
B	control / R0	2,186
C	target / R1	511
D	control / R1	34
excluded	guard + excluded	2,267
合计		9,108（纳入 6,841）

⟶ 出处 `351_all/351_audit.json "group_counts" · 351_all/351_bin_counts.csv`。group_tight 全量：target 4,621、control 2,220、guard 214、excluded 2,053 ⟶ 出处 `label_README.md:7`；A+C = 4,621、B+D = 2,220，与 Stage 10 的 target/control 一致。

标注包自检：0-2 档通过率 38.6%（普通规则）/ 25.5%（tight），超过 10% 上限（检查未通过，两种标签仍都写出）；guard 通过率 98.1%；低字母概率顺序 0.0%；team_b 与 team 的通过标签一致率 83.6%（kappa 0.63）⟶ 出处 `label_README.md:8-9`。

8 局限与质疑

- 0-2 档 tight 通过率 25.5% ≈ 固定偏好某一选项的随机水平 25%，低引用档的"认得"可能大部分是猜 ⟶ 出处 `README_first.md:28-30`。
- 高引用档 control 极少（4097+ 档 0 篇 B、0 篇 D；2049-4096 档 B 8、D 1）⟶ 出处 `351_bin_counts.csv`，所以 A/B 配对在高档用尽。
- 开放回答有 8.2% 被截断、0.5% 拒答；400 篇用 batch size 1 重答，45.5% 文本不同（batch 噪声）⟶ 出处 `step1_README.md:11-13`。
- 标签由另一套代码 `elm` 生成，elm 源码与两次运行的 `run.json` 都不在集群上。生成设置与匹配规则已由 Dreaming Up Authors 代码包核对：开放题提示与 dua `stage1_generate.py` 逐字相同，thinking 关、贪心、`MAX_NEW_TOKENS = 96`（stage1_generate.py:171；743 条截断回答全为 96 token）；dua NameMatcher + L4 阈值复算 (m, h, u) 与标注包 9,095/9,108 一致 ⟶ 出处 附录 J 第 4、8 行 · g-labels-open、g-labels-match **复核过**。panel.py 的剥离规则只能从数据推断 **未知/需确认：向组长确认（metrics/panel.py 的名字切分与剥离规则原文，U04）**
- 多选题干扰团队的具体抽取规则（"相近"的阈值）只在 label_README 里有一句描述，dua 代码包里也没有这种干扰；bin_window / over_cap / et_al_mismatch 的含义已从数据推断（见 g-labels-mc）。**未知/需确认：向组长确认（干扰团队的匹配阈值原文与随机种子，U06）**

9 状态

已完成 2026-10-15 标注包；`351_prepare_all.py` 2026-10-17 运行。Git `e2c7156`。

1.2 Pilot50 抽样与 A/B 配对（Stage 4–7 共用）

1 问题与作用

为 Stage 5–7.4 冻结一个 50 篇的探索样本（A20/B20/C7/D3）和 20 对同领域同引用档的 A/B 配对。Stage 6、7.1、7.2、7.3、7.4 都复用这同一批 ID 和配对。

2 材料

`351_all/351_four_groups.csv`（6,841 篇，sha256 `c7c79bbb…a180`）；Stage 4 的 20 篇（Benchmark20，A8/B8/C3/D1，见 4.1）⟶ 出处 `351_stage5_crossed_v2/cohort/pilot50_v1.json → manifest.parent_csv_sha256, stage4_raw_sha256`。

3 工具

`05_stage5/351_stage5.py prepare`（CPU；配对用 networkx `max_weight_matching`）。

4 步骤

1. 强制保留 Stage 4 的 20 篇（组数必须是 A8/B8/C3/D1，否则报错）⟶ 出处 351_stage5.py:52-53。配额 A20/B20/C7/D3 ⟶ 出处 :60，即新增 A12/B12/C4/D2 共 30 篇。
2. **"种子 35150" 不是随机抽样**：每组其余论文按 `sha256("35150" + paper_id)` 的十六进制字符串排序（`stable_order`），这是确定性的哈希顺序 ⟶ 出处 351_stage5.py:7, :34, :63。 复核过 （2026-10-30：手册对 Pilot50、8A、8B 都写"哈希排序"，不写"随机抽样"；真正用种子随机数的是 Stage 4 的 20 篇、7.2 的随机错误作者池和 Stage 11 的样本）
3. 先为 Stage 4 的每篇 A（或 B）在同 (citation_bin, field) 里找一篇新的 B（或 A），按 |Δ年份| + 2×|Δ作者数| 最小、再按 paper_id 选 ⟶ 出处 :70-77。
4. 再按哈希顺序遍历新 A，为每篇在同 (bin, field) 桶里找代价最小的新 B，成对加入 ⟶ 出处 :78-85。
5. 仍不足配额时按哈希顺序补齐（C、D 也在这一步）⟶ 出处 :86-90。
6. **A/B 配对**：对选出的 20 A × 20 B 建二部图，只在 field 与 citation_bin 完全相同时连边，边权 = −(|Δ年份| + 2×|Δ作者数|)，求**最大基数**下的最大权匹配（先最大化对数，再最小化代价），不放回 ⟶ 出处 :126-145。C/D 不配对，只作描述。
7. 写出 `cohort/pilot50_v1.csv`、`matching/pilot50_ab_pairs.csv`、`controls/pilot50_controls.csv` 和带各文件 sha 的 `pilot50_v1.json`；已存在则拒绝覆盖 ⟶ 出处 :158-169。

5 公式

配对代价 $c(a,b) = |year_a - year_b| + 2 \cdot |n_authors_a - n_authors_b|$；$SMD = (\bar{x}_A - \bar{x}_B) / \sqrt{((s_A^2 + s_B^2)/2)}$。

6 对照

无（这是抽样步骤）。`validate_plan` 在每次 run/analyze 前重新核对 ID 顺序和各文件 sha，改动即报错 ⟶ 出处 351_stage5.py:171-189。

7 结果

- 50 篇 = 原 Stage 4 的 20 篇 + 新增 30 篇（A 8+12、B 8+12、C 3+4、D 1+2）⟶ 出处 351_stage5_crossed_v2/cohort/pilot50_v1.csv 列 stage4_existing（本手册统计）。
- 20 对 A/B 全部成对（P001–P020）。平均 |Δ年份| 1.65（最大 4），平均 |Δ作者数| 1.15（最大 7）；年份 SMD +0.016，作者数 SMD +0.287 ⟶ 出处 matching/pilot50_ab_pairs.csv 列 year_gap, author_count_gap；SMD 为本手册重算。
- 引用档分布：5-8 档 10 篇、0-2 档 6、65-128 档 6、3-4 档 5、其余各 3–4 篇；没有 257-512、2049-4096、4097+ 档 ⟶ 出处 pilot50_v1.csv 列 citation_bin（本手册统计）。

8 局限与质疑

- "同领域同档"不等于平衡：作者数 SMD 0.287 偏大，P005 作者数差 7、P009 差 5。
- n=20 对，统计功效很低；L56 等层在之后多次被看过，Pilot50 上的一切都只能算探索。
- Stage 4 的 20 篇不是为配对抽的，后加的 30 篇是围绕它们挑的，样本不是简单随机样本。

9 状态

已完成 （冻结于 Stage 5 prepare）。Git `e2c7156`。

Stage 2 · 模型、提示词与打分协议

2.1 固定模型与读出协议

1 问题与作用

保证后续每个 Stage 在同一模型、同一提示和同一打分规则下进行。Stage 2 没有独立目录和独立脚本，协议散在 Stage 4/5/6 的 config 里 ⟶ 出处 `~/jlens_work/STAGES.md "Stage 2"`。

2 材料

见上文"共同约定"表；协议记录文件：`351_all/351_stage4_artifacts/config/run_config.json`、`351_stage5_crossed_v2/config/protocol.json`、`351_stage6/config/protocol.json`。

3 工具

`jlens` 包（本地 `/home/zhang.zhiqi2/projects/jacobian-lens/jlens/`），`jlens.from_hf(hf, tokenizer)` 包装 HF 模型，`lens.apply(model, prompt, layers, positions=[-1], max_seq_len=512, use_jacobian=…)` ⟶ 出处 `05_stage5/351_stage5.py:236-238, :203-204`。

4 步骤

1. 加载模型（revision 固定）和 lens，记录 lens sha256；runtime_hash = sha(protocol_hash, lens_sha256, revision, layers)，续跑时不一致即拒绝 ⟶ 出处 `351_stage5.py:216-225`。
2. 每个上下文：提示切词一次（上限 512 token，超出即报错），名字 token 逐个接在后面；对每个前缀做一次前向，同时得到 J-Lens、Vanilla 各 7 层和 Final 的 logits。
3. 打分两类：**第一个 token**（只给提示，看名字第一个 token 的名次或 logit）；**后续 token**（给出正确名字前缀 $G_{1..j-1}$，看 G_j）。

5 公式

$r(t) = 1 + \#\{v : \text{logit}_v > \text{logit}_t\}$；作者分 = 对其 token 平均 1/r；论文-层分 = 对作者平均（作者等权）；7 层积分 = 7 个源层的算术平均，L63 不计入 ⟶ 出处 `351_stage6/config/protocol.json "author_mean","paper_layer","paper_integrated"`。这就是 JMRR（Jacobian MRR）。

6 对照

每次读出都同时算 Vanilla 和 Final，作为 J-Lens 的固定参照。

7 结果

协议在 Stage 4–8 保持一致：Stage 7.1B 冻结时核对 Stage 5 helper 的 sha256 为 `4fd4c9fb7027e6fe0127d94efe653c52c349cbdab13fc45d2ffdec4e7940c339`，Stage 8C 的 runtime 也记录同一 helper sha 和同一提示词 ⟶ 出处 `07_1_stage7_1/351_stage7_1B_terminal.py:13` · `351_stage8/config/stage8C_runtime.json "stage5_code_sha256","stage5_prompt"`。

8 局限与质疑

- 读出提示词（"Who are the authors … ({year})?\nAuthors:"，无 chat template）与标注时的开放回答提示（chat template + system，"which was published in {year}"）**不同**。读出测的是这一种格式下的内部状态。
- 名字前加空格独立切词，可能和模型自然生成时的切分不同（协议里明确写"not the tokenizer's natural joint encoding"）⟶ 出处 `04_stage4/351_stage4_readout.py:50-55`。
- 并列取最好名次，对并列多的低层 logits 有利。

9 状态

已完成 （无独立实验）。Git `e2c7156`。

Stage 3 · J-Lens 搭建与验证（Exp 1–7，含 provenance audit）

比较两个 lens：Public-557（Neuronpedia，557 条 WikiText-103 提示拟合，L0–L62）和 Self-100（自己拟合，100 条，6 层 L24/32/40/48/56/62）。共有层是 6 层，L16 不参与配对比较。Exp 编号沿用已发布报告第一部分（来源是 laptop 会话的 Validation_Record 文档，集群上找不到原件）。**未知/需确认：向用户确认（Exp 1–7 编号来源文档 Validation_Record_351_Completeness_Checked.docx 不在集群上）**

3.0 Self-100 拟合与 Public lens 的早期诊断（冒烟、作者特异性、任务验证）

1 问题与作用

先让 lens 在 Qwen3-32B 上跑通，并得到一个自己拟合、来源可查的 lens（Self-100），供 Exp 1–7 比较。

2 材料

- 拟合语料：`/scratch/zhang.zhiqi2/jlens/fitting_corpus/wikitext103_qwen32b_1000.jsonl`（1,000 条，sha256 `1af8dbff…0b06`），取索引 0–99，无随机选择 ⟶ 出处 `qwen32b_jlens_prelim_100_config.json · 351_provenance_audit/reports/audit_report.md "Audit 3"`。
- 公共 lens 冒烟：1 篇论文 1 个金标准首 token，L16/24/32 ⟶ 出处 `01_public_lens_validation/351_public_jlens_smoke.py:20`。
- 作者特异性：A/B/C/D 各 1 篇（seed 351），每篇 1 个金标准首 token vs 1 个同领域同档错误作者首 token ⟶ 出处 `351_author_specificity.py:93 · 351_all/351_author_specificity.jsonl（4 行）`。

3 工具

`jlens.fit`（本地 jacobian-lens 包），H200 GPU 拟合 Self-100 ⟶ 出处 `qwen32b_jlens_prelim_100.log "GPU: NVIDIA H200"`。

4 步骤

1. Self-100 拟合：源层 24/32/40/48/56/62 → 目标 L63，dim_batch 8，max_seq_len 128，skip_first 16 ⟶ 出处 `qwen32b_jlens_prelim_100_config.json`。检查点续跑，最后两条每条约 65–

68 s，"fit: done, 100 prompts" ⟶ 出处 `qwen32b_jlens_prelim_100.log`。产物 `qwen32b_jlens_prelim_100.pt`，sha256 `268aba15…40e3`。

2. 公共 lens 任务验证 `351_validate_public_lens.py` → 直接 `device_map='cuda'` 在 40GB 卡 OOM；`…_offload.py` 用 `device_map='auto'` 报 meta tensor 错；`…_offload_fixed.py` 改为显式 29 层 GPU 后跑通 ⟶ 出处 `351_all/351_public_task_validation.log`, `351_offload_validation.log`, `351_offload_fixed_validation.log`。

5 公式

拟合估计量见 Exp 6。

6 对照

无（准备步骤）。

7 结果

作者特异性 4 篇只给了单篇名次（例如 A 篇 L32 J-Lens 金标准名次 11,828，错误作者 725），是诊断，不做推断 ⟶ 出处 `351_all/351_author_specificity.log`。

8 局限与质疑

前两版任务验证脚本没有产出可用结果。

9 状态

Self-100 拟合、冒烟、作者特异性 **已完成**；`351_validate_public_lens.py` 与 `…_offload.py` 两次运行 **作废**（OOM / meta tensor 报错，由 `…_offload_fixed.py` 取代）。

3.1 Exp 1 · 数值检查

1 问题与作用

lens 矩阵能否加载、形状对、数值有限。

2 材料

Self-100 的 6 个 5120×5120 矩阵；Public-557 的 63 个矩阵。

3 工具

`02_self_lens_validation/351_self_jlens_validation.py --mode numerical`（函数 `numerical`，:71-86）。

4 步骤

逐层读矩阵，记录 shape、dtype、是否全有限、min/max/mean/std、Frobenius 范数、max_abs；任一不有限或形状不对即报错。

5 公式

$\| J \|_F = \sqrt{\Sigma J_{ij}^2}$。

6 对照

无。

7 结果

层	Self-100 $\| J \|_F$	有限
L24	39.62	是
L32	49.59	是
L40	86.40	是
L48	104.89	是
L56	92.88	是
L62	80.36	是

⟶ 出处 `351_self100_validation/numerical/matrices.csv 列 fro_norm, finite`。Self-100 6/6 通过。

已发布报告写"Public 63/63 通过，L48 范数 105.8 vs 104.9"。2026-10-30 用 CPU 从 lens 文件重算各 lens 的逐层 Frobenius 范数（作业 11500587）：**105.8 是 Public lens 的 L48 范数按 float32 求和的值**（105.81；float64 精确值 106.07），104.9 是 Self-100 的 float32 值（104.89）。L48 精确值（float64）：Public 106.07、Self-100 105.16、split-half A 半 105.86、B 半 105.49、Stage 9 fitA 99.62；所有矩阵都有限。⟶ 出处 `351_stage8/recompute_2026-10-30/stage3_lens_frobenius_norms.csv 列 frobenius_fp64 / frobenius_fp32（行 layer=48）· stage3_lens_frobenius_norms_11500587.log · ~/jlens_work/08_stage8/recompute_2026-10-30/stage3_lens_frobenius_norms.py` 复核过 （U22 已解决；本手册旧稿"105.86 实为 split-half A 半"的说法有误，已删除）

8 局限与质疑

范数相近不代表方向一致，更不代表读出正确。

9 状态

已完成 （Self-100）；Public 部分 复核过 （2026-10-30 重算范数，作业 11500587）。

3.2 Exp 2、4 • 读出保真度（KL、Top-10、final-top1 名次）

1 问题与作用

中间层读出能否近似模型最终的下一个 token 分布。分两种文本：通用短文本（Exp 2）和论文提示（Exp 4）。

2 材料

- 通用：20 条自写提示（如 "Explain why a bicycle remains easier to balance…\nAnswer:"），max_seq_len 128 ⟶ 出处 `02_self_lens_validation/351_self_jlens_validation.py:13-34, :89`。
- 论文：20 篇论文的读出提示，按引用档轮转抽（`random.Random(74219)` 打乱每档后轮流取），max_seq_len 512 ⟶ 出处 `351_self_jlens_validation.py:40-50`。
- Public 的对应结果取自 `351_all/351_public_fidelity.jsonl`、`351_paper_prompt_fidelity.jsonl`（同一批提示、同一种子，脚本核对 seed=74219）⟶ 出处 `:154, :236-237`。

3 工具

`351_self_jlens_validation.py --mode generic/paper/report`；Public 侧 `351_public_lens_fidelity.py`、`351_paper_prompt_fidelity_offload.py`。

4 步骤

每条提示在最后位置读出 6 层 lens logits 与 final logits，按函数 `metric` 计算四个指标；Self–Public 在同一提示上配对，bootstrap 求 95% CI。

5 公式

$KL(P_{final} \| P_{lens}) = \Sigma_v P_{final}(v)[\log P_{final}(v) - \log P_{lens}(v)]$（nats）；Top-10 重合 = |top10(lens) ∩ top10(final)| / 10；final-top1 RR = 1 / (lens 中 final 第一名 token 的名次) ⟶ 出处 `351_self_jlens_validation.py:58-64`。

6 对照

Vanilla（logit lens）在同一提示上的同一指标。

7 结果

层	通用 Self–Public KL [95% CI]	论文 KL Public / Self / Vanilla	论文 Top-10 Public / Self / Vanilla
L24	−0.379 [−1.129, +0.520]	9.460 / 8.286 / 9.422	0 / 0 / 0
L32	−1.221 [−1.954, −0.488]	11.135 / 7.180 / 8.810	0 / 0 / 0
L40	−1.443 [−2.264, −0.629]	8.869 / 6.869 / 8.299	0 / 0 / 0
L48	−0.301 [−0.543, −0.052]	8.479 / 8.722 / 9.922	0 / 0 / 0

L56	−0.134 [−0.383, +0.123]	5.207 / 5.124 / 8.286	0.040 / 0.045 / 0.005
L62	−0.018 [−0.041, +0.005]	0.584 / 0.578 / 0.808	0.345 / 0.355 / 0.720

⟶ 出处 `351_self100_validation/summary/paired_bootstrap_ci.csv`（task=generic, metric=kl_final_to_lens_nats）· `summary/metrics.csv`（task=paper）· `351_all/351_paper_fidelity.log` 末段（Public 论文 KL、Top10）。Public 论文 KL = Self KL − Δ，例如 L24 8.286 + 1.175 = 9.460

Self-100 在 6/6 个通用层、5/6 个论文层 KL 更低（论文 L48 Self 更差，Δ = +0.243 [+0.003, +0.520]）。论文 L62：两个 lens 的 KL 都低于 Vanilla，但 Top-10 重合 0.345/0.355 远低于 Vanilla 0.720。

8 局限与质疑

- 每类只有 20 条提示，CI 未做多重校正。
- KL 低不代表排名对：分布保真度和排名保真度不能互换。
- Public 的拟合文本未知，WikiText 来源可能与通用提示风格重合（见 Exp 5）。

9 状态

已完成。Git `e2c7156`。

3.3 Exp 3 · 金标准 vs 错误作者（20 篇）

1 问题与作用

在作者任务上，两个 lens 谁更能把真作者排在错误作者前面。

2 材料

20 篇论文（按引用档轮转，`random.Random(351)`），每篇 1 个金标准（作者列表中第一个"有效全名"：≥2 词、≥8 字符、无数字、无"."）+ 1 个错误作者（同领域同档、按年份最近的前 60 篇中，token 数差最小、首 token 与金标准不同的名字）⟶ 出处 `01_public_lens_validation/351_validate_public_lens_offload_fixed.py:15-16, :46-83`。Self 侧复用 Public 侧完全相同的论文、名字和 token ⟶ 出处 `351_self100_gold_false/archive/config/comparison_config.json "protocol": reuse_original_public_pair_and_strict_tokenization_v1`。

3 工具

`351_validate_public_lens_offload_fixed.py`（Public）、`02_self_lens_validation/351_self_gold_false.py`（Self）。

4 步骤

对金标准和错误名字分别做 teacher forcing，记录每个 token 的 log-prob。

5 公式

首 token 胜 = $1[\text{logp}_{\text{gold}}(t_1) > \text{logp}_{\text{false}}(t_1)]$；全名胜 = $1[\text{mean}_j \text{logp}_{\text{gold}} - \text{mean}_j \text{logp}_{\text{false}} > 0]$ ⟶ 出处 `351_validate_public_lens_offload_fixed.py:93`。

6 对照

Vanilla、Final 同时算。

7 结果

层	Self 首 token 胜率	Public 首 token	Self 全名胜率	Public 全名
L24	0.70	0.70	0.70	0.65
L32	0.55	0.60	0.55	0.55
L40	0.40	0.50	0.55	0.45
L48	0.55	0.45	0.70	0.70
L56	0.75	0.75	0.55	0.50
L62	0.50	0.45	0.55	0.55

⟶ 出处 `351_self100_gold_false/archive/summary/layer_summary.csv` 列 `first_win_rate`, `full_win_rate`。Final：首 token 0.50，全名 0.60。

8 局限与质疑

20 篇、每篇 1 个错误名字，参照太窄；全名胜率用了 teacher forcing，不是自由回忆；两个 lens 互不占优。

9 状态

已完成。

3.4 Exp 5 · 泄漏审计（provenance audit 第 3 部分）

1 问题与作用

拟合文本和评估提示是否重叠。

2 材料

Self-100 拟合的 100 条文本；40 条评估提示（20 通用 + 20 论文）；Public lens 的 provenance 元数据。

3 工具

`03_provenance_audit/351_jlens_provenance_audit.py`（CPU）。

4 步骤

规范化后完全匹配；8 词 n-gram 重叠标记；读两个 .pt 的键和元数据。

5 公式

无。

6 对照

无。

7 结果

Self-100 vs 40 条评估提示：0 次完全匹配，0 次 8 词重叠标记。Public-557 的 557 条拟合文本没有随 checkpoint 提供，逐条重叠"UNDETERMINED" ⟶ 出处 `351_provenance_audit/reports/audit_report.md "Audit 3"`。

8 局限与质疑

文本不重叠不等于分布独立；Public lens 无法做"无泄漏"结论。

9 状态

已完成 。

3.5 Exp 6 · 拟合协议与源码对比（provenance audit 第 4 部分）

1 问题与作用

两个 lens 的估计公式和拟合条件是否可比。

2 材料

本地 `jacobian-lens/jlens/fitting.py` 与 Neuronpedia 版本 `neuronpedia_utils/jlens/jlens/fitting.py` 的 diff（519 行）⟶ 出处 `03_provenance_audit/351_jlens_fitting_diff.txt (wc -l = 519)`；Public checkpoint 内嵌的 provenance。

3 工具

同上脚本 + `diff`。

4 步骤

逐参数比对，标 SAME / DIFFERENT / UNKNOWN_PUBLIC ⟶ 出处 `351_provenance_audit/summary/protocol_comparison.csv`。

5 公式

两份代码的估计量：每个输出维注入 one-hot cotangent 到所有有效目标位置，反传得到源位置梯度，在源位置上取平均，逐提示累加后除以 n_done（文档字符串描述，见 diff 头部）。

6 对照

无。

7 结果

参数	Self-100	Public-557
提示数	100（固定）	557（人工停止）
源层	6 层	L0–L62 共 63 层
dim_batch	8	64
max_seq_len	128	128
停止规则	无	stop_at_delta 0.002，未达到；最后 final_mean_rel_change 0.006215
数据	WikiText-103 本地 1000 条前 100	Salesforce/wikitext, wikitext-103-raw-v1, train, max_chars 2000

⟶ 出处 `protocol_comparison.csv · audit_summary.json → public_config ( "prompts_fitted": 557, "final_mean_rel_change": 0.006215, "detail": "Stopped by the operator at 557 prompts…" )`。Public provenance 还写明：Δmean 从约第 300 条起停在 5e-3 到 8e-3，估计 557 条时平均 Jacobian 的相对标准误约 13%。

注：protocol_comparison.csv 把 Public 的 target_layer、skip_first 标成 UNKNOWN_PUBLIC，但同一 checkpoint 的 provenance 字段其实写了 `target_layer: 63, skip_first: 16, jlens_commit 22f0412f…`，只有 model_revision 确实未知 ⟶ 出处 `audit_summary.json → checkpoint.neuronpedia.provenance`。

8 局限与质疑

公式可比，但数据、层数、停止方式都不同，所以 557 与 100 的差别不能归因于提示数量。

9 状态

已完成。

3.6 Exp 7 · Self-100 历史 split-half（50 + 50）

1 问题与作用

同一配方用不同拟合文本，lens 是否一致。

2 材料

Self-100 语料：A 半 = 偶数索引 0..98，B 半 = 奇数索引 1..99（交错，非随机），各 50 条 ⟶ 出处 `351_self100_split_half/fits/config_A.json · reports/split_half_report.md`。评估用 Exp 2/4 的同 20 + 20 条提示。

3 工具

`02_self_lens_validation/351_self_jlens_split_half.py`（拟合 + 矩阵比较）、`351_split_readout_compare.py`（读出比较）。

4 步骤

两半各自用原 Self-100 设置拟合（6 层，dim_batch 8，max_seq_len 128，skip_first 16）；比较矩阵余弦、相对 Frobenius 差；再在 40 条评估提示上配对比较 KL 和 final-top1 RR。

5 公式

$\cos(A,B) = \langle \mathrm{vec}\, J_A, \mathrm{vec}\, J_B \rangle / (\|J_A\| \, \|J_B\|)$；相对差 $= \|J_A - J_B\| / \|J_B\|$（按 csv 列名推断，脚本中的分母未逐行核对）。

6 对照

无外部对照；只比较 A 与 B。

7 结果

拟合用时 A 1,971.33 s、B 1,503.67 s ⟶ 出处 `fits/complete_A.json, complete_B.json "elapsed_seconds"`。

层	cos(A,B)	相对 Frobenius 差	论文 KL A / B
L24	0.678	0.807	7.238 / 9.509
L32	0.849	0.549	6.180 / 8.176
L40	0.939	0.348	6.666 / 7.697
L48	0.980	0.198	8.383 / 9.041
L56	0.996	0.093	5.086 / 5.165
L62	0.99985	0.018	0.588 / 0.567

⟶ 出处 `351_self100_split_half/summary/matrix_stability.csv · readout_comparison/reports/readout_stability_report.md`。论文 L48 上 B 的 final-top1 RR 更高（0.001354 vs 0.001028）。

8 局限与质疑

后层一致可能只是因为离 L63 近；只有一次拆分，无法估计拆分间变异；评估提示少。这一检验后来由 Stage 9 FullFit 设计取代（但 Stage 9 没做完）。

9 状态

已完成 。

Stage 4 · 作者读出基准（Benchmark20）

4.1 Benchmark20 逐层读出与单篇主导审计

1 问题与作用

正确作者在模型内部有没有可读出的信号；平均值是否被少数论文拉高。

2 材料

20 篇（A8/B8/C3/D1），93 位作者，0 位因切词跳过 ⟶ 出处 `351_all/351_stage4_readout.log "authors_scored 93 authors_skipped 0"`。抽样（**真正的种子随机**，不是哈希排序）：seed 3514，每组每档用 `random.Random(f"3514:{组}:{档}")` 打乱，然后在 13 个引用档间轮流各取一篇直到配额，最后按 A,B,C,D 交错排列 ⟶ 出处 `04_stage4/351_stage4_readout.py:14-40 (select), :383, :397`。

3 工具

`04_stage4/351_stage4_readout.py`（GPU）、`351_stage4_audit.py`（CPU）。Public-557 lens。

4 步骤

1. 每篇对作者去重；每位作者名字切词（上限 16 token，超出则跳过）⟶ 出处 `:386, :57`。
2. 对名字每个前缀做一次前向（同一前缀缓存复用），记录 J-Lens、Vanilla 7 层和 Final 中目标 token 的名次与 log-prob。
3. 每篇写一行 JSONL，可续跑。
4. 审计脚本：按层算均值、中位数、最大单篇占比、去掉最大一篇后的均值；A−B 未配对 bootstrap。

5 公式

主指标：第一作者（作者列表第一位）首 token 的 RR = 1/r，论文等权平均。辅助：所有作者首 token RR；条件全名 token MRR（teacher forcing）。最大单篇占比 = $\max_i RR_i / \Sigma_i RR_i$。

6 对照

Vanilla 与 Final。尚无错误作者或错误标题对照（审计报告明确写"NOT RUN"）⟶ 出处 `351_stage4_artifacts/audit/audit_report.md`。

7 结果

层	首 token RR J-Lens	首 token RR Vanilla	条件全名 J-Lens	条件全名 Vanilla

L16	0.000221	0.000057	0.001819	0.000970
L24	0.000046	0.000025	0.000895	0.000656
L32	0.000219	0.000023	0.002404	0.002487
L40	0.001442	0.000032	0.005481	0.009703
L48	0.001333	0.000023	0.056041	0.012760
L56	0.007040	0.000300	0.047070	0.052046
L62	0.055923	0.196579	0.397721	0.401723

⟶ 出处 `351_all/351_stage4_readout.log "STAGE 4 BENCHMARK" 段`。L56 J-Lens 第一作者 RR 均值 0.00704，中位数 0.00092，最大一篇占总和 71.03%，去掉后均值 0.00215 ⟶ 出处 `351_stage4_artifacts/audit/audit_report.md 表 1 行 L56`。平均 90.68 s/篇（最长 229.69 s），GPU 峰值 29.5 GiB ⟶ 出处 `351_stage4_readout.log`。

8 局限与质疑

20 篇，四组极不平衡；A–B 区间全部跨 0（未配对）；高均值由单篇驱动。这直接促成了 Stage 5 起的对照设计。

9 状态

已完成 2026-10-15（报告生成时间 2026-10-18T08:51 UTC，⟶ 出处 `run_config.json "report_created_utc"`）。Git `e2c7156`。

Stage 5 · 交叉标题–作者对照

5.1 2 标题 × 2 作者交叉对照（Pilot50）

1 问题与作用

作者首 token 得分高，是因为标题对了，还是名字本身好猜。

2 材料

Pilot50（见 1.2）。每篇论文 P（第一作者 G）配一篇来源论文 Q（第一作者 F），F 和替换标题都来自同一篇 Q ⟶ 出处 `05_stage5/351_stage5.py:98-125`。

3 工具

`351_stage5.py run`（GPU，Public-557）、`analyze`（CPU）。

4 步骤

1. Q 的条件：与 P 同 field、同 citation_bin；不是 P 本身、标题不同；Q 的作者团队与 P 的作者按小写全名字符串不相交；F 的第一个名字成分与 G 不同。按 |Δ年份|、|Δ标题长度（字符）|、|Δ名字长度（字符）|、sha256("35150"+Q_id) 依次排序取第一 ⟶ 出处 `:103-117`。
2. 两个上下文：P 的真标题和 Q 的标题，年份都用 P 的年份 ⟶ 出处 `:253-254`。
3. 每个上下文取同一组 logits 中 G 首 token 与 F 首 token 的 logit ⟶ 出处 `:257-261`。
4. 首 token 相同的论文不进入判别分析（实际 0 篇）⟶ 出处 `:308` · `reports/pilot50_audit_manifest.json "n_same_first_token": 0`。

5 公式

M(上下文) = logit(G_1) − logit(F_1)；标题特异优势 T = M(P 的标题) − M(Q 的标题)（单位：logit）⟶ 出处 `:264`。若只是 G 本身更好猜，T 抵消为 0。A–B 在 20 对上逐对相减；积分终点 = 7 层平均（7 层都齐才计入）⟶ 出处 `:319-324`。Bootstrap 2,000 次，用排序后的第 50、第 1950 个值（约 2.45%/97.45% 分位）⟶ 出处 `:277-280`。

6 对照

交叉设计本身就是对照（同一 F 在两个标题下）；Vanilla 与 Final。

7 结果

- 50 篇全部完成，单篇约 11.5 s ⟶ 出处 `stage5_run.log`。

- 全部 50 篇 J-Lens：L56 +1.167 [+0.533, +1.838]，L62 +1.077 [+0.363, +1.816]；L16–L48 均接近 0（L40 +0.178 [+0.041, +0.342]）；Final +0.874 [+0.208, +1.581]；Vanilla L56 +0.759 [−0.006, +1.481] ⟶ 出处 `351_stage5_crossed_v2/summary/layer_summary.csv 行 group=ALL`。
- 匹配 A−B（J-Lens，7 层积分）：+0.081 [−0.426, +0.648]，正向对 9/20；Vanilla 积分 +0.129 [−0.317, +0.603] ⟶ 出处 `summary/matched_ab_summary.csv 行 layer=integrated`。

8 局限与质疑

只有 1 个错误名字和 1 个替换标题，参照太窄（→ 7.2、7.3）；logit 差没有归一化；无多重比较校正；Q 的作者团队相交只按名字字符串判断。

9 状态

已完成 2026-10-15。Git `e2c7156`。

Stage 6 · 完整作者名的条件 JMRR

6.1 全部作者、全部 token 的 teacher-forced JMRR（Pilot50）

1 问题与作用

从第一作者首 token 扩展到全部作者的全部名字 token 后，A/B 是否有差别。

2 材料

Pilot50；227 个"论文 × 作者"实例，972 个 token 实例，最长名字 10 token ⟶ 出处 `351_stage6/raw/token_layer_scores.csv（本手册统计）`。

3 工具

`06_stage6/351_stage6.py`（GPU + CPU），Public-557。

4 步骤

1. 复用 Stage 4 的 20 篇：要求提示词完全相同、组别相同、作者顺序和数量一致、token 名次一致，任一不符即重算 ⟶ 出处 `351_stage6.py:72-114 (stage4_reuse)`；结果 20/20 复用，30 篇新算 ⟶ 出处 `351_stage6/summary/summary.json`。
2. 每位作者单独打分：对名字 $[G_1, G_2, \ldots]$，在提示后给 G_1 打分，在提示 + G_1 后给 G_2 打分，依此类推。提示上限 512 token；名字上限 32 token（超长报错，不截断）⟶ 出处 `:173-181, :382`。
3. 15 种读出（J-Lens 7 层、Vanilla 7 层、Final）× 972 token = 14,580 行 ⟶ 出处 `token_layer_scores.csv（14,580 行）`。

5 公式

作者 JMRR = $\text{mean}_{\text{token}}(1/r)$；论文-层 JMRR = $\text{mean}_{\text{作者}}$；7 层积分 = $\text{mean}_{\text{层}}$。组均值 bootstrap 2,000 次（种子 35161+…），匹配 A–B 每层 bootstrap（种子 35162+层+len(mode)）⟶ 出处 `:47, :275, :302`。

6 对照

Vanilla；与 Stage 5 交叉对照的论文级拼接表（不混合）⟶ 出处 `summary/stage5_stage6_paper_join.csv`。

7 结果

组	n	7 层积分 J-Lens JMRR	95% CI

A	20	0.084931	[0.075009, 0.095752]
B	20	0.068310	[0.056929, 0.079632]
C	7	0.105820	[0.079544, 0.137856]
D	3	0.050166	[0.032803, 0.082428]

⟶ 出处 `351_stage6/reports/experiment_report.md 表 "Actual group estimates"`。匹配 A−B：J-Lens L62 +0.0966 [+0.0282, +0.1630]，Vanilla L62 +0.0907 [+0.0161, +0.1676]；其余各层区间都跨 0 ⟶ 出处 `summary/matched_ab_bootstrap.csv`。

8 局限与质疑

条件全名给了名字前缀，不等于无提示回忆；作者间、token 间不独立；n=20 对。

9 状态

已完成 2026-10-15。只读复核 `351_stage6.py audit-reuse`： Reusable Stage4 20 of 20 复核过 ⟶ 出处 `~/jlens_work/README.md "Read-only re-checks"`。Git `e2c7156`。

Stage 7 · 排除替代解释（7.1–7.4）

7.1 第一个 token vs 后续 token（CPU 重分解）

1 问题与作用

Stage 6 的 A−B 差来自名字开头，还是给出正确前缀之后的后续 token。

2 材料

Stage 6 原始输出 `pilot50_full_author.jsonl`（sha256 `89551cf9…215a`）、Pilot50 配对表（sha256 `9ebfc7e2…1537`）⟶ 出处 `351_stage7_1/reports/experiment_report.md`。不重跑模型。

3 工具

`07_1_stage7_1/351_stage7.py`（CPU）。

4 步骤

1. 每位作者拆成：第一个 token 的 RR、后续 token 的平均 RR、全部 token 的平均 RR ⟶ 出处 `351_stage7.py:65-81`。
2. 后续 token 只对多 token 作者平均，单 token 作者不补 0 ⟶ 出处 `:86-87`。
3. 作者等权 → 论文；配对 A−B；bootstrap 10,000 次，种子 35171，按配对重抽 ⟶ 出处 `:14, :30-38, :181`。
4. 与 Stage 6 历史聚合值逐一核对：750 行，最大差 1.11×10^{-16}。

5 公式

first_jmrr、continuation_jmrr、all_jmrr 同 JMRR 定义，只是 token 集合不同。

6 对照

Vanilla。

7 结果

终点（J-Lens）	7 层积分 A−B [95% CI]	积分正向对	L62 A−B [95% CI]	L62 正向对
第一个 token	+0.001366 [−0.011430, +0.014861]	12/20	+0.014112 [−0.071572, +0.109842]	12/20
后续 token	+0.020644 [+0.002232, +0.038503]	17/20	+0.121454 [+0.043441, +0.197383]	15/20

全部 token	+0.016620 [+0.002640, +0.029887]	15/20	+0.096579 [+0.028582, +0.161619]	17/20

⟶ 出处 351_stage7_1/reports/experiment_report.md 两张表 · summary/layer_and_integrated_effects.csv。Vanilla 积分：首 +0.014693 [−0.002449, +0.033563]，后续 +0.016844 [−0.002067, +0.033707]。

补充：第一作者首 token RR 与全部作者首 token RR 的论文级相关（J-Lens）：L40 r = 0.691、L56 r = 0.723、L62 r = 0.771（本手册从 `351_stage6/raw/token_layer_scores.csv` 重算，与报告一致；报告原始出处文件未找到）。

8 局限与质疑

后续 token 的差异以给出正确前缀为条件，不是自由回忆；排在后面的作者，提示里并不包含前面的作者名（每位作者单独接在 "Authors:" 后），所以"首 token"对每位作者都是无前缀的；原因当时未解释。

9 状态

已完成 2026-10-15。Git `e2c7156`。

7.1B 终端读出（全名前缀 × 21 种标题）

1 问题与作用

计划把 7.1 的后续 token 优势也放到"真标题 vs 20 个错误标题"的框架里检验。

2 材料

Stage 6 的 227 位作者全部名字前缀 × 21 个标题条件（真标题 + 7.3 冻结的 20 个错误标题），共 15,519 个上下文 ⟶ 出处 07_1_stage7_1/351_stage7_1B_terminal.py:92-…（freeze，要求 len(tasks)==15519 且 authors==227）。

3 工具

`351_stage7_1B_terminal.py freeze → run → check`；读出复用 Stage 5 helper（sha 校验）。

4 步骤

冻结 plan/authors/tasks 并写 manifest；逐上下文 GPU 读出，每个上下文一个 JSON；`check` 做结构检查和真标题 parity（与 Stage 6 的名次/log-prob 比较）。

5 公式

计划与 7.3 相同（真 − 20 个错误标题平均），未实现到分析阶段。

6 对照

真标题 parity：1,380 行，log-prob 最大差 0.0，名次最大差 0 ⟶ 出处 `351_stage7_1B_terminal/smoke_check.json`。

7 结果

只完成 1,166 / 15,519 个上下文（约 7.5%），中位 6.0 s/上下文；最后写入 2026-10-22 08:31；没有估计任何标题效应（check 状态 "Structural check complete … No full-experiment title effect estimated."）⟶ 出处 `smoke_check.json` · `raw/`（1,166 个文件）· `run.log`。

8 局限与质疑

没有结果可用；不得与 7.1 的完成结果合并（仓库 ARCHIVE_NOTES 明确要求）⟶ 出处 `jlens-authorship-recall_update/ARCHIVE_NOTES.md:138`。停止方式已查实：run.log 共 5 次启动，最后一次从 443 续跑到 "SAVED 723" 后中断，日志里没有报错或 Traceback；最后一个作业 11296093@d1029（06:54 开始，时限 120 min）在 2026-10-22 08:31:23 由用户本人账号（uid 103706）结束（"CANCELLED by 103706"），离时限还有约 23 分钟，不是超时；之后没有续跑，10-23 只做了结构检查。⟶ 出处 `351_stage7_1B_terminal/run.log` · `sacct -j 11296093` · `smoke_check.json` · 附录 I U23 **未知/需确认：向用户确认（是否有意放弃 7.1B，U23；该问题后来由 Stage 10 的全量 token 分析覆盖）**

9 状态

计划未做（运行到 7.5% 后停止，未分析）。不在 git 里。

7.2 金标准 vs 100 个错误作者

1 问题与作用

真标题下，金标准首 token 能否赢过很多真实存在的错误名字（而不只是 Stage 5 的一个）。

2 材料

Pilot50 每篇：第一作者金标准 + 50 个匹配错误作者 + 50 个随机错误作者，共 5,050 条候选；名字来自 6,841 篇的作者全集，排除目标论文的全部作者 ⟶ 出处 `351_stage7_2/summary/quality.json` · `07_2_stage7_2/351_stage7_2.py:62-83`。

3 工具

`351_stage7_2.py`（CPU，只用 tokenizer，种子 35172）、`351_stage7_2_score.py audit → run → analyze`（GPU）。

4 步骤

1. 匹配分级（名字 token 数相同为 tier 0–3 的必要条件）：tier 0 同领域、同 token 数、年份差 ⩽ 2；tier 1 年份差 ⩽ 5；tier 2 同领域同 token 数任意年份；tier 3 只同 token 数；tier 4–6 为更宽松的后备 ⟶ 出处 `351_stage7_2.py:95-104`。按 (tier, sha 种子哈希) 排序取前 50 ⟶ 出处 `:106-107`。
2. 随机池：去掉匹配池后，`random.Random(stable_int(35172, pid, "random"))` 无放回抽 50 ⟶ 出处 `:108-111`。
3. 每篇 1 个真标题上下文，15 种读出，同一组 logits 上比较金标准首 token 与 100 个错误首 token。

5 公式

金标准胜率 $W = (1/100)\,\Sigma_k\,[1(g > f_k) + 0.5 \cdot 1(g = f_k)]$，g、$f_k$ 为首 token 的 logit；首 token 相同记平局 0.5 ⟶ 出处 `351_stage7_2_score.py:129-130`。A–B 配对 bootstrap 10,000 次，种子 351720 + len(subset) + len(mode) + len(pool) ⟶ 出处 `:131-135, :171`。

6 对照

匹配池 vs 随机池；Vanilla、Final；敏感性：唯一 token 胜率、去平局胜率（`paper_layer_pool.csv` 列 unique_token_win_rate、non_collision_win_rate）。

7 结果

- 实际匹配分级 2,159 / 182 / 135 / 24（tier 0/1/2/3）⟶ 出处 `summary/quality.json "matching_tiers"`。24 个 tier-3 全部在同一篇 A 论文（1e987cda…，配对 P004）⟶ 出处 `candidates/pilot50_author_candidates.csv`（本手册统计）。
- 5,000 个错误候选中 156 个与金标准首 token 相同（A 79、B 62、C 9、D 6）；每篇 100 个错误名字平均只有 45.76 个不同首 token ⟶ 出处 `quality.json "first_token_collision_count_by_group" · 45.76 为本手册从 candidates csv 统计`。
- 75,000 行配对记录 = 50 × 100 × 15 ⟶ 出处 `summary/candidate_pairwise_scores.csv`。
- J-Lens 合并池 A–B：L40 +0.150 [+0.013, +0.289]，L48 +0.193 [+0.071, +0.318]，L56 +0.187 [+0.015, +0.345]，L62 +0.038 [−0.144, +0.215] ⟶ 出处 `reports/stage7_2_gpu_report.md · summary/matched_ab_bootstrap.csv`。
- 7 层积分 A–B：J-Lens +0.105、Vanilla +0.106（点估计本手册从 `matched_pair_layer.csv` 重算一致）。报告给的区间 [−0.012, +0.228] / [−0.005, +0.217] 在 7.2 输出里没有，2026-10-30 改用重算值：pool = combined，20 对配对 bootstrap，B = 20,000，百分位 95%，J-Lens +0.105 [−0.012, +0.229]（种子 20261031）、Vanilla +0.106 [−0.006, +0.216]（种子 20261032）；另用种子 1–200 各算一次，区间端点的 Monte-Carlo 范围下端 [−0.015, −0.010] / [−0.011, −0.002]，原报告的数落在范围内。⟶ 出处 `351_stage7_3/recompute_2026-10-30/stage7_interval_recompute.csv 行 "7.2 integrated7 A-B jlens/vanilla pool=combined" ·`

`~/jlens_work/07_3_stage7_3/recompute_2026-10-30/stage7_recompute_intervals.py` 复核过（U24 已解决）

- 去掉 P004 后，7.3 匹配池 7 层积分 A−B = +0.0226（本手册从 `351_stage7_3/summary/integrated_pair_differences.csv` 重算，19 对）；报告区间 [−0.018, +0.069] 改用重算值：J-Lens、pool = combined，19 对配对 bootstrap，B = 10,000，种子 20261039：+0.022 [−0.018, +0.068]（pool = matched 为 +0.023 [−0.017, +0.068]，种子 20261037）。⟶ 出处 `351_stage7_3/recompute_2026-10-30/stage7_interval_recompute.csv 行 "7.3 integrated7 A-B drop P004"` · `~/jlens_work/07_3_stage7_3/recompute_2026-10-30/stage7_recompute_intervals.py` 复核过（U25 已解决）

8 局限与质疑

只看首 token，100 个名字只有约 46 种首 token，"赢 100 个名字"其实是赢约 46 个 token；匹配未控制首 token 的基础频率；A−B 积分区间跨 0；无多重校正。

9 状态

已完成。只读复核 `351_stage7_2_score.py audit`：CANDIDATE HASH/COHORT VERIFIED 复核过 ⟶ 出处 `~/jlens_work/README.md`。Git `e2c7156`。

7.3 打乱标题参照（20 个错误标题）

1 问题与作用

减去"错误标题下也能赢"的背景后，真标题还有多少优势；A 是否比 B 受益更多。

2 材料

每篇 20 个仅按元数据选的错误标题，共 1,000 个上下文；使用 7.2 完全相同的候选、提示格式和 15 种读出，年份保持目标论文年份 ⟶ 出处 `351_stage7_3/reports/freeze_audit.md` · `stage7_3_null_report.md`。

3 工具

`07_3_stage7_3/351_stage7_3.py freeze/audit`（CPU，种子 35173）、`351_stage7_3_score.py audit/run/analyze`（GPU）。

4 步骤

1. 候选来源论文：不是目标本身、标题不同、作者团队与目标不相交；按 (tier, |Δ年份|, |Δ标题 token 数|, sha(35173, 目标, 来源)) 排序取前 20，同一目标内标题去重 ⟶ 出处 `351_stage7_3.py:83-91`。tier 0 同领域同档、年份差 ⩽ 2、标题 token 差 ⩽ 5；tier 1 放宽到 ⩽ 5/⩽ 10；tier 2 放宽引用档（已发布报告描述；tier 函数本身未逐行核对）。
2. 每个错误标题上下文用 7.2 的同一 101 个候选计算 W。

3. 论文内：标题优势 = W(真) − 20 个 W(错) 的平均；配对 A–B bootstrap 10,000 次，种子 35173 ⟶ 出处 351_stage7_3_score.py:156-158。

5 公式

$TA_{p,\ell} = W_{p,\ell}$(真标题) − (1/20) $\Sigma_{k=1..20} W_{p,\ell}$(错误标题 k)；经验尾部 = (1 + #{k : $W_k \geqslant W_{真}$}) / 21（描述性，不是校准 p 值）。

6 对照

20 个错误标题本身就是对照；Vanilla、Final；敏感性：去掉来源作者出现在错误作者池中的标题。

7 结果

选题质量：tier 0/1/2 = 334/487/179；平均 |Δ年份| 1.852，平均 |Δ标题 token| 4.358，同领域 100%，同档 82.1%；199/1,000 个来源作者出现在错误作者池中 ⟶ 出处 summary/quality.json。

层（J-Lens，合并池）	真标题 W	20 错误标题均值	真 − 参照	正向篇数
L16	0.469	0.471	−0.002	26/50
L24	0.475	0.471	+0.005	26/50
L32	0.502	0.499	+0.003	23/50
L40	0.529	0.526	+0.004	29/50
L48	0.541	0.526	+0.015	27/50
L56	0.616	0.523	+0.093	35/50
L62	0.623	0.556	+0.067	32/50

⟶ 出处 summary/paper_null_distribution.csv（pool=combined；本手册按 mode 求均值）。Vanilla L56 +0.106（33/50），Final +0.072（30/50）。

- L56 标题优势 +9.34 pp。报告区间 [+4.75, +13.95] 在 7.3 输出中没有，改用重算值：50 篇论文 bootstrap，百分位 95%，B = 20,000，种子 20261046：**+9.34 pp [+4.77, +14.00]**（B = 10,000、种子 20261045 为 [+4.91, +14.01]；Monte-Carlo 种子 1–200 的下端范围 +4.62 到 +4.88）；留一法均值 +8.64 到 +9.99（与报告"8.6 到 10.0"一致）。⟶ 出处 351_stage7_3/recompute_2026-10-30/stage7_interval_recompute.csv 行 "7.3 L56 J-Lens title advantage" · 351_stage7_3/recompute_2026-10-30/stage7_l56_loo.csv · ~/jlens_work/07_3_stage7_3/recompute_2026-10-30/stage7_recompute_intervals.py 复核过（U26 已解决）
- 匹配 A–B 7 层积分：J-Lens +0.0177 [−0.0211, +0.0626]，Vanilla +0.0315 [−0.0090, +0.0757]，均跨 0 ⟶ 出处 summary/integrated_ab_bootstrap.csv 行 pool=combined。报告的"L56 +0.113（跨 0）、L32 −0.032 [−0.058, −0.006]"在 `matched_ab_bootstrap.csv` 中，未逐行核对。
- 去掉来源作者重叠：J-Lens 7 层积分标题优势 +0.0262 → +0.0283（本手册重算，列 advantage_no_source_false_overlap）。

- 0-2 引用档 6 篇（A2/B2/C1/D1）L56 平均 +0.224，5/6 为正，0/6 的经验尾部 ⩽ 0.05（K=20 时最小可能值 1/21 ≈ 0.048）⟶ 出处 `351_stage7_4/summary/floor_L56_low_six.csv` · `overview.json "floor_low_six_L56"`。
- 报告"三种候选加权下合并标题优势 +0.026 到 +0.032、6 个 A–B 区间全部跨 0"改用重算值（真标题胜率取 7.2、20 个错误标题胜率取 7.3，同一加权，7 层平均）：J-Lens 标题优势 作者加权 +0.026 [+0.011, +0.043]、唯一 token +0.032 [+0.018, +0.046]、去平局 +0.027 [+0.011, +0.043]（50 篇 bootstrap，B = 10,000，种子 20261047/49/51）；对应 A–B（20 对）+0.018 [−0.021, +0.063]、+0.015 [−0.022, +0.057]、+0.018 [−0.021, +0.064]，Vanilla 三个 A–B 为 +0.032 [−0.010, +0.076]、+0.029 [−0.011, +0.071]、+0.033 [−0.009, +0.076]，6 个区间全部跨 0，与报告一致。⟶ 出处 `351_stage7_3/recompute_2026-10-30/stage7_interval_recompute.csv 行 16–27（weight=author/unique/noncollision）` · `351_stage7_3/recompute_2026-10-30/stage7_weighting_title_advantage.csv` 复核过 （U27 已解决）

8 局限与质疑

20 个错误标题是按元数据挑的，不可交换，尾部比例不是 p 值；同一来源标题可被不同目标复用，参照上下文之间不独立；L56 在此前已被看过（赢家诅咒）。

9 状态

已完成 （按 mtime：7.3A 冻结 2026-10-18 11:49，打分输出 `raw/shuffled_firsttoken_scores.jsonl` 2026-10-18 13:56；STAGES.md 写"7.3 的打分在 2026-10-21 完成"，与 mtime 不一致，可能指后来的复核）。只读复核 `351_stage7_3_score.py audit`：STAGE 7.3A + STAGE 7.2 VERIFIED 复核过 。Git `e2c7156`。

7.4 跨阶段审计与 J-Lens 增量

1 问题与作用

不同 Stage 的测量是否一致；同一篇论文上 J-Lens 是否比 Vanilla 多读出标题信息。

2 材料

Stage 5–7.3 的冻结输出（只读），50 篇、20 对、1,000 个参照上下文 ⟶ 出处 `351_stage7_4/summary/overview.json`。

3 工具

`07_4_stage7_4/351_stage7_4.py`（CPU）；bootstrap 20,000 次，种子 741 ⟶ 出处 `351_stage7_4.py:10-13`。

4 步骤

1. Stage 5 的 L56 logit 交叉对比 vs 7.3 的 L56 标题优势，论文级 Pearson/Spearman。

2. 每篇每层：J-Lens 标题优势 − Vanilla 标题优势；积分后论文 bootstrap；留一法。

3. 该增量的匹配 A−B。

4. 引用档分段描述。

5 公式

增量 $\Delta_p = \text{mean}_\ell[TA^J{}_{p,\ell} - TA^V{}_{p,\ell}]$。

6 对照

Vanilla 就是对照。

7 结果

诊断	估计	不确定性 / 样本
Stage 5 vs 7.3（L56 J-Lens）	Pearson 0.445，Spearman 0.471	n = 50，符号一致 64%（32/50）
J-Lens − Vanilla 积分标题优势	+0.00221	[−0.00753, +0.01241]，中位数 −0.0019，24/50 为正，留一法 −0.000046 到 +0.003494
增量的匹配 A−B	−0.01387	[−0.03320, +0.00358]，20 对

⟶ 出处 `351_stage7_4/summary/overview.json` · `jlens_increment_bootstrap.csv` 行 `integrated` · `cross_stage_correlations.csv` 行 `jlens,56`

8 局限与质疑

Stage 5 与 7.3 对比单位、参照数都不同，中等相关不说明谁错；所有结论探索性；论文间依赖（共享作者、候选、来源标题）未处理。

9 状态

已完成 2026-10-18。Git `e2c7156`。

Stage 8 · 200 篇新论文的独立验证

8.0 冻结的分析计划与 PROTOCOL（主终点问题）

1 问题与作用

在看到任何新论文的读出之前，冻结样本、对照和分析方案，使 Stage 8 成为唯一的非探索性检验。

2 材料

文件	时间（mtime）	sha256	主终点写法
`351_stage8/config/analysis_plan.json`（8A 冻结，"status": FROZEN_BEFORE_STAGE8_SCORING）	2026-10-18 14:29:47	`7ff662e07798de70c09e30c387f675db0c47c6e6723270f590809e4874942b9c`，记录在 8A manifest 中	"primary_endpoint": 7 层平均的 [真标题胜率 − 20 个错误标题胜率均值]；"secondary_l56": L56 为 pilot 选出的次要终点；"multiplicity": 主终点 1 次比较，7 个逐层比较探索性并做 Holm
`351_stage8/config/manifest.json`（8A manifest）	2026-10-18 14:29:47	`7e4b878e9ee037fe822cc42821218459ff48b203cd026de6995a79ca13486f55`	—（含 analysis_plan、papers、pairs、matching_audit 的 sha）
`08_stage8/351_stage8_README.md`	2026-10-18 14:29	`ec1de9a1…f28396`	第 5 条：主检验 = 7 层积分 A−B；"L56 is a pilot-selected secondary endpoint"；逐层 Holm
`08_stage8/351_stage8_PROTOCOL.md`	2026-10-18 21:14	`3a3a3d65dc500b2ff7255701f55cae8c74ff4aa7402c119619dda259721fac6f`（文件首行原为 `cat > ~/351_stage8_PROTOCOL.md <<'MD'`，即 heredoc 原样存入，说明是从聊天里粘到终端写出的）	"L56 is the predeclared replication layer"；"Integrated seven-layer statistics are secondary"
`351_stage8/config/stage8B_manifest.json`	2026-10-18 21:25	`b3c8c5a7…5309`	—
`stage8C_runtime.json`（打分开始）	2026-10-18 21:47	—	—

⟶ 出处 `ls --time-style=full-iso /scratch/zhang.zhiqi2/jlens/351_stage8/config · sha256sum ~/jlens_work/08_stage8/* · analysis_plan.json 全文`

3 工具

`351_stage8.py freeze/audit`（sha256 `5dce8e48…7ba6`）写 analysis_plan 和 manifest；重复 freeze/audit 会核对所有哈希，拒绝静默修改 ⟶ 出处 `351_stage8_README.md "Files and commands"`。

4 步骤

PROTOCOL.md 冻结的内容（逐条）：RQ1 L56 真 − 20 个错误标题平均，论文为单位，论文 bootstrap；RQ2 100 对 L56 优势(A) − 优势(B)，按配对重抽，不得独立重抽 A、B；RQ3 低（0-2）− 高（129+），A、B 分开再组平衡；固定 7 层，L56 为预设复现层，不得按 Stage 8 结果换层；J-Lens vs Vanilla 用相同上下文和候选；8B 每篇 1 金标准 + 50 匹配 + 50 随机错误作者 + 20 个错误标题（共 20,200 / 4,000 条）；8C 4,200 个上下文，Public lens sha `0af5ccf2…13d8`；8D 10 项必做分析（含首 token 碰撞敏感性、名字 token 数、错误标题匹配质量、离群、与 pilot 比较）；完成标准：4,200 全部打分或记录失败、三个 RQ 全部检验、不做事后换层 ⟶ 出处 `08_stage8/351_stage8_PROTOCOL.md:53-687`。

5 公式

见 8D。

6 对照

—

7 结果

两份文件对主终点的规定相反：哈希锁定的 analysis_plan.json（14:29）把 7 层积分定为主终点、L56 为次要；同日 21:14 写的 PROTOCOL.md（不在任何 manifest 里）改为 L56 为主。两者都早于 8B 冻结（21:25）和 8C 打分（21:47 起），所以改动发生在看到 Stage 8 结果之前，但改动在 pilot 已看过 L56 之后。8D 按 PROTOCOL 报告 L56 为主（`351_stage8D.py:242-270`，报告写"L56 is the predeclared main layer"）；冻结计划的积分终点后来由 8D supplement 补算（见 8.5）。

git 记录：git 里没有任何提交改动过主终点。analysis_plan.json 与 PROTOCOL.md 都是在 `e2c7156`（2026-10-22 批量上传）首次入库；`3a4f605`（2026-10-21 PDT，分支 review/stage8-fixes）只删掉 PROTOCOL.md 第一行 heredoc、在 README 里注明冻结计划中 L56 是次要终点、加入 supplement；该分支已于 2026-10-30 合入 main（`a4c1f0a`，2026-10-30 11:52 EDT）。集群上没有 10-23 之前的 Claude 记录，~/.bash_history 也没有线索。冻结主终点（7 层积分）、Holm 与逐对剔除的结果见 8D、8D supplement。⟶ 出处 `git -C /scratch/zhang.zhiqi2/jlens-authorship-recall_update show --stat 3a4f605 · git log --oneline main (a4c1f0a "Merge review/stage8-fixes") · PROGRESS.md 2026-10-30 12:30 条` 复核过

8 局限与质疑

导师可能问"主终点是不是事后换的"：时间戳表明换在打分之前，但换的依据（pilot 上 L56 最强）本身就是 pilot 结果。**未知/需确认：向用户确认（10-18 14:29 → 21:14 之间把主终点从 7 层积分改为 L56 的决定由谁、依据什么做出，是否有书面记录；U28，用户正在笔记本上查 ~/.claude/projects/*/*.jsonl 与网页聊天记录）**

9 状态

已完成（冻结）。 复核过（本手册核对时间戳与哈希）。Git `e2c7156`。

8A 队列冻结（200 篇 / 100 对）

1 问题与作用

选出与 Pilot50 不重叠的新论文，做独立复现。

2 材料

Stage 5 的 `full_6841_manifest.csv`（sha256 `a63316ef…bcb`）和 Pilot50 ID（排除）⟶ 出处 `351_stage8/config/manifest.json "inputs"`。只用 A 与 B（R0），不用 C/D。

3 工具

`08_stage8/351_stage8.py freeze → audit`，SEED = 3518001 ⟶ 出处 `351_stage8.py:8`。

4 步骤

1. 排除 50 篇 Pilot 目标；按 (field, citation_bin) 分格，格内 A、B 各按 (年份, sha 种子哈希) 排序 ⟶ 出处 `:34-43`。**这是确定性的哈希排序，不是随机抽样：** `rank_seed` = sha256("3518001|…") 的前 16 位十六进制 ⟶ 出处 `351_stage8.py:30 rank_seed` 复核过 。
2. 格内贪心最近邻、不放回：从较少的一侧逐篇，在另一侧作者名单不相交的论文中选代价最小者；代价 = 10×|Δ年份| + 3×|Δ作者数|，再按 sha(3518001, a, b) ⟶ 出处 `:31-33, :44-55`。贪心，不是最大基数匹配。
3. 引用段：0-2 → low，起点 ≥ 129 → high，其余 mid ⟶ 出处 `:26-29`。所有候选对生成后再按配额取：每段内按 (代价, sha 哈希) 排序取前 25/50/25 ⟶ 出处 `:56-61`。
4. 写 `cohort/stage8_papers.csv`、`matching/stage8_ab_pairs.csv`、analysis_plan、manifest。

5 公式

$c(a,b) = 10|\Delta \text{year}| + 3|\Delta \text{n_authors}|$。

6 对照

—

7 结果

100 对 = 200 篇：low 25、mid 50、high 25；可用候选对 low 194、mid 1,154、high 314；入选对平均 |Δ年份| 0.0、平均 |Δ作者数| 0.0（即全部年份、作者数完全相同）；无拒绝 ⟶ 出处 `351_stage8/summary/matching_audit.json`。

8 局限与质疑

目标论文独立，但公共 lens 和 6,841 篇来源宇宙与 pilot 共享（README 明确"not full data-source or model independence"）；配额是在候选对中按代价最小取，偏向"容易精确匹配"的格（如作者数少

的论文)。

9 状态

已完成 2026-10-18。只读复核 `351_stage8.py audit`：STAGE 8 VERIFIED 复核过。Git `e2c7156`。

8B 错误作者与错误标题冻结

1 问题与作用

在打分前为新论文冻结 7.2/7.3 式的对照。

2 材料

200 篇目标；6,841 篇作者与标题全集。

3 工具

`351_stage8B.py freeze/audit`，SEED = 3518201 ⟶ 出处 `351_stage8B.py:16` · `stage8B_manifest.json "seed"`。

4 步骤

1. 错误作者排除目标论文全部真作者。匹配预分级：0 同领域且同 token 数，1 只同领域，2 只同 token 数，3 其余；按 (预分级, 种子哈希) 取前 600 入短名单，再按 (预分级, |Δ年份|, |Δtoken 数|, 种子哈希) 取 50 个匹配 ⟶ 出处 `:298-341`；其余按种子哈希取 50 个"随机"错误作者 ⟶ 出处 `:343-345`——这里的"随机"也是按 sha256(3518201, 论文, "random", 候选) 排序取前 50 的**确定性哈希顺序**，不是伪随机数抽样 ⟶ 出处 `351_stage8B.py:80, 366–370` 复核过 （与 7.2 的随机池不同：7.2 用 `random.Random(stable_int(35172, pid, "random"))` 真正抽样)。（与 7.2 的分级规则也不同：8B 不把年份放进 tier。）
2. 错误标题：不同来源论文、不用目标标题、排除与目标共享作者的论文；tier 0 同领域同档、1 同领域、2 其他；按 (tier, |Δ年份|, |Δ标题 token|, 种子哈希) 取 20；提示里保留目标年份 ⟶ 出处 `:505-540`。

5 公式

—

6 对照

—

7 结果

评分前检查	结果

候选记录	20,200（200 金 + 10,000 匹配 + 10,000 随机）；错误标题 4,000
匹配错误作者 tier 0	9,905 / 10,000（99.05%）
错误标题 tier 0（同领域同档）	4,000 / 4,000
金标准–错误作者首 token 相同	673
错误标题来源作者在错误池中	760 / 4,000（19.0%）
标题 token 长度差	均值 7.37，中位数 5，最大 57
年份差	均值 2.33，中位数 2，最大 20

⟶ 出处 351_stage8/config/stage8B_manifest.json · controls/stage8_control_quality.csv（列 matched_tier0, candidate_first_token_collisions 求和）· controls/stage8_shuffled_titles.csv（列 match_tier, source_author_in_false_pool, title_token_gap, year_gap；本手册统计）

8 局限与质疑

错误标题的 token 长度差比 7.3 大（均值 7.37 vs 4.36）；19% 来源作者重叠需要敏感性分析（8D 做了）。

9 状态

已完成 2026-10-18 21:25。Git `e2c7156`。

8C GPU 打分

1 问题与作用

在 4,200 个冻结上下文上读出金标准与 100 个错误作者首 token 的分数。

2 材料

每篇 21 个上下文（真标题 + 20 个错误标题），每篇固定一组首 token 候选 ID（101 个候选首 token 去重排序）⟶ 出处 351_stage8C.py:126。

3 工具

`351_stage8C.py audit → run`，复用 Stage 5 的 `readout`（helper sha `4fd4c9fb…c339`），29 层 GPU，Public-557 ⟶ 出处 config/stage8C_runtime.json。

4 步骤

每个上下文：提示切词（< 512），一次读出得到 15 种模式（J-Lens 7 层 + Vanilla 7 层 + Final L63）下所有候选 ID 的 logit，写 JSONL；不打后续 token ⟶ 出处 351_stage8C.py:360-400。runtime 有一个 `stage8C_runtime.json.before_mode_fix`（21:40），说明打分前修过一次模式列表。

5 公式

—（分数在 8D 中转成胜率）

6 对照

Vanilla、Final 同上下文同候选。

7 结果

4,200 / 4,200 上下文全部完成，单上下文约 6.1 s ⟶ 出处 `351_stage8C_full.log 末行 · raw/stage8C_context_scores.jsonl（4,200 行，sha256 ace40bc2…7b23，见 stage8D/output_manifest.json）`。

mode_fix 的内容（U29 已解决）：两份 runtime 逐字段只差 3 处——modes 第一项 "final" → "final_L63"，以及随之变化的 protocol_hash（b00dfa98… → 272715c9…）和 runtime_hash（62bc49cb… → 6dcf3961…）。时间顺序：before_mode_fix 10-18 21:40:20 → 正式版 21:47:31 → 8C 第一次写入打分在其后。用旧版跑第 1 条时，在保存前就报 "Unexpected readout modes"（351_stage8C.py:378）并拒绝；换成正式版后才保存第 1 条（"existing=0"）。打分文件 4,200 行的 runtime_hash 全部是 6dcf3961…。即**只改了模式名，发生在任何打分保存之前**。⟶ 出处 `diff 351_stage8/config/stage8C_runtime.json{.before_mode_fix,}（第 15、31、34 行）· 351_stage8C_smoke.log · raw/stage8C_context_scores.jsonl runtime_hash 列 · 附录 I U29` 复核过

8 局限与质疑

只有首 token。

9 状态

已完成（runtime 写于 2026-10-18 21:47，`raw/stage8C_context_scores.jsonl` 最后写入 2026-10-20 09:05，按 mtime）。Git `e2c7156`。

8D 冻结统计分析（RQ1–RQ3、J–V）

1 问题与作用

按 PROTOCOL 检验三个预设问题。

2 材料

8C 原始分数；8A 配对；8B 候选与错误标题。

3 工具

`351_stage8D.py audit → analyze`（CPU），bootstrap 10,000 次，SEED = 3518401（RQ1 用 +1、RQ2 +2、RQ3 +3）⟶ 出处 `351_stage8D.py:9, :34-49, :242-270`。

4 步骤

1. 每篇每模式：W(真)、20 个 W(错)，TA = W(真) − 均值；同时算去碰撞、去来源重叠、短标题（标题 token 差 ⩽ 10）三种敏感性 ⟶ 出处 :186-236。
2. RQ1：200 篇 L56 TA 的论文 bootstrap。
3. RQ2：100 对 L56 TA(A) − TA(B) 的配对 bootstrap。
4. RQ3：低、高各 25 对，每对取 (A+B)/2，低段均值 − 高段均值，两段分别重抽 ⟶ 出处 :253-270。
5. J − V：200 篇 L56 $TA^{J} - TA^{V}$。

5 公式

$W = (1/100)\,\Sigma_k[1(g > f_k) + 0.5 \cdot 1(g = f_k)]$ ⟶ 出处 :51-54；TA 同 7.3；描述性尾部 $(1 + \#\{W_k \geqslant W_{真}\})/21$。

6 对照

20 个错误标题；Vanilla；敏感性见上。

7 结果

问题	估计	95% CI	N	正 / 负
RQ1 L56 真 − 错误标题	+5.18 pp	[+2.93, +7.48]	200 篇	127 / 73
RQ2 匹配 A − B	+4.98 pp	[+0.32, +9.91]	100 对	55 / 45
RQ3 低 − 高	+2.45 pp	[−3.69, +8.60]	25 + 25 对	—
L56 J-Lens − Vanilla	−1.38 pp	[−3.48, +0.68]	200 篇	96 / 104

⟶ 出处 351_stage8/stage8D/summary/primary_questions.csv · jlens_minus_vanilla.csv · reports/stage8_results.md

- 引用段（L56 TA）：低 50 篇 +6.83 pp [+2.55, +11.59]；中 100 篇 +4.75 pp [+1.94, +7.66]；高 50 篇 +4.38 pp [−1.07, +9.99] ⟶ 出处 summary/layer_and_band_summary.csv 行 jlens_L56。
- RQ2 分布：中位数 +4.41 pp；最大正向一对 S8P094（高引用段）+84.4 pp；去掉最大的 k 对后剩余均值：k=0 +4.98、1 +4.18、2 +3.53、3 +2.94、5 +1.90、10 −0.39 pp（本手册从 `summary/matched_pair_differences.csv` mode=jlens_L56 重算，与报告一致；这是事后诊断，不在冻结计划里）。
- 敏感性（200 篇合并）：去平局 +5.32 pp，去来源重叠 +5.15 pp，短标题 +4.82 pp（199 篇）⟶ 出处 summary/sensitivity.csv。配对重算的点估计：去平局 +5.03、去来源重叠 +4.53、标题差 ⩽ 10 +4.79 pp（本手册重算一致）；报告的对应区间 [+0.15, +9.87]、[−0.34, +9.41]、[−0.39, +9.80] 在 8D 输出中没有，改用重算值（配对 bootstrap，B = 10,000，百分位 95%，种子 20261034/35/36）：去平局 +5.03 [+0.23, +9.95]（100 对）、去来源重叠 +4.53 [−0.15, +9.53]（100 对）、标题差 ⩽ 10 +4.79 [−0.24, +9.83]（99 对）；Monte-Carlo 种子 1–100 的下端范围分别为 +0.08…+0.41、−0.39…−0.08、−0.41…−0.09，原报告的数落在范围内。⟶ 出处

351_stage8/recompute_2026-10-30/s8_unsourced_intervals.csv 行 "8D paired sensitivity L56 A-B" · ~/jlens_work/08_stage8/recompute_2026-10-30/stage8_frozen_plan_recompute.py（作业 11500770） 复核过 （U30 已解决）

8 局限与质疑

- RQ2 对少数正向大对极敏感：去掉 10 对即变号。
- 冻结计划要求的 Holm 校正和 leave-one-pair-out 在 8D 和 supplement 里都没跑（analysis_plan.json "multiplicity"、"sensitivity"），**2026-10-30 已事后补算**（见下）。
- 首 token 判别不是完整作者身份；错误标题不可交换。

冻结主终点、Holm、逐对剔除（2026-10-30 事后补算，作业 11500770，CPU，不改冻结文件） → 出处 ~/jlens_work/08_stage8/recompute_2026-10-30/stage8_frozen_plan_recompute.py · 351_stage8/recompute_2026-10-30/s8_endpoints.csv、s8_holm.csv、s8_lopo_summary.csv、manifest.json 复核过 （U17 已解决）

终点（J-Lens）	计划中的地位	N	估计 [95% CI]（pp）	与原文件
RQ1 7 层积分标题优势	主终点（analysis_plan primary_endpoint）	200 篇	+2.33 [+1.50, +3.19]	原种子 3518501 精确复现 supplement
RQ2 7 层积分匹配 A–B	主组检验（primary_group_test）	100 对	+2.41 [+0.91, +4.00]	原种子 3518502 精确复现
RQ1 L56	计划次要；PROTOCOL.md 为主	200 篇	+5.18 [+2.93, +7.48]	原种子 3518402 精确复现 8D
RQ2 L56 A–B	计划次要；PROTOCOL.md 为主	100 对	+4.98 [+0.32, +9.91]	原种子 3518403 精确复现 8D
RQ3 7 层积分 低 − 高（新增，同 8D RQ3 方法）	analysis_plan floor_primary	25 + 25 对	+0.78 [−1.53, +3.18]	以前没算过（种子 3518504）

Holm（族内 step-down，α = 0.05）：原始 p = 双侧 bootstrap p，$p = \min(1, 2 \cdot \min(1 + \#\{b \leqslant 0\}, 1 + \#\{b \geqslant 0\}) / (B + 1))$，b 为 B = 100,000 次 bootstrap 均值（与区间同一重抽单位），种子 20261031 + 检验序号；另给符号翻转随机化检验的 p 作为第二种原始 p。**冻结计划只规定了 bootstrap 区间，没有规定 p 值公式和检验族的划分，二者都是我们的选择**。结果：

- F0 两个主终点：RQ1 积分 $p = 2 \times 10^{-5}$，RQ2 积分 A–B p = 0.0019（Holm 两项后仍 < 0.05）。
- F1 计划写明的"7 个逐层 A–B（J-Lens）"：只有 L62 通过（+8.83 pp，p 0.0041 → p_{Holm} 0.029）；L48 +2.64 pp（p 0.028 → 0.17）、L56 +4.98 pp（p 0.038 → 0.19）都不通过；其余层原始 p ⩾ 0.37。符号翻转 p 结论相同（L62 p_{Holm} 0.041）。
- F2（另一种读法：7 个逐层标题优势 RQ1）：L56（+5.18）与 L62（+8.80）通过（p_{Holm} 0.00014），L48 +1.37 不通过（0.22）。F3（PROTOCOL.md 的 L56 RQ1 + RQ2 两项）：两项都通过（RQ2 p_{Holm} 0.037）。

逐对剔除（leave-one-pair-out）：计划只写了 "leave-one-pair-out"，未进一步定义；实现为每次去掉 100 对中的 1 对，RQ2 = 剩下 99 对的 A−B 均值，RQ1 = 剩下 198 篇的均值，每次再做 B = 10,000 的 bootstrap（种子 20261032 + k）。积分 A−B：+2.14 到 +2.61，100/100 次区间不含 0；积分 RQ1：+2.17 到 +2.43，100/100；L56 A−B：+4.18 到 +5.50，无变号，但 14/100 次区间跨 0（去掉 S8P018 时下端 −0.29）；L56 RQ1：+4.87 到 +5.41，100/100。⟶ 出处 `s8_lopo_summary.csv`（`n_ci_excluding_0` 列）· `s8_lopo_by_pair.csv`

9 状态

已完成 2026-10-20（stage8D 目录 mtime 10-20 09:35）。Git `e2c7156`。冻结主终点 / Holm / LOPO 2026-10-30 事后补算 复核过。

8D supplement · 冻结计划中的 7 层积分终点

1 问题与作用

补算哈希锁定的 analysis_plan.json 规定、但 8D 没算的主终点（7 层积分）。

2 材料

只读 8D 的 `paper_layer_advantage.csv`（sha256 `ea8211b5…3945`）与 `stage8_ab_pairs.csv`（`9505d324…539`）；不读原始分数 ⟶ 出处 `08_stage8/351_stage8D_supplement.py:45-46` · `supplement_manifest.json "inputs"`（与当前文件 sha256 一致，本手册核对）。

3 工具

`351_stage8D_supplement.py`（CPU），SEED 3518501，10,000 次 ⟶ 出处 `:17-18`。

4 步骤

每篇 7 层 TA 求平均；论文 bootstrap（RQ1 积分）、配对 bootstrap（A−B 积分）、J−V 积分；7 个逐层 A−B 只描述、不做 Holm。

5 公式

$TA^{int}_p = (1/7)\ \Sigma_{\ell\in\{16,\ldots,62\}}\ TA_{p,\ell}$。

6 对照

Vanilla 积分。

7 结果

终点	估计	95% CI	正 / N
J-Lens 7 层积分 TA（冻结主终点）	+2.33 pp	[+1.50, +3.19]	122/200
J-Lens 7 层积分 A−B（冻结主组检验）	+2.41 pp	[+0.91, +4.00]	61/100

Vanilla 7 层积分 TA	+2.35 pp	[+1.43, +3.32]	122/200
Vanilla 7 层积分 A–B	+2.25 pp	[+0.50, +4.09]	58/100
积分 J – V	−0.02 pp	[−0.69, +0.64]	98/200

⟶ 出处 `/scratch/zhang.zhiqi2/jlens_github_publish/experiment_files/351_stage8/stage8D_supplement/frozen_plan_endpoints.csv · commit 3a4f605（分支 review/stage8-fixes）`。该分支已于 2026-10-30 按用户决定合入 main（`a4c1f0a`），输出现在也在 `jlens-authorship-recall_update/experiment_files/351_stage8/stage8D_supplement/`（frozen_plan_endpoints.csv、supplement_manifest.json）；`/scratch/zhang.zhiqi2/jlens/351_stage8/` 下仍没有这份输出，但 2026-10-30 的重算（`351_stage8/recompute_2026-10-30/s8_endpoints.csv`）用原种子精确复现了它。⟶ 出处 `git log --oneline main · PROGRESS.md 2026-10-30 14:00 条 U31` 复核过

8 局限与质疑

方向与 L56 一致，积分效应约为 L56 的一半；补算发生在看过 8D 结果之后（2026-10-21），但终点是事先冻结的，计算只读 8D 中间表。

9 状态

已完成 （2026-10-21）；2026-10-30 合入 main（a4c1f0a），U31 已解决。

8E 外部事后审计（2026-10-30 从原始 logits 重算）

1 问题与作用

用不同的候选加权（作者加权、唯一 token、去掉与金标准同首 token 的错误作者）重算 RQ1/RQ2 的 L56 结果，并检查错误标题 token 长度差的影响。

2 材料

原 8E 的脚本、种子、逐篇 CSV 在集群和两个 git 克隆中都没有找到，只有已发布报告里的一张表（作者加权 +5.18 [2.90, 7.51]、唯一 token +4.58 [2.53, 6.67] 等）和"斜率 +0.12 pp/token"。2026-10-30 改为从 8C 原始首 token logits 重算：`351_stage8/raw/stage8C_context_scores.jsonl`（4,200 行）+ 冻结候选 `controls/stage8_author_candidates.csv`；斜率用 `stage8D/summary/context_level_scores.csv`。⟶ 出处 `351_stage8/recompute_2026-10-30/manifest.json inputs（含 sha256）`

3 工具

`~/jlens_work/08_stage8/recompute_2026-10-30/stage8_frozen_plan_recompute.py`（作业 11500770，CPU）。

4 步骤

每个上下文按 7.2 的胜率公式计算金标准首 token 对错误集合的胜率（平局 0.5），四种加权：author（每个错误作者一票）、unique（相同首 token 只算一次）、no_gold_tie（去掉与金标准首 token 相同的错误作者）、unique_no_gold_tie；TA = 真标题 − 20 个错误标题均值；RQ1 论文 bootstrap、RQ2 配对 bootstrap，B = 10,000，种子 20261037–20261044；Monte-Carlo 种子 1–100 给区间端点范围。斜率：论文固定效应下 (真标题胜率 − 错误标题胜率) 对 |标题 token 差| 的 OLS，4,000 个上下文，按论文整群 bootstrap（种子 20261132）。

5 公式

W 同 8D；斜率 = 论文内去均值后的 OLS 系数（pp / token）。

6 对照

作者加权版本应与 8D 的 L56 点估计相同（是：+5.18 / +4.98）。

7 结果

加权	RQ1 L56（200 篇）	RQ2 L56 A−B（100 对）	已发布报告
author	+5.18 [+2.91, +7.44]	+4.98 [+0.34, +9.81]	+5.18 [2.90, 7.51] / +4.98 [0.28, 9.77]
unique	+4.58 [+2.52, +6.68]	+5.40 [+1.19, +9.70]	+4.58 [2.53, 6.67] / +5.40 [1.10, 9.73]
no_gold_tie	+5.32 [+3.07, +7.72]	+5.03 [+0.27, +9.98]	+5.32 [2.98, 7.70] / +5.03 [0.22, 9.92]
unique_no_gold_tie	+4.64 [+2.52, +6.72]	+5.46 [+1.19, +9.84]	+4.64 [2.56, 6.77] / +5.46 [1.10, 9.88]

错误标题 token 长度差斜率（J-Lens L56）：+0.124 pp/token [+0.015, +0.240]（报告 +0.12 [+0.015, +0.236]）。点估计与报告全部相同；加权表中报告的 16 个区间端点都落在 bootstrap Monte-Carlo（种子 1–100）范围内（斜率没有做 Monte-Carlo 范围）。⟶ 出处 `351_stage8/recompute_2026-10-30/s8_unsourced_intervals.csv 行 "8E …" · s8_8E_weighting_by_paper.csv（逐篇）· PROGRESS.md 2026-10-30 13:00 条`

8 局限与质疑

这是对原 8E 数字的独立重算，原 8E 脚本仍找不到，无法证明与原实现逐行相同；四种加权和斜率都是看过 8D 之后的事后分析，只作敏感性。

9 状态

复核过：2026-10-30 从原始 logits 重算（作业 11500770），U32 已解决。原 8E 方案未在 Explorer 上执行、脚本不存在这一点不变 ⟶ 出处 `~/jlens_work/STAGES.md "Stage 8" 表 8E 行`。

Stage 9 · FullFit 拟合可靠性

9A.1 WikiText 拆分冻结（Fit A100 / Fit B100 / Holdout100）

1 问题与作用

为"换一批数据拟合 lens 结论是否稳定"准备互不重叠的两套拟合集和一个留出集。

2 材料

Salesforce/wikitext，wikitext-103-raw-v1，train ⟶ 出处 `351_stage9/config/plan.json "source"`。

3 工具

`/scratch/zhang.zhiqi2/jlens/351_stage9.py prepare → audit → preflight`（sha256 `1e2900ef…3878`，与 `09_stage9/` 备份一致）。

4 步骤

1. 空白规范化；去掉长度不在 100–1,200 字符内的段落和形如 "= … =" 的标题行 ⟶ 出处 `351_stage9.py:50, :223-224`。
2. 按规范化文本 sha256 去重 ⟶ 出处 `:51`。
3. 哈希键排序后用 `random.Random(3519001)` 打乱，依次切出 fit_A 100、fit_B 100、holdout 100 ⟶ 出处 `:55-62`。
4. 写 plan.json 与 manifest.json（sha256 `951f5ad3…850c`）。

5 公式

—

6 对照

—

7 结果

三份各 100 条，互不重叠 ⟶ 出处 `351_stage9/config/manifest.json "counts"`。

8 局限与质疑

哈希不重叠不保证来源文档独立；不能证明与 Public-557 的拟合文本不重叠（plan.json "interpretation"）；按字符筛选，实际 token 长度不一（如有提示只有 48 或 87 token）。

9 状态

已完成 2026-10-18。不在 git 里。

9A.2 Fit A（含修复）

1 问题与作用

在 fit_A 上拟合 7 层 lens。它后来成为 Stage 10 的 fitA lens。

2 材料

fit_A 100 条 + 1 条替补。

3 工具

`stage9_runner.py`、`stage9_fullfit.py`、`stage9_A_recover.py`、`stage9_a_repair.py`（从 `/scratch/zhang.zhiqi2/jlens/` 运行，按绝对路径互相加载）。

4 步骤

1. 配置：源层 L16/24/32/40/48/56/62 → L63，dim_batch 1，max_seq_len 128，skip_first 16，每条提示存一次检查点 ⟶ 出处 `runner_runs/full_A_repair_101_100valid/repair_manifest.json`。
2. 单条 7 层基准：Jacobian 320 s，墙钟 5:50.72（350.72 s），最大常驻内存 65,071,380 kB（≈ 62.06 GiB）⟶ 出处 `351_stage9_benchmark_A1_7layers.log`。正式第一批 4 条每条 325/323/325/322 s（均值 323.75 s），max_d_mean 0.674 → 0.578 → 0.474 ⟶ 出处 `351_stage9_full_A.log`。
3. 分多次 GPU 会话续跑（progress_000_004 … progress_047_056 等）。`jlens.fit` 跳过了索引 66 的提示（只有 14 个 token，少于 skip_first+1）⟶ 出处 `351_stage9_A_repair.log "SKIP-AWARE AUDIT PASS … skipped_index 66, skipped_token_length 14"`。
4. 修复（协议 A_APPEND_ONE_V1）：保留原检查点（n_done 94、next_idx 95），在冻结列表后追加 1 条经核验的 128 token WikiText 替补（train 第 3 行，"Senjo¯no Valkyria 3 …"），续跑到 next_idx 101、n_done 100；估计方法不变 ⟶ 出处 `repair_manifest.json · 09_stage9/stage9_a_repair.py:1-5`。
5. 保存 lens 后重新加载，与检查点均值比较，相对误差 ⩽ 0.01 才算通过。

5 公式

$J_\ell = (1/n_{done})\ \Sigma_{prompts}\ J_\ell(prompt)$，每条提示内对有效源位置 16 … seq_len−2 平均，FP32 累加（7 个矩阵约 700 MiB）⟶ 出处 `09_stage9/351_stage9_README.md "2026-10-18 revision"`。

6 对照

续跑方法检验：从检查点续跑与一次跑完在 L48/L56 上差 0.0，容差 0.001，PASS ⟶ 出处 `351_stage9/reports/stage9_resume_comparison.json`（工程检验，不是留出集比较）。

7 结果

状态 `VERIFIED_100_VALID_OF_101_CANDIDATES`，n_done 100，next_idx 101，skipped 1；各层 reload 相对误差 2.05×10^{-4}–2.60×10^{-4}；lens sha256 `4881b830b1e10b920ff7882b53e2fbf5c66235668b93e6c54a4c01aa32cf4444`，完成于 2026-10-22 05:33 ⟶ 出处 `runner_runs/full_A_repair_101_100valid/final_status.json`。Stage 10 的 `config/lenses.json` 登记的 fitA 就是这个文件，sha 一致 ⟶ 出处 `351_stage10/config/lenses.json:2-6`。

8 局限与质疑

替补文本是 WikiText train 的第一篇文章，是否与 fit_B/holdout 重复只靠哈希检查；只有一个拟合，无法单独回答稳定性问题。

9 状态

已完成。只读复核 `351_stage9.py audit`：STAGE 9 AUDIT PASS 复核过。不在 git 里。

9A.3 Fit B

1 问题与作用

在不重叠的 fit_B 上用同一配方拟合，与 Fit A 比较。

2 材料

fit_B 100 条。

3 工具

同 Fit A（`stage9_fullfit.py run --half B`）。

4 步骤

与 Fit A 相同配置（fitting_config.json 写于 2026-10-20 05:57）。

5 公式

同 Fit A。

6 对照

—

7 结果

检查点 `runner_runs/full_B_100_7layers/checkpoint.pt`：n_done = 66，next_idx = 66，最后写入 2026-10-21 19:12，此后没有续跑 ⟶ 出处 本手册用 `torch.load(mmap)` 读取检查点元数据 · `ls --time-style=full-iso`。

停止原因（U33 已解决）：最后一段从 51/100 续跑（作业 11289356@d1028，17:04:46 开始，时限 120 min，加载权重用了 18 分钟且有部分参数被 offload 到 CPU），作业在 19:05:30 到达 2 小时时限（TIMEOUT），各 step 在 19:16:35 被 signal 9 结束；在此之前日志最后写到 "prompt 66/100"（19:12:03），没有报错；checkpoint.pt 最后写入 19:12:14，完好，可以续跑。之后的 GPU 时间转去做 Fit A 修复和 7.1B。⟶ 出处 `351_stage9_full_B.log` 尾部 · `sacct -j 11289356` · `ls --full-time runner_runs/full_B_100_7layers/` · 附录 I U33 复核过

8 局限与质疑

没有 Fit B，Stage 9 的核心问题"换一批数据拟合，结论稳不稳"没有回答。**用户 2026-10-30 决定答辩前不补做**；本手册的局限写明：Fit B 停在 66/100、未使用，lens 稳定性只有间接证据（Self-100 split-half、Stage 10 三把 lens 一致）。⟶ 出处 `PROGRESS.md 2026-10-30 14:00 条 U33`

9 状态

计划未做 （66/100 因作业时限中断；答辩前不补做，用户决定）。

9B / 9C · 留出集比较与影响因素

1 问题与作用

9B：在冻结的 `prompts/holdout.jsonl`（100 条）上比较 Fit A 与 Fit B 的读出。9C：提示格式、token 类型、Final-logit anchor 等因素。

2 材料

holdout.jsonl（sha256 `8b958f47…f852`）⟶ 出处 `351_stage9/config/manifest.json`。

3 工具

没有实现代码。

4 步骤

README 中列了计划的比较方式：独立校准提示上的 token 名次 Spearman / 成对一致率、逐层标题优势、轨迹可复现性、自拟合 vs 公共 vs Vanilla；9C 的 token 类型（首字母/姓开头/后缀/标点）、提示改写、Final-logit anchor ⟶ 出处 `09_stage9/351_stage9_README.md "Validity gates" 第 5、6 条`。均未执行。

5 公式

未定义（未执行）。

6 对照

计划以 Public lens 与 Vanilla 为参照（未执行）。

7 结果

没有任何留出集比较结果；`reports/` 中只有续跑工程检验 `stage9_resume_comparison.json`。

8 局限与质疑

Stage 10 中三种 Jacobian lens（public、self100、fitA）结论一致，只是间接支持。

9 状态

计划未做 ⟶ 出处 `~/jlens_work/STAGES.md "Stage 9" 表（2026-10-27 核实）`。

Stage 10：全量逐层读出（9,108 篇论文、149,999 个名字 token、7 种标题条件）

代码：冻结脚本 `~/jlens_work/10_stage10/351_stage10.py`（sha256 `9107c80b…dbdd3479`）+ GPU 入口 `351_stage10_fix1.py` + CPU 补充分析 `stage10_extra*.py`；结果根目录 `/scratch/zhang.zhiqi2/jlens/351_stage10/`。 ⟶ 出处 sha256sum ~/jlens_work/10_stage10/351_stage10.py = 9107c80b13fd…；git 仓库只有一个版本 b4b76ed（code/scratch_scripts/351_stage10.py，sha 相同）；raw/final/L63、raw/logit/L55、raw/cond_false_0/jlens_public/L62 全部 72 个 npz 的 meta.script_sha256 = 9107c80b13fd（本文起草时读取）

注意一个版本细节：`config/manifest.json` 记录的冻结时脚本 sha256 为 `847e7dde…`（2026-10-23T15:44:01），与现在文件（`9107c80b…`）不同；`config/conditions_manifest.json` 记录的是 `9107c80b…`。所有 raw 输出都由 `9107c80b…` 版本产生，protocol 的 sha256（`9fafd1d4…`）每次读取都校验未变（`verify()`，351_stage10.py:189–196）。2026-10-30 复查的时间线：`config/manifest.json` 写于 10-23 15:44:01（记录 847e7dde）→ `351_stage10.py` mtime 10-23 17:00:31 → freeze-conditions 作业 11333739（日志 17:03）生成 conditions_manifest.json（created 17:03:05，记录 9107c80b），即脚本在 15:44–17:00 之间被改过，17:03 起所有运行都用 9107c80b。在 /scratch 与 ~ 下 1,709 个 .py/.bak、4 个仓库全部分支的 1,932 个 .py blob、~/.claude 的 file-history/backups 里都找不到 sha 为 847e7dde 的文件；本会话转录从 10-23 20:31 EDT 才开始，覆盖不到那次修改。⟶ 出处 附录 I U34 **未知/需确认：向 Stage 10 原作者（10-23 下午的会话，可能在 laptop 上）或用户确认 847e7dde → 9107c80b 之间改了什么（U34；laptop 上若有副本，sha 应为 847e7dde7f2005dd…918f2b05）** ⟶ 出处 /scratch/zhang.zhiqi2/jlens/351_stage10/config/manifest.json（script_sha256, protocol_sha256）· config/conditions_manifest.json（script_sha256）

0. 共同设定（以下各实验共用；本节不是独立实验）

项目	实际做法	出处
模型	Qwen/Qwen3-32B，revision `9216db5781bf21249d130ec9da846c4624c16137`，bf16，整模放在 GPU（禁止 CPU/disk offload），A100-SXM4-80GB；transformers 5.x 下 `hf_device_map` 不存在导致 parity 作业 11334054 崩溃，fix1 改为从参数/buffer 读设备位置，其余不变；另加权重预读（只影响速度）	⟶ 出处 351_stage10.py:26–27, load_model() 371–386；351_stage10_fix1.py:1–48；raw npz meta.gpu = ['NVIDIA A100-SXM4-80GB']
论文队列	标注包 `paper_labels_with_papers.csv`（2026-10-15）全部 9,108 篇，按 paper_id 排序；每 128 篇一个 shard，共 72 个 shard	⟶ 出处 351_stage10.py:31–35, freeze() 104–179；config/manifest.json: n_papers 9108, n_shards 72

分组	group_tight：target 4,621、control 2,220、excluded 2,053、guard 214；recall_level 交叉：target R0 4,110 / R1 511；control R0 2,186 / R1 34	⟶ 出处 cohort/papers.parquet（group_tight × recall_level 交叉表，本文起草时计算）

提示词	`Who are the authors of the paper titled "{title}" ({year})?`
	`Authors:`
	
	tokenizer 默认编码（Qwen3 不加 BOS），后接 `encode(" " + author, add_special_tokens=False)`；**每位作者单独一个上下文**（作者串按 ";" 切分、strip，拼写原样保留）。这个问法沿用 Stage 1–9；**选择过程未记录，未系统比较其他问法**（U11，用户 2026-10-30 决定照此写）
		⟶ 出处 351_stage10.py:29（PROMPT）；config/protocol.json: context, authors；PROGRESS.md 2026-10-30 14:00 条 U11；g-prompts-list

打分单元	33,811 个作者上下文、149,999 个名字 token（平均 4.44 token/作者）；排除规则（名字 > 32 token 或上下文 > 512 token）实际排除 0 位作者	⟶ 出处 config/manifest.json: n_contexts 33811, n_tokens 149999, n_excluded_authors 0；351_stage10.py:30–31
输入与位置	input_ids = prompt_ids + name_ids[:-1]；位置 len(prompt)−1+j 预测名字第 j 个 token（教师强制）。第 0 个名字 token 由提示词最后一个 token ":" 预测	⟶ 出处 config/protocol.json: input_ids；351_stage10.py run() 562–563
token 角色	按 " "+author 的字符偏移分词到词：最后一个词 = 姓（surname）；小写在 PARTICLES 里 = particle；匹配 `^[^\W\d_]\.?$` = initial；其余 = given；纯标点 token = punct；词的第一个 token 加 _first，其余 _cont。计数：surname_cont 39,995、surname_first 33,810、punct 28,701、initial_first 28,382、given_first 9,431、given_cont 9,357、particle_first 323	⟶ 出处 351_stage10.py:80–101 token_roles()；cohort/name_tokens.parquet role 计数（本文起草时计算）
名次	rank = 1 + #{词表中 logit 严格大于 gold logit 的 token}（bf16 lm_head 输出转 float）；并列数另存（ties），名次按有利方向计	⟶ 出处 351_stage10.py:442–449 score()；config/protocol.json: rank, ties
主指标	nlr = log10(1/rank) = −log10(rank)（0 = 第 1 名，−1 = 第 10 名，最小约 −5.18）。论文分 = 该论文所有被打分作者的所有名字 token 的 nlr 平均（mean_nlr）；组间只比均值不比总和。原 protocol 主指标是 Σ1/rank，2026-10-23 的分析计划修正为 log10(1/rank)	⟶ 出处 config/analysis_plan.json（primary_metric, amends, reason）；351_stage10.py:634–640 METRICS, paper_scores() 651–667
引用档	13 个引用档（0-2, 3-4, 5-8 ⋯ 4097+）；target−control 只在两组都 ⩾ 30 篇的档内比较（实际 10 档：0-2, 3-4, 5-8, 9-16, 17-32, 33-64, 65-128, 129-256, 257-512, 513-1024）	⟶ 出处 351_stage10.py:642 MIN_STRATUM；summary/B_target_vs_control_within_bin.csv 列 strata、n_strata = 10
统计	档内合并差：Σ_b w_b (Ā_b − C̄_b)，w_b ∝ n_t n_c/(n_t+n_c)；bootstrap 在"档 × 组"内重抽论文，B = 2000，取 2.5/97.5 百分位	⟶ 出处 351_stage10.py:670–691 stratified_diff()；645 B_BOOT

10-1 全量逐层读出（真标题；logit lens + 3 种 Jacobian lens + 真实输出）

1 问题与作用

在名字位置上，模型在哪一层开始"拿着"这篇论文的作者？如果目标组（写不全作者但 4/4 认出名单）在中间层就已有作者信息，那么中间层的读出应该比对照组更容易命中正确名字。本实验给出每个读出 × 层的读出水平（6.1）和真标题下的 target−control 档内差（6.2）。

2 材料

全部 9,108 篇、33,811 个作者上下文、149,999 个 token；真标题条件。主轮读出 × 层 = 4 种 lens 读出 × 6 层（L24, 32, 40, 48, 56, 62）+ final L63 = 25 对；与 6 种其它条件合计 7 条件 × 72 shard × 25 对 = 12,600 个 npz 文件。 ⟶ 出处 `351_stage10.py:42 DEFAULT_LAYERS；main() 914 --readouts 默认 jlens_public,jlens_self100,jlens_fitA,logit,final；报告 §5（12,600 文件）`

3 工具

读出	定义	lens 文件 / 拟合
logit	model.unembed(residual_l)（final norm + lm_head），= JacobianLens.apply(use_jacobian=False)	—
jlens_public	lens.transport(residual_l, l) 后 unembed	Neuronpedia 公开 lens `public_neuronpedia/qwen3-32b/jlens/Salesforce-wikitext/Qwen3-32B_jacobian_lens.pt`，source_layers 63 层，n_prompts 557
jlens_self100	同上	`qwen32b_jlens_prelim_100.pt`，6 层（24…62），n_prompts 100
jlens_fitA	同上	Stage 9 `351_stage9/runner_runs/full_A_repair_101_100valid/lens.pt`，7 层（16…62），n_prompts 100
final	model.unembed(residual_63) = 模型真实下一个 token logits	—

residual = 第 l 个 decoder block 的输出（jlens `ActivationRecorder` 约定）。快速路径：一次前向记录所需层的残差，再对所有名字位置一起做读出；每个 J_l 放在持有 block l 的 GPU 上。 ⟶ 出处 `config/protocol.json: readouts, residual；config/lenses.json；351_stage10.py:407–439 open_lenses()/fast_readout()`

4 步骤

1. `freeze`：冻结队列、token 表、上下文、shard、protocol、lens 登记（sha256）。 ⟶ 出处 `351_stage10.py:104–179`
2. `parity`（GPU 门）：快速路径 vs 逐前缀 `lens.apply`，见 10-9。 ⟶ 出处 `351_stage10.py:464–508；stage10_parity.sbatch；stage10_decision.sbatch（日志 logs/decision_11363838.log）`

3. `run`：每个 shard、每个条件，对每个作者上下文跑一次前向，存每个 (readout, layer, shard) 的 rank/ties/logprob/top1，写临时文件再 os.replace；已存在的文件跳过。运行前逐篇核对提示词编码与冻结 ids 一致。⟶ 出处 `351_stage10.py:523–581`
4. 调度：自重排的 worker 和"队列池"（dispatcher + 每 job 一块 + 工作窃取），作业日志在 `logs/pool/dispatcher_*.log`（11338002, 11338033, 11338068, 11363842, 11374266）。⟶ 出处 `~/jlens_work/10_stage10/README.md` ; `stage10_pool.py, stage10_steal.py`
5. `analyze`（CPU）：读全部完整的 (readout, layer)，算论文分、A/B/C/D/E 各表。作业 11373362。⟶ 出处 `351_stage10.py:738–766` ; `logs/analyze_s10_analyze_main_11373362.log`

5 公式

nlr_{t} = −log10(rank_t)；论文 p 的 mean_nlr(p) = 该论文全部 token 的平均；组均值 = 论文均值的平均（bootstrap 论文，B=2000）；target−control = Σ_b w_b [mean_{target,b} − mean_{control,b}]，w_b = n_t n_c/(n_t+n_c) 归一化。⟶ 出处 `351_stage10.py:651–667, 670–691, 694–705`

6 对照

- 组间对照：control（同样写不全、但认不出名单）；只在同一引用档内比（引用与分组高度相关）。
- 敏感性：去掉 0-2、3-4 两个低引用档（B_sens_drop_low_bins）；用 plain group 代替 group_tight；档 × 领域分层（每格 ⩾10）；group_both（500 篇第二次抽样）。⟶ 出处 `351_stage10.py:752–756`
- 读出间对照：logit lens（无 lens 代码）和 final（真实输出）作参照。

7 结果

真实输出 L63 的论文均值：target −0.708 [−0.717, −0.700]，control −1.004 [−1.018, −0.989]，guard −0.329，excluded −0.887。⟶ 出处 `summary/A_group_layer.csv ( readout=final, 列 mean_nlr_mean/lo/hi )`

真标题 target−control（档内，10 档，target 3,734 / control 2,182 篇）：final +0.247 [0.230, 0.266]；L62：jlens_public +0.225、self100 +0.226、fitA +0.226、logit +0.247；中间层 Jacobian lens 为负（L48：public −0.133、self100 −0.153、fitA −0.096，CI 均 < 0），logit lens 在 L48 前约为 0（L48 −0.011 [−0.027, 0.005]）。⟶ 出处 `summary/B_target_vs_control_within_bin.csv ( 列 mean_nlr_diff/lo/hi, n_a, n_b )`

8 局限与质疑

- 测的是教师强制下给正确名字打的分，不是生成。
- 每位作者单独接在 "Authors:" 后打分，没有其他作者作上下文（E3 另测连写）。
- 名次用 bf16 logits，并列按有利方向计。

- 真标题水平混有"名字好不好猜"，所以要看 10-3 的标题特异信号。
- Jacobian lens 中间层 target−control 为负，说明 lens 读出的名字信息不区分论文（见 10-7）。

9 状态

已完成 （12,600 个文件，analyze 11373362）。团队聚类 bootstrap（Stage 13 `stage13_cluster_ci.py`，作业 11443686/11443696）点估计不变。⟶ 出处 PROGRESS.md 2026-10-28 14:30 / 12:35 条；381_stage13/reports/cluster_ci.md

10-2 细层扩展 L50–L55、L57–L61

1 问题与作用

主轮只有 L48、L56、L62，看不出目标组优势是突然在 L62 出现还是逐层积累。扩展补齐中间细层，定位起点。

2 材料

同一队列、7 个条件全部；只算 jlens_public 和 logit 两种读出（self100、fitA 只拟合了 6–7 层，加层需要重新拟合）；层 L50–55、L57–61（11 层 × 2 读出 = 22 对）。

3 工具

同 10-1，入口 `351_stage10_fix1.py run --readouts jlens_public,logit --layers 50,…`；扩展专用 sbatch：`stage10_ext_wait / ext_parity / ext_worker / ext_analyze.sbatch`。⟶ 出处 ~/jlens_work/10_stage10/stage10_ext_worker.sbatch:2–3, 16–20；README.md 第 18 行

4 步骤

1. 扩展 parity（parity_ext 作业 11338000）：12 个上下文、52 个位置，jlens_public 与 logit 在 L50–55、57–61 全部 PASS（容差 0.25，最大 |Δlogit| 0.25，出现在 public L52/L58/L61、logit L61）。⟶ 出处 reports/parity.json runs[0]（created 2026-10-24T15:18:12, status PASS）；logs/parity_ext_11338000.log
2. ext_worker：7 条件 × 指定 shard，自重排直到 remaining=0，然后提交 ext_analyze。⟶ 出处 stage10_ext_worker.sbatch:16–28
3. ext_analyze（作业 11378377）重新跑 analyze / analyze-title：它们会自动收进所有完整的 (readout, layer)，新层与主轮层并列。扩展前的分析快照保存在 `snapshot_main_round/`。⟶ 出处 stage10_ext_analyze.sbatch:3；logs/analyze_s10x_analyze_11378377.log

5 公式

同 10-3（标题特异 = 真标题 − 5 个假标题平均）和 10-4（档内 target−control）。

6 对照

主轮 L48/L56/L62 作两端锚点；logit lens 与 public J-lens 互为对照。

7 结果

标题特异 target−control（档内）：logit L48 +0.004 [−0.001, 0.008]，L50 +0.007 [0.001, 0.014]，L52 +0.018，L54 +0.030，L56 +0.065，L58 +0.114，L60 +0.178，L61 +0.212，L62 +0.229；jlens_public L48–L54 为负（L50 −0.050 [−0.067, −0.033]），L55–56 约 0，L57 起为正 +0.029 [0.009, 0.048]，L61 +0.207。 ⟶ 出处 summary_title/B_spec_target_vs_control_within_bin.csv（列 spec_mean_nlr_diff/lo/hi，readout ∈ {logit, jlens_public}，layer 48–62）

8 局限与质疑

- 只有两种读出；三种 Jacobian lens 的细层一致性没有检验。
- "从 L50 开始积累"由 logit lens 给出；public lens 的转正点晚到 L57，两者不同。

9 状态

已完成

10-3 标题特异信号（真标题 vs 5 个匹配假标题 vs 删去标题）

1 问题与作用

把"名字本身好猜"从"认识这篇论文"中扣掉：同一组名字 token，在真标题、5 个借来的假标题、无标题三种提示下打分，配对相减。

2 材料

7 个条件：true；false_0…false_4（同领域其他论文的标题，年份保留为目标论文年份）；delete：

```
Who are the authors of the paper ({year})?
Authors:
```

假标题匹配（每篇 K=5 个供体）：同领域；硬排除 = 自身、任何共享姓氏（含共享作者）、标题词 Jaccard（≥4 字母小写词）> 0.6；代价 = |Δ年份|/5 + |Δ作者数|/max(1, 0.25n) + |Δ标题 token 数|/max(1, 0.2 · len) + 1.5 · |Δ引用档|（**这些权重是经验设定，没有做过敏感性分析**，U13，用户 2026-10-30 确认 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条；h-false-cost）；按 4 级容差（strict / medium / loose / nearest）逐级做 `scipy.optimize.linear_sum_assignment` 最小代价指派（加哑列）；false_0 = 级别最好、代价最小的那个。实际指派：strict 35,898、medium 1,638、loose 2,565、nearest 5,439；单个供体最多被用 7 次；有 5 个 strict 供体的论文 6,056 篇。 ⟶ 出处 351_stage10.py:243–248（K_FALSE, DELETE_PROMPT, TIERS, NEAR_DUP_JACCARD），255–307 match_false_titles()（cost 在 281 行），310–349 freeze_conditions()；

config/conditions_manifest.json（tier_summary, donor_use_max=7, papers_by_n_strict_donors）；作业 freeze_conditions 11333739

3 工具

同 10-1 的 5 种读出；`analyze-title`（CPU）。⟶ 出处 351_stage10.py:838–890；作业 11375853（logs/analyze_s10_analyze_title_11375853.log）

4 步骤

1. 每个条件单独跑 10-1 的 run（同一 token 集合，只换提示词前缀）。冻结时检查 true 条件的 ids 与 base freeze 完全相同、每个条件上下文 ⩽ 512。
2. 每个条件算论文分（mean_nlr 及各角色均值），在 (paper, readout, layer) 上对齐（缺任一条件就不进入）。
3. 得到每篇论文的 spec、total、topic、spec_strict（只对 strict 级假标题平均）和 tail_fraction。

5 公式

对每个指标 m（mean_nlr 等）：
spec(p) = m_true(p) − (1/5) Σ_{k=0..4} m_{false_k}(p)　（标题特异）
total(p) = m_true(p) − m_delete(p)；topic(p) = mean_k m_{false_k}(p) − m_delete(p)
tail_fraction(p) = (1 + #{k: m_{false_k} ⩾ m_true}) / 6（参考尾比例，不是 p 值）⟶ 出处 351_stage10.py:860–870

6 对照

- 5 个匹配假标题（主对照）；无标题（名字先验，见 10-5）。
- 阴性对照：标点 token 的标题特异信号应接近 0（negative_control_by_role）。
- spec_strict：只用 strict 级假标题，检查匹配质量。

7 结果

论文均值（jlens_public）：target L48 −0.007、L56 +0.145、L62 +0.560，final +0.481；control L48 +0.038、L56 +0.123、L62 +0.235，final +0.200。标点 token 的标题特异在所有读出 × 层最大约 0.014（public L56）。⟶ 出处 summary_title/A_group_layer.csv（列 spec_mean_nlr_mean）；summary_title/negative_control_by_role.csv（spec_mean_nlr_punct_mean）

8 局限与质疑

- 假标题保留目标论文年份，因此"年份"不在对比里。
- 匹配有 5,439 个 nearest 级（容差最松），供体最多重复 7 次。
- 假标题同领域、不共享姓氏，但仍可能"主题相近"，所以 topic 项 = 假标题 − 无标题，被单独报告。

9 状态

已完成

10-4 标题特异信号的 target − control（引用档内）及与多选题成绩的等级关系

1 问题与作用

本阶段关键比较：目标组的优势在哪一层出现，是否是标题特异的（不能用名字好猜解释）。

2 材料

summary_title/paper_title_contrasts.parquet（每篇 × 读出 × 层的 spec 等）；10 个可比档内 target 3,734、control 2,182 篇。

3 工具

`stratified_diff`（seed 3511，B=2000）；`graded()`：mc_team_a（多选题 4 种顺序中答对次数 0–4），只用 R0/R1 论文，指标在引用档内中心化后求各等级均值和 Spearman。 ⟶ 出处 351_stage10.py:708–722 graded(), 883–886

4 步骤

1. B_spec_target_vs_control_within_bin：spec_mean_nlr、spec_author_mean_nlr、spec_mean_nlr_surname_first、spec_strict_mean_nlr。
2. 敏感性：去掉 0-2、3-4 档（剩 8 档）。
3. C_graded：真标题水平（summary/）和标题特异（summary_title/）两个版本。

5 公式

Δ = Σ_b w_b [spec̄_{target,b} − spec̄_{control,b}]，w_b ∝ n_t n_c/(n_t+n_c)，档内两组都 ⩾ 30；CI = 档 × 组内重抽论文的百分位区间。graded：x_c = x − mean_{bin}(x)，Spearman(mc_team_a, x_c)。

6 对照

去低档敏感性；spec_strict（只用 strict 假标题）；按 token 角色拆分（10-6）。

7 结果

读出	L24	L48	L56	L62	L63
jlens_public	−0.002	−0.044 [−0.058, −0.030]	+0.007	+0.230 [0.215, 0.245]	
jlens_self100	−0.004	−0.054 [−0.069, −0.040]	+0.003	+0.231 [0.216, 0.246]	

jlens_fitA	−0.003	−0.037 [−0.048, −0.026]	+0.009	+0.228 [0.213, 0.242]	
logit	+0.007	+0.004	+0.065 [0.050, 0.080]	+0.229 [0.213, 0.244]	
final					+0.207 [0.195, 0.220]

⟶ 出处 summary_title/B_spec_target_vs_control_within_bin.csv（spec_mean_nlr_diff/lo/hi；n_strata 10，n_a 3734，n_b 2182）

去掉 0-4 低档（8 档，3,327 / 1,475 篇）：final +0.212 [0.199, 0.226]，L62 四种读出 +0.236 到 +0.238。⟶ 出处 summary_title/B_sens_drop_low_bins.csv

等级关系（真标题 mean_nlr，final，n=8,894）：档内中心化均值 0 次 −0.137、1 次 −0.122、2 次 −0.047、3 次 −0.018、4 次 +0.076，Spearman ρ = 0.268（报告写 0.27）；同样分析用标题特异时 ρ = 0.329。⟶ 出处 summary/C_graded_mc_team_a.csv 与 summary_title/C_graded_mc_team_a.csv（readout=final, metric=mean_nlr / spec_mean_nlr, spearman_rho_within_bin）

8 局限与质疑

- 高引用档几乎没有 control（4097+ 档 0 篇），比较主要来自中低档。
- 低档"认出"接近固定偏好的 25%，因此做了去低档敏感性。
- 多选分组本身不稳（标注包 500 篇重测一致率 83.6%，kappa 0.63），会稀释组差。⟶ 出处 报告 §3.2 ④（标注包检查）
- bootstrap 按论文独立重抽，没有考虑同一作者团队的多篇论文相关；Stage 13 的团队聚类 bootstrap 复核过点估计。

9 状态

已完成 复核过 （R0-only 见 10-8；团队聚类 CI 见 381_stage13/reports/cluster_ci.md）

10-5 名字先验（name prior）分解

说明：分析脚本是 `~/jlens_work/12_stage12/stage12_cpu.py`（Stage 12 CPU 作业 11413574），但只用 Stage 10 的冻结结果；报告 §6.12 把它标为 Stage 11 · CPU。⟶ 出处 stage12_cpu.py:1–3, main() 37–80；PROGRESS.md 2026-10-27 Stage 12 条（job 11413574）

1 问题与作用

不给标题时，名字本身有多好猜？target−control 差距里有多少来自名字先验、多少来自主题、多少来自"这篇论文"？先验会不会就是"认得出名单"的原因？

2 材料

summary_title/paper_title_contrasts.parquet 中各角色的 true / false（5 个假标题平均）/ delete 论文分；papers.parquet 的 mc_team_a、recall_level、authors。

3 工具

pandas；档内合并差 `strat()`（与 Stage 10 相同规则，≥30 篇/档，B=2000，seed 3712）；Spearman（scipy）。

4 步骤

1. A：对每个读出 × 层 × 角色（全部、surname_first、surname_cont、given_first、initial_first、punct），按组求 prior、topic、spec、true 均值，并做档内 target−control。
2. B：只取 R0/R1 且 mc_team_a 有值的论文（n=8,894），prior 和 spec 在引用档内中心化，与 mc_team_a 求 Spearman，500 次 bootstrap 求 CI。
3. C：姓氏常见度 = 该姓在全部队列作者中出现次数的 log10，按论文平均，与 prior/spec 求 Spearman。

5 公式

prior = m_delete；topic = m_{false} − m_delete；spec = m_true − m_{false}；true = m_true，故 true = prior + topic + spec（恒等式，同一组 token）。 ⟶ 出处 `stage12_cpu.py:43`

6 对照

标点 token（应只靠先验）；按角色拆分。

7 结果

final，全部 token：所有论文 prior −1.089、topic −0.100、spec +0.378、true −0.812；target−control 档内：prior +0.087 [0.070, 0.103]、topic −0.047 [−0.055, −0.038]、spec +0.207 [0.195, 0.220]、true +0.247 [0.229, 0.265]。surname_first：prior 差 +0.022 [−0.019, 0.063]，spec 差 +0.423 [0.393, 0.454]。B：prior 与多选答对次数 ρ = +0.085 [0.063, 0.106]，spec ρ = +0.329 [0.312, 0.346]。C：姓氏常见度与 prior ρ = −0.215（全部论文）。 ⟶ 出处 `/scratch/zhang.zhiqi2/jlens/371_stage12/summary_cpu/A_prior_decomposition.csv ( readout=final , group ∈ all/target/control/tc_* ) · B_prior_vs_recognition.csv · C_surname_commonness.csv`

8 局限与质疑

- "无标题"提示仍含年份；先验是"给定年份和提示格式"的先验。
- 姓氏常见度只用本队列内的频次，不是语料频次。

9 状态

已完成 （属于 Stage 12 代码，此处只作 Stage 10 数据的方法说明）

10-6 名字格式与 token 位置 / 角色（X1 及其两项后续）

1 问题与作用

标题特异信号落在名字的哪个 token 上？是否主要由第 1 个 token 决定？知道第 1 个 token（或第一个词）后，后面的 token 是否容易很多？

2 材料

主轮 7 条件 raw（5 种读出，层 24/32/40/48/56/62 + final）；name_tokens 的 token_index、word_index、role。格式 = 该作者第 1 个 token 的角色：initial_first 24,739 位作者（8,275 篇有此类作者）、given_first 8,876 位（4,157 篇）。 ⟶ 出处 `summary_extra/X1_by_format.csv ( n_tokens, n_papers )`

3 工具

`stage10_extra.py`（X1，作业 11376473）、`stage10_extra_format.py` 与 `stage10_extra_cond_role.py`（作业 11376890）。均读冻结 raw，CPU。 ⟶ 出处 `logs/extra_11376473.log, logs/extra_fmt_11376890.log`

4 步骤

1. X1_levels / X1_tc：按位置（第 1 个 token / 后续 token；第一个词 / 后续词）在每篇论文内平均，算真标题水平、spec、档内 target−control。 ⟶ 出处 `stage10_extra.py:95–108`
2. X1_cond：以作者上下文为单位，第 1 个 token（或第一个词的所有 token）是否都在前 10 名，比较后续 token 的均值；CI 用按论文聚类的 bootstrap（multinomial 论文权重，B=2000）。 ⟶ 出处 `stage10_extra.py:110–135`
3. X1_by_format：按格式 × 位置，读出 jlens_public/logit 的 L56、L62 和 final，算水平、spec、target−control。 ⟶ 出处 `stage10_extra_format.py:52–61`
4. X1_cond_by_first_role：X1_cond 的"第一个词"版本按格式拆开。 ⟶ 出处 `stage10_extra_cond_role.py:87–98`

5 公式

位置子集 S 下论文值 = 该论文属于 S 的 token 的 nlr 平均；spec = true − mean(false_0..4)；target−control 同 10-4（MIN_N=30，B=2000，seed 3512/3514/3513）。条件差 = mean(later | first in top10) − mean(later | not)。

6 对照

教师强制下后续 token 天然更好猜（前缀已给出），因此用 spec（减假标题）而不是水平来比较位置；标点角色作阴性对照（10-3）。

7 结果

final L63：initial 格式第 1 个 token spec +0.112 [0.104, 0.120]、target−control +0.048 [0.028, 0.069]；后续 token spec +0.447、target−control +0.253 [0.237, 0.269]。given 格式第 1 个 token spec +0.358、target−control +0.174 [0.136, 0.213]；后续 +0.396、+0.202 [0.173, 0.232]。X1_cond（final，全部）：第 1 个 token 进前 10 时后续 token −0.740，未进 −0.780，差 +0.041 [0.029, 0.052]；按第一个词混合格式时差为 −0.113。 ⟶ 出处 summary_extra/X1_by_format.csv（readout=final）· summary_extra/X1_cond.csv（readout=final, group=all, unit=token/word）· X1_cond_by_first_role.csv

8 局限与质疑

- "第 1 个 token 是否进前 10"是观察性分组，两组作者在格式、长度上不同，按格式分开后结论才稳定。
- 只用主轮 6 层；细层没有做这些拆分。
- 姓的第一个 token 从不出现在第 1 个位置（stage10_extra_format.py:62–65 打印的 share at token_index 0），所以"位置"与"角色"两种分法不能互换。

9 状态

已完成

10-7a 引用数的影响（X2）

1 问题与作用

引用越低，模型越不认识论文吗？影响在水平上还是在标题特异上？

2 材料

同 10-6 主轮数据，13 个引用档 × 组（all/target/control）。

3 工具

`stage10_extra.py` X2 段（作业 11376473）。 ⟶ 出处 stage10_extra.py:137–157

4 步骤

1. 每档每组的水平和 spec（bootstrap 均值）。
2. 论文指标对 log10(citations+1) 的 Spearman（秩相关系数用 Pearson-on-ranks 计算），1,000 次 bootstrap CI。

5 公式

ρ = corr(rank(log10(c+1)), rank(x))。

6 对照

组内相关（target、control 分开），排除"组别 × 引用"混杂。

7 结果

final：全部论文 ρ(spec) = 0.451 [0.433, 0.467]、ρ(level) = 0.317；target 0.328 / 0.181；control 0.187 / 0.084。 ⟶ 出处 `summary_extra/X2_spearman.csv（readout=final）；分档值见 summary_extra/X2_by_bin.csv`

8 局限与质疑

高档 control 很少（2049-4096 档 9 篇、4097+ 0 篇），分档曲线在高档只有 target。

9 状态

已完成

10-7b 三种 Jacobian lens 与 logit lens 的比较（X3）

1 问题与作用

是否有某种 lens 能在中间层读出 logit lens 读不出、且偏向目标组的论文信息？

2 材料

主轮 4 种 lens 读出 × 6 层，全部论文，同一 token 配对。

3 工具

`stage10_extra.py` X3 段。 ⟶ 出处 `stage10_extra.py:159–176`

4 步骤

1. 每层每指标（水平、前 10 命中率、spec）的读出均值与两两配对差（bootstrap 论文）。
2. 每种 lens 的 (spec_lens − spec_logit) 做档内 target−control。

5 公式

Δ_pair = mean_p [x_{r1}(p) − x_{r2}(p)]；Δ_tc = strat_diff(spec_lens − spec_logit)。

6 对照

logit lens 为基准（不含任何 lens 代码）。

7 结果

(lens − logit) 标题特异的 target−control：L48 public −0.047 [−0.062, −0.033]、self100 −0.058、fitA −0.041；L56 −0.058 / −0.062 / −0.055；L62 约 0（+0.001 / +0.002 / −0.001）。 ⟶ 出处

summary_extra/X3_tc_spec_vs_logit.csv；均值与配对差见 X3_means.csv、X3_pairwise.csv

8 局限与质疑

三种 lens 均在通用文本（WikiText-103）上拟合，不是在作者提示上拟合；Stage 12 F2 另拟合了作者提示 lens（不在本部分）。

9 状态

已完成

10-8 R0 / R1 分层（Stage 10 + Stage 11）

1 问题与作用

target/control 都把 R0（一个作者都没写对）和 R1（写对少数，HR2 > 0.5）合在一起；target 有 511 篇 R1（11%），control 只有 34 篇（1.5%）。目标组的优势会不会来自"本来就能写出部分作者"的论文?

2 材料

Stage 10 paper_title_contrasts.parquet、paper_scores.parquet；Stage 11 raw/patch、patch_title、knockout、knockout_name、titlepos；标注包 open_results.jsonl（开放回答生成的名字）。Stage 11 样本中 R1 只有 41 篇（target 36、control 5）。

3 工具

`stage10_extra_r0.py`（R0-only 敏感性，复用冻结 `stratified_diff` 和 `stage11_analyze`；日志 logs/r0_sensitivity.log）和 `stage10_extra_r0r1.py`（作业 11429580）。stage10_extra_r0.py **没有单独的作业号**：它是 Claude 在会话里用 Bash 直接运行的（输出写于 10-27 18:57:08–18:58:47 EDT），运行在 Claude 所在的交互作业 11371284（bash@c0591，short）里，CPU。⟶ 出处 附录 I U35 · `ls --full-time 351_stage10/summary_r0/ logs/r0_sensitivity.log` 复核过 （U35 已解决）

4 步骤

1. R0 view：A（target R0）− B（control R0），档内 ≥30；R1 view：C（target R1）− B，档内每格 ≥10；C − A（target 内 R1 − R0，≥10）；D（control R1，34 篇）只做描述。⟶ 出处 stage10_extra_r0r1.py:68–111
2. Stage 11 曲线（E1、E1t、E2、E2n）按子组重算，比值 CI 用 `boot_ratio`（每组 ≥3 篇才算）。⟶ 出处 stage10_extra_r0r1.py:176–220
3. E4b probe 预测与 Stage 11 完全相同（同折、同 α 规则），只是汇总按 recall 拆分。⟶ 出处 stage10_extra_r0r1.py:223–273
4. R1 论文内部逐作者：标注包只存匹配数 m，于是重建匹配（姓 + 名首字母，两种名序，一对一），只保留重建数 = m 的论文；在论文内比较写对 vs 没写对的作者（配对 bootstrap）。⟶ 出

处 stage10_extra_r0r1.py:45–64, 115–167

5 公式

同 10-4 的档内差；Stage 11 部分同 11-2/11-4 的比值公式。逐作者：d(p) = mean_{written} spec − mean_{missed} spec，对论文 bootstrap。

6 对照

pooled（R0+R1）作参照；R1 view 用 R0 control 作参照（control R1 太少）；去掉第一作者的版本。

7 结果

R0 view final 标题特异 +0.199 [0.187, 0.212]（3,423 / 2,151 篇；pooled +0.207）；C − A +0.110 [0.086, 0.133]（485 / 3,204）；R1 内部写对 − 没写对 +0.199 [0.168, 0.232]（495 篇），去掉第一作者 +0.204 [0.165, 0.241]（333 篇）。重建匹配与 m 一致率：R0 99.9%、R1 99.3%、R2 98.1%。Stage 11 R0-only：E1 L48 0.81 / 0.87（target/control），E2 从 L0 损失 0.84 / 0.81。 ⟶ 出处 351_stage10/summary_r0r1/tc_R0.csv、C_minus_A.csv、R1_author_written_minus_missed.csv（readout=final）；reports/r0_r1_analysis.md 第 244–248 行；361_stage11/summary_r0/E1_recovery.csv、E2_knockout.csv；reports/r0_sensitivity.md

8 局限与质疑

- 逐作者匹配是重建的，不是标注包原样数据（R1 不一致的 0.7% 被丢弃）。
- stage10_extra_r0.py 与 stage10_extra_r0r1.py 都算过 C−A，两次 CI 上限略不同（0.135 vs 0.133），因为 bootstrap 随机序列不同。 ⟶ 出处 summary_r0/target_R1_minus_R0_within_bin.csv vs summary_r0r1/C_minus_A.csv

9 状态

已完成

10-9 一致性检查（parity）与 Stage 6 交叉核对

1 问题与作用

快速路径（整段一次前向，所有名字位置一起读出）是否与参考路径（每个前缀单独前向，Stage 5/6 用的 `JacobianLens.apply`）给出相同的 logits/名次？这是 GPU 主轮的放行门。

2 材料

随机抽 12 个作者上下文（seed 10），共 52 个名字位置；每个 (readout, layer) 都比较。 ⟶ 出处 351_stage10.py:464–475；--n 默认 12（main() 917）

3 工具

参考路径 `apply_on_ids()`：把 `model.encode` 临时替换为固定 ids，调用 `lens.apply(model, '', layers=[l], positions=[-1], max_seq_len=512, use_jacobian=…)`；final 用 apply 的第二个返回值（真实最终 logits）。⟶ 出处 `351_stage10.py:477–486, 511–520`

4 步骤

1. parity 作业 11334054 崩溃（transformers 5.x 无 hf_device_map）→ fix1。
2. 主 parity（作业 11347585、11350174，结果数值完全相同）：容差 0.25，10/25 对超过 0.25，最大 0.3671875（final@L63）→ FAIL。
3. 2026-10-25 用户批准把容差放宽到 0.5（事后调整），写入 parity.json 的 tolerance_amendment 和 config/analysis_plan.json 的 amendments；原 run 保留原状态，整体状态按新容差重算为 PASS。
4. decision 门（作业 11363838）放行主轮。
5. crosscheck：与 Stage 6 raw 的重叠 token 比较 token id、rank、logprob。⟶ 出处 `351_stage10.py:613–631`

5 公式

d(r,l) = max over 位置 |logit_fast − logit_ref|（全词表）；rank mismatch = 两路径 gold 名次不等的位置数。PASS 条件：所有对 d ⩽ tol。⟶ 出处 `351_stage10.py:489–497`

6 对照

final@L63 不经过任何 lens 代码，作为"纯 bf16 前向差异"的参照。

7 结果

主 parity 最大 |Δlogit|：final@L63 0.367；jlens_public 0.188–0.344；self100 0.164–0.336；fitA 0.180–0.336；logit 0.156–0.229。放宽理由（原文）：差异来自 bf16 前向舍入（整段一次前向 vs 逐前缀），确定性，在 logit 量级 20–40 时约 1–2 个 bf16 ulp；所有名次 < 42 的位置完全一致；名次不一致的都在 ⩾42，log10(1/rank) 最多变 0.070。Stage 6 交叉：972 个重叠 token id 100% 一致，名次完全相同 77–95%（报告 §5）。⟶ 出处 `reports/parity.json ( runs[1], runs[2] max_abs_logit_diff ; tolerance_amendment.reason ) ; reports/crosscheck_stage6.csv`

8 局限与质疑

- 容差是看到结果后放宽的（事后调整），已在分析计划和报告 §8 里披露。
- parity 只能发现两条路径不一致的错误，不能发现两条路径共有的错误。
- 只抽了 12 个上下文。

9 状态

已完成 （PASS，按 0.5 容差；原始 FAIL 记录保留）

Stage 11：干预实验（替换、阻断、标题位置读出与 probe、作者连写）

代码：`~/jlens_work/11_stage11/361_stage11.py`（freeze / preflight / smoke / work / remaining / analyze）与 `stage11_analyze.py`；结果 `/scratch/zhang.zhiqi2/jlens/361_stage11/`。Stage 11 通过 `351_stage10_fix1.py` 加载冻结的 Stage 10 模块（同一模型加载、同一 lens、同一 `score()`），只读 Stage 10 的队列和提示词并校验 sha256。 ⟶ 出处 361_stage11.py:44–62, freeze() 104–138, verify() 141–146

版本：全部 1,033 个 raw 单元的 meta.script_sha256 = `f8adb8d9…` = git c86112f 版本；当前文件 `613eb813…` = git 13ae7ec，与 c86112f 的差别只在 `stop_queue`（加 s12 作业名）和 `work()`（无单元时交给 Stage 12），计算函数完全相同。protocol.json 记录的冻结时脚本 sha256 为 `26e3f817…`，不在 git 历史中。 ⟶ 出处 raw/*/shard_*.npz meta（本文起草时统计：chain 504、knockout 24、knockout_name 24、patch 24、patch_title 24、probe_vocab 1、titlepos 432，全部 f8adb8d9，protocol sha 17c6c37a = sha256(config/protocol.json)）；git diff c86112f 13ae7ec -- code/scratch_scripts/361_stage11.py

冻结版到运行版的差异（U36 已解决）：26e3f817 不在 git，但从本会话转录重放了 10-26T17:15Z 至 10-27T07:12Z 之间所有写 361_stage11.py 的操作（1 次 Write、13 次 python 就地替换、1 次 sed -i），逐步算 sha256：第 5 步之后 = 26e3f817（随即重跑 freeze），中间经过 8624a895（= git b4b76ed，作验证），第 42 步之后 = f8adb8d9（= git c86112f），两端与记录的 sha 精确吻合。diff 共 239 行，**只加了补充 1 的单元（E1t patch_title、E2n knockout_name、E4 titlepos、probe_vocab）和调度 / 自检逻辑**（smoke 改为 PASS/WARN 继续、ERROR 才 stop_queue，单元认领，只在 GPU 分区停池）；patch hook 加了 ('at', pos) 分支、ko_mask 加了默认 None 的参数，原 E1 patch / E2 knockout / chain 三个函数的计算代码没有改动。重放得到的两个版本和 diff 存档在 `351_stage10/reports/methods/research_scripts/s11replay/`（v_005_26e3f817.py、v_042_f8adb8d9.py、diff_26e3f817_to_f8adb8d9.diff）。 ⟶ 出处 附录 I U36 · research_scripts/s11replay/ 复核过

0. 共同设定（E1/E1t/E2/E2n 样本与打分；本节不是独立实验）

项目	实际做法	出处
样本	只用 target、control 都 ≥30 篇的引用档（10 档）；每档每组随机抽 min(60, n_t, n_c) 篇（numpy default_rng(11)）；共 593 target + 593 control = 1,186 篇；9 档各 120 篇，513-1024 档 106 篇；按 citation_bin、group、paper_id 排序后每 50 篇一个 shard，共 24 shard；名字 token 共 19,077 行	⟶ 出处 361_stage11.py:56, 108–117；config/protocol.json sample；config/sample.parquet（本文起草时统计）；raw/patch 合并 19,077 行
假标题	只用 false_0（匹配代价最小的一个），不对 5 个假标题平均	⟶ 出处 361_stage11.py:57 FALSE_FOR_PATCH

上下文	与 Stage 10 相同：每位作者单独，T = 真标题 prompt_ids + name[:-1]，F = false_0 prompt_ids + name[:-1]；打分位置 = 每个名字 token 的前一位置	⟶ 出处 361_stage11.py:238–241
输出打分	所有干预都在模型最终输出（block 63 输出经 unembed）处取 gold 名字 token 的名次（Stage 10 的 `score()`）	⟶ 出处 361_stage11.py:192–193 ranks(), 196–223 run_final()
标题位置	与标题字符区间（'titled "' 之后到最后一个 '" (' 之前）有重叠的 token；若标题以 "." 结尾，"." 与右引号合成一个 token '."'，该 token 算作标题 token	⟶ 出处 361_stage11.py:89–94 title_positions()；本文起草时用 tokenizer 抽查 3 篇（见 11-2）
统计	nlr = −log10(rank)；先在每篇论文内对它的所有 token（所有作者）平均，再跨论文；比值的 CI 为配对 bootstrap 论文（B=2000，seed 3611），每次重抽后计算均值之比	⟶ 出处 stage11_analyze.py:8–23

11-1 E1 替换（activation patching）：标题之后的整段尾部，true → false_0，逐层

1 问题与作用

论文信息在第几层已经全部位于"标题之后的位置"上？如果把真标题运行中这些位置在第 l 层的残差复制到假标题运行里，输出能恢复多少真假差距？

2 材料

1,186 篇样本（0 节）；层 PATCH_LAYERS = [24, 32, 40, 48, 50, 52, 54, 56, 57, 58, 59, 60, 61, 62, 63]。⟶ 出处 361_stage11.py:52

3 工具

- **复制的状态**：第 l 个 decoder block 的输出（残差流），即 `model.layers[l]` 的 forward hook 输出，与 Stage 10 的 residual 定义相同；不是注意力输出、不是 MLP 输出、不是 KV。⟶ 出处 361_stage11.py:203–209（register_forward_hook on model.layers[l]，h[e, -n:] = t）；record() 226–231（ActivationRecorder at=PATCH_LAYERS）
- 真标题运行先用 `record()` 一次记下全部 15 层的残差；假标题运行复制 15 份成一个 batch，第 i 份只在 PATCH_LAYERS[i] 层被替换。
- 真、假两个基线运行各为 batch 1（`run_final(C, T, posT)`、`run_final(C, F, posF)`）。

4 步骤

1. **对齐（freeze 时冻结）**：tail_prompt = min(common_suffix(真 ids, 假 ids), len(真)−1−max(真标题位置), len(假)−1−max(假标题位置))，即两条提示词末尾完全相同的 token 段，并截断到不进入任一提示词的标题；< 3 则报错。⟶ 出处 361_stage11.py:97–100 common_suffix(), 123–125

2. 每个作者上下文的替换长度 tail = tail_prompt + len(name) − 1，替换方式为"从末尾对齐"：`h[e, -tail:] = acts_true[l][-tail:]`。
3. **"标题之后的 token"具体是哪些**：tail_prompt 段 + 全部名字输入 token。样本中 tail_prompt = 8（478 篇）或 9（708 篇）。抽查 3 篇的解码：tail 9 = `'"', ' (', '2','0','0','7', ')?\n', 'Authors', ':'`；tail 8 = 标题以 "." 结尾、右引号并入标题最后一个 token `."'` 时的 `' (', 4 个年份数字, ')?\n', 'Authors', ':'`。在这 3 篇里 tail_prompt 正好等于真提示词中最后一个标题 token 之后的全部 token。名字输入 token 是 name[0..n−2]（最后一个名字 token 不作为输入），因此**包含名字位置**，所有打分位置都在被替换的区间内。⟶ 出处 `config/sample.parquet tail_prompt 分布；本文起草时 tokenizer 解码 sample 第 0、300、700 行；361_stage11.py:240`
4. 同一批次里每份在各自一层被替换，向后正常计算到 block 63，取名字位置的输出 logits 并打分。
5. X6（Stage 13）为 false_1…false_4 重算对齐时使用同一公式：`do_xf_patch()` 中 `tails.append(min(s11.common_suffix(pt, pf), len(pt) - 1 - max(json.loads(r.title_pos)), len(pf) - 1 - max(tf)))`，其中 `tf = s11.title_positions(c.tok, pfx, pf)`，上下文由 `ctx11()` 提供（Stage 11 的样本、标题位置和条件提示词）。⟶ 出处 `~/jlens_work/13_stage13/381_stage13.py:329–337 ctx11(), 340–354 do_xf_patch()（公式在 350 行）；git 6d667ab`

5 公式

对每篇论文 p，X_p = 该论文全部名字 token 的 nlr 平均（X ∈ {true, false, patched(l)}）。
恢复率 recovery(l) = mean_p[P_p(l) − F_p] / mean_p[T_p − F_p] （先 token 在论文内平均，再跨论文平均，最后两个均值相除，不是逐篇比值的平均）
gain(l) = mean_p[P_p(l) − F_p]
CI：配对 bootstrap 论文（同一组重抽样本同时用于分子和分母）。target − control：恢复率之差（两组独立 bootstrap，不分档）；gain 之差另用档内合并 `strat_diff`。 ⟶ 出处 `stage11_analyze.py:52–74 patch_section(), 20–29 boot_ratio()/boot_ratio_diff(), 32–41 strat_diff()`

6 对照

- L63 替换 = 输出本身被替换，必须恰好恢复 1（自检）。全量数据中 L63 名次与真运行 19,077/19,077 行相同。⟶ 出处 `本文起草时由 raw/patch/*.npz 计算 (rank_patch[:, L63] == rank_true).mean() = 1.0`
- 真运行名次与 Stage 10 真标题 final 名次一致（smoke：1.00），假运行与 Stage 10 false_0 一致（1.00）。⟶ 出处 `reports/smoke.json checks.patch_true_matches_stage10, patch_false_matches_stage10`
- E1t（只换标题最后一个 token）作为位置对照。

7 结果

组	L24	L40	L48	L50	L52	L54	L63	真−假（分母）
all（1,186）	0.077	0.291	**0.825** [0.807, 0.842]	0.932	**0.996** [0.989, 1.003]	1.001	1.000	0.316
target（593）	0.070	0.267	0.803 [0.784, 0.824]	0.911	0.986 [0.979, 0.994]	1.001	1.000	0.418
control（593）	0.092	0.337	0.866 [0.835, 0.903]	0.974	1.016 [1.002, 1.031]	1.003	1.000	0.214

即"L48 恢复 83%"= all 0.825；"L52 起 100%"= all L52 0.996、L54 起 0.998–1.001（target 到 L54 才到 1.00）。gain 的 target−control（档内）从 L52 起约 +0.19 到 +0.20。 ⟶ 出处 `361_stage11/summary/E1_recovery.csv`（列 `group, layer, recovery, rec_lo, rec_hi, true_minus_false`）；`E1_target_minus_control.csv`（`gain_tc`）

8 局限与质疑

- 只用 false_0 一个假标题 → Stage 13 X6 用 false_1…4 重做，时间点对 5 个假标题都成立（L48 各标题 0.788–0.803、L52 0.984–0.990，target）。⟶ 出处 `381_stage13/reports/x6_report.md`；`PROGRESS.md 2026-10-29 00:55`
- 替换区间同时包含年份、"Authors:" 和名字位置，因此"恢复"不能单独归于名字位置；单位置替换（P1）在 Stage 12 做。
- 替换后的运行是 batch 15，假基线是 batch 1，GPU 批量 GEMM 的舍入不同，会给 $\bar{P}-\bar{F}$ 带来噪声；PROGRESS 估计约 0.01（相对 ~0.3 的效应）。⟶ 出处 `PROGRESS.md 2026-10-27 05:20` 条
- 替换按"从末尾对齐"，真假提示词标题长度不同，被复制状态的绝对位置（RoPE 位置）与目标位置不同。
- 约 10% 论文的真假差距 $|\text{gap}| < 0.05$、10% 为负；比值取"均值之比"是稳定的，逐篇比值的平均会被极端值拉高（L52 逐篇中位数 0.988，逐篇均值 1.31）。⟶ 出处 `PROGRESS.md 2026-10-28 13:05 Stage 11 review item 5`

9 状态

已完成 复核过 （X6：5 个假标题；R0-only；团队聚类 CI）

11-2 E1t 只替换标题最后一个 token（amendment 1）

1 问题与作用

论文信息是否已经在标题位置（最后一个标题 token）上？如果是，从中间层替换这一个位置就应能恢复。

2 材料

同 E1 样本；PT_LAYERS = [8, 16, 24, 32, 40, 44, 48, 52, 56, 60, 62, 63]。⟶ 出处 361_stage11.py:58；config/protocol_amendment_1.json "E1t patch_title"

3 工具

同 E1 的 block 输出 hook，但用 `('at', position)` 模式只写一个位置：`h[e, jf] = acts_true[l][it]`。⟶ 出处 361_stage11.py:206, 308–326

4 步骤

1. it = 真提示词标题位置的最后一个；jf = 假提示词标题位置的最后一个（两者绝对位置一般不同）。
2. 真运行记录 12 层残差；假运行复制 12 份，第 i 份在 PT_LAYERS[i] 层把位置 jf 换成真运行位置 it 的残差。
3. 输出处对全部名字 token 打分。

5 公式

同 E1 的 recovery，但**分母和分子里的"假"基线换成同一批次中 L63 那一份**（L63 替换标题位置后，下游已没有任何层能读它，所以它就是一次未替换的假运行）；原 batch-1 假运行保存为 false_batch1。⟶ 出处 stage11_analyze.py:56–60

6 对照

L63 必须等于假运行（自检）；E1 的真运行与 E1t 的真运行必须相同（smoke 1.00）。

7 结果

all：L8 0.119、L16 0.145、L24 0.142、L32 0.136、L40 0.139、L44 0.107、L48 0.069、L52 0.027、L56 0.004、L62 0.001。target 与 control 形状相同（control 略高，例如 L24 0.175 vs 0.125）。⟶ 出处 summary/E1t_recovery.csv

8 局限与质疑

- **GPU 自检未达阈值（WARN）**：smoke 中 L63 那一份与 batch-1 假运行名次相同的比例只有 0.7037（54 行中 38 行；阈值 0.99）。全量数据中为 59.4%（19,077 行），平均 |Δlog10 rank| 0.0105。原因判断：批量 12 与批量 1 的 bf16 GEMM 舍入不同；CPU 真权重检查（作业 11396804）中 E1t L63 == false 完全精确。处理：按用户规则记 WARN 继续计算，分析改用同批 L63 作基线。⟶ 出处 reports/smoke.json checks.patch_title_L63_equals_false = 0.7037；smoke/raw/patch_title/shard_000.npz（54 行，38 相等）；raw/patch_title 全量（本文起草时计算）；logs/realcheck_11396804.log 第 20 行 "PASS E1t patch at L63 == false (exact)"；stage11_analyze.py:56–60

- 只换一个 token；论文信息可能分散在全部标题 token 上，所以低恢复不等于标题位置没有信息（E4b probe 另测）。
- 复制的向量来自位置 it、放到位置 jf，位置编码不同（RoPE 在注意力里加，残差本身不含显式位置向量）。

9 状态

已完成 （含 WARN 记录）

11-3 E2 注意力阻断（knockout）：标题之后的所有位置不能注意标题 token

1 问题与作用

模型在哪几层从标题读取论文信息？从第 l 层起禁止读标题，若输出不再受影响，说明到第 l 层时需要的信息已被搬走。

2 材料

同 E1 样本，**只在真标题运行上做**；KO_LAYERS = [0, 24, 32, 40, 48, 50, 52, 54, 56, 58, 60, 62, 64]，**64 = 什么都不挡**，但走同一套显式 mask 路径，作为基线。⟶ 出处 `361_stage11.py:53`

3 工具

- **方法**：显式加性 attention mask。对每个 decoder layer 注册 `register_forward_pre_hook(..., with_kwargs=True)`，把 `kwargs['attention_mask']` 替换为形状 [B, 1, T, T] 的张量：允许处为 0，禁止处为 `torch.finfo(bf16).min`（不是 −inf）；第二维为 1，因此**对所有注意力头生效**。允许集合 = 因果下三角 ∧ ¬阻断块。⟶ 出处 `361_stage11.py:210–216, 254–270 ko_mask()`
- 实现为 transformers 5.19.0 的 sdpa 注意力；传入显式 mask 时 sdpa 走 is_causal=False 且不走 GQA 快路径（与不加 mask 的 Stage 10 前向不是同一内核）。⟶ 出处 `logs/realcheck_11396804.log "attn sdpa"；…/transformers/integrations/sdpa_attention.py:37, 136`

4 步骤

1. **挡哪些位置**：query = 从 max(标题位置)+1 到序列末尾的每个位置（右引号（若单独成 token）、" ("、年份、")?\n"、"Authors"、":"、全部名字输入 token）；key = 全部标题 token 位置。query→key 这些格子被置为 finfo.min。标题前缀（"Who are the authors of the paper titled \""）仍可被注意，标题 token 之间的注意也不动。⟶ 出处 `361_stage11.py:258–260`
2. **从第 l 层到哪层**：从第 l 层到最后一层 L63（含）。batch 中第 i 份在 layer ≥ KO_LAYERS[i] 时使用阻断 mask，否则使用普通因果显式 mask；l = 64 那一份所有层都不阻断。⟶ 出处 `361_stage11.py:262–269（key = tuple(layer >= k for k in ko_layers)）`
3. 一个 batch 13 份同时前向，输出处对名字 token 打分。

5 公式

K_p(l) = 论文 p 在"从 l 层起阻断"下的 nlr 均值；T_p、F_p 取自 E1 的 batch-1 真/假运行（按 row_id 合并）。

损失比例 loss(l) = mean_p[K_p(64) − K_p(l)] / mean_p[T_p − F_p] （同样先论文内平均，再跨论文平均，再相除；分子用同批的 l=64 基线）

drop(l) = mean_p[K_p(64) − K_p(l)]；CI 为配对 bootstrap 论文；target−control 为两组损失比之差。

⟶ 出处 `stage11_analyze.py:83–109 ko_section()`

6 对照

- l = 64 显式 mask 基线（消除"换内核"的影响）。
- smoke：从 L0 阻断必须降低分数（ko_L0_lowers_score = 0.124 > 0，通过）。
- E2n（只挡 ":" 和名字位置）。
- X7（Stage 13）：同样的阻断施加在 false_0 运行上，扣除"模型被带偏"的通用损伤。

7 结果

起始层	L0	L24	L32	L40	L48	L50	L52	L56
all	**0.831** [0.801, 0.862]	0.712	0.372	0.142	0.064	0.018	0.007	0.006
target	0.839 [0.808, 0.872]	0.708	0.357	0.139	0.066	0.020	0.006	0.003
control	0.816 [0.751, 0.885]	0.719	0.402	0.148	0.060	0.013	0.010	0.011

"从 L0 阻断损失 0.83"= all 0.831。⟶ 出处 `summary/E2_knockout.csv`（列 `group, start_layer, loss_fraction, lo, hi`）；`E2_target_minus_control.csv`

8 局限与质疑

- **GPU 自检未达阈值（WARN）**：smoke 中 l=64 基线与 Stage 10 真标题名次相同的比例 0.7037（阈值 0.75），平均 |Δlog10 rank| 0.0081（阈值 0.05，通过）。原因判断：显式 mask 换了 sdpa 内核、且 batch 13，bf16 舍入不同。CPU 真权重检查中"显式 mask 基线 == 不加 mask 的真运行（同设备）"为 1.00。⟶ 出处 `reports/smoke.json checks.ko_baseline_matches_stage10 = 0.7037, ko_baseline_mean_abs_dlog10rank = 0.0081`；`361_stage11.py:436`（注释）, 469；`logs/realcheck_11396804.log 第 11 行`
- 分子用同批 l=64 基线，分母用 batch-1 的真/假运行，两者来自不同内核。
- 阻断是"离分布"干预，可能造成与论文无关的损伤 → X7：S = T − F 后，target S(L0) = 1.000、S(L52) = 0.018，时间点不变。⟶ 出处 `381_stage13/reports/x7_report.md`（`WARN：kF64 与 Stage 11 假运行名次相同 0.5963`）
- 只挡标题 token 作为 key；标题信息若已在 L<l 层流入标题之后的位置，阻断不影响它（这正是设计要测的）。

9 状态

已完成 复核过 （X7、R0-only）

11-4 E2n 只阻断 ":" 和名字位置（amendment 1）

1 问题与作用

标题信息是直接从标题传到名字位置，还是先经过标题之后的中间 token（右引号、年份、"Authors"）？

2 材料

同 E2；KO_LAYERS 同 E2。⟶ 出处 `config/protocol_amendment_1.json "E2n knockout_name"`

3 工具

`ko_mask(..., query_from=len(pt) - 1)`：query 只从提示词最后一个 token ":" 开始（":" + 全部名字输入位置）；key 仍是全部标题 token。⟶ 出处 `361_stage11.py:329–340（query_from 在 337 行）, 259`

4 步骤

1. 同 E2，只是 query 集合缩小；右引号、" ("、年份、")?\n"、"Authors" 仍可注意标题。
2. 从第 l 层到 L63 阻断，l=64 为基线。

5 公式

同 E2 的 loss(l)。

6 对照

E2n 的 l=64 基线必须等于 E2 的 l=64 基线（smoke 1.00，CPU 精确相等）。⟶ 出处 `reports/smoke.json ko_name_baseline_equals_ko_baseline = 1.0；logs/realcheck_11396804.log 第 24 行`

7 结果

all：L0 0.153 [0.136, 0.172]、L24 0.120、L32 0.097、L40 0.073、L48 0.051、L50 0.014、L52 0.006。target/control 相近（L0 0.158 / 0.143）。⟶ 出处 `summary/E2n_knockout.csv`

8 局限与质疑

- 若右引号并入标题 token（'"'），它属于 key（被挡的对象），不属于中间 token。
- 只能说明"大部分不是直接从标题到名字"，不能指出具体经过哪一个中间位置（Stage 12 P1 单位置替换另测）。

9 状态

已完成

11-5 E3 作者连写（chain）vs 单独打分（isolated）

1 问题与作用

主分析每位作者单独打分。若全部作者连写，前面的作者是否让后面的更好猜？标题特异信号和 target−control 差是否仍在？

2 材料

全部 9,108 篇、33,811 个作者（preflight 核对）、7 个 Stage 10 条件；按 Stage 10 shard（128 篇）× 7 条件 = 504 个单元。⟶ 出处 `361_stage11.py:154–157, 614`；`PROGRESS 中 504 chain 单元`

3 工具

`chain_ids()`：ids = 提示词 + A1 的 token + [","] + A2 的 token + …，去掉最后一个 token；每个名字 token 与 Stage 10 完全同样的 token 化（" "+author），逗号 token id 必须为 [11]。读出 CHAIN_PLAN = final@63 + logit 与 jlens_public 在 L40, 48, 52, 56, 58, 60, 62。⟶ 出处 `361_stage11.py:54–55, 187–188, 286–305`

4 步骤

1. 一次前向（Stage 10 `fast_readout`），对所有名字 token 打分；输出保留 Stage 10 的 row_id，因而能与单独打分逐 token 配对。
2. preflight：每个打分位置的下一个 token 等于 gold；最长序列 ⩽ 1024。
3. 分析：作者位置 1、2、3、4+（第 4 位及以后合并）；每篇论文 × 位置 × 条件求均值。

5 公式

gain_true = chain_true − iso_true；spec = true − mean(false_0..4)（chain 与 iso 各自算）；gain_spec = chain_spec − iso_spec；target−control 用档内合并。⟶ 出处 `stage11_analyze.py:226–260`

6 对照

第 1 位作者连写与单独应相同（smoke：final 名次相同 0.833，平均 |Δlog10 rank| 0.0029；CPU 同设备 1.00）。

7 结果

final：作者 2 位 gain_true target +0.173、control +0.118；4+ 位 +0.234 / +0.183。标题特异 target：位置 4+ 从单独 0.421 降到连写 0.260。target−control（档内，连写）：2 位 +0.151、3 位 +0.124、4+ 位 +0.114。⟶ 出处 `summary/E3_chain.csv ( readout=final )`

8 局限与质疑

- 连写顺序固定为标注包作者顺序；"谁和谁是合作者"（换成真实合作者）没有在模型内部检验。
- smoke 第一作者一致率 0.833 > 阈值 0.75（通过），差异同样归因于序列长度不同导致的 bf16 舍入。

9 状态

已完成

11-6 E4 标题位置的 lens 读出（amendment 1）

1 问题与作用

在标题最后一个 token 上用 lens 读，能否读到这篇论文的作者（中间层）？

2 材料

全部 9,108 篇；条件 true、false_0…false_4（不含 delete）；只跑提示词（不接名字），每篇一次前向；6 条件 × 72 shard = 432 单元。⟶ 出处 `361_stage11.py:62 TP_CONDS, 355–384 do_titlepos()`

3 工具

TP_PLAN = logit 与 jlens_public 在 TP_LAYERS = [16, 24, 32, 40, 44, 48, 52, 56, 58, 60, 62] + final@63。
⟶ 出处 `361_stage11.py:59–60`

4 步骤

1. 两个位置：标题最后一个 token（title）和提示词最后一个 token ":"（colon）。
2. gold：每位作者的第 1 个 token（name[0]，kind = first）和姓的第一个 token（kind = surname，若存在）。对同一位置的同一读出 logits，分别求每个 gold 的名次。
3. true 与 false_0 两个条件另存 PROBE_LAYERS 层的残差（fp16）供 E4b 使用。

5 公式

每篇论文 × kind：对作者平均 nlr；spec = true − mean(false_0..4)；target、control 各自 bootstrap 均值；target−control 档内合并。⟶ 出处 `stage11_analyze.py:112–136`

6 对照

colon 位置的 final@63 必须等于 Stage 10 第 1 个名字 token 的名次（smoke 1.00、|Δ| 0）；同一 gold 在 5 个假标题下作对照。

7 结果

kind = first、title 位置的 target−control：logit 各层 −0.014 到 +0.025（CI 都含 0）；jlens_public L16–L56 −0.029 到 +0.036（CI 都含 0），L60 +0.037 [0.015, 0.060]、L62 +0.035。target 自身 spec 最大约 0.03（public L60–62）。⟶ 出处 `summary/E4_titlepos_lens.csv ( kind=first, position=title )`

8 局限与质疑

- title 位置的 final@63 是模型在该位置预测下一个标题 token/右引号的分布，与作者无关，仅作参照。
- 一个位置的 lens 读出只用于名字第一个 token，不读整名。

9 状态

已完成

11-7 E4b 标题位置线性 probe（Stage 11 原版）

1 问题与作用

lens 读不出，但标题最后一个 token 的残差里是否线性地含有这篇论文的作者信息？

2 材料

- **输入 X**：**真标题运行**中、标题最后一个 token、第 l 层的 block 输出残差（PROBE_LAYERS = [16, 24, 32, 40, 44, 48, 52, 56, 60]），fp16 存储，每篇 1 个 5120 维向量；共 9,108 篇。
- **目标 Y**：同一**真标题运行**中、":" 位置的 L63 残差（block 63 输出，未经 final norm），5120 维。也就是说 probe 学的是"从标题位置的状态预测模型自己在 ':' 处的最终状态"，目标不是 gold token。
- **评分词表**：probe_vocab 单元存的 final norm 权重（RMSNorm，eps = 1e-6）和 lm_head 中"队列内所有作者第 1 个 token ∪ 所有姓的第一个 token"的行，共 4,314 行。
- **测试输入**：同一篇论文 false_0 运行（借来的标题、年份不变）在同一位置、同一层的残差。

⟶ 出处 `361_stage11.py:61, 343–352 probe_vocab, 377–383 ; stage11_analyze.py:150–163 ; raw/probe_vocab npz ( eps 1e-06 , lm_head_rows (4314, 5120) )`

3 工具

numpy 闭式 ridge 回归；5 折交叉验证。⟶ 出处 `stage11_analyze.py:150–202 probe_section()`

4 步骤

1. **折**：`fold = np.random.default_rng(11).integers(0, 5, len(pid))`，按论文随机独立分配（各折大小不强制相等），所有层共用。⟶ 出处 `stage11_analyze.py:164`

2. **标准化**：μ、σ 在**全部 9,108 篇**的真标题 X 上计算（含测试折；σ 加 1e-6）。ym = 训练折 Y 的均值。⟶ 出处 `stage11_analyze.py:176, 181`
3. **λ（ridge alpha）的选取**：候选 α ∈ {0.1, 1.0, 10.0}，实际惩罚 = α · tr(G)/d · I（$G = A^TA$，A = 训练折标准化 X，d = 5120）；**只在第 1 折（k=0）上**用该折留出集的 MSE（预测 Y vs 真 Y）选 α，之后 5 折都用它；每层单独选。结果所有层都选中 α = 1.0。⟶ 出处 `stage11_analyze.py:182–188`；`summary/E4_probe.csv 列 alpha`
4. $W = (G + \alpha \cdot \mathrm{tr}(G)/d \cdot I)^{-1} A^T(Y_tr - ym)$；对测试论文分别用其真标题 X 和 false_0 X 做预测（同一个 W、同一个 μ、σ）。
5. **预测要经过 final norm + unembedding**：$h = \hat{y} / \mathrm{sqrt}(\mathrm{mean}(\hat{y}^2) + \mathrm{eps}) \odot w_norm$；logits = h · W_vocab^T（只算 4,314 个 token）。⟶ 出处 `stage11_analyze.py:165–166`
6. **打分的是第一个名字 token，不是全部 token**：gold = 该论文每位作者的第 1 个 token（kind = first，即 " "+author 的 name[0]；73% 的作者以首字母开头，此时 gold 就是首字母 token），在 4,314 词表内求名次 1 + #{logit > gold logit}，取 −log10，再对该论文的作者平均。姓的第一个 token 虽在词表里，但不作为 gold。⟶ 出处 `stage11_analyze.py:163, 167–171`

5 公式

$score(p, x) = mean_{a \in authors(p)} -\log10(1 + \#\{v \in V: \ell_v(x) > \ell_\{g_a\}(x)\})$，$\ell(x) = RMSNorm(probe(x)) \cdot W_V^T$

spec(p) = score(p, X_true) − score(p, X_false_0)（减的是 **false_0 一个**假标题，不是 5 个假标题平均；E4 lens 读出那里才是 5 个平均）

target−control：档内合并（≥30/档，B=2000）。⟶ 出处 `stage11_analyze.py:189–195`；对比 `E4 lens 的 spec 在 122 行（5 个平均）`

6 对照

false_0 输入（同 probe、同 gold）；Stage 12 X1 加了置换 gold 和只用假标题训练两种对照（11-8）。

7 结果

spec target：L16 +0.043、L24 +0.047、L40 +0.066、L52 +0.080、L56 +0.082；control 约低 0.01–0.02；target−control：L16 +0.019 [0.005, 0.033]、L24 +0.024、L32 +0.035、L56 +0.039 [0.022, 0.056]。⟶ 出处 `summary/E4_probe.csv`

8 局限与质疑

- 按论文随机分折，同一作者团队的论文可能分在训练和测试两边。
- **标题复用泄漏**：测试论文 A 的 false_0 标题是另一篇论文 D 的真标题；若 D 在训练折，probe 训练时已见过这个标题作为输入。这一问题在 Stage 13 复核（11-8）。
- μ、σ 用了全部论文（含测试折）的 X，是轻微的信息泄漏（只影响标准化，不涉及 Y）。Stage 13 复核脚本沿用同样写法（`10_stage10/stage10_extra_r0r1.py:244`）。2026-10-30 已改为"只在训练折内拟合标准化"重算，结论不变，见 11-8。

- gold 只用第 1 个 token，多为首字母；而 10-6 显示首字母几乎不带标题特异信号，所以此 probe 测量的是信号最弱的那部分 token。
- 目标 Y 是 ":" 处的状态（预测第一作者的第 1 个 token），却用来给所有作者的第 1 个 token 打分。
- 输入取自"含年份的提示词"的前向，但在标题最后一个 token 处年份尚未出现（因果），所以输入不含年份。

9 状态

已完成；结论措辞已被 11-8 的复核版取代（数值层面"目标组更高"保留）。

11-8 E4b 的对照与复核（Stage 12 X1 对照；Stage 13 按作者团队分折 + 剔除标题复用）

1 问题与作用

E4b 的信号是"这篇论文"的，还是"这类标题配这类名字"的泛泛关联？是否来自同一作者团队跨折或同一标题跨折的泄漏？

2 材料

同 11-7 的残差和词表。Stage 12 X1 只做 L24、L40、L56；Stage 13 对 E4b 全部 9 层和 X1 三层都重算 4 种配置（paper/group 分折 × 剔除/不剔除）。

3 工具

- Stage 12 `probe_controls()`： ⟶ 出处 ~/jlens_work/12_stage12/stage12_cpu.py:111–162
- Stage 13 `stage13_e4b_group.py`（作业 11443587，short，16 CPU / 96G）：原样载入 `stage10_extra_r0r1.probe`（与 Stage 11 预测相同）和 `stage12_cpu.probe_controls` 的源码，只把分折那一行替换掉，并在形成每篇 spec 之后注入两行（记录 + 可选剔除）。⟶ 出处 ~/jlens_work/13_stage13/stage13_e4b_group.py:1–10, 43–61 ; stage13_e4b_group.sbatch ; git 0dde4de
- 团队分折 `group_folds(pid, k=5, seed=13, donors=None)`。⟶ 出处 ~/jlens_work/13_stage13/stage13_cpu.py:80–109

4 步骤

1. **置换 gold 对照**（X1）：probe 不变，把评分用的 gold 换成随机另一篇论文的作者（perm_any），或同领域随机另一篇（perm_field），seed 3712。⟶ 出处 stage12_cpu.py:123–127, 150–151
2. **"只用假标题训练"**（X1 ftrain）：训练输入 = 训练折论文的 **false_0 运行**标题最后一个 token、第 l 层残差（XF[tr]）；训练目标 = 同一 **false_0 运行**中 ":" 位置的 L63 残差（YF[tr] = Cf[:, −1]）；

μ、σ 取自训练折的 XF；α 固定 1.0。测试时对测试论文的真标题 X 和 false_0 X 各预测一次，gold 仍是测试论文自己的作者第 1 个 token。⟶ 出处 `stage12_cpu.py:118–119, 138, 142–148`

3. X1 中"probe (as in Stage 11)"一行：真运行训练，但 α 固定 1.0、μ/σ 取自训练折（与 Stage 11 原版略有不同）。⟶ 出处 `stage12_cpu.py:111, 143–145`
4. **团队分析**：两篇论文只要共享任一作者全名（NFKD → ASCII、小写、去非字母、至少 2 个词；同名者也合并，偏保守）就并成一组（并查集）；组按大小从大到小（同大小按 seed 13 打乱后的顺序）整体放入当前负载最小的折。此处未传 donors，因此组内不按供体合并。⟶ 出处 `stage13_cpu.py:80–109` ; `stage13_e4b_group.py:26`
5. **剔除标题复用（逐条规则）**：只考虑 false_0 的供体（DONOR = false_titles.parquet 中 k = 0）。对测试折 k 中的论文 A：
   - 真运行训练的 probe（"probe (as in Stage 11)"、E4b 原版、两种置换 gold 对照）：若 A 的 false_0 供体 D 在 F 中且 F[D] ≠ k（D 在训练折）→ 剔除。理由：A 的假标题 = D 的真标题，已作为训练输入出现过。
   - 假运行训练的 probe（名称含 "FALSE-title"）：若存在训练折论文与 A 共用同一个 false_0 供体（同一假标题已作为训练输入出现）→ 剔除；或者存在训练折论文的 false_0 供体就是 A 本身（A 的真标题曾作为别人的假标题出现在训练里）→ 剔除。
   - 剔除的方式是把该论文的 spec 置为 NaN，然后照常做 bootstrap 和档内合并。

   ⟶ 出处 `stage13_e4b_group.py:21, 30–41 leak_mask(), 55–59`
6. λ 规则：E4b 部分沿用 Stage 11 规则（第 1 折留出 MSE 在 {0.1, 1, 10} 中选，按新折重新选；4 种配置各层均为 1.0）；X1 部分固定 1.0。⟶ 出处 `E4b_4configs.csv 列 alpha` ; `stage12_cpu.py:111`
7. sanity：paper / none 配置必须复现已发表数（E4b 最大差 0.0000；X1 对照最大差 0.0080，来自置换与 bootstrap 的随机序列不同）。⟶ 出处 `381_stage13/reports/e4b_split_compare.md 最后一行`

5 公式

同 11-7：spec = score(X_true) − score(X_false_0)；spec_all = 全部论文 spec 的 bootstrap 均值；tc_pooled = 档内 target−control。

6 对照

两种置换 gold（应为 0）；4 种配置互为对照（paper vs group、剔除 vs 不剔除）。

7 结果

样本量（团队分折 + 剔除）：真运行训练的 probe 在全部 9,108 篇中保留 **1,863** 篇（target 896、control 499、excluded 433、guard 35）；target−control 比较用 896 + 499 = 1,395 篇（R0/R1 拆分：target 788 + 108，control 491 + 8）；假运行训练版本剔除更严，保留 1,568 篇（target 810、control 399）。paper 分折 + 剔除：1,850 篇（target 945、control 447）。剔除比例约 0.80（真运行训练）和 0.83（假运行训练，团队分折）。⟶ 出处 `381_stage13/summary_cpu_group/perpaper_probe_spec.parquet`

（excl=none, dropped=False 计数，本文起草时按 group_tight 统计）；E4b_4configs.csv 列 T0_n/T1_n/C0_n/C1_n；reports/e4b_split_compare.md 第 3–10 行；PROGRESS.md 2026-10-28 17:55

层	tc：paper / 不剔除	tc：group / 剔除（复核版）	spec_all probe（group / 剔除）	spec_all 假标题训练（group / 剔除）	置换 gold（group / 剔除）
L24	+0.024 [0.010, 0.038]	+0.052 [0.017, 0.088]	+0.050 [0.039, 0.062]	+0.050	−0.002 / +0.001
L40	+0.032	+0.065 [0.027, 0.102]	+0.066 [0.053, 0.079]	+0.057	−0.004 / +0.006
L56	+0.039	+0.075 [0.038, 0.115]	+0.080 [0.068, 0.094]	+0.074	−0.003 / −0.005

复核版 target−control 在 L24–L60 为 +0.036 到 +0.078，CI 都在 0 以上；L16 为 +0.021 [−0.011, 0.054]（含 0）。⟶ 出处 381_stage13/reports/e4b_split_compare.md（两张表）；summary_cpu_group/E4b_4configs.csv、X1controls_4configs.csv

只在训练折内拟合标准化的重算（2026-10-30，作业 11500917，CPU；tiny 测试 11500538）：原版 E4b probe 的 μ、σ 用全部 9,108 篇（含测试折）计算（`10_stage10/stage10_extra_r0r1.py:244`，Stage 11 原版同样在 `stage11_analyze.py:176`）；改为每折只用训练折（含第 0 折的 α 搜索），折、剔除规则、α 网格、打分与 bootstrap 种子都不变。X1 对照本来就在训练折上标准化（`stage12_cpu.py:143`），重跑后逐位复现。结果：E4b target−control（合并）最大差 0.0020，全部论文的标题特异信号最大差 0.0075；36 个（配置, 层）格子中 α 没有变化；**确认版（团队折 + 剔除标题复用）没有任何"区间是否含 0"的翻转**：全部论文信号 L40 +0.066 → +0.061、L56 +0.080 → +0.075，区间都 > 0；target−control 差 ⩽ 0.0015，**L44 下界变为 +0.000（边界）**（+0.0345 [+0.000, +0.071]）。唯一的翻转在确认版之外（团队折、不剔除、L16）。建议正文采用新值并注明此重算。⟶ 出处 381_stage13/reports/e4b_trainstd_compare.md（"Max |new - orig|"行与 Confirmatory 两张表）· 381_stage13/summary_cpu_trainstd/compare_E4b.csv、compare_E4b_allpapers.csv · 13_stage13/stage13_e4b_trainstd.py · PROGRESS.md 2026-10-30 15:40 条 复核过

8 局限与质疑

- 剔除后只剩约 20%，CI 明显变宽；剩下的论文不是随机子集（其供体恰好落在同一折）。
- 只剔除 false_0 的供体关系；false_1…4 不进入 probe，因此无关。
- 团队分折没有把供体并入同组（donors=None），供体泄漏完全靠剔除规则处理。
- 原 6.12 表中"只用假标题训练更强"（L24 +0.068、L40 +0.081、L56 +0.094）来自泄漏，复核后为 +0.050 / +0.057 / +0.074，与普通 probe 相当或略低。作废 （原"更强"的说法）⟶ 出处 报告 6.12 表注；PROGRESS.md 2026-10-28 13:30
- 11-7 列出的 gold 只用第 1 个 token 的设计问题在复核中未改；μ/σ 全样本的问题已于 2026-10-30 按训练折标准化重算（见上），结论不变。

9 状态

复核过 （团队分折 + 剔除标题复用为确认性配置；作业 11443587；训练折标准化重算 11500917）

11-9 GPU smoke、CPU 真权重检查与 preflight（质量控制）

1 问题与作用

在花 GPU 前确认代码路径正确（L63 自检、基线一致、与 Stage 10 的名次一致），并在第一块 GPU 上做一次小规模全流程自检。

2 材料

smoke：样本中每组前 2 篇（共 4 篇，54 个名字 token）跑 patch / knockout / patch_title / knockout_name；另取 2 篇跑 chain 与 titlepos；写入临时根 `361_stage11/smoke/`。CPU realcheck：样本中名字 token 最少的 1 篇（2 个 token），真 Qwen3-32B 权重 + 真 public lens，CPU bf16。⟶ 出处 `361_stage11.py:416–465` ; `stage11_cpu_realcheck.py:1–30` ; `smoke/raw/*/shard_000.npz`

3 工具

`smoke_run()`（第一个领到 GPU 的 worker 运行，其他等待最多 30 分钟）；`stage11_cpu_realcheck.py`（作业 11396804，d0146，1,582 s）；`preflight`（CPU）。

4 步骤

1. preflight：protocol 与 Stage 10 输入未变；样本数与每档平衡；标题位置解码回标题、tail 长度可复现；chain ids 每个打分位置预测的就是 gold；chain 最长 ≤1024；33,811 个作者上下文齐全；lens 含 chain 层。⟶ 出处 `361_stage11.py:587–616`
2. GPU smoke（2026-10-27 05:17，作业 11363847，d1029 A100-80GB）：计算 15 项检查并与阈值比较；status = PASS / WARN / ERROR。⟶ 出处 `reports/smoke.json ( created 2026-10-27T05:17:52 ) ; PROGRESS.md 2026-10-27 05:20`
3. 2026-10-27 用户规则：数值一致性检查未达阈值只记 WARN、保留数值和阈值、继续计算；只有代码崩溃（ERROR）才 `stop_queue`（scontrol hold，不取消）。⟶ 出处 `361_stage11.py:475–481, 485–496` ; `PROGRESS.md 2026-10-27 03:10`

5 公式

各项检查 = 两组名次逐行相等的比例，或平均 |Δlog10 rank|；patch_recovery_L63 = E1 恢复率公式在 L63 的值。

6 对照

CPU realcheck 在同一设备上比较（排除 GPU 内核差异），用于判断 GPU WARN 是舍入还是 bug。

7 结果

检查	值	阈值	判定
patch_true_matches_stage10 / patch_false_matches_stage10	1.000 / 1.000	⩾0.9	通过

patch_L63_equals_true；patch_recovery_L63	1.000；1.000	⩾0.99；|·−1| <0.02	通过
ko_baseline_matches_stage10	**0.7037**（54 行中 38）	⩾0.75	**未达（WARN）**
ko_baseline_mean_abs_dlog10rank	0.0081	⩽0.05	通过
ko_L0_lowers_score	0.124	>0	通过
chain_first_author_matches_stage10；mean |Δ|	0.833；0.0029	⩾0.75；⩽0.05	通过
patch_title_true_equals_patch_true	1.000	⩾0.99	通过
patch_title_L63_equals_false（E1t 的 L63 元素）	**0.7037**（54 行中 38）	⩾0.99	**未达（WARN）**
ko_name_baseline_equals_ko_baseline	1.000	⩾0.99	通过
titlepos_colon_matches_stage10_first_token；mean |Δ|	1.000；0.0	⩾0.75；⩽0.05	通过
titlepos_has_residuals；probe_vocab_size	true；4,314	—；>1000	通过

经过：两项略低于阈值 → status = WARN，按规则继续计算。解释：E1t 的 12 份替换运行是一个 batch，假基线是 batch 1，批量 GEMM 舍入不同；E2 的基线走显式 mask 的另一个 sdpa 内核且是 batch 13。CPU realcheck 13/13 PASS，其中 "E1t patch at L63 == false (exact)" 与 "E2 explicit-mask baseline == unmasked true (same device) 1.00"，确认不是 bug。分析随之调整：E1t 改用同批 L63 元素作假基线（stage11_analyze.py:56–60）。不影响结论的依据：E1 L63 恰好恢复 1、E1t L63 恰好为 0（定义使然）。 ⟶ 出处 `reports/smoke.json`；`smoke/raw/patch_title/shard_000.npz`、`knockout/shard_000.npz`（54 行）；`logs/realcheck_11396804.log` 第 7–31 行；`PROGRESS.md` 2026-10-27 05:20、06:17

8 局限与质疑

- smoke 只有 4 篇、54 个 token；比例阈值（0.75、0.99）是在看到数据前写死的，GPU 上 E1t 的那一项在全量数据中为 59.4%（不是偶然的小样本波动）。
- E1（batch 15 vs batch-1 假基线）没有类似的同批基线，现有 raw 中也没有 E1 的同批未替换元素，无法事后计算；它的批量舍入噪声只有估计值**约 0.01，远小于主效应**（E1 L48 恢复 0.825）。**用户 2026-10-30 决定答辩前不补测**；GPU 空闲时按 X4 的做法补（同一批里加一个不替换的元素）。 ⟶ 出处 `PROGRESS.md` 2026-10-30 14:00 条 U37 计划未做

9 状态

已完成 （smoke = WARN，已披露；realcheck PASS 13/13；1,033/1,033 单元完成，分析作业 11385983）

Stage 12 总览：单元构成、共用设定、GPU 型号守卫

范围与代码

Stage 12 是 Stage 11 的后续 + 名字先验测量（2026-10-27 启动，用户原话 "run everything except a second model"）。它只读 Stage 10 的冻结 cohort（9,108 篇论文、33,811 个作者上下文、149,999 个名字 token、72 个 shard）和 Stage 11 的样本（1,186 篇），输出写到 `/scratch/zhang.zhiqi2/jlens/371_stage12/`。 ⟶ 出处 `~/jlens_work/12_stage12/371_stage12.py:1-23（docstring）· /scratch/zhang.zhiqi2/jlens/351_stage10/config/manifest.json（n_papers / n_contexts / n_tokens / n_shards）· /scratch/zhang.zhiqi2/jlens/361_stage11/config/protocol.json → sample.n=1186, n_target=593, n_control=593`

文件	作用
`12_stage12/371_stage12.py`	freeze / work / remaining / analyze；GPU 单元 patch_pos (P1)、patch_comp (P2)、prior_mc (N1)、prior_pure (N2)、alens_read (F2)；F1 作者提示 lens 拟合 `fit_step()`
`12_stage12/stage12_analyze.py`（+ .sbatch，gate 作业 11413617）	GPU 单元的统计分析 → `summary/*.csv`、`reports/stage12_report.md`
`12_stage12/stage12_cpu.py`（+ .sbatch，作业 11413574）	CPU：名字先验 A/B/C、Stage 11 探针对照 X1 → `summary_cpu/*.csv`、`reports/stage12_cpu_report.md`
`12_stage12/stage12_v100.py`（+ .sbatch）	V100 上只跑 prior 单元 + smoke parity；写好但**从未用于真实计算**（见下）
`12_stage12/stage12_keeper.py`（作业 11413494）、`stage12_work.sbatch`	队列 keeper：填满 A100 提交位（90 分钟 s12w 作业），在 lens 拟合完成前始终保留一个 s12fit

git（GitHub 镜像 `/scratch/zhang.zhiqi2/jlens-authorship-recall_update`，文件位于 `code/scratch_scripts/`）：13ae7ec（Stage 12 代码首版 = freeze 版本）、95dd94d（P1 8/9-token tail 修复）、37b8b7b（任何作业可接手拟合）、d569682（GPU 型号守卫 + Stage 13 hand-over）、663ac1c（F2 剔除 3 篇同名作者论文）、fb8be6f（Stage 12 完成：报告）。 ⟶ 出处 `git log --oneline -- '*stage12*' '*371_stage12*'`

单元总数 408 的构成

单元类型	实验	切分	单元数	行数（实测）
patch_pos	P1	Stage 11 样本 24 个 shard（每 shard 50 篇，`n11 = 24` 硬编码）	24	19,077 个名字 token（4,307 个作者上下文）
patch_comp	P2	同上 24 shard	24	19,077

prior_mc	N1	2 个提示（delete、true）× Stage 10 的 72 shard（每 shard 128 篇）	144	72,864 = 9,108 篇 × 4 选项 × 2 提示
prior_pure	N2	Stage 10 的 72 shard	72	149,999 个名字 token（全 cohort）
alens_read	F2	6 个条件（true + false_0..false_4）× 24 shard；F1 完成（`alens/lens.pt` 存在）后才开放	144	每条件 19,077
合计			**408**	

⟶ 出处 371_stage12.py:70-80 units()/all_units() · 实测：raw/patch_pos 24、raw/patch_comp 24、raw/prior_mc 144、raw/prior_pure 72、raw/alens_read 144 个 npz（2026-10-30 计数）· PROGRESS.md "2026-10-28 20:35 — [Stage 12] DONE (408/408)" F1 拟合本身不是"单元"，不计入 408；它由 `try_fit()` 在任意 A100-80GB 作业里推进。 ⟶ 出处 371_stage12.py:304-317

共用设定（P1/P2/F2 都依赖的 Stage 11 定义）

- **提示**（Stage 10 冻结）：`Who are the authors of the paper titled "{title}" ({year})?\nAuthors:`；delete 条件：`Who are the authors of the paper ({year})?\nAuthors:`。 ⟶ 出处 10_stage10/351_stage10.py:29 PROMPT · cohort/conditions/prompts.parquet（condition=='delete' 第一行实测）
- **每个作者单独成一个上下文**：输入 = 提示 token + 该作者名字 token 去掉最后一个（`T = pt + name[:-1]`），在位置 `len(pt)-1+j` 读出预测 `name[j]` 的 logits（teacher forcing），所有名字 token 都打分。 ⟶ 出处 371_stage12.py:111-112, 150-151 · 361_stage11.py:80-86 contexts_of()
- **false 标题**：patching 用 `false_0`（Stage 10 匹配的第一个假标题；年份保持目标论文的年份，只换标题）。 ⟶ 出处 361_stage11.py:57 FALSE_FOR_PATCH · 351_stage10.py:339 false_title_year='target paper year (title swapped only)'
- **对齐尾部（aligned tail）**：true 与 false_0 提示的公共后缀，截到两个提示都在最后一个标题 token 之后开始：`S = min(common_suffix(pids,fids), len(pids)-1-max(tp), len(fids)-1-max(tf))`，存为 sample.parquet 的 `tail_prompt`。实测 708 篇 S=9、478 篇 S=8。 ⟶ 出处 361_stage11.py:117-127 · 361_stage11/config/sample.parquet tail_prompt 计数
- **标题 token 位置**：与 `titled "` 和最后一个 `" (` 之间字符重叠的 token。 ⟶ 出处 361_stage11.py:89-94 title_positions()
- **打分**：rank = 1 + #(logit > gold logit)（并列不算更大），指标 log10(1/rank) = −log10(rank)。 ⟶ 出处 351_stage10.py:442-448 score() · 361_stage11.py:192-193 ranks() · stage12_analyze.py:8 nlr()
- **模型**：Qwen/Qwen3-32B，revision 9216db5781bf21249d130ec9da846c4624c16137，bf16，64 层，d_model 5120。 ⟶ 出处 351_stage10.py:27-28 · HF config.json
- **统计**：所有 bootstrap B=2000，按论文重抽；GPU 分析 seed 3713，CPU 分析 seed 3712。target−control 一律"按引用 bin 分层"：只用 target、control 各 ≥30 篇的 bin，bin 内差值按 $n_t \cdot n_c/(n_t+n_c)$ 加权。 ⟶ 出处 stage12_analyze.py:7, 12-30 · stage12_cpu.py:11, 15-28

GPU 型号守卫与 V100 smoke parity

2026-10-28 04:30 前后 A100-80GB 4.5 小时无空位，用户同意"参数、条件、数据不变即可"用别的 GPU。`work()` 里加了守卫： ⟶ 出处 371_stage12.py:287-302, 310, 326, 331 · PROGRESS.md "2026-10-28 04:30 — [Stage 12] no A100-80GB for 4.5 h" · 修改前备份 371_stage12/logs/371_stage12.py.bak_20261028_v100 · commit d569682

```
gpus = [torch.cuda.get_device_name(i) ...]
big = 每张卡 total_memory >= 70 GiB
same_arch = 每张卡名字含 'A100'
a100 = big and same_arch          # 单张 A100-80GB：全部单元 + F1 拟合
not a100 且 same_arch（如 2×A100-40GB）：全部前向单元，不拟合
not same_arch（V100、H200 等）：只做 prior_mc / prior_pure
```

任何非 "a100" 作业开始时先重算 `('prior_mc','delete',0)`（已在 A100-80GB 上算过），比较 `mean_nlr`：行一致且 max|diff| < 0.05 记 PASS，否则 WARN（只记录，不停作业），写入 `logs/v100_smoke.jsonl`。`stage12_v100.py smoke` 做同样的比较（加 prior_pure shard 0 的形状/rank≥1 检查）。 ⟶ 出处 371_stage12.py:296-302 · stage12_v100.py:22-40

实际情况：408 个单元的 npz meta 里 GPU 全是单张 `NVIDIA A100-SXM4-80GB`（实测 2026-10-30）；`logs/v100_smoke.jsonl` 不存在，只有 `logs/v100_smoke.mocktest_20261028.jsonl`（CPU mock 测试行，不是真实 GPU 结果）。V100 不可行：QOS 每作业最多 2 GPU，2×32GB < 65.5GB bf16 权重；`stage12_v100.py` "written but unused"。两个被 scontrol 改成 `gpu:a100:2` 的作业 11414108、11428778 启动时 Stage 12 已无单元，直接交给 Stage 13。 **计划未做** （V100 路径） ⟶ 出处 PROGRESS.md 2026-10-28 04:30 条 · logs/work_11414108.log、work_11428778.log 第 3 行 "nothing left to claim"

脚本 hash 注意：`config/protocol.json` 记录的 `script_sha256 = d695fa…` 是 freeze 版（= commit 13ae7ec），之后脚本改过 3 次（P1 valid 修复、any-job 拟合、GPU 守卫/hand-over），当前文件 sha256 = 070a02…。单元 npz meta 里的 `script_sha256` 是 Stage 11 脚本的 hash（613eb8…，`s11.Ctx.env` 写的），**不能**从 meta 区分是哪个 Stage 12 版本算的。比对 13ae7ec→95dd94d 和备份→当前的 diff：计算函数只有 `do_patch_pos` 在 95dd94d 改过（加 valid mask），其余改动都在 `work()` / `units()` 调度逻辑。 ⟶ 出处 git diff 13ae7ec 95dd94d -- code/scratch_scripts/371_stage12.py · diff logs/371_stage12.py.bak_20261028_v100 当前文件 · 361_stage11.py:187-188 env

P1 单个位置替换（patch_pos）

1 问题与作用

Stage 11 E1 把 false_0 运行里"标题之后的整段尾部 + 全部名字位置"的残差一次性换成 true 运行的，L52 起恢复约 100%。P1 问：这份"论文信息"具体在哪个 token 上？逐个位置只换一个，看每个位置单独能恢复多少。⟶ 出处 `371_stage12.py:5-8 docstring`

2 材料

- Stage 11 样本 1,186 篇（593 target / 593 control，10 个引用 bin，seed 11），24 shard × 50 篇；4,307 个作者上下文，19,077 个名字 token（其中 3 个上下文只有 1 个名字 token）。⟶ 出处 `361_stage11/config/protocol.json · raw/patch_pos/*.npz row_id 合计 · name_tokens.parquet 按 (paper, author) 计数`
- 两条提示：true 标题、false_0 标题（同年份）。
- 层 PP_LAYERS = [32, 40, 44, 48, 50, 52, 56]。⟶ 出处 `371_stage12.py:35`

3 工具

Qwen3-32B bf16，单张 A100-SXM4-80GB；`jlens.hooks.ActivationRecorder` 记录 decoder block 输出；替换用 `model.layers[l].register_forward_hook`（hook 在 decoder block 本身，替换的是 block 的输出 = 该层之后的残差流）。⟶ 出处 `361_stage11.py:196-223 run_final()、226-231 record()`

4 步骤

1. 对每个作者上下文：T = true 提示 + name[:-1]，F = false_0 提示 + name[:-1]；读出位置 posT / posF = 提示最后一个 token 起往后 len(name) 个位置。⟶ 出处 `371_stage12.py:111-112`
2. 跑一次 T，记录 PP_LAYERS 各层所有位置的残差 `acts[l]`；再跑一次 T 得到 true 的最终输出 rank（`rank_true`）。⟶ 出处 `371_stage12.py:114, 131`
3. 构造固定的 12 个"项目"（item），位置均为 T 坐标，用 `pF = pT − len(pt) + len(pfi)` 映射到 F：

item	精确定义（代码）	典型 token（9-token 尾部，如 2018 年）
none	不替换（同一 batch 内的未替换 false 运行，作基线）	—
p*k*, k=1..9	从提示末尾倒数第 k 个 token：`have = {f'p{S-i}': [len(pt)-S+i] for i in range(S)}`，即 p1 = 位置 len(pt)−1（最后一个提示 token），p*S* = 对齐尾部第一个 token。只替换这一个位置。	p1 = `:`；p2 = `Authors`；p3 = `)?\n`；p4 = 年份第 4 位（`8`）；p5 = 年份第 3 位（`1`）；p6 = 年份第 2 位（`0`）；p7 = 年份第 1 位（`2`）；p8 = `(`；p9 = 标题的右引号 `"`

names	`range(len(pt), len(T))`：所有名字输入位置（name[0]..name[n−2]），**不含** `:`	—
all	`range(len(T)-tail, len(T))`，tail = S + len(name) − 1：整段对齐尾部 + 全部名字位置（= Stage 11 E1）	—

尾部只有 8 个 token 的论文（478 篇；右引号和标题最后一个 token 合并，尾部从 `(` 开始）没有 p9：p9 不替换并标 `valid=False`，分析时剔除。p9 有效行 10,566 / 19,077。Qwen 的分词器把年份拆成单个数字（实测 tail_tokens）。⟶ 出处 371_stage12.py:36 MAX_TAIL=9, 118-123 · raw/patch_pos/*.npz 的 items、valid、tail_tokens（实测最常见：`["\"", " (", "2","0","1","8", ")?\n", "Authors", ":"]` 222 个上下文）

4. 元素 = 7 层 × 12 item = 84 份 F 的拷贝；元素 (l, item) 在第 l 层 block 输出上把 item 的位置换成 T 的 `acts[l][pT]`（每个元素只在一层、只换该 item 的位置）。按 48 一批分块前向（两块：48 + 36）。每个元素在最终输出上对 gold 名字 token 求 rank。⟶ 出处 371_stage12.py:124-137 · 361_stage11.py:205-209（('at', position) 分支）
5. 保存 row_id、rank_true、rank_patch[n, 7, 12]、valid、tail_tokens。⟶ 出处 371_stage12.py:140-141

5 公式

对 token t，nlr = −log10 rank。对每个 (层 l, item i)，先在论文内对有效 token 取平均，再跨论文：

```
num_p = mean_t [ nlr_patch(t, l, i) − nlr_none(t, l) ]
den_p = mean_t [ nlr_true(t)        − nlr_none(t, l) ]
recovery(l, i) = mean_p(num_p) / mean_p(den_p)        （比值的均值，不是均值的比值逐篇）
95% CI：论文重抽 2000 次，对每次重抽重算比值，取 2.5/97.5 百分位
```

注意 `none` 按层取（同一层所在的 batch 块里的未替换拷贝），分母也随层变。⟶ 出处 stage12_analyze.py:12-14 boot_ratio()、46-59

6 对照

- **none**：同一 batch 内的未替换 false_0 运行，作为每层的 0 点。
- **all**：应复现 Stage 11 E1。实测 P1 all 与 Stage 11 E1 recovery（all 组）：L32 0.143 / 0.144，L40 0.292 / 0.291，L48 0.825 / 0.825，L50 0.933 / 0.932，L52 0.996 / 0.996，L56 0.999 / 0.998。复核过 ⟶ 出处 summary/P1_patch_pos.csv（item=all, group=all）· 361_stage11/summary/E1_recovery.csv（group=all）
- 上线前 tiny 模型 CPU 干跑 14/14 PASS（含 "all = Stage 11 E1 尾部替换"、"none = false 运行"），8/9-token 修复后用一篇 8-token、一篇 9-token 论文重跑 14/14 PASS。⟶ 出处 PROGRESS.md "2026-10-27 ~15:10" 和 "~15:40" 条
- 年份在 true/false 运行里相同（年份 token 落在公共后缀里），因此年份位置本身不会造成 true/false 差异。⟶ 出处 PROGRESS.md 2026-10-28 13:05 条 "Year: identical in true and false runs" · 351_stage10.py:339

7 结果

recovery（全部论文，95% CI）：

item	L32	L40	L44	L48	L50	L52	L56
p9 右引号	+0.011	+0.024	+0.027 [+0.022, +0.033]	+0.011	+0.004	+0.002	−0.000
p5 年份第 3 位	+0.012	+0.033	+0.071 [+0.063, +0.079]	+0.062	+0.035	+0.027	−0.000
p1 `:`	+0.005	+0.008	+0.011	+0.074 [+0.065, +0.083]	+0.157	+0.172 [+0.156, +0.187]	+0.130
names	+0.094	+0.175	+0.236	+0.612 [+0.594, +0.630]	+0.726	+0.847 [+0.830, +0.864]	+0.878
all	+0.143	+0.292	+0.429	+0.825	+0.933	+0.996	+0.999

p2、p3、p4、p6、p7、p8 在所有层都 ⩽ 0.008。target / control 分组在 CSV 的 group 列。⟶ 出处 `371_stage12/reports/stage12_report.md §P1 · summary/P1_patch_pos.csv · 作业：P1 24/24 由 11413557、11413569（d1029）等在 2026-10-27 完成（PROGRESS "2026-10-27 17:28"）`

按 token 角色拆分（2026-10-30 用原始数据重算，U38 已解决）：用 `371_stage12/raw/patch_pos/shard_*.npz`（19,077 行）与 `351_stage10/cohort/name_tokens.parquet` 的 role 列，按 stage12_analyze.py:46–58 的同一估计量（先按论文平均，再算 Σnum/Σden，recovery = (nlr_patch − nlr_none)/(nlr_true − nlr_none)）重算，CPU 秒级。本手册编辑于 2026-10-30 下午重新运行脚本，数字一致。⟶ 出处 `351_stage10/reports/methods/research_scripts/u38_u40_recompute.py · 附录 I U38` 复核过

token 角色	行数	":"（p1）单独替换 L48 / L52	names 替换 L48 / L52	true − false 差距 gap
initial_first（首字母）	3,479	**0.502 / 0.917**	0.153 / 0.223	0.114
given_first（名的首 token）	1,378	**0.349 / 0.940**	0.013 / 0.036	0.293
surname_first（姓的首 token）	4,306	0.017 / 0.051	**0.567 / 0.925**	0.548
surname_cont（姓的后续 token）	4,940	0.013 / 0.040	0.810 / 0.990	0.518
全体	19,077	0.074 / 0.172	0.612 / 0.847	0.316

即名字第一个 token（首字母或名）的恢复几乎全来自 ":" 位置，姓的 token 主要来自名字位置；"全体"一行与 summary/P1_patch_pos.csv 一致。punct（标点）的 gap 只有 0.009，比值不稳定，不解读。PROGRESS 2026-10-28 14:45 的预览值（首字母 0.50/0.90、名 0.37/0.95、姓名字位置 0.55/0.92）与重算值相差 ⩽ 0.03，原预览脚本没有保存，以重算值为准。

8 局限与质疑

- **names 的因果可及范围**：名字的第一个 token 是在 `:`（p1）位置预测的，位于所有名字位置之前；因果注意力下替换 names 不可能影响第一个 token 的预测。因此"names 恢复 0.85"主要来自第 2 个及以后的名字 token，第一个 token 只能由 p 项恢复。3 个单 token 上下文的 names 项是空替换。⟶ 出处 `371_stage12.py:112, 120（推导）`
- 单位置 recovery 不可相加：多个位置之间有冗余（names 0.85 + p1 0.17 > 1 at L52）。
- p5（年份第 3 位）在 L44 达到 0.071。**只描述，不解读**（用户 2026-10-30 决定，U39）。⟶ 出处 `PROGRESS.md 2026-10-30 14:00 条 U39`
- 只用了 false_0 一个假标题；Stage 13 X6 用 false_1..4 重做了 E1（整段尾部），但没有重做单位置版本。
- 评分是 teacher-forced 下的 rank，不是生成。
- 每层只替换一层；跨层组合未测。

9 状态

已完成 24/24 shard（2026-10-27），分析 2026-10-28 19:55（gate 作业 11413617）。"all" 与 Stage 11 E1 吻合 复核过 。首次运行时作业 11371622 在 8/9-token 尾部上崩溃（np.concatenate 11 vs 12 项），修复后（commit 95dd94d）才产出现有数据。⟶ 出处 `PROGRESS.md "2026-10-27 ~15:40" 条 · logs/analyze_11413617.log 末行 "STAGE12 ANALYZED"`

P2 注意力 vs MLP 组件替换（patch_comp）

1 问题与作用

Stage 11 E1 说明论文信息在 L24–L50 间到达标题之后的位置。P2 问：是注意力把标题信息"搬"过来，还是这些位置上的 MLP 自己"算"出来？做法：只把某一层的注意力输出或 MLP 输出换成 true 运行的，其余照常。⟶ 出处 371_stage12.py:9-11

2 材料

与 P1 相同：Stage 11 样本 1,186 篇、4,307 个上下文、19,077 个名字 token；true vs false_0。层 PC_LAYERS = [8, 16, 24, 28, 32, 36, 40, 44, 48, 52, 56]；组件列表 comps = ['none'] + 每层的 attn*、*mlp*，共 23 项。⟶ 出处 371_stage12.py:37, 146 · raw/patch_comp/*.npz comps

3 工具

PyTorch forward hook 挂在 **子模块** 上：

- 注意力：`model.layers[l].self_attn`（HF `Qwen3Attention`）。其 forward 返回 `(attn_output, attn_weights)`，其中 `attn_output = self.o_proj(...)`，即 **o_proj 之后、加回残差之前**的注意力输出。hook 取/改 `o[0]`。
- MLP：`model.layers[l].mlp`（`Qwen3MLP`），输出 = `down_proj(act(gate_proj(x)) * up_proj(x))`，即 **down_proj 输出、加回残差之前**。
- Decoder block 里：`h = residual + self_attn(input_layernorm(residual))[0]`，然后 `h = h + mlp(post_attention_layernorm(h))`。所以替换后的组件输出在本层被**加回 false 运行自己的残差流**（注意力输出加在 block 输入残差上；MLP 输出加在注意力之后的残差上），下游所有层照常重算。

⟶ 出处 371_stage12.py:156-157（记录）、166-171（替换）· transformers 5.19.0 models/qwen3/modeling_qwen3.py:271-272（o_proj）、81（down_proj）、296-314（decoder layer 残差相加）。注：运行时的 transformers 版本未写入 meta，此处是当前 conda 环境的版本

4 步骤

1. 每个作者上下文 T、F、posT/posF 同 P1；`tail = S + len(name) − 1`（对齐提示尾部 + 全部名字输入位置，即"标题之后的全部 tail"，与 P1 的 all 相同）。⟶ 出处 371_stage12.py:150-151
2. 跑 T 一次，在 PC_LAYERS 每层的 self_attn 和 mlp 上挂记录 hook，存 `o[0][0, -tail:]`（注意力）和 `o[0, -tail:]`（MLP）——即最后 tail 个位置的组件输出；同时得到 rank_true。⟶ 出处 371_stage12.py:154-160

3. 把 F 复制 23 份放进一个 batch；第 e 份（comps[e] = attn/或 mlp/）在第 l 层对应子模块的输出上执行 `t[e, -tail:] = 记录值`（只改这一份、这一层、这个组件、最后 tail 个位置）；第 0 份 'none' 不改。⟶ 出处 371_stage12.py:161-174
4. 在最终输出（block 63 + final norm + lm_head）对每个名字 token 求 rank。⟶ 出处 361_stage11.py:219-222

5 公式

```
recovery(comp, l) = mean_p mean_t[nlr_patch − nlr_none] / mean_p mean_t[nlr_true − nlr_none]
（none = 同一 batch 第 0 份；CI 同 P1：论文重抽 2000 次）
```

⟶ 出处 stage12_analyze.py:68-77

6 对照

- in-batch 'none'（未替换 false 运行）作为 0 点。
- P2 没有"全替换 = 1"的内置上界检查（替换单层单组件不应恢复 1），当时也**没有做显式的 sanity 检查**（例如 L63 或"替换为自身 = 不变"；logs 下只有 P1 的 smoke 项）。
- **从数据看 hook 生效（2026-10-30 重算，U40 部分解决）**：各替换列与 none 列相比，L8–L44 平均 Δnlr 在 −0.001 到 +0.008 之间（基本是噪声），L48–L56 明显上升（+0.020 到 +0.070，如 attn48 +0.069、mlp56 +0.070），与 recovery 表一致；基线 none 列与 Stage 11 假标题运行 rank_false 逐行相同的只有 59.4%，但全部行的名次都在 2 倍以内、平均 Δnlr 0.0000、p99 |Δnlr| 0.146，P1 自己的 none 列在不同层之间也只有 59.7% 逐行相同，所以是批大小不同带来的 bf16 噪声，不是 hook 泄漏；rank_true 与 Stage 11 100% 相同。⟶ 出处 351_stage10/reports/methods/research_scripts/u38_u40_recompute.py（371_stage12/raw/patch_comp，19,077 行 × 23 列）· 371_stage12.py:146–170 · 附录 I U40 是否补做显式检查待用户决定。**未知/需确认：向 用户 确认（是否补做 P2 显式 sanity，例如 L63 或"替换为自身"，U40）**

7 结果

层	attention	MLP
L8–L44	⩽ +0.023（L24 +0.023 [+0.020, +0.028] 为最大）	⩽ +0.009
L48	**+0.231 [+0.221, +0.244]**	+0.068 [+0.061, +0.075]
L52	+0.109 [+0.101, +0.118]	**+0.178 [+0.168, +0.190]**
L56	+0.058 [+0.052, +0.064]	**+0.224 [+0.208, +0.239]**

顺序：L48 注意力先搬，L52–56 MLP 后算。⟶ 出处 reports/stage12_report.md §P2 · summary/P2_patch_comp.csv（comp, layer, group=all）

8 局限与质疑

- 只替换单层单组件；层是稀疏的（L44→L48→L52），L45–47、L49–51 未测，"L48 注意力"可能代表相邻几层。
- 替换的是组件对残差的增量，作用在 false 运行的残差上：如果 false 残差本身已偏离，相同增量的效果不同（非线性）。
- 替换范围是整段 tail（含 `:` 和名字位置），没有按位置拆分组件；"注意力从哪个源位置读"没有做（没有 head 级、没有注意力模式分析）。
- 各层 recovery 不可相加。

9 状态

已完成 24/24（2026-10-27 至 10-28；17:28 时 15/24），分析 2026-10-28 19:55。⟶ 出处 PROGRESS.md "2026-10-27 17:28" 条 · summary/P2_patch_comp.csv 时间戳

N1 多选选项先验（prior_mc）

1 问题与作用

Stage 1 的多选题（选出真作者名单）把论文分成 target/control。质疑：模型选中真名单，会不会只是因为真名单"听起来更像"（名字先验高），而不是认得这篇论文？N1 把 4 个选项都当作作者链打分，分别在不给标题（delete）和给真标题时比较。⟶ 出处 371_stage12.py:12-14

2 材料

- 全部 9,108 篇论文（Stage 10 的 72 shard）。
- 选项来自标注包 `labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/mc_team.jsonl`（9,108 行，kind=team）：每篇 4 个选项，**option 0 = 真作者名单**（`correct_index` 全部为 0，实测），option 1–3 = 另外 3 篇论文的真实作者团队（`false_sources` 记录来源论文）；名字格式全部为 `initial_surname`（如 "S. Bree"）。这里用的是规范顺序（option 0..3），不是多选题实际展示的 4 种顺序。⟶ 出处 371_stage12.py:183-187 mc_options() · mc_team.jsonl（实测 correct_index、format 计数）· 标注包 step1_README.md:53 "team (the three wrong options are the real author lists of other papers)"
- 两种提示：`delete`（`Who are the authors of the paper ({year})?\nAuthors:`）和 `true`（带真标题）。⟶ 出处 371_stage12.py:192

3 工具

`s10.fast_readout(..., [('final', 63)])`：一次前向，只取最终输出 logits。⟶ 出处 351_stage10.py:425-439 · 371_stage12.py:203

4 步骤

1. 对每篇论文、每个选项 oi=0..3：序列 = 提示 ids；对选项里每个名字 n（第 2 个起先加逗号 token `,`，id 11），把 `' ' + 名字` 分词后逐 token 追加，每个 token 的读出位置是它前一个位置。得到 "Authors: A1, A2, …" 作者链；逗号不打分。⟶ 出处 371_stage12.py:196-201 · 361_stage11.py:187-188（comma == [11]）
2. 序列超过 1000 token 跳过（实测 0 篇被跳过：72,864 行 = 9,108×4×2）。
3. 前向 `ids[:-1]`，每个名字 token 求 rank；存每个选项的 `mean_nlr`（全部名字 token 的平均 log10(1/rank)）、`first_author_nlr`、`first_token_nlr`、n_tokens。⟶ 出处 371_stage12.py:203-210

5 公式

只用 recall_level ∈ {R0, R1} 的论文（8,894 篇；其中 target 4,621、control 2,220，其余为 excluded 组，计入 "all"）。按论文、按选项透视 `mean_nlr`：

```
margin_p        = mean_nlr(option 0) − mean( mean_nlr(option 1), mean_nlr(option 2), mean_nlr(option 3) )
most_plausible_p = 1  if mean_nlr(option 0) >= max(option 1..3)  else 0     （并列算真名单赢）
报告：margin 和 most_plausible 的论文平均 + bootstrap CI；机会水平 0.25
target − control：margin 按引用 bin 分层（见总览）
Spearman( mc_team_a , margin − 该 bin 内 margin 均值 )   mc_team_a = 多选 4 种顺序中答对的次数 (0–4)
```

⟶ 出处 stage12_analyze.py:84-98 · summary/N1_prior_mc.csv n 列 · 标注包 label_README.md:27 mc_team_a 定义

6 对照

- delete vs true：同一组选项，无标题时的差别 = 纯名字先验；有标题时的差别 = 先验 + 标题。
- 机会水平 0.25（4 选 1）。
- distractor 是真实的作者团队（不是编造的名字），所以名字分布接近。

7 结果

提示	组	n	margin	真名单 most plausible 比例
delete	all	8,894	+0.004 [−0.002, +0.009]	0.255 [0.246, 0.264]
delete	target	4,621	+0.017 [+0.008, +0.024]	0.269 [0.256, 0.282]
delete	control	2,220	−0.011 [−0.023, −0.000]	0.248 [0.230, 0.266]
delete	target − control（bin 内）		+0.032 [+0.017, +0.048]	
delete	Spearman(margin, mc_team_a)		+0.045	
true	all	8,894	+0.313 [+0.305, +0.319]	0.625 [0.615, 0.635]
true	target	4,621	+0.444 [+0.435, +0.453]	0.771 [0.758, 0.783]
true	control	2,220	+0.125 [+0.113, +0.137]	0.406 [0.384, 0.427]
true	target − control（bin 内）		+0.236 [+0.218, +0.253]	
true	Spearman(margin, mc_team_a)		+0.273	

无标题时真名单在 25.5% 的论文里得分最高（≈ 机会 25%）；多选成功不是由名字"像不像"驱动。⟶ 出处 summary/N1_prior_mc.csv · reports/stage12_report.md §N1

8 局限与质疑

- 打分格式是 "Authors: A1, A2, …" 的 teacher-forced 链式 rank，不是多选题本身的字母概率；两者是不同任务。
- 名单长短不同（选项来自不同论文，作者数不同），mean_nlr 对长名单是更多 token 的平均；链后部的名字受前面名字影响（同一团队内部的共现先验）。
- Spearman 没有给 CI（代码只算点估计）。
- 名字格式是标注包的 "首字母. 姓"，Stage 10 单作者读出用的是全名。2026-10-30 全量比对（33,811 对）：两者来自同一份作者列表，每篇人数与顺序相同（9,108/9,108）；"名的首字母 + '. ' + 最后一个词"与 mc_team 精确相同的 **33,644 对（99.51%）**，另 158 对差在姓氏前缀（mc_team 保留整个前缀姓，如 'F. V. van Dam' → 'F. van Dam'），9 对是后缀或标点。⟶ 出处 `351_stage10/cohort/contexts.parquet × labels_20261015/…/mc_team.jsonl (options[correct_index])` · 附录 I U41 **复核过** （U41 已解决）

9 状态

已完成 144/144（2026-10-28；18:28 时 103/144，约 31 s/shard）。⟶ 出处 `PROGRESS.md "2026-10-28 17:28"、"18:28" 条`

N2 纯名字先验（prior_pure）

1 问题与作用

Stage 10 的 delete 条件（无标题）仍有年份和"论文作者"语境。N2 用一个完全不提论文的提示，测每个作者名字本身的先验，并与 delete、true 配对比较：target 组的优势有多少来自名字本身？ ⟶ 出处 371_stage12.py:15-16

2 材料

全部 9,108 篇、149,999 个名字 token（行 id 沿用 Stage 10 的 row_id，可与 Stage 10 true/delete 逐 token 配对）。提示原文：

```
The following is the name of a researcher.
Name:
```

（Python 字符串 `'The following is the name of a researcher.\nName:'`） ⟶ 出处 371_stage12.py:38 PURE_PROMPT · config/protocol.json pure_prompt

3 工具

`fast_readout`，读出计划 PURE_PLAN = final@63 + logit lens @ L48/56/62 + 公开 J-lens（jlens_public，557 条 Salesforce-wikitext 提示拟合）@ L48/56/62。 ⟶ 出处 371_stage12.py:39 · 351_stage10/config/lenses.json public.n_prompts=557

4 步骤

1. 每个作者：ids = `tok(PURE_PROMPT).input_ids + name[:-1]`，读出位置 = 提示最后一个 token 起 len(name) 个位置（与 Stage 10 同样的单作者 teacher forcing，只是前缀换成纯名字提示）。 ⟶ 出处 371_stage12.py:214-218
2. 每个读出求 rank，存 `final@63`、`logit@48/56/62`、`jlens_public@48/56/62`。 ⟶ 出处 371_stage12.py:219-221
3. 分析：取 Stage 10 冻结输出 `final@63` 的 true 和 delete rank（72 shard），按 row_id 对齐。 ⟶ 出处 stage12_analyze.py:108-114

5 公式

```
每篇论文：pure / delete / true = 该论文所有名字 token 的 log10(1/rank) 平均（模型输出）
组均值：直接平均（无 CI）
year_prompt = mean(delete − pure)
target − control：pure、delete、true 各自按引用 bin 分层 + bootstrap CI
```

⟶ 出处 stage12_analyze.py:115-122

6 对照

同一 token 的三个条件配对（pure → delete → true），差值逐层剥离：pure = 名字本身；delete − pure = "论文作者 + 年份"语境；true − delete = 标题。

7 结果

组	n 篇	pure	delete（无标题）	true
all	9,108	−1.229	−1.089	−0.812
target	4,621	−1.215	−1.061	−0.708
control	2,220	−1.261	−1.150	−1.004

target − control（bin 内）：pure +0.047 [+0.031, +0.062]；delete +0.087 [+0.070, +0.103]；true +0.247 [+0.229, +0.266]。由此 pure / true = 0.047/0.247 ≈ 19%，delete / true ≈ 35%（STATUS §3 的说法，是这两个数的比值）。⟶ 出处 summary/N2_prior_pure.csv（tc_pure、tc_delete、tc_true 行）· reports/stage12_report.md §N2 · STATUS.md §3 Stage 12 "≈ 19% of the true-title gap (35% with the no-title prompt)"

8 局限与质疑

- logit lens / 公开 J-lens 的 L48/56/62 读出已经算了并存在 raw 里，但 `stage12_analyze.py` 只分析了 final@63。 计划未做 （层剖面分析）⟶ 出处 stage12_analyze.py:109 只读 'final@63'
- 组均值没有 CI；"19%"是两个点估计之比，没有对比值本身做 bootstrap。
- "纯"提示仍然是英文研究者语境，不是零语境；结果依赖提示措辞，只测了一种。

9 状态

已完成 72/72（2026-10-28）。⟶ 出处 raw/prior_pure 72 个 npz

F1 作者提示 lens 拟合（alens_fit）

1 问题与作用

Stage 10 用的公开 J-lens 是在维基文本上拟合的，中间层读不出作者名。质疑：会不会是 lens 的拟合分布不对？F1 在"提示 + 真作者名单"这种作者文本上重新拟合一个 Jacobian lens，供 F2 读出。⟶ 出处 371_stage12.py:17-18

2 材料

- 40 篇论文，从 Stage 10 cohort 中 `group_tight ∈ {target, control}` 且**不在 Stage 11 样本**里的论文中随机抽取（`np.random.default_rng(12).choice(len(pool), 40, replace=False)`）。实测 31 target / 9 control，年份 1955–2018。⟶ 出处 371_stage12.py:40, 52-58 · config/fit_prompts.parquet
- 每条文本 = Stage 10 提示 + 空格 + 真作者名单（作者用 `, ` 连接）。例（第一条原文）：

  ```
  Who are the authors of the paper titled "Laccase: Properties and
  applications" (2009)?
  Authors: V. Madhavi, S. Lele
  ```

  token 长度 27–104（中位 50），没有超过 max_seq_len=128 的；有效位置合计 1,566 个。⟶ 出处 371_stage12.py:56 · fit_prompts.parquet text 列（Qwen3 分词器实测）
- 冻结：`config/fit_prompts.parquet` sha256 be0c3ab7…（写在 protocol.json fit.sha256）。⟶ 出处 config/protocol.json

3 工具

`jlens.fit`（本地 jacobian-lens 包，`~/projects/jacobian-lens/jlens/fitting.py`）。参数：`source_layers = [24, 32, 40, 44, 48, 52, 56, 62]`，`target_layer = 63`，`dim_batch = 1`，`max_seq_len = 128`，`skip_first = 16`，`checkpoint_every = 1`，可续跑。⟶ 出处 371_stage12.py:233-234 · protocol.json fit

4 步骤

1. 每个作业按剩余预算估计能拟合几条（`n = max(1, int((budget_min − 6)/7))`），从 checkpoint 续跑到 `min(40, done + n)`，更新 `alens/progress.json`。⟶ 出处 371_stage12.py:225-235
2. 任何 A100-80GB 作业都可以通过 claim 文件 `logs/claims/alens_fit` 接手拟合（2026-10-27 18:45 起），剩余预算需 ⩾20 分钟；出错写 `logs/fit_error_*.json`，之后所有作业停止拟合（未发生）。⟶ 出处 371_stage12.py:304-317 · logs/ 无 fit_error 文件

3. 拟合进度（作业日志）：11413557 0→10（2026-10-27 15:29）、11413572 10→20→22、11413584 22→32→35、11413586 35→40，2026-10-28 17:12:17 "author lens COMPLETE"，写 `alens/lens.pt`。最后一段 5 条用 1,344 s（269 s/条）。⟶ 出处 logs/work_11413557.log、work_11413572.log、work_11413584.log、work_11413586.log · alens/progress.json

5 公式

对每个源层 l，拟合矩阵 $J_l \in \mathbb{R}^{5120\times5120}$：

```
对每条提示：对每个输出维 d（dim_batch=1 → 每条提示 5120 次反向），
  在所有有效目标位置同时注入 one-hot cotangent e_d 于 h_63，反向得到 ∂(Σ_{p'≥p}
h_63[p'])_d / ∂h_l[p]，
  在有效源位置 p ∈ [16, seq_len−2] 上取平均 → 该条提示的 J_l 的第 d 行
J_l = 40 条提示的平均
读出：lens_l(h) = unembed( J_l · h )      （unembed = final RMSNorm + lm_head）
```

⟶ 出处 jlens/fitting.py:1-21（模块 docstring）、45-72 valid_position_mask()、100-210 jacobian_for_prompt()、365/386（求和与平均）

6 对照

对照是 Stage 10 的公开 lens（557 条 wikitext 提示，L0–L62）。F1 当时**没有做拟合质量检查**（如 held-out 上的 top-1 一致率；日志只有 "author-lens fit: a -> b of 40"，checkpoint 只存了 J 的总和，做不了 split-half）。

事后一致性比较（2026-10-30，CPU，mmap 逐层读，U42 部分解决）：F1 与 public / fitA / self100 在共同层上的矩阵余弦。本手册编辑于 2026-10-30 下午重新运行脚本，数字一致。⟶ 出处 351_stage10/reports/methods/research_scripts/u42_lens_compare.py · 附录 I U42 复核过

层	F1 vs public	F1 vs fitA	F1 vs self100	fitA vs public	self100 vs public
L24	0.296	0.269	0.278	0.795	0.853
L32	0.294	0.295	0.296	0.846	0.916
L40	0.357	0.354	0.365	0.892	0.953
L44	0.511	—	—	—	—
L48	0.688	0.679	0.700	0.969	0.987
L52	0.848	—	—	—	—
L56	0.923	0.923	0.927	0.997	1.001
L62	0.996	0.996	0.998	1.009	1.010

即 F1 在 L24–L48 与三个 WikiText lens 一致性差（余弦约 0.27–0.70），而三个 WikiText lens 彼此很一致；到 L56/L62 才接近。（单精度累加有约 0.01 的误差，故有 1.01 这样的值。）口径核对：public 的 ‖J−I‖/‖I‖ 在 L62 为 0.482，与 Neuronpedia config.yaml 的 final_identity_distance 0.4821 吻合。这种差别可能来自提示分布不同，也可能来自只用 40 条造成的噪声，现有数据无法区分。**未知/需确认：向 用户 确认（是否补做 F1 lens 的对半拟合以区分噪声与提示分布，需要 GPU，U42）**

7 结果

产出 `371_stage12/alens/lens.pt`（8 个源层），2026-10-28 17:12 完成。读出结果见 F2。⟶ 出处 `alens/progress.json done=40`

8 局限与质疑

- 只有 40 条提示（公开 lens 557 条），J 的估计方差较大；提示很短（中位 50 token），skip_first=16 后每条平均约 39 个有效位置。
- 拟合平均覆盖了提示里所有位置（标题、年份、名字），不是只针对名字位置。
- 拟合集 31/9 target/control 不平衡。
- 拟合集与评估集论文不重叠，但按作者团队不完全独立：3 篇评估论文与拟合集有同名作者（"qiang xu"、"j schwartz"、"j braun"，可能是同名不同人），已在 F2 中剔除，未重拟合（会浪费约 4 GPU 小时，当时已拟合 35/40）。⟶ 出处 `config/alens_eval_exclude.json · PROGRESS.md "2026-10-28 12:00" 条`

9 状态

已完成 （40/40）。

F2 作者提示 lens 读出（alens_read）

1 问题与作用

用 F1 的作者提示 lens 重做 Stage 10 风格的读出：如果换一个"懂作者文本"的 lens，中间层能否读出标题特异的作者信号？同时在最后一个标题 token 上读。⟶ 出处 371_stage12.py:19-20

2 材料

- 评估样本：Stage 11 样本 1,186 篇，6 个条件 true + false_0..false_4（delete 不在内），每条件 19,077 个名字 token。
- **剔除**：`config/alens_eval_exclude.json`（2026-10-28 写）—— 规则 "Stage 11 sample papers sharing any full author name with the 40 author-lens fit papers"，3 篇（均为 target）：8baa8249…（"qiang xu"）、d3104dd7…（"j schwartz"）、debd096c…（"j braun"）。剔除后 1,183 篇（590 target / 593 control）。⟶ 出处 `config/alens_eval_exclude.json` · `stage12_analyze.py:136-139` · commit `663ac1c`

3 工具

F1 的 `alens/lens.pt`（`jlens.JacobianLens.load`，每层 J 放到该层所在 GPU）；`ActivationRecorder` 记录 FIT_LAYERS 的 block 输出；`L.transport(h, l)` = J_l · h，再 `model.unembed`。⟶ 出处 371_stage12.py:242-248, 259-264

4 步骤

1. 对每个条件、每篇样本论文：取该条件提示的最后一个标题 token 位置 `it = title_positions(...)[-1]`（false 条件就是假标题的最后一个 token）。
2. 对每个作者：ids = 该条件提示 + name[:-1]，一次前向，记录 8 个源层。
3. name 读出：在名字读出位置（`len(pids)-1+j`）对 h 做 transport + unembed，求每个名字 token 的 rank。
4. title 读出：在 `it` 位置对 h 做 transport + unembed，得到一个 logits 向量，复制 len(name) 份，对该作者的每个名字 token 求 rank（即在标题位置上，所有名字 token 的排名）。⟶ 出处 371_stage12.py:255-266

5 公式

```
每篇论文、每个（层，位置）：x_c = 该论文所有名字 token 的 log10(1/rank) 平均，c ∈ {true, false_0..4}
spec = x_true − mean(x_false_0 .. x_false_4)        （标题特异信号）
报告：target、control 各自 spec 的平均 + bootstrap CI；target − control 按引用 bin 分
```

层
只用 6 个条件都存在、且不在剔除名单里的论文

⟶ 出处 stage12_analyze.py:129-149

6 对照

- 5 个假标题均值作为基线（同一 token、同一论文配对）。
- 对照 lens：Stage 10 公开 lens 在同样设计下的结果（Stage 10 / Stage 11 E4）。但 F2 分析里**没有**把两种 lens 放在同一张表里直接比较。

7 结果

位置	层	target spec	control spec	target − control（bin 内）
name	L24	+0.011	+0.022	−0.011 [−0.023, +0.002]
name	L44	−0.004	+0.037	−0.041 [−0.061, −0.020]
name	L48	+0.009	+0.067	−0.058 [−0.088, −0.028]
name	L52	+0.036	+0.098	−0.062 [−0.096, −0.028]
name	L56	+0.143	+0.144	−0.001 [−0.038, +0.036]
name	L62	+0.489	+0.252	**+0.238 [+0.210, +0.267]**
title	L24–L52	+0.031 ~ +0.061	0.000 ~ +0.108	CI 均含 0（−0.053 ~ +0.039）
title	L56	+0.049	+0.020	+0.030 [+0.000, +0.060]

全部 16 行见 `summary/F2_author_lens.csv`。结论：作者提示 lens 也读不出中间层的 target 特异作者信号；target − control 只在 L62 为正。⟶ 出处 summary/F2_author_lens.csv · reports/stage12_report.md §F2 · STATUS.md §3 F2

8 局限与质疑

- F1 只有 40 条提示，"读不出"也可能是 lens 质量问题（没有拟合质量检查，见 F1）。
- 中间层 control 的 spec 反而比 target 大（L44–52 的 target − control 显著为负）。**只描述**（用户 2026-10-30 决定，U43）；一个可能的方向（未检验）：作者提示 lens 偏向常见名字。另外 F1 在这些层与 WikiText lens 一致性差（见 F1），读出本身的可靠性也有限。⟶ 出处 PROGRESS.md 2026-10-30 14:00 条 U43
- title 位置的读出是在标题最后一个 token 上，对所有名字 token（包括后续 token）打分，后续名字 token 在该位置上本不该被预测，值偏低且噪声大。
- 剔除按"完整名字字符串相同"，不能发现拼写不同的同一人。

9 状态

已完成 144/144（2026-10-28 19:28 时剩 132，约 20:00 完成），分析 19:55（含剔除）。⟶ 出处 PROGRESS.md "2026-10-28 19:28"、"20:35" 条

Stage 12 CPU 分析（stage12_cpu.py）：名字先验 A/B/C 与探针对照 X1

1 问题与作用

- **A**：把 Stage 10 模型输出上的作者信号拆成三段：先验（无标题）→ 主题（假标题 − 无标题）→ 论文特异（真标题 − 假标题），按 token 角色和组。
- **B**：名字先验能否解释多选识别（mc_team_a）？
- **C**：名字常见程度（姓氏频率）与先验、论文特异信号的关系。
- **X1**：Stage 11 标题位置线性探针（E4b）的对照：换 gold、在假标题对上训练。

⟶ 出处 stage12_cpu.py:1-3 · 作业 11413574（short，16 CPU，64G）

2 材料

- A/B/C：Stage 10 冻结汇总 `351_stage10/summary_title/paper_title_contrasts.parquet`（每篇论文 × 读出 × 层，含 true / false（5 个假标题均值）/ delete 的 mean_nlr，按角色细分）；`cohort/papers.parquet`（authors、field、mc_team_a、recall_level）。⟶ 出处 stage12_cpu.py:36-37 · 351_stage10.py:839-864（contrast 定义：spec = true − mean(false_k)，topic = mean(false_k) − delete）
- X1：Stage 11 `raw/titlepos/true` 与 `false_0` 的残差（最后标题 token 在 PROBE_LAYERS 的残差 `resid_title`；`:` 位置 PROBE_LAYERS + L63 的残差 `resid_colon`，fp16），`raw/probe_vocab`（final norm 权重、cohort 中所有作者首 token / 姓氏首 token 的 lm_head 行）。gold = 每个作者名字的第一个 token（kind=='first'）。⟶ 出处 stage12_cpu.py:111-120 · 361_stage11.py:355-391 do_titlepos()

3 工具

numpy / pandas / scipy（spearmanr）；ridge 回归用闭式解 `np.linalg.solve`。⟶ 出处 stage12_cpu.py:143-144

4 步骤

1. **A**：对每个 (读出, 层, 角色)：prior = delete，topic = false − delete，spec = true − false，true；按 all/target/control 平均；target − control 对四项各做 bin 内分层 + bootstrap。角色：all_tokens、surname_first、surname_cont、given_first、initial_first、punct。另存按 bin（A_prior_by_bin.csv）和按层（logit / jlens_public，A_prior_by_layer.csv）。⟶ 出处 stage12_cpu.py:39-72

2. **B**：R0/R1 且 mc_team_a 非空的论文（8,894 篇），prior 和 spec（模型输出，all tokens）先在引用 bin 内去均值，再与 mc_team_a 求 Spearman；CI = 500 次论文重抽。⟶ 出处 stage12_cpu.py:74-83
3. **C**：每个作者取最后一个空格分隔的词（小写）作姓氏，在全 cohort 作者中计数；每篇论文 = 各作者 log10(计数) 的平均；与 prior、spec 求 Spearman；target − control 的姓氏常见度做 bin 内分层。⟶ 出处 stage12_cpu.py:89-99
4. **X1**：层 L24/40/56；5 折（`default_rng(11).integers(0,5)` 按论文随机分折）。ridge：X = 标题位置残差（训练集标准化），Y = `:` 位置 L63 残差，λ = 1.0 × trace(A^TA)/d。预测的 L63 残差 → RMSNorm（norm 权重、eps）→ 名字词表 lm_head 行 → gold 首 token 在名字词表内的 rank → log10(1/rank)，对作者平均。spec = 探针(true 标题残差) − 探针(false_0 标题残差)。四种设定：
   - probe（同 Stage 11）：true 对训练；
   - gold = 随机另一篇论文的作者（全局置换，seed 3712 的 rng）；
   - gold = 同领域随机另一篇论文的作者；
   - 在 false_0 对（X=false 标题残差，Y=false 运行 `:` L63）上训练。

   ⟶ 出处 stage12_cpu.py:111-158

5 公式

```
A:  prior = nlr_delete ; topic = nlr_false − nlr_delete ; spec = nlr_true −
nlr_false ; ( nlr_false = 5 个假标题均值 )
B:  rho = Spearman( mc_team_a , metric − mean_bin(metric) )
C:  log_surname_freq_p = mean_authors log10( count(surname) )
X1: W = (AᵀA + λI)⁻¹ Aᵀ(Y − ȳ),  λ = α·tr(AᵀA)/d,  α = 1
    score = mean_gold −log10(1 + #{名字词表中 logit 更大的 token})
    spec  = score(true 残差) − score(false_0 残差)
```

6 对照

X1 的两个 gold 置换是"信号应为 0"的负对照；在假标题上训练是检验探针是否只是学到了"真标题"的通用特征。

7 结果

A（模型输出，all tokens）：target prior −1.061 / topic −0.128 / spec +0.481 / true −0.708；control −1.150 / −0.053 / +0.200 / −1.004。target − control（bin 内）：prior +0.087 [+0.070, +0.103]，topic −0.047 [−0.055, −0.038]，spec +0.207 [+0.195, +0.220]；surname_first 上 prior +0.022 [−0.019, +0.063]（不显著）、spec +0.423 [+0.393, +0.454]。⟶ 出处 reports/stage12_cpu_report.md §A · summary_cpu/A_prior_decomposition.csv (readout=final)

B：prior 与 mc_team_a 的 ρ = +0.085 [+0.063, +0.106]；spec ρ = +0.329 [+0.312, +0.346]（n = 8,894）。⟶ 出处 summary_cpu/B_prior_vs_recognition.csv

C：姓氏常见度与 prior ρ = −0.215（all），与 spec ρ = −0.091；target 姓氏比 control 少见：−0.199 [−0.233, −0.166]。 ⟶ 出处 summary_cpu/C_surname_commonness.csv

X1（spec，全部论文）：

层	probe	gold=随机论文	gold=同领域随机	假标题上训练
L24	+0.045 [+0.040, +0.050]	−0.002	+0.002	+0.068 [+0.063, +0.073]
L40	+0.062 [+0.056, +0.067]	−0.003	−0.004	+0.081 [+0.076, +0.087]
L56	+0.081 [+0.075, +0.087]	+0.001	+0.000	+0.094 [+0.088, +0.100]

⟶ 出处 summary_cpu/X1_probe_controls.csv · reports/stage12_cpu_report.md §X1

X1 的复核（Stage 13）：按作者团队分折 + 剔除"探针训练时见过该测试论文的提示标题"（title reuse，随机分折下约 80% 测试论文受影响）重跑后：置换对照仍 ≈0、probe 本身成立；但"在假标题上训练更强"**不成立**（剔除后 L24/40/56 = +0.050/+0.057/+0.074，与 probe 持平或更低），原因是假标题供体泄漏。 作废 （仅"假标题训练更强"这一句） 复核过 （置换对照） ⟶ 出处 381_stage13/reports/e4b_split_compare.md（group / dropped 列；sanity 行 max|diff| 0.0080）· 作业 11443587 · PROGRESS.md "2026-10-28 12:00"、"12:35" 及 ~19:10 前的复核条

8 局限与质疑

- A 的 "false" 是 5 个假标题均值，假标题来自同领域匹配论文，"topic" 混入了匹配质量。
- C 的姓氏取"最后一个词"，对复姓、东亚名字顺序、"Jr." 等处理粗糙；只在本 cohort 内计频。
- X1 原版按论文随机分折，存在标题复用泄漏和同团队泄漏（已由 Stage 13 E4b 复核处理）。
- X1 只看 L24/40/56 三层；gold 只用首 token。

9 状态

A/B/C 已完成 （2026-10-27 14:56，作业 11413574）。X1 复核过 （Stage 13 E4b 团队分折），其中"假标题训练更强" 作废 。 ⟶ 出处 logs/cpu_11413574.log 末行 "STAGE12 CPU DONE"

Stage 13：中间层装的如果不是名字，是什么（M1–M5，X1–X7）

0. 共用部分：路径、样本、切分、区间、预注册、GPU 调度

路径缩写（本节以下都用）：`C13` = `~/jlens_work/13_stage13/`（代码）；`R13` = `/scratch/zhang.zhiqi2/jlens/381_stage13/`（结果）；`R10` = `/scratch/zhang.zhiqi2/jlens/351_stage10/`；`R11` = `/scratch/zhang.zhiqi2/jlens/361_stage11/`；`PROG` = `~/jlens_work/PROGRESS.md`。git 镜像里这些脚本在 `code/scratch_scripts/`（仓库 `/scratch/zhang.zhiqi2/jlens-authorship-recall_update`），除 `stage13_gpu_followup.py` 外与工作副本逐字相同（2026-10-30 用 diff 核对）；`stage13_gpu_followup.py` 的工作副本（b7/b2 两种批大小版）比 git 最新提交 44d084f 新，尚未同步。⟶ 出处 `diff code/scratch_scripts/* vs C13/* · git log --oneline -- code/scratch_scripts/stage13_gpu_followup.py → 44d084f, 8d1a5c0`

来由

用户 2026-10-28 指出"lens 读不出"≠"不存在"，由此开 Stage 13。三个假设：H1 中间层主要表示任务/格式；H2 主要表示"这是哪篇论文"（论文身份）；H3 作者信息已在、只是不在词表方向上。⟶ 出处 `PROG 2026-10-28 ~07:15 条 · stage1_11_report.html Part IV · C13/README.md`

共用样本

名称	内容	出处
全队列	9,108 篇论文（8 个领域：biology, chemistry, computer science, materials science, mathematics, medicine, physics, psychology）；group_tight：target 4,621 / control 2,220 / excluded 2,053 / guard 214。M1、M2、M2b、M2c、M3 用全队列。	⟶ 出处 `R10/cohort/papers.parquet`（列 `field, group_tight`），2026-10-30 读取计数
Stage 11 样本	1,186 篇（target 593 / control 593），24 个 shard。X1、X3、X4、X6、X7 用它。	⟶ 出处 `R11/config/sample.parquet`
配对表 pairs	1,178 / 1,186 篇找到搭档 B（另 8 篇无合格候选而丢掉）。B：同领域、作者问法下标题 token 数相同、作者姓氏无交集、有 Stage 11 名字 token 上下文；随机对照 C：任意领域、标题长度相同、与 A 和 B 都无共同姓氏。随机种子 1313。tune 596 / test 582（control test 294、tune 295；target test 288、tune 301）；1,141 个团队，跨 tune/test 的团队 0 个。	⟶ 出处 `C13/381_stage13.py:143-178 freeze()` · `R13/config/pairs.parquet`（列 `partner, random_c, n_title_tokens, team, split`）

按作者团队分组（group_folds）

审计（2026-10-28 11:45）发现两处泄漏：false_0 运行里 2,004 个借出标题同时出现在训练和测试里；有 2,319 对论文共享至少一位作者全名却分在不同折。修法 `group_folds`：作者名先做 NFKD→ASCII→小写→只留字母和空格，至少两个词才算"全名"；用并查集把共享任一全名的论文并成一组（同名不同人也并，保守）；false_0 运行再把行与它展示的借出论文绑在一起；整组按"大的先放、放进当前最小的折"分到 k 折（打乱用 seed）。全队列得到 7,247 组；折大小 1,821–1,822；0 个名字、0 个借出标题跨折。⟶ 出处 C13/stage13_cpu.py:80-109 group_folds · PROG 2026-10-28 11:45 条 · R13/reports/cluster_ci.md 第 3 行（7247 groups）

X1/X3/X4 的 tune/test：`group_folds(k=2, seed=1314)` 只用它的团队编号；然后在 target、control 内各自把团队洗牌（seed 1314），按团队整体依次放进 tune，直到累计篇数达到该组一半，余下放 test。⟶ 出处 C13/381_stage13.py:165-177

区间

预注册原文是"按论文 bootstrap 2,000 次"，补充 2 改为"按作者团队整群 bootstrap，团队版为主"。Stage 13 的 M3/X1/X4/X6/X7 分析脚本都用团队整群 bootstrap：对团队做多项式抽样得到每篇论文的权重 W[draw, paper]，比率类统计量在每次抽样里用加权和重算，2,000 次，取 2.5/97.5 百分位。M2/M2b/M2c 不在此列（见各节）。⟶ 出处 config/prereg.json general_rules · prereg_addendum2.json "intervals" · C13/stage13_x1x4_analyze.py:31-33 team_w · C13/stage13_m3_analyze.py:96

预注册文件（只读，sha256 已于 2026-10-30 用 sha256sum -c 核对通过）

文件	实际写于（EDT）	sha256（前 8 位）	主要内容
prereg.json	2026-10-28 10:39（文件内 written 字段手填为 11:00，有误）	18409398	M2c、M3、X1、X3、X4、X5 的判定规则；声明 M1/M2/M2b 已先跑（探索性）；"X2 跑之前另写补充"。
prereg_addendum1.json	10:50（written 字段 11:40）	60804 5ca	所有 probe 用团队折；false_0 按借出论文分组；X1/X3/X4 tune/test 按团队；团队版为确认性。
prereg_addendum2.json	11:05（written 字段 12:30）	377b7963	替换 M3 身份指数规则（只看 I(l)−I(L4)，迁移比 T 为主）；替换 X1 选层与判定规则；M2c 改为单向；X3 加词重合对照；E4b；全部区间改团队版。
prereg_addendum3.json	11:41（written 字段 14:00）	e01aa6e3	X6、X7 的定义与判定；X6 先跑、X7 最后跑。
prereg_addendum4.json	11:55（written 字段 14:40）	ac77da9f	X3 主对照改为词重合近邻 D（G'）；按 token 角色报告；措辞规则（teacher forcing 下的偏好，不是生成过程）。

prereg_addendum5	—	—	计划未做 尚未写定：草稿已发给用户，等用户确认（X3 排除 27 条完全相同改写、改用 jaccard2、37 条姓氏误拒维持拒绝、意义检查改走备用方案——因为判断器 v2 不可用）。⟶ 出处 R13/config/ 目录无此文件；PROG 2026-10-29 04:40、12:40 条，2026-10-30 14:00、15:00 条

⟶ 出处 R13/config/prereg*.json 与 .sha256。"实际写于"取自会话记录、文件 mtime 与 git 提交时间（三者一致）；文件内的 written 字段是当时会话手填的，比实际晚 20 分钟到 2 小时 45 分钟，不能当作写定时间引用（U18 已解决）。⟶ 出处 PROG 2026-10-30 12:30 条 · h-prereg-files 复核过

GPU 单元与调度

GPU 部分全部在 `381_stage13.py work` 里按"单元"领取（claim 目录 `R13/logs/claims/`），每个单元存一个 `raw/<类型>/[条件/]shard_XXX.npz`，已存在就跳过；某类单元崩溃会写 `logs/broken_<类型>.json` 并停用该类型（2026-10-30 查：没有任何 broken 文件）。单元顺序：X6 xf_patch（先 false_0 shard 0 复现检查）→ M3 task 72 → X1 xpatch 24 → X4 pscope 24 → X3 para_gen 24 →（仅当 `logs/para_ok` 存在）para_run 24 → X7 ko_false 24。它们作为 Stage 12 作业的"转交"运行（`371_stage12.py handover13`，开关 `R13/logs/accept_handover`）。模型 Qwen3-32B bf16，单卡 A100-80GB（task 单元的 meta 记录了 2 张 A100-80GB）。⟶ 出处 C13/381_stage13.py:59-68 units, 395-416 work · R13/raw/*/shard_*.npz 的 meta 字段

脚本版本：除 6 个 xf_patch shard 外，所有 GPU 输出的 meta.script_sha256 = a7b01a8f（即当前 `381_stage13.py`，git fb8be6f）。xf_patch 有 4 个 shard（false_1 shard 1–4）出自 448b78c2（git 54d8ac2，备份 `R13/logs/381_stage13.py.before_logfix_20261028`，与当前版只差日志那一行），2 个 shard（false_0 shard 0、false_1 shard 0）的 meta 记 613eb813。**613eb813 是转交时记录的 Stage 11 脚本哈希，不是 Stage 13 的版本**：它就是当前的 `~/jlens_work/11_stage11/361_stage11.py`（= git 13ae7ec），副本存在。原因：这两个 shard（10-28 19:46:02/19:46:10）由 Stage 12 worker 交接运行，`handover13` 把 Stage 12 的上下文 C.env 传给 Stage 13（371_stage12.py:360），这个 env 由 361_stage11 的 Ctx 生成，`script_sha256 = sha_file(__file__)` 指向 361_stage11.py（361_stage11.py:189）。实际计算代码应为当时的 381_stage13.py，即备份 448b78c2（= git 54d8ac2，推断）。这是记录口径问题；false_0 shard 0 与 Stage 11 逐值相同。⟶ 出处 R13/raw/xf_patch/*/shard_*.npz 全部 97 个 meta · 371_stage12.py:350–365 · 361_stage11.py:189 · 附录 I U44 · PROG 2026-10-28 20:35 条 复核过（U44 已解决）

预检：tiny 模型（2 层随机 Qwen3 + 真 tokenizer）`dry` 从 14/14 逐步扩到 31/31（含"B 用自己的状态替换 = corrupt""在最后一层替换所有位置 = clean""X7 不挡的一列 = 普通 false_0 运行"）；全流程测试 `stage13_pipeline_test.py` 作业 11459594 PIPELINE PASS 48/48（7 种单元都走 claim→compute→save→log→读回）。⟶ 出处 C13/381_stage13.py:419-487 dry · R13/logs/pipetest_11459594.log 末行 · PROG 第 232、344、488 行

M1　lens 在名字位置排第一的是什么 token

1 问题与作用

Stage 10 的 lens 在中间层读不出正确名字。M1 问：那它读出的是什么？如果是"格式/句末"之类，支持 H1 或说明 lens 在中间层没在读内容；并看读出是否随标题而变（即使读错）。

2 材料

Stage 10 已存的逐 token 读出：每个名字 token × 每种读出 × 每层的 top-1 token 与正确 token 的名次。名字 token 表 149,999 行（9,108 篇）；每个条件读入 7,049,953 行（token×读出×层）。条件：true、false_0、delete。读出：final（L63）、jlens_public、jlens_self100、jlens_fitA、logit；层数随读出不同（public 和 logit 有 L24–L62 的细层，self100/fitA 只有 24/32/40/48/56/62）。⟶ 出处 R10/cohort/name_tokens.parquet · R13/logs/cpu_11438885.log 第 41-43 行 · R13/summary_cpu/M1_top1_classes.csv

3 工具

`C13/stage13_cpu.py m1()`（第 40-77 行），CPU 作业 11438885；分组重跑作业 11443403 只把 M1 的 csv 复制到 summary_cpu_group（M1 不涉及切分）。⟶ 出处 C13/stage13_group.sbatch（cp 那一行）

4 步骤

1. 把 top-1 token 解码、去空格、小写后按优先级归类：**correct**（名次=1）→ **name-like**（是队列里任一作者名字首 token 的 id）→ **punct/format**（空串或只含非字母数字）→ **task word**（固定词表 TASK：author(s)、paper(s)、title(d)、et、al、by、published、journal、university、study、and、the、of、in、who、are、is、was、were、written、name(s)、unknown、none、n、a、an）→ **title word**（在本条件提示的 token id 第 8 到倒数第 8 个之间出现过，作者注释为"粗略的标题区间"）→ **other**。⟶ 出处 stage13_cpu.py:36-37, 47-55, 62-66
2. 按（读出、层）算各类比例，分"每个名字的第一个 token"和"全部名字 token"两版。
3. M1b：同一（row、读出、层）在 true 与 false_0 下 top-1 是否不同的比例。⟶ 出处 stage13_cpu.py:74-76

5 公式

share(c | readout, layer) = #{token：类别 = c} / #{token}；M1b = mean[top1_true ≠ top1_false_0]。

6 对照

无专门对照；true/false_0/delete 三条件并列；M1b 本身是"读出是否依赖标题"的对照。

7 结果

（真标题，名字首 token）Jacobian lens 在 L24–L48 几乎 100% 是 punct/format（如 ".\n"".\n\n""...\n\n"）；public 从 L51 起 name-like 0.665、L52 0.694，correct 只有 0.001（L52）到

0.019（L62）；logit lens 在 L24–L52 几乎全是 other（0.991–1.000），name-like 从 L57 的 0.113 升到 L62 的 0.867。M1b：L24–L48 top-1 随标题变的比例 0.28–0.64（fitA 0.283–0.455，public 0.318–0.525，logit 0.536–0.636）。⟶ 出处 R13/reports/stage13_cpu_report_group.md M1、M1b 表 · R13/summary_cpu/M1_title_sensitivity.csv 具体的 token 例子（工程技术、您好、Robert、Zhang 等）出自 PROG 2026-10-28 08:30 条，csv 里只有类别比例，没有 token 频次表。⟶ 出处 PROG 第 453-461 行

8 局限与质疑

- "title word"用固定的 [8:-8] 截取标题，作者自注"rough"；不同条件模板长度相同，但没有逐篇验证截取边界。
- name-like 定义很宽（队列任一作者名字首 token），常见名字和泛化 token 都会算进去。
- 只说明 lens 在中间层读出什么，不能说明信息是否存在（这正是 M2–X4 要回答的）。

9 状态

已完成 预注册把 M1 标为探索性（prereg.json status_at_writing）；无判定规则，无偏离。

M2　残差 probe：领域、年份、引用数、是否认得出（按论文 vs 按团队切）

1 问题与作用

在标题最后一个 token（此时年份还没出现，因果注意力看不到后面）和 ":" 位置，用线性 probe 读"不是作者名字"的论文属性。读得出年份、引用数、"是否认得出"，说明这些位置表示的是"这篇论文"。

2 材料

Stage 11 titlepos 运行存下的残差：true 与 false_0 两条件，全部 9,108 篇；标题位置 9 层（16, 24, 32, 40, 44, 48, 52, 56, 60），":" 位置同 9 层 + L63。提示是 Stage 10 的问法 `Who are the authors of the paper titled "{title}" ({year})?\nAuthors:`，年份在标题之后，所以 ":" 位置已经看到年份。false_0 运行展示借出论文的标题，但年份是目标论文的。⟶ 出处 R11/raw/titlepos/{true,false_0}/shard_*.npz（resid_title, resid_colon, resid_paper_id, probe_layers）· stage13_cpu.py:168-173 注释 · R13/summary_cpu_group/M2_probes.csv 的 layer 列

3 工具

`C13/stage13_cpu.py m2()`（第 149-200 行）。按论文切：作业 11438885（标题位置有效）→ 加年份对照后重跑 11439283；按团队切（SPLIT=group）：作业 11443403；对比表 `stage13_split_compare.py`（日志 cmp_11443607.log）。⟶ 出处 PROG 2026-10-28 08:00、09:35、11:45、14:30 条 · R13/logs/

4 步骤

1. 标签取自"提示里标题所属的论文"：true 用本篇，false_0 用借出论文（donor）。目标：field（8 类 one-hot，准确率）、year（R^2）、log10(1+citations)（R^2）、去掉年份后的 log citations（先用 5 年一箱的年份 one-hot 做 ridge 拿残差）、recognised = target vs control（0/1，只在 true 运行、只用 target+control 的 6,841 篇，按引用数箱内 AUROC）。⟶ 出处 `stage13_cpu.py:159-180 · M2_probes.csv n 列（9108 / 6841）`
2. 每个（条件、位置、层、目标）：ridge_cv —— 训练折上标准化；alpha 在 {0.01, 0.1, 1, 10, 100, 1000}×trace(G)/d 里，在第 0 折上按测试 MSE 选，然后所有折都用它；输出每篇的留出预测。⟶ 出处 `stage13_cpu.py:112-123`
3. 打乱标签对照：同一折、按层固定种子打乱 Y 再跑一次。⟶ 出处 `stage13_cpu.py:193-194`
4. 年份基线：只用提示中的年份（5 年一箱 one-hot）预测 citations、citations|year、recognised。⟶ 出处 `stage13_cpu.py:181-184`
5. ":" 位置的年份 probe 是泄题（年份就在提示里），只作阳性对照（note 列标 LEAK）。
6. **按论文切**：`np.random.default_rng(13).integers(0,5,n)`，每篇独立随机分折（各折大小不严格相等）。**按团队切**：`group_folds(lab_ids)`，k=5、seed=13；false_0 时 lab_ids 是借出论文，所以同一借出标题只在一折。⟶ 出处 `stage13_cpu.py:166-167`

5 公式

acc = mean[argmax p = argmax y]；$R^2 = 1 - \Sigma(y-p)^2 / \Sigma(y-\bar{y})^2$；auc_bin = 各引用数箱内 AUROC 按箱大小加权平均（只用两类都 ⩾10 篇的箱）。CI：对留出预测按论文重抽样 **200** 次（rng 3813），probe 不重拟合。⟶ 出处 `stage13_cpu.py:126-146`

6 对照

打乱标签；年份基线；":" 位置年份（泄题阳性对照）；按论文 vs 按团队两种切分对比。

7 结果（团队切，确认性）

真标题 · 标题位置	L16	L24	L48	L52	打乱
field acc	0.698	0.722	0.716	0.719	0.13–0.14
year R^2	0.383	0.451	0.555	0.561	≈0
log citations R^2	0.286	0.370	0.530	0.520	≈0
citations | year removed	0.275	0.356	0.521	0.510	≈0
recognised AUROC	0.620	0.632	0.671	0.669	0.48–0.51

年份基线：citations R^2 0.047，citations|year −0.001，recognised 0.555。":" 位置 year R^2 0.974–0.982（泄题）。L48 例：year 0.555 [0.537, 0.573]，n=9,108。⟶ 出处 `R13/reports/stage13_cpu_report_group.md "true title run" 两表 · R13/summary_cpu_group/M2_probes.csv（condition=true, position=title, layer=48）`

团队整群区间（2026-10-30 补算，U45）：脚本 `C13/stage13_m2_teamci.py`，作业 11503585（tiny 11501115 因 CPU 型号不同折分配不同而作废、11503471 通过）。用同一团队折、同一 α 网格与特征重算留出预测（存于 `R13/summary_cpu_teamci/preds/`），按作者团队（= probe 的折分组）有放回重抽，NB = 2,000，种子 20261030。结果：**176 个点估计与原表全部一致**（最大 |差| 1.1×10^{-16}）；有原区间的 171 行，团队 / 论文区间宽度比平均 **1.15**（中位 1.10，0.86–1.58），true 条件 1.02、false_0 条件 1.32（标题位置 field 1.42、log citations 1.39 最宽）；"区间不含 0"与"区间高于机会水平"**都是 0 个翻转**。年份基线行现在也有区间（如 recognised AUROC 0.555 [0.534, 0.573]）。⟶ 出处 R13/reports/m2_teamci.md（Point-estimate check、Interval width ratio、Flips 三段）· R13/summary_cpu_teamci/M2_teamci.csv（width_ratio 按 condition 求均值：true 1.021、false_0 1.318）· R13/logs/m2teamci_full_11503585.log · PROG 2026-10-30 17:30 条 复核过

按论文 vs 按团队：真标题运行相差 ≤0.005（如 L48 year 0.556 vs 0.555）；false_0 运行团队切后降约 0.1（L48 借出年份 0.623→0.535，citations 0.588→0.488），所以按论文切的 false_0 数字作废，但团队切后仍与真标题运行可比（"表示跟着提示里展示的那篇论文走"）。⟶ 出处 R13/reports/split_compare.md 第 119-152 行

8 局限与质疑

- M2 原报告的 CI 是按论文 bootstrap、只 200 次，不是补充 2 要求的团队整群 2,000 次；报告头仍写"5-fold CV by paper"（团队版报告也是这句，属硬编码的旧标题）。⟶ 出处 stage13_cpu.py:142-146, 209 2026-10-30 已按补充 2 补算团队整群区间（见第 7 项末段），U45 已解决。
- 团队折的复现性：`group_folds()` 对同样大小的组排序不稳定（pandas value_counts），不同 CPU 型号上折分配可能不同（d0026 上点估计最多差 0.006；c0622 上精确复现），补算用的折已固定保存在 `R13/summary_cpu_teamci/folds_group.npz`。⟶ 出处 PROG 2026-10-30 17:30 条 · h-stats-team
- docstring 列了"first-author first token"目标，但 targets 列表里没有实现；作者首 token probe 在 E4b（`stage13_e4b_group.py`，属 Stage 11 复核）。⟶ 出处 stage13_cpu.py:13 vs 178-180
- recognised 用 ridge 回归 0/1 标签再算 AUROC（不是 logistic）。
- 年份、引用数可能来自标题用词风格而非"认得这篇"——这是 M2b、M2c 的任务。
- false_0 的团队分组只按借出论文及其团队，不包括行本身的目标论文（":" 位置上看到的是目标论文年份）。

9 状态

已完成 复核过（团队切重跑）。预注册：M2 为探索性（prereg.json），补充 1 规定团队切为确认性 —— 已照做；补充 2 的"团队区间"原来没有用到 M2 上，2026-10-30 已补算（作业 11503585），0 个翻转。

M2b　TF-IDF 标题词基线

1 问题与作用

年份/引用数是"认得论文"还是只靠标题用词（领域、时代用语）就能猜？用纯词袋模型做下限。

2 材料

与 M2 同：9,108 篇真标题运行，标题最后一个 token 的残差（9 层）；标题文本；年份；引用数；group_tight；citation_bin。⟶ 出处 C13/stage13_m2b.py:16-21

3 工具

`C13/stage13_m2b.py`（53 行）。按论文切：作业 11441993；团队切：11443403。sklearn TfidfVectorizer、Ridge。

4 步骤

1. (a) 词基线：每折在训练折上拟合 TfidfVectorizer(ngram_range=(1,2), min_df=2, sublinear_tf=True)，Ridge(alpha=1.0)（不调参）预测 year、log citations，得留出预测 pw。⟶ 出处 stage13_m2b.py:37-42
2. (b) 残差 probe：同 M2 的 ridge_cv（同折）。
3. (c) 词 + 残差：pw + ridge_cv(X, y − pw)。⟶ 出处 stage13_m2b.py:45-51
4. 认得出 vs 认不出：每个引用数箱内（target、control 各 ⩾30 篇）比较绝对误差均值之差，权重 n_a · n_c/(n_a+n_c)。⟶ 出处 stage13_m2b.py:23-34

5 公式

R^2 同 M2；abs_err_target_minus_control = Σ_bin w · (mean|e|_target − mean|e|_control) / Σw（负 = target 更准）。

6 对照

词基线本身就是对照；无打乱，无 CI。

7 结果（团队切）

TF-IDF：year R^2 0.103，citations 0.075；残差 probe：year 0.383（L16）→ 0.561（L52），citations 0.286 → 0.530（L48）。"词 + 残差"反而低（year 0.231–0.399）。认得出的论文年份误差小：残差 probe −0.68 到 −1.21 年，但词基线已经是 −0.971 年；引用数无差别（−0.024 到 +0.003）。⟶ 出处 R13/summary_cpu_group/M2b_words_vs_probe.csv（按论文切版：TF-IDF year 0.108、cit 0.067，词基线 target−control −0.883 年。⟶ 出处 R13/summary_cpu/M2b_words_vs_probe.csv）

8 局限与质疑

- TF-IDF 是很弱的文本模型，L16 残差本身已是强文本模型；所以"残差远超 TF-IDF"不能说明"认得论文"。作者自己在 PROG 记下这一点，并转向 M2c（BERT）和 M3 的 L4/L8。⟶ 出处 PROG 2026-10-28 10:30 条
- "词 + 残差"低于"残差"，说明在留出预测的残差上再拟合并不合适（叠加方式本身有损），不应解读为"词有害"。
- 没有 CI。

9 状态

已完成 探索性（prereg.json）；补充 2 的 X3 词重合规则与此无关。无偏离。

M2c　BERT 基线（没背过这些论文的文本模型）

1 问题与作用

如果一个没记住这些论文的模型（bert-base-uncased）只看标题文本也能读出同样水平的年份/引用数/"认得出"，那 Qwen 读出的就可能只是标题风格。

2 材料

9,108 篇标题文本（与 M2 同顺序）；同样的目标与折。⟶ 出处 C13/stage13_m2c.py:15-16

3 工具

`C13/stage13_m2c.py`（37 行）；bert-base-uncased，13 个 hidden state（嵌入层 + 12 层），按 attention_mask 对 token 求均值（含 [CLS]/[SEP]），max_length 96，batch 64。按论文切：作业 11443232；团队切：11443403。⟶ 出处 stage13_m2c.py:17-24

4 步骤

1. 每层特征 → M2 的 ridge_cv（同 alpha 网格、同折）→ year R^2、log citations R^2、citations|year R^2（年份残差化用 5 年箱 one-hot、按论文随机折 seed 13，团队版也一样）、recognised AUROC（箱内）。⟶ 出处 stage13_m2c.py:25-36
2. 取 BERT 最好一层与 Qwen 标题位置（L16–L60）最好一层比较。

5 公式

预注册（原）：BERT ⩾ Qwen − 0.05 → 风格解释；Qwen 高出 ⩾0.10（R^2）或 ⩾0.05（AUROC）→ 超出风格。补充 2 改为**单向**：只有"BERT 追上 Qwen（⩾ Qwen − 0.05）"作数，支持风格；Qwen 高出只作参考，不算 Qwen 认得论文的证据。⟶ 出处 prereg.json M2c · prereg_addendum2.json M2c

6 对照

BERT 本身就是对照模型；同折同 ridge。

7 结果（团队切；按论文切差 ≤0.02）

目标	BERT 最好层	Qwen 标题位置最好层	按规则
year R^2	0.240（L12）	0.561	BERT < Qwen−0.05：不支持风格解释
log citations R^2	0.188	0.530	不支持风格解释
citations | year removed	0.182	0.521	不支持风格解释
recognised AUROC	0.632	0.671	0.632 ⩾ 0.621：**支持风格解释**

⟶ 出处 R13/summary_cpu_group/M2c_bert_baseline.csv（各 target 的 value 最大值）· R13/summary_cpu_group/M2_probes.csv（true, title, 各 target 最大值）· PROG 2026-10-28 13:30 条

8 局限与质疑

- "最好层"在同一数据上取最大值，两边都有轻微选择偏差。
- BERT 见过 Wikipedia（会记载名作），Qwen 读文本也更强，所以 Qwen > BERT 不能当认得论文的证据——补充 2 已据此改为单向。
- 结论只到"认得出与否大体能从标题本身预测"，对 year/citations 只能说"BERT 达不到"。

9 状态

已完成 预注册在数据前写好（prereg.json 写时 M2c 已提交未出结果）；补充 2 在结果出来之前（12:30，结果 13:30）改为单向。按改后规则执行，无其他偏离。

M3　同一论文三种问法：迁移比 T 与身份指数 I

1 问题与作用

同一个标题放进三个不同问题里。如果中间层在标题位置表示的是"这篇论文"（H2），换问题不应改变它：probe 跨问法迁移、同篇论文跨问法余弦高；如果表示的是任务（H1），换问法就分开。

2 材料

9,108 篇 × 3 种问法 = 27,324 次前向（72 个 shard × 128 篇）。三种问法原文（实际使用的 `TASKS`；注意脚本开头 docstring 里写的旧问法与之不同，以 TASKS 为准）：

```
authors: Who are the authors of the paper titled "{t}"?\nAuthors:
year   : In what year was the research paper titled "{t}" published?\nYear:
field  : In what research field is the paper titled "{t}"?\nField:
```

三者都不含年份；标题前都是 9 个 token（`PREFIX_TOKENS = 9`，freeze 时逐个模板 assert），所以标题在三种问法里占相同的绝对位置（X1 跨问法替换需要这一点）。⟶ 出处 C13/381_stage13.py:46-51, 149-151

每次前向存：15 层（4, 8, 16, 24, 28, 32, 36, 40, 44, 48, 52, 56, 60, 62, 63）在"标题最后一个 token"和"最后的 ':'"两个位置的残差（fp16，5120 维），数组 [paper, task, layer, pos, 5120]。year 问法后面 teacher-force 真实年份（' '+年份的 token），authors 问法后面 teacher-force 第一作者名字的第一个 token，记录它们在输出端的 log10(1/rank)（year_nlr、author_first_nlr）。⟶ 出处 381_stage13.py:45, 80-117 · R13/raw/task/shard_000.npz resid 形状 (128, 3, 15, 2, 5120)

3 工具

GPU：`381_stage13.py` 单元 task（72/72，claim 均为作业 11428778，2026-10-29 01:05 完成）。分析：`C13/stage13_m3_analyze.py`，作业 11472899（先 --test：3 shard、L4/L24，384 篇，全流程；再全量）。另有 `stage13_x1_similarity.py`（作业 11474830）用同一缓存算"换问法距离 ÷ 换论文距离"，放在 X1 一节。⟶ 出处 R13/logs/claims/task_★ · R13/logs/m3_11472899.log 第 4、121 行 · PROG 2026-10-29 01:05、09:30 条

4 步骤

1. 一次性把 72 个 shard 写进 memmap 缓存 `summary_m3/resid_cache.f16`。⟶ 出处 stage13_m3_analyze.py:31-42
2. 折：`group_folds(pids)`，默认 k=5、seed=13 —— 5 个团队折。⟶ 出处 stage13_m3_analyze.py:93
3. 目标 Y = [year, log10(1+citations), field 8 列 one-hot]，一个 ridge 同时拟合所有列。⟶ 出处 :94-95
4. 对每个（层、位置）：在问法 a 的训练折上标准化（用 a 的均值、方差），alpha 在 {0.1, 1, 10, 100, 1000, 10000}×trace(G)/d 中选，选法：**第 0 折**上、在问法 a 内，按"各列 MSE/Var(Y) 的平均"最小；每个 a 只选一次，此后各折沿用。然后用 a 拟合的权重去预测**问法 b** 的留出论文（b 的状态用 a 的均值方差标准化）。全量运行中所列层的 alpha 都是 1.0。⟶ 出处 stage13_m3_analyze.py:20, 58-75 · summary_m3/m3_transfer.csv alpha 列
5. 身份指数：每个问法、每层先减该问法均值再单位化，算同篇跨问法余弦减去异篇跨问法平均余弦。⟶ 出处 :48-55
6. 团队整群 bootstrap 2,000 次（seed 31），对留出预测/每篇 d_i 重加权，probe 不重拟合；再算相对 L4 的差。⟶ 出处 :96, 120-126

5 公式

迁移比：T(l,pos,target) = mean over 6 个有序对 (a≠b) of score(训练 a → 测试 b) / score(训练 b → 测试 b)，score 为 R^2（year、log citations）或准确率（field 的 one-hot ridge 取 argmax）。bootstrap 时每次抽样分别算比再对 6 对平均。⟶ 出处 stage13_m3_analyze.py:78-84, 105-117

身份指数：u = 问法 a 的居中单位向量，v = 问法 b 的；d_i = cos(u_i, v_i) − ½[mean_{j≠i} cos(u_i, v_j) + mean_{j≠i} cos(v_i, u_j)]；I(l,pos) = mean_i d_i，再对 3 个问法对平均。只看 I(l) − I(L4)。⟶ 出处 stage13_m3_analyze.py:5-7, 48-55

6 对照

L4 作为"纯文本下限"：所有判断都相对 T(L4)、I(L4)。":" 位置作为对照位置。

7 结果

标题位置：T 在 L16–L40 为 year 0.92–0.95、citations 0.92–0.96、field 0.99，按阈值属 H2；但 T(L4) 已是 0.983 / 0.976 / 0.998，T − T(L4) 在 L16–L40 为 −0.03 到 −0.06（year，区间不含 0）。":" 位置：year、citations 的 T 为负（跨问法 R^2 为负），field 0.26–0.36 → H1（任务特异）。身份指数：标题位置 I(L4) = 0.978，I(l) − I(L4) 在所有层都 < 0（L24 −0.011，L48 −0.052）；":" 位置 −0.04 到 −0.64。within-prompt：标题位置 year R^2 从 L4 0.130 到 L48 0.543。⟶ 出处 R13/reports/m3_report.md 全部表 · R13/summary_m3/m3_transfer.csv, m3_identity.csv

解读（作者在 PROG 的原话要点）：三个提示只在标题前 9 个 token 不同、标题完全相同，所以标题位置的 T≈1 只说明"换问法不变"，不说明有超出文本的论文身份。⟶ 出处 PROG 2026-10-29 10:10 条

8 局限与质疑

- 标题最后一个 token 在三种问法里看到的上下文只差前缀 9 个 token，T 高几乎是构造决定的；T(l) − T(L4) 和 I(l) − I(L4) 都为负，说明中间层并没有比 L4 更"跨问法一致"。
- alpha 对所有目标列一起选（一个 ridge），对 field 与 year 可能不是各自最优。
- year_nlr、author_first_nlr（模型是否知道年份/作者首 token）**数据已存但未使用**（用户 2026-10-30 决定照此写明，U46）。⟶ 出处 PROG 2026-10-30 14:00 条 U46
- ":" 位置 T 为大负数（如 L4 year −20.6），是跨问法 R^2 为负除以小的 within R^2 所致，数值本身不宜读太细。

9 状态

已完成 预注册：prereg.json M3（H2：I>0 从 L8–L24 起且 T⩾0.8 于 L16–L40；H1：I≈0 或 T<0.5；L4 为词汇下限；按论文 5 折）。补充 1 改为团队折，补充 2 把 I 规则替换为只看 I(l)−I(L4)、T 为主并看 T−T(L4)、区间改团队版。实际按补充 1/2 执行。报告里的"Pre-registered reading"段落只按 T 的绝对阈值（L16–L40 区间下限 ⩾0.8 → H2）给出，T−T(L4) 和 I−I(L4) 的判读写在表里和 PROG，没有自动判定语句。⟶ 出处 stage13_m3_analyze.py:142-149

M4　作者专用 lens（Stage 12 F1/F2）

1 问题与作用

计划中的定义：用作者问法提示（不是 WikiText）拟合的 Jacobian lens；如果它在中间层读得出名字而 WikiText lens 读不出，说明信息在、只是基底不同（H3）。⟶ 出处 stage1_11_report.html Part IV

实验表 M4 行 · STATUS.md §3 Stage 13 "M4: the Stage 12 author lens"

2 材料

拟合集：Stage 11 样本之外随机 40 篇 target/control 论文（seed 12），提示 = Stage 10 问法 + 真实作者列表；评估：Stage 11 样本，真标题 vs 5 个假标题；评估时排除 3 篇与拟合集共享作者全名的论文（`371_stage12/config/alens_eval_exclude.json`）。⟶ 出处 PROG 2026-10-28 12:00、12:35 条

3 工具

Stage 12 代码 `~/jlens_work/12_stage12/` 的 F1（拟合，40/40）/F2（alens_read）单元；不在 Stage 13 代码里。

4 步骤

见 Stage 12 F1/F2 一节（本节不重复）。Stage 13 没有为 M4 另做计算。

5 公式

标题特异信号 = 真标题 − 假标题的读出差，target − control；定义见 Stage 12 一节。

6 对照

control 组；5 个假标题。

7 结果

名字位置 target − control 在 L24–L56 ⩽ 0（L44–L52 为 −0.041 到 −0.062，区间不含 0），只在 L62 为 +0.238 [0.210, 0.267]；标题位置除 L56（+0.030 [0.000, 0.060]）外区间都含 0 或为负。即作者专用 lens 也读不出中间层的标题特异作者信号。⟶ 出处 /scratch/zhang.zhiqi2/jlens/371_stage12/reports/stage12_report.md 第 73-92 行 · 371_stage12/summary/F2_author_lens.csv

8 局限与质疑

- 只有 40 篇拟合论文；读不出仍不等于不存在（与 M1 同理）。
- prereg.json 没有 M4 的判定规则，M4 的结论是描述性的。

9 状态

已完成 （作为 Stage 12 F1/F2 完成，2026-10-28 20:35）。无 Stage 13 预注册条款，故无偏离可言。

M5　逐位置 probe / patchscope / 名字位置上的真假判别（提议）

1 问题与作用

PROG 原文："M5 (per-position probes, patchscope, true/false at name positions) is proposed and will be decided after M1–M3."报告 Part IV 提到"真/假判别要在上下文里已经出现作者名字的位置上做（M5③）"，因为在标题和 ":" 位置真假标题都是真实论文、设计上不可区分。⟶ 出处 PROG 第 491 行 · stage1_11_report.html Part IV "关于区分真标题和假标题"

2 材料

—

3 工具

无代码（C13 下没有 M5 相关文件）。

4 步骤

未定义到可执行的程度。

5 公式

—

6 对照

—

7 结果

无。

8 局限与质疑

M5 的 patchscope 部分后来在一定程度上由 X4 覆盖（X4 在标题/":" 位置，不在名字位置）。M5①②③的完整定义只在报告里以"M5③"被引用，没有全文。**处置（用户 2026-10-30 决定，U47）**：如果 M5 问的是与 X4 同一个问题，就并入 X4；否则放弃并写明理由。截至 2026-10-30 还没有逐项判断哪些部分与 X4 相同，所以目前没有任何 M5 计算。⟶ 出处 PROG 2026-10-30 14:00 条 U47

9 状态

计划未做（与 X4 重合则并入 X4，否则放弃）。无预注册条款。

X1　跨问法标题 patching（因果）

1 问题与作用

M3 是相关性。X1 做因果检验：把 A 的标题状态（分别来自作者、年份、领域问法）塞进"作者问法 + B 的标题"的运行里，看能否把 A 的第一作者带回来。如果来自年份/领域问法的状态和来自作者问法的状态恢复得一样好，那么第 l 层标题位置上的状态是与任务无关的论文身份。

2 材料

pairs.parquet 的 1,178 篇（test 582 / tune 596）。clean = 作者问法 + A 的标题；corrupt = 作者问法 + B 的标题（同领域、同 token 数、无共同姓氏）；random C = 无关论文 C 在作者问法下的标题状态。层 XP_LAYERS = 4, 8, 16, 24, 32, 40, 48。 ⟶ 出处 C13/381_stage13.py:121, 210-234 · R13/reports/x1_report.md 第 3 行

3 工具

GPU 单元 xpatch 24/24（claim 作业 11428778、11453127）；分析 `C13/stage13_x1x4_analyze.py x1()`（第 40-77 行），作业 11472859，修 X4 格式后重跑 11472901。相似度检查 `C13/stage13_x1_similarity.py`，作业 11474830（用户 2026-10-29 第 5 项）。 ⟶ 出处 R13/logs/claims/xpatch_* · R13/logs/x1x4_*.log · R13/logs/userchecks_11474830.log

4 步骤

1. 标题区间 spanA = 位置 9 … 标题最后一个 token（`title_span`：PREFIX_TOKENS 到 it）；A、B、C 的区间长度相同（构造保证；不同则跳过）。 ⟶ 出处 381_stage13.py:138-140, 216-220
2. 记录 A 在三种问法下、C 在作者问法下，XP_LAYERS 各层标题区间的残差（block 输出）。
3. 对每个来源 src ∈ {authors, year, field, random_c}：一次前向 batch=7，第 i 个元素在第 XP_LAYERS[i] 层把 corrupt 运行**所有标题位置**的 block 输出换成来源状态（forward hook 改 out[e, pos]）。 ⟶ 出处 381_stage13.py:181-197, 227-232
4. 打分：teacher forcing 第一作者名字的全部 token（Stage 11 contexts_of 给出的 token 序列），在最后一层取对应位置 unembed，每个 token 的 log10(1/rank) 取平均。clean、corrupt 用 batch=1 单独算。 ⟶ 出处 381_stage13.py:200-207, 218
5. 分析：test（确认性）、tune、all 三份；团队整群 bootstrap 2,000 次（seed 101）。

5 公式

恢复率 R(src,l) = mean_papers[score(src@l) − score(corrupt)] / mean_papers[score(clean) − score(corrupt)]；bootstrap 时分子分母各自加权求和。差值 R(authors,l) − R(year,l)、R(authors,l) − R(field,l)、R(authors,l) − R(random_c,l)。 ⟶ 出处 stage13_x1x4_analyze.py:44-54

判定（补充 2）：≤L16 只是"文本替换"阳性对照，不下结论；l ≥ 24：若 a−year 与 a−field 两个差的区间都在 [−0.1, 0.1] 内且 a−random_c 区间下限 > 0 → task-independent；若 a−year 下限 > 0.1 →

task-specific；否则 undecided。零假设检查：任一 |R(random_c)| > 0.1 记 NULL FLAG。⟶ 出处 stage13_x1x4_analyze.py:55-61 · prereg_addendum2.json X1

相似度检查（"换问法距离 ÷ 换论文距离"，用 M3 缓存的 9,108 篇）：对每层、每位置、每对问法 (a,b)：同篇跨问法原始余弦 cos(x_a, x_b)（不居中）；相对距离 ‖ x_a − x_b ‖ / ‖ x_a ‖；同问法 a 下与另一篇（固定随机置换，seed 5）的余弦和相对距离 ‖ x_a − x_a^{perm} ‖ / ‖ x_a ‖；**question share** = mean(同篇换问法相对距离) / mean(换论文相对距离)（0 = 问法不改变状态，1 = 换问法与换论文移动一样远）；再对 3 对问法取平均。pos 0 = 标题最后一个 token（X1 替换整个标题区间，这里只用最后一个 token 代表）。⟶ 出处 C13/stage13_x1_similarity.py:1-9（docstring）, 174-187

6 对照

random C（Hana 的建议：同长度无关论文）；L4–L16 作为文本替换的阳性对照；tune/test 按团队分。dry 检查："B 用自己的状态替换 = corrupt"、"最后一层替换所有位置 = clean"。⟶ 出处 381_stage13.py:455-463

7 结果（test，确认性；clean − corrupt = 0.347）

层	R authors	R year	R field	R random C	a − year	a − field	判定
L4	1.004	1.003	1.006	−0.149	+0.001	−0.002	文本对照 · NULL FLAG
L24	0.991	0.985	0.992	−0.097	+0.006 [−0.004, +0.015]	−0.001	task-independent
L40	0.921	0.917	0.915	−0.007	+0.003	+0.006	task-independent
L48	0.424	0.405	0.417	+0.122	+0.019 [+0.012, +0.026]	+0.008	task-independent · NULL FLAG

NULL FLAG：test L4/L8/L16 的 R(random C) 为 −0.149/−0.139/−0.132，L48 为 +0.122；a − random C 始终 ⩾ +0.30。⟶ 出处 R13/reports/x1_report.md test 表与末行 · R13/summary_x1x4/x1_recovery.csv

相似度（标题最后 token）：同篇两问法余弦 L24 0.983、L40 0.977、L48 0.953；不同论文同问法余弦约 0.25–0.47（L4–L48）；question share L4 0.060、L24 0.147、L40 0.165、L48 0.268、L52 0.329。":" 位置相反：share 在 L4–L44 为 2.9–4.9（问法主导）。用户据此认可的措辞："问法让标题状态移动约 15–27%（以换论文为 1），但替换效果不变"。⟶ 出处 R13/reports/x1_similarity.md · R13/summary_m3/x1_similarity.csv · PROG 2026-10-29 12:10、12:40 条

8 局限与质疑

- 替换的是**全部**标题位置；三种问法里标题文本完全相同、前缀只差几个 token，相似度检查显示 L24–L40 状态本来就相近（share 0.15–0.17），所以"恢复一样好"部分是构造所致；但 share 不为 0 而恢复差 ⩽0.02，说明问法带来的那部分差异不影响作者恢复。
- L48 的 a − year +0.019 区间不含 0，但在 ±0.1 等价界内，按规则仍判 task-independent（规则是等价界，不是差异检验）。

- random C 不为 0（L4–L16 为负、L48 为正），预注册要求"所有恢复相对它来读"——报告里给了 a − random C 列。
- clean/corrupt 用 batch 1，替换运行用 batch 7，bf16 下批大小不同会有舍入差（与 X4/X7 的问题同源），L4 的 R≈1.004 可能部分来自此。
- 相似度检查的置换可能把少数论文配给自己（未排除不动点），影响可忽略；只用标题最后一个 token 代表整个区间。

9 状态

已完成 预注册：prereg.json X1 原规则（在 tune 上选 R(authors)−R(random_c) 最大的一层，再在 test 上要求 R(year)、R(field) 超过 R(random_c) 且与 R(authors) 相差 ≤0.1）被补充 2 替换为"全曲线、不选层、≤L16 不下结论、L24 起等价界判定"；tune/test 由补充 1 改为按团队。实际按补充 1/2 执行。相似度检查不在预注册内（用户事后要求）。

X2　只替换子空间（因果）

1 问题与作用

计划：用 M2/M3 的 probe 找出"论文身份"方向（领域、年份、引用）和"作者首 token"方向，只替换或抹掉这些方向，看作者能否被带过去或被破坏；只换身份方向就够 → H2；必须换作者方向 → H3。依赖 M3 残差，约 1 GPU 小时，"等 M3 结果后排队"。 ⟶ 出处 stage1_11_report.html Part IV 实验表 X2 行 · PROG 第 477 行

注意：Stage 10 也有一个叫"X2"的分析（引用数分箱，`stage10_extra.py`），与此无关。

2 材料

—

3 工具

无代码。

4 步骤

—

5 公式

—

6 对照

—

7 结果

无。

8 局限与质疑

prereg.json 规定"X2 跑之前另写补充"，该补充未写；H3 因此没有因果检验。**X2 暂停**（用户 2026-10-30 决定，U48）。⟶ 出处 PROG 2026-10-30 14:00 条 U48

9 状态

计划未做 （暂停）

X3　同义改写标题 vs 词重合近邻

1 问题与作用

把标题换成意思相同、用词不同的改写，如果模型内部状态和作者得分仍接近原标题、而不如一个词重合同样多的别篇论文标题接近，说明它认的是"这篇论文"而不是字面。Stage 13 里被视为最关键的实验。

2 材料

Stage 11 样本 1,186 篇的改写（para_gen 24/24，claim 作业 11374284、11453127）。⟶ 出处 R13/raw/para_gen/shard_000…023.npz (paper_id, title, paraphrase, auto_ok, word_jaccard) · R13/logs/claims/para_gen_*

3 工具

生成与自动检查：`381_stage13.py` do_para_gen / para_checks；NLI：`C13/stage13_para_nli.py`（作业 11475175）；Qwen 判断器：`C13/stage13_gpu_followup.py --sample --judge`（GPU 作业 11485349）。para_run（真正的实验）：do_para_run 已写好，但只在 `R13/logs/para_ok` 存在时才排队（2026-10-30 该文件不存在，raw/para_run 不存在）。⟶ 出处 381_stage13.py:66, 268-325

4 步骤

(a) 生成：原始文本（不用对话模板，避免 thinking），贪心（do_sample=False），max_new_tokens=60，pad=eos；解码后只取第一行、去首尾空白和引号。提示原文：⟶ 出处 381_stage13.py:125-126, 276-286

```
Rewrite the title of a research paper using different words while keeping exactly
the same meaning. Do not add any information.
Original title: {t}
Rewritten title:
```

(b) 自动检查 para_checks：通过需同时满足 —— 改写非空；年份字符串不出现在改写里；每位作者的姓（作者名按空格切后的最后一段，小写）不作为**子串**出现在改写的小写文本中；字符长度比 0.5–

2.0。另算原版 word Jaccard（只记录，不参与通过判定）。⟶ 出处 381_stage13.py:268-273

结果：1,186 条中 auto_ok 1,142（96.3%），被拒 44 条。本节作者逐条复算拒因（互不重叠）：37 条因姓氏子串命中，其中 36 条是词内子串（如 "li" 在 "application" 里、"le"、"lo"、"he"、"lu"、"tan"、"ding"），1 条是整词 "and"（某作者姓氏为 "And"）；2 条含年份（原标题本身含年份，如 "…from 2012"）；5 条长度比越界（其中 2 条是 "Sure, here's a rewritten title…" 这类开场白）。所以"误拒 37 条"的原因是子串匹配而非整词匹配。作者提议保留这些保守拒绝。⟶ 出处 R13/summary_x3/para_nli.csv（auto_ok）与 R10/cohort/papers.parquet 复算 · PROG 2026-10-29 04:40 条 **用户 2026-10-30 决定：37 条姓氏误拒维持拒绝**，写入补充 5（U49）。⟶ 出处 PROG 2026-10-30 14:00 条 U49

(c) 完全相同的改写：标题与改写都小写、去掉所有非 [a-z0-9] 字符后相等即"identical"。共 27 条，全部 auto_ok。用户同意在分析中排除（2026-10-29 10:40）；但 do_para_run 目前只按 auto_ok 过滤，排除尚未写入代码，计划随补充 5 一起改。⟶ 出处 C13/stage13_para_nli.py:32 · para_nli.csv identical 列 · PROG 2026-10-29 10:40 条第（3）项 · 381_stage13.py:313

(d) Jaccard 分词：原版 `w()` / `title_words()`：小写，非字母数字变空格，只保留长度 >3 的词，不还原词形。于是 "of"、"M22" 被丢掉，symmetric ≠ symmetrical，样本 #3 "Symmetric generation of M22" → "Symmetrical creation of M22" 算成 0.00；13 个标题只剩 ≤2 个计数词，130 个 ≤4 个。修正版 jaccard2：小写、非 [a-z0-9] 变空格、去掉 sklearn ENGLISH_STOP_WORDS、任意长度都保留、简单去后缀词形还原（按顺序试 ically, ation, ities, ness, ment, ical, ally, ing, ity, ies, ive, al, ed, es, ic, ly, s，词长需 > 后缀长+2），#3 变为 0.50。分布：原版中位数 0.571（q10 0.231，q90 0.875），jaccard2 中位数 0.625（q10 0.273，q90 1.0）。⟶ 出处 381_stage13.py:269, 289 · C13/stage13_para_nli.py:15-26 · para_nli.csv（word_jaccard, jaccard2）· PROG 2026-10-29 12:40 条（2d）

(e) 近邻 D（lexical_neighbour）：同领域、不同论文、无共同姓氏，词 Jaccard（用原版 title_words）≥ 改写的 Jaccard 中最接近的一篇；若没有 ≥ 的，取最高的。它用的是原版分词，所以受 (d) 的问题影响；但 para_run 还没跑，D 还没有被实际匹配过，因此目前**没有结果受影响**；修正计划随补充 5 一起改代码。⟶ 出处 381_stage13.py:292-304, 315 · PROG 第 103 行

(f) NLI 双向蕴含：cross-encoder/nli-deberta-v3-large，原→改、改→原两个方向，batch 16、max_length 256，两个方向的 argmax 都是 entailment 才算通过。⟶ 出处 C13/stage13_para_nli.py:12, 35-46 结果：auto_ok 且非 identical 的 1,115 条中 96 条（8.6%）不通过（全部 1,186 条中 100 条不通过）。在 20 条人工样本上：漏掉 #3（术语替换）、#15、#4；抓到 #5；误拒 #2、#11、#13、#14（某一方向 <0.5，如 #2 正向 0.000、反向 0.994）。结论：单靠 NLI 不够。⟶ 出处 R13/summary_x3/para_nli.csv（nli_both, p_entail_fwd, p_entail_bwd）· R13/logs/para_review_sample20.csv · PROG 2026-10-29 13:45 条

(g) Qwen3-32B 判断器（用户 2026-10-29 同意）：盲测 A/B，两种顺序各问一次（A=原/B=改，A=改/B=原），对话模板、关闭 thinking；分数 = 第一个答案 token 上 P(Yes)/(P(Yes)+P(No))（只在 "Yes"、"No" 两个 token id 上做 softmax）；两种顺序都 >0.5 才算"同义"。提示原文：⟶ 出处 C13/stage13_gpu_followup.py:25-29（JUDGE）, 56-72（judge）

```
Do these two research paper titles have exactly the same meaning?
Rules: any added, removed or changed specific information counts as a change. A
```

```
technical term replaced by a near-synonym that does not keep its precise
technical sense (for example "generation" of a group replaced by "creation")
counts as a change. Only rewording that keeps every fact and every technical
term's precise meaning counts as the same meaning.
Title A: {a}
Title B: {b}
Answer with Yes or No.
```

校准（作业 11485349，20 条用户人工标注过的样本 `logs/para_review_sample20.csv`）：两种顺序都通过的只有 4/20（#8、#9、#19、#20）。混淆表：

人工标签	判断器：同义	判断器：改变
同义（16 条）	4	12
改变（3 条：#3、#5、#15）	0	3
边界（#4）	0	1

一致率 7/19 = 0.37（不计 #4）。方向没有反：3 条"改变"的两顺序平均 P(Yes) 为 0.000–0.009，16 条"同义"为 0.024–0.940，按平均分完全分开（排序完美）；问题在阈值 0.5 且要求两种顺序都过太严，加上顺序不对称很大（如 #14：0.881 vs 0.182）。全量判断没有启动。⟶ 出处 R13/followup/judge_sample.csv（p_yes_title_first, p_yes_para_first, judge_both）· R13/logs/fugpu_11485349.log "judge: 20 items, both-yes 0.20" · PROG 2026-10-30 两条 这 20 条人工标签（A 组）已存档为 `R13/logs/para_review_sample20_labels.csv`（U50 已解决）。⟶ 出处 PROG 2026-10-30 12:30 条 U50

自评偏差：改写和判断都是同一个 Qwen3-32B；对策为盲 A/B、两种顺序、用人工标签校准、并要求独立的 NLI 也通过（"补充 5 的规则将要求两者都过"）。⟶ 出处 stage13_gpu_followup.py:3-9（docstring）

(h) 新 20 条人工标注（B 组）：`R13/logs/para_review_sample20_b_labels.csv`（已排除前 20 条和完全相同的改写），标注来源一栏写"人工确认（参考了 Claude 的建议）"：**保留 11、边界 6（B6、B7、B8、B13、B16、B19）、改意思 3（B11、B15、B20）**，各条附理由。⟶ 出处 para_review_sample20_b_labels.csv 列 human_label、reason、label_source（本次计数）· PROG 2026-10-30 14:00 条 复核过

(h′) 判断器 v2（`C13/stage13_judge2.py`，tiny 11501023 通过，GPU 作业 11501072）：提示写入用户的 5 条标准（同一研究对象、同一范围、术语不被换成普通词或别的术语、数字/名字/缩写都在、关系方向不变；只是换说法或换语序仍算 Yes）；盲 A/B、两种顺序、chat 模板、关闭 thinking，分数 = P(Yes)/(P(Yes)+P(No))；在 A 组 20 条上按两顺序平均 P(Yes) 选阈值使一致率最大（边界算"不保留"），阈值 0.024，**校准一致率 0.90**；在 B 组 20 条（用户在判断器看到之前标好）上检验：**一致率 0.55**（边界算不保留），去掉 6 条边界后 0.79（14 条）——判断器把 20 条全判为保留，包括 6 条边界和 3 条改意思。用户的规则是低于 0.80 就不用，所以**判断器 v2 不用**，改走备用方案（人工核对 + 有/无该部分样本的敏感性分析），补充 5 草稿待用户确认。⟶ 出处 R13/followup/judge2_summary.json（threshold、calibration_agreement、test_agreement_*、test_confusion、verdict）· judge2_sample.csv · stage13_judge2.py:1–11, 78–86 · R13/logs/judge2_11501072.log · PROG 2026-10-30 15:00 条 复核过 作废 （判断器路线）

(i) para_run（计划）：对每篇 A，在作者问法里分别放原标题、改写、搭档 B、近邻 D 四个版本，存 15 层 × 2 位置残差和第一作者得分。⟶ 出处 `381_stage13.py:307-325`

5 公式

原版 Jaccard = |W(t) ∩ W(p)| / max(1, |W(t) ∪ W(p)|)，W = 长度 >3 的小写字母数字词；jaccard2 同式，W 换成去停用词、任意长度、去后缀的词集。计划中的主指标（补充 4）：G′(l,pos) = cos(原, 改写) − cos(原, D)，要求 G′(l) > 0 且 G′(l) − G′(L4) > 0（团队区间不含 0），tune 上找、test 上确认；行为：第一作者得分 改写 − D > 0。原指标 G = cos(原, 改写) − cos(原, B) 降为次要。补充 2 另要求在回归里用词 Jaccard 和"共享稀有词比例"（稀有 = 队列标题文档频率 <10）作协变量。⟶ 出处 `prereg.json X3` · `prereg_addendum2.json X3` · `prereg_addendum4.json X3`

6 对照

近邻 D（主）；搭档 B（次）；词 Jaccard 协变量；意义检查（NLI + Qwen 判断器 + 人工 20+20）。

7 结果

只有生成和意义检查的结果（见上）。X3 主实验无结果。

8 局限与质疑

- 姓氏检查是子串匹配，误拒 37 条（不会放进错误样本，只是少了样本，偏保守）。
- 原版 Jaccard 低估词重合，若不修，近邻 D 会被匹配得词重合偏高/偏低，直接影响主对照。
- 意义检查没有可接受的自动标准：NLI 误拒 4/20、漏检 3/20；第一版判断器方向对但阈值过严；判断器 v2 在新 20 条上一致率 0.55 < 0.80，不用。
- 改写与判断用同一模型（自评）。

9 状态

para_gen 已完成；NLI 已完成；判断器 v1 校准与 v2 检验已完成，**判断器 v2 不用**（0.55 < 0.80）；补充 5 计划未做（草稿待用户确认）；para_run 计划未做（未放行，`logs/para_ok` 不存在）。预注册：prereg.json 的 X3 inclusion 要求"自动检查 + 人工抽查（para_ok）"——正在执行中；排除完全相同、换 Jaccard、意义检查规则都属于将写的补充 5，现在还没有实际偏离（因为主实验没跑）。用户已定的放行原则：判断器一致率 ≥ 80% 才用，否则走备用方案（人工核对 + 有/无敏感性），写好补充 5 后再放 para_ok；判断器已不可用，所以走备用方案。⟶ 出处 `PROG 2026-10-30 14:00 条 U51、15:00 条` **未知/需确认：向用户确认（补充 5 的最终条款，草稿已发出，U51）**

X4　Patchscope：从论文自己的状态解码年份、第一作者、领域

1 问题与作用

把 A 在作者问法（不含年份）下某层的状态注入一个只有占位符 X 的提示，看模型能否据此说出 A 的年份、第一作者、领域。年份从不出现在源提示里，所以 D(年份) > 0 只能来自认出这篇论文。H2 预测

年份/领域最早可解码的层 ⩽ 第一作者。

2 材料

pairs 的 1,178 篇（tune 596 / test 582）。源状态：A 的作者问法提示（同 M3 的 authors 模板），取第 l 层在**标题最后一个 token**（pos=title）或**最后的 ":"**（pos=colon）的 block 输出；对照源：搭档 B 的同一状态。层 PS_LAYERS = 8, 16, 24, 32, 40, 48, 56。目标提示（PS_TARGETS 原文）：⟶ 出处 C13/381_stage13.py:122-124, 243-265

```
year  : Paper: X\nThe paper was published in the year
author: Paper: X\nThe first author of the paper is
field : Paper: X\nResearch field:
```

答案：year = ' '+年份 的 token；author = 第一作者名字的全部 token（Stage 11 contexts_of）；field = ' '+领域名 的 token。⟶ 出处 381_stage13.py:237-240

3 工具

GPU 单元 pscope 24/24（claim 作业 11453127）；分析 `stage13_x1x4_analyze.py x4()`（第 93-146 行），作业 11472859 / 11472901。文本基线 `stage13_x4_textbase.py`（11474830，阳性对照 11475014）、`stage13_x4_textbase2.py`（11475517）。逐 token 复算 `stage13_gpu_followup.py`（11485349 失败停止；tiny 作业 11485182、11485266、11499990、11500050）。

4 步骤

1. 找占位符：目标提示 tokenize 后 offset 覆盖 "X" 的那个 token 的位置 xi（dry 检查唯一且解码为 "X"）。⟶ 出处 381_stage13.py:246-248, 466-468
2. none：不注入，teacher forcing 打分。
3. self / partner × title / colon：一次前向 batch=7，第 i 个元素在第 PS_LAYERS[i] 层把 xi 位置的 block 输出换成源状态（同一层注入）；打分同 X1：答案每个 token 的 log10(1/rank) 取平均。⟶ 出处 381_stage13.py:256-263
4. 分析：tune/test/all，团队整群 bootstrap 2,000 次（seed 202）；最早可解码层在 tune 上找（区间下限 >0 的最小层），在 test 同一层确认（下限 >0）。认得出：全部论文上，按引用数箱内 target − control 的 (self − partner)，箱内两组各 ⩾10 篇，权重 n_a · n_b/(n_a+n_b)，团队 bootstrap（seed 303+l）。⟶ 出处 stage13_x1x4_analyze.py:80-122

5 公式

D(answer,pos,l) = mean_papers score(self@pos@l) − mean_papers score(partner@pos@l)，score = mean_tokens log10(1/rank)。⟶ 出处 stage13_x1x4_analyze.py:101

文本基线 —— kNN：见第 7 项中的式子。LR：D_text2 = ½ Σ_{role} [log10 p_m(tok_role | title_A) − log10 p_m(tok_role | title_B)]；"可由文本解释的份额" = 1 − 截距 / mean(D_Qwen)（D_Qwen 对

D_text 的一元线性回归)。⟶ 出处 stage13_x4_textbase.py:80-85 · stage13_x4_textbase2.py 文件第 105-106 行

6 对照

partner 源（同领域、同长度）；none；文本基线两版；逐 token 复算自检。

7 结果

答案	位置	tune 上最早层	test 上该层 D	确认
first author	title	L8	+0.046 [+0.035, +0.059]	是
year	title	L40	+0.013 [+0.008, +0.018]	是
field	title	无	—	—
author	colon	L24	+0.001 [−0.000, +0.003]	否
field	colon	L40	+0.003 [−0.002, +0.007]	否
year	colon	无	—	—

按规则顺序"不确定"（三者未全部可解码）；但第一作者（L8）早于年份（L40），与 H2 预测的方向相反。title 位置 author 的 D 在 L16–L40 为 +0.056 到 +0.068，L56 掉到 +0.001。认得出：author/title 的 target − control 在 L8–L40 为 +0.040 到 +0.060（区间不含 0），year 都含 0。⟶ 出处 R13/reports/x4_report.md · R13/summary_x1x4/x4_earliest.csv, x4_decodability.csv, x4_recognition.csv

文本基线 1（kNN，`stage13_x4_textbase.py`）：标题表示用 TF-IDF（1–2 gram，min_df 2）或 BERT（最后一层均值、居中、单位化）；对每个查询标题在全队列 9,108 个标题中找余弦最近的 k=50 个（排除查询标题本身和 A 的整个作者团队），p(s | t) = (Σ_k sim_k · [姓_k = s] + α · prior(s)) / (Σ_k sim_k + α)，α=1，prior = 队列第一作者姓氏频率；D_text = log10 p(s_A | A 标题) − log10 p(s_A | 搭档标题)。结果：TF-IDF +0.007 [−0.013, +0.028]，BERT +0.006 [−0.025, +0.038]；Qwen D(L8)（全部 1,178 篇）+0.048，与文本 D 的相关 ≈0。但阳性对照（EXCLUDE_TEAM=0，允许同团队）也没有信号：TF-IDF −0.025，BERT −0.159（BERT 为负是因为搭档标题可以检索到 A 本身），所以**阳性对照未通过**，这一版的"文本解释不了"不能成立。⟶ 出处 C13/stage13_x4_textbase.py:19-43 · R13/reports/x4_textbase.md, x4_textbase_posctrl.md · PROG 2026-10-29 12:10、13:10 条

文本基线 2（监督 LR，`stage13_x4_textbase2.py`）：从标题文本预测第一作者的 Qwen 名字 token，两个角色分开：given_first（名字首 token）、surname_first（姓的首 token）。特征 (a) BERT 均值池化、标准化，(b) TF-IDF 1–2 gram 经 TruncatedSVD 降到 300 维、标准化。类 = 训练折中该角色出现 ⩾5 次的 token id，其余并成 OOV 类（按训练频率分摊概率）。5 个团队折；C 在 {0.03, 0.3, 3} 中、第 0 折上按平均 log10 p 选（结果都选 0.03）。阳性对照：留出 log10 p 是否高于频率先验。D_text = 两个角色的 [log10 p(tok | 本标题) − log10 p(tok | 搭档标题)] 的平均，用 A 被留出的那一折的模型。结果：given_first **通过**（BERT +0.025，TF-IDF+SVD +0.091 log10，2,442 篇）；surname_first **未通过**（−0.163 / −0.104，9,108 篇）。D_text：BERT +0.048 [+0.025, +0.072]（mean/sd 0.12，胜率 0.559），TF-IDF +0.013 [−0.006, +0.034]；与 Qwen D(L8) 的逐篇相关 +0.003 / +0.005；Qwen D(L8) 对文本分回归的截距 +0.048，"可由文本解释的份额" 0.00；文本无帮助处 Qwen D(L8) 仍为 +0.049。⟶

出处 C13/stage13_x4_textbase2.py（文件第 16、37、58-88 行）· R13/reports/x4_textbase2.md · R13/summary_x1x4/x4_textbase2_posctrl.csv · PROG 2026-10-29 14:00 条

逐 token 自检失败与修法：原 X4 只存了作者答案各 token 的平均分。为了按 token 角色报告（补充 4），`stage13_gpu_followup.py` 在 L8、L40 的 title 位置重算每个 token 的分数，并要求逐 token 平均复现已存的 author@{self,partner}@title@{8,40}（容差 0.05 log10）。第一版只在 L8、L40 两层注入，即 **batch=2**，而原 X4 在一次前向里注入全部 7 层（**batch=7**）。GPU 样本运行 11485349（shard 0，50 篇，200 个 篇×源×层 值）：max |diff| 0.0683 > 0.05 → 写 `followup/X4TOK_CHECK_FAILED`（内容 "max |diff| 0.0683"），退出码 3，未进入全量。本节作者用 `followup/x4tok/shard_000.csv` 与 `raw/pscope/shard_000.npz` 复算：200 个值 max |d| 0.0683，平均 |d| 0.0068，平均 d −0.0003，sd 0.011，>0.05 的 2 个，>0.02 的 9 个，逐位相同只 2%；D(self−partner) 与已存值的相关 L8 0.993、L40 0.983；shard 0 的 L8 平均 D：batch 2 为 +0.0039，已存 +0.0061。诊断：bf16 下批大小不同导致的舍入差。⟶ 出处 R13/followup/X4TOK_CHECK_FAILED · R13/logs/fugpu_11485349.log 末两行 · R13/followup/x4tok/shard_000.csv · PROG 2026-10-30 两条

修法（2026-10-30 写进工作副本，之后在 GPU 上跑过，结果见下段）：每个 shard 同时算 b7（注入全部 7 个 PS_LAYERS，与原 X4 完全相同）和 b2（只注入 L8、L40），存到 `followup/x4tok2/`。样本规则：b7 的逐 token 平均必须复现已存分数（代码中仍以 max |diff| < 0.05 判定），否则写 X4TOK_CHECK_FAILED 并退出（致命）；全量规则：若 L8 的平均 D 在 b7 与 b2 之间相差 > 0.005，写 `X4TOK_BATCH_SENSITIVE` 并停止（退出码 4）。用户把这个自检定为"正确性检查，不是旁支"，所以与一般的"容差不达标只记 WARN"规则不同，这里是致命的。tiny 测试 11499990、11500050 完成。⟶ 出处 C13/stage13_gpu_followup.py:75-100, 124-146（文件行号）· PROG 2026-10-30 第（1）项、"x4tok check made fatal" 条 · sacct

修法在 GPU 上的结果（2026-10-30）：样本 11500083（shard 0）b7 与已存分数逐值相同（exact share 1.000，max |d| 0）→ CHECK PASS，说明之前的不一致完全来自批大小。全量 `--all` 作业 11500437：b7 与已存 X4 分数 **4,712 个值完全相同**（CHECK PASS）；b2 与 b7 逐 token 最大差 0.602、平均 0.010；L8 平均 D（1,178 篇）**b7 +0.0484、b2 +0.0490，差 0.0006 < 0.005**（L40 +0.0694 / +0.0699），批大小不影响结论。⟶ 出处 R13/logs/fugpu_11500437.log:30–34 · R13/followup/x4tok2/shard_000–023.csv · PROG 2026-10-30 11:50、15:00 条 复核过

按 token 角色（补充 4；b7、L8、title 位置、团队 bootstrap 种子 47）：

第一作者的 token 角色	篇数	Qwen D(L8) [95% 团队区间]
姓的首 token（surname_first）	1,178	**+0.066 [+0.046, +0.088]**
名的首 token（given_first）	357	+0.018 [−0.003, +0.041]（含 0）
首字母（initial_first）	880	+0.017 [+0.006, +0.029]

在名的首 token 上，BERT 文本基线（只看有 given_first 的 357 篇）+0.088（均值/标准差 0.13）**大于** Qwen 的 +0.018（均值/标准差 0.09），两者逐篇相关 0.07。Qwen 的 L8 信号主要在姓上，而姓这个角色上文本基线的阳性对照没有通过。所以**不能排除 L8 的第一作者信号可以由标题文字解释**（措辞待用户最终确认，这里先写事实）。⟶ 出处 R13/summary_x1x4/x4_rolesplit.csv（~/jlens_work/13_stage13/stage13_x4_rolesplit.py，团队 bootstrap 2,000 次，种子 47）

8 局限与质疑

- 源状态和注入都在同一层；注入单一位置。D 很小（log10 名次 0.01–0.07 量级），解读为"内部偏好"，不是生成。
- field 在 test 的 L8 有 +0.074 [0.005, 0.143]，但 tune 上未过，按规则不算。
- 文本基线 1 阳性对照失败；基线 2 只在名字首 token 上通过、姓氏上失败，所以"文本解释不了 Qwen 的姓名信号"只在名字首 token 上有对照支撑。基线 2 的搭档标题可能在训练折中（会推高 D_text）。
- 本节作者复核代码发现：基线 2 对没有 given_first token 的论文（1,178 篇中 821 篇；第一作者只有首字母的情形），given_first 一项仍按 OOV 类概率算了 log p(本) − log p(搭档)（与姓名无关的差），并进入 D_text 的两角色平均；这 821 篇的 BERT given_first 项平均 +0.027，全部非零。⟶ 出处 `stage13_x4_textbase2.py 文件第 80-83 行（ya 含 −2 的行仍走 logp_matrix 的 OOV 分支）· R13/summary_x1x4/x4_textbase2_perpaper.csv 复算` 已由总编按实际角色重算（见本节末"总编更正"），结论不变（U52 已解决）。
- 按 token 角色的结果来自全量逐 token 复算（见上）；角色拆分的数字已存为结果文件 R13/summary_x1x4/x4_rolesplit.csv（stage13_x4_rolesplit.py，2026-10-30 重跑与会话内数字一致）。
- 修法上线后 `--all` 只要 `followup/X4TOK_CHECK_FAILED` 存在就拒绝启动（stage13_gpu_followup.py 第 105 行）；b7 样本通过后，这个标记已改名为 `followup/X4TOK_CHECK_FAILED.resolved_b7_reproduces_exactly_2026-10-30`（未删除），全量随即运行（U53 已解决）。⟶ 出处 `ls R13/followup/ · PROG 2026-10-30 11:50 条`

9 状态

pscope 与分析 **已完成**；文本基线 **已完成**（基线 1 阳性对照失败，基线 2 部分通过）；逐 token 复算 **复核过**（第一次 batch 2 自检失败；b7 修法全量 11500437 逐值复现、批大小不敏感）；按 token 角色报告已完成（X4）。预注册：prereg.json X4（D、tune 找 test 确认、H2 顺序、认得出）+ 补充 1（团队 tune/test）+ 补充 2（团队区间）——照做，无偏离；补充 4 的 token 角色报告 2026-10-30 完成；文本基线与逐 token 复算是事后加的（用户 2026-10-29/30 要求），不在预注册内。

总编更正（2026-10-30，CPU 复算，脚本未改）：只用每篇论文实际具有的角色重算 D_text（没有 given_first 的 821 篇只算 surname_first）。出处：⟶ 出处 `381_stage13/summary_x1x4/x4_textbase2_perpaper.csv 列 lp_bert_given_first / lp_bert_surname_first / lp_tfidf_svd_*；given_first 归属取自 351_stage10/cohort/name_tokens.parquet（author_index=0, role=given_first）；团队 bootstrap 2,000 次，种子 43；PROGRESS.md 2026-10-30 11:40 条`

比较	篇数	D_text（团队 95% 区间）	均值/标准差	胜率	与 Qwen D(L8) 逐篇相关
BERT，原版（含错误项）	1,178	+0.048 [+0.025, +0.072]	0.12	0.559	+0.003

BERT，修正后（有哪项算哪项）	1,178	+0.048 [+0.014, +0.083]	0.08	0.542	+0.014
BERT，只看 given_first（有该 token 的论文）	357	+0.088 [+0.022, +0.155]	0.13	0.583	−0.013
TF-IDF+SVD，修正后	1,178	+0.015 [−0.014, +0.043]	0.03	0.499	−0.011
Qwen D(L8)，全部	1,178	+0.048 [+0.040, +0.057]	0.31	0.615	—
Qwen D(L8)，有 given_first 的 357 篇	357	+0.037 [+0.022, +0.053]	0.24	0.588	—

注意：阳性对照中 given_first 通过的 2,442 篇是全体 cohort 里有该 token 的论文，X4 样本中只有 357 篇。之前对用户报告的"0.12 vs Qwen 0.31"用的是含错误项的数字，已于同日更正。结论（文本信号与 Qwen 逐篇信号不相关）不变。

X5　层序量化

1 问题与作用

计划：在同一批论文上画三条曲线——论文身份能被检索（M3 跨问法 top-1 检索）、作者首 token 能被 probe 读出（M2/E4b）、名字能被 lens 读出（Stage 10）——标出各自开始的层。H2 预测：身份最早，然后 probe 读出作者，最后 lens 读出名字（L57–L61）。⟶ 出处 `prereg.json X5 · stage1_11_report.html Part IV X5 行`

2 材料

计划用 M3 + X4 + 已有数据，CPU。

3 工具

无代码。

4 步骤

—

5 公式

—

6 对照

—

7 结果

无。

8 局限与质疑

M3 显示标题位置的跨问法一致性在 L4 就已最高（不随层上升），"身份何时出现"在 M3 上可能无法定义起点。**用户 2026-10-30 决定：X5 在 M3、X4 全量完成后用已有数据在 CPU 上做**（U54；M3 已完成，X4 逐 token 全量 11500437 已完成），尚未开始。⟶ 出处 PROG 2026-10-30 14:00 条 U54

9 状态

计划未做（有预注册条款 prereg.json X5；待在 CPU 上做）

X6　Stage 11 E1 尾部替换换成另外 4 个假标题

1 问题与作用

Stage 11 E1（把真标题运行的尾部状态换进假标题运行，看从哪层起能恢复真作者）只用了 false_0。X6 用 false_1…false_4 重做，看 L48/L52 的时序是否只是 false_0 特有。

2 材料

Stage 11 样本（1,186 篇，名字 token 19,077 行）；false_0 用 Stage 11 原输出 `R11/raw/patch`；false_1–4 用 `R13/raw/xf_patch/false_j/` 各 24 shard；外加 xf_patch false_0 shard 0 作复现检查（共 97 个单元）。层 = Stage 11 PATCH_LAYERS：24, 32, 40, 48, 50, 52, 54, 56, 57, 58, 59, 60, 61, 62, 63。⟶ 出处 ~/jlens_work/11_stage11/361_stage11.py:52 · R13/raw/xf_patch/（97 个 npz）

3 工具

GPU：`381_stage13.py do_xf_patch`（第 340-354 行）直接调用 Stage 11 的 `s11.do_patch`，只临时改 `FALSE_FOR_PATCH`，并按 Stage 11 冻结时的公式为每个假标题重算对齐尾部长度 = min(公共后缀长, 真提示中标题之后的长度, 假提示中标题之后的长度)。claim 作业：11413886、11374283、11385971、11405680、11414108、11414120、11415844、11416869、11428778、11430308。分析：`C13/stage13_x6_analyze.py`，作业 11461197（先自检再正式）。⟶ 出处 R13/logs/claims/xf_patch_* · C13/stage13_x6_analyze.sbatch

4 步骤

1. **复现检查**：单元 ('xf_patch','false_0',0) 与 Stage 11 `raw/patch/shard_000.npz` 比较——row_id、layers、rank_true、rank_false、rank_patch（516 行 × 15 层）**逐个名次完全相同**（PROG 记录，本节作者 2026-10-30 用 np.array_equal 再核一次，5 个数组全部 True）。⟶ 出处 PROG 2026-10-28 21:30 条 · R13/raw/xf_patch/false_0/shard_000.npz vs R11/raw/patch/shard_000.npz
2. 每行 nlr = log10(1/rank)，先在论文内对名字 token 平均。⟶ 出处 stage13_x6_analyze.py:21, 45-47
3. 每个假标题、每组（target/control/all）、每层算 R；团队整群 bootstrap 2,000 次（seed 13），5 个标题共用同一套抽样，所以平均和"标题 j − false_0"是配对的。⟶ 出处 :50-70

4. 分析脚本自检：把 5 个标题都指向 Stage 11 false_0 的 shard（符号链接，`R13/test_x6/`），要求 R 与 Stage 11 E1 相同、标题间差为 0：max|diff| 2.22e-16，差值 0 → PASS。⟶ 出处 `C13/stage13_x6_analyze.sbatch` · `R13/logs/x6_11461197.log`（SELFTEST 行）

5 公式

R(l) = mean_papers(nlr_patched@l − nlr_false) / mean_papers(nlr_true − nlr_false)；平均 R = 5 个标题 R 的简单平均（bootstrap 同样逐次平均）。⟶ 出处 `stage13_x6_analyze.py:60-66`

判定（补充 3）：5 个标题平均 R(L48) ∈ [0.70, 0.95] 且平均 R(L52) ⩾ 0.95，并且每个假标题 |R(L48) − R_false_0(L48)| ⩽ 0.10 且 R(L52) ⩾ 0.90。⟶ 出处 `prereg_addendum3.json X6` · `stage13_x6_analyze.py:85-92`

6 对照

false_0 本身（与 Stage 11 相同）；标题间配对差；target/control/all。

7 结果

平均 R(L48)：target 0.794 [0.777, 0.811]，control 0.876，all 0.821；平均 R(L52)：0.987、1.012、0.995。每个标题 L48 都在 false_0 的 0.02 以内（target 0.788–0.803）。三组都 **HOLDS**。分母 true − false：target 0.418–0.439，control 0.205–0.221。⟶ 出处 `R13/reports/x6_report.md` · `R13/summary_x6/x6_recovery.csv` · `R13/summary_x6/x6_verdict.json`

8 局限与质疑

- xf_patch 的 6 个 shard 出自较早的脚本版本（见第 0 节），据记录只差日志行。
- 只换了假标题，问法仍是 Stage 10 含年份的问法；年份在真/假运行中相同。

9 状态

已完成 预注册（补充 3）在 X6 无数据时写好；完全按其执行，无偏离。

X7　在假标题运行上做同样的注意力阻断

1 问题与作用

Stage 11 E2：从第 l 层起禁止注意到标题，真作者得分掉多少。阻断本身是"离开分布"的破坏，可能不论标题真假都会伤害得分。X7 在 false_0 运行上做同样的阻断，把"破坏"减掉，剩下的才是论文特有的损失。

2 材料

Stage 11 样本，19,077 行名字 token。真运行的阻断结果用 Stage 11 `R11/raw/knockout`；假运行新算 `R13/raw/ko_false/false_0/` 24 shard（claim 作业 11374284、11454922）；分母用 Stage 11 `raw/patch` 的 rank_true、rank_false。阻断层 KO_LAYERS = 0, 24, 32, 40, 48, 50, 52, 54, 56, 58, 60,

62, 64（64 = 什么都不挡，但仍走显式 mask）。⟶ 出处 ~/jlens_work/11_stage11/361_stage11.py:53 · C13/stage13_x7_analyze.py 文件第 35-37 行

3 工具

GPU：`381_stage13.py do_ko_false`（第 357-369 行），用 Stage 11 的 ko_mask 与 run_final（batch = 13 个阻断设置，加性 mask：从标题之后的查询位置到所有标题位置置为 dtype 最小值），在 false_0 提示上 teacher force 真论文的作者名。分析：`C13/stage13_x7_analyze.py`，作业 11470471（自检里把 `D.T` 写成了 pandas 转置而崩溃）→ 修为 `D['T']` 并把自检包成只能 WARN，作业 11470524。⟶ 出处 361_stage11.py:254-271 ko_mask · PROG 2026-10-29 07:40 条

4 步骤

1. 按 row_id 合并 patch（true/false）、Stage 11 knockout（T）、ko_false（F）。
2. 每行 nlr = log10(1/rank)，论文内平均。
3. 团队整群 bootstrap 2,000 次（seed 17），T、F、S 用同一套抽样。⟶ 出处 stage13_x7_analyze.py 文件第 50 行
4. 自检（只 WARN）：T 必须复现 Stage 11 `summary/E2_knockout.csv` 的 loss_fraction：max|diff| 9.9e-17；F 中"不挡"一列（kF64）应与 Stage 11 的 rank_false 逐行相同：只有 59.63% 的行相同 → WARN。

5 公式

T(l) = mean(kT64 − kT_l) / mean(true − false)；F(l) = mean(kF64 − kF_l) / mean(true − false)；S(l) = T(l) − F(l)（论文特有损失）。k 均为论文平均的 nlr。⟶ 出处 stage13_x7_analyze.py 文件第 3-8、56-60 行

判定（补充 3）：S(L52) ⩽ 0.05 且 S(L0) ⩾ 0.5；另报告 S(L0)/T(L0)。⟶ 出处 prereg_addendum3.json X7 · stage13_x7_analyze.py 文件第 81-86 行

6 对照

F 本身就是"离开分布破坏"的对照；target/control/all。

7 结果

组	S(L0)	F(L0)	S(L52)	判定
target	+1.000	−0.161	+0.018 [+0.010, +0.027]	HOLDS
control	+1.012	−0.196	+0.051 [+0.033, +0.068]	**DOES NOT HOLD**（超阈值 0.001）
all	+1.004	−0.173	+0.029 [+0.021, +0.038]	HOLDS

F < 0：在假标题运行里挡住（假）标题反而帮助真作者。control 的 S 在 L54 为 0.018，L56 起为 0。⟶ 出处 R13/reports/x7_report.md · R13/summary_x7/x7_knockout.csv · R13/summary_x7/x7_verdict.json

WARN 的来源：本节作者复算 19,077 行：kF64 与 Stage 11 rank_false 相同的比例 0.5963，|Δlog10 rank| 平均 0.0105、p99 0.146，带符号平均 −0.0003（无偏）。原因：Stage 11 的 false 运行是

batch=1、不传显式 mask（`run_final(C, F, posF)`），ko_false 的 kF64 是 batch=13、传显式加性 mask，bf16 下舍入不同。按规则只记 WARN，继续分析。⟶ 出处 `361_stage11.py:244, 281` · `stage13_x7_analyze.py` 文件第 45-46 行

8 局限与质疑

- control 的 S(L52) 0.051 只超阈值 0.001，属边界；区间 [0.033, 0.068] 跨过 0.05。
- S(L0) ≈ 1 而 T(L0) 0.83：减去的 F 是负的，所以"剩余份额" S(L0)/T(L0) = 1.19–1.24 > 1，不能读成"占 L0 损失的百分比"。
- F 只在 false_0 一个假标题上做（X6 用了 5 个，X7 只用 1 个）。
- kF64 与 false 运行不完全相同（批大小舍入），T 与 F 的基线并非逐位一致；带符号差 −0.0003，影响很小。

9 状态

已完成 预注册（补充 3）在 X7 无数据时写好；按其执行。control 未达标按原样报告，未改阈值；kF64 不一致按"容差不达标记 WARN 继续"处理。无偏离。

附录 G1　术语表

每条一句话定义，右列链接到正文中给出完整定义或数字的位置。同一缩写在不同 Stage 指不同东西的（X1、X2、X3、A/B），分条写明。

G1.1 论文、标签与分组

术语	定义	正文位置
9,108 篇 / cohort	Dreaming Up Authors（EMNLP 2026）研究的分层评测样本（8 领域 × 13 引用档，每格最多 100 篇），由组长的标注包原样提供，不是本项目抽的。	g-data-origin、g-data-package
标注包 / elm	组长用 elm 流程在 Qwen3-32B 上做开放回答与 4 选 1 多选、并打出全部标签列的数据包（zhiqi_qwen3-32b_labels_2026-10-15）；run.json 与 elm 代码不在包里。	g-data-package、s19-s1-cohort
M / H / U	开放回答经姓名匹配后：M = 写对的真作者数，H = 编造的名字数，U = 漏掉的真作者数。	g-labels-match
HR2	(H + U) / (M + H + U)，开放回答的"错漏比例"；R2 的阈值是 HR2 ⩽ 0.5。	g-labels-match、g-labels-recall
R0 / R1 / R2	开放回答的回忆等级：R0 = 一个真作者都没写出（M = 0）；R1 = HR2 > 0.5 且 M > 0（写对少数）；R2 = HR2 ⩽ 0.5（基本写对）。全样本 8,298 / 596 / 214。	g-labels-recall
多选 4 种顺序（ABDC 等）	同一组 4 个选项按 [[0,1,3,2],[1,2,0,3],[2,3,1,0],[3,0,2,1]] 显示，真名单依次出现在 A、C、D、B，每个字母一次。	g-labels-mc
mc_team_a	4 种顺序中"真字母概率最高且字母总概率 ⩾ 0.5"的顺序数（0–4）。	g-labels-mc
mc_team_p_true	4 种顺序下真字母原始概率（未归一化）的平均。	g-labels-mc
initial_surname	多选选项的名字格式：名缩成首字母、去掉中间名首字母（"Karin G. Coifman" → "K. Coifman"）。	g-labels-mc、s12-n1
target / control / excluded / guard（group_tight）	Stage 10 起用的收紧分组：target = R0/R1 且 a = 4 且 p_true ⩾ 0.5（认出名单但写不出作者）；control = R0/R1 且 a ⩽ 1；guard = R2；excluded = 其余。人数 4,621 / 2,220 / 2,053 / 214。	g-labels-groups、s1011-s10-common
group（普通规则）	较宽的分组（target = R0/R1 且 a ⩾ 3），只在 Stage 10 敏感性分析里用。	g-labels-groups、s1011-s10-main
A / B / C / D 组（行为分组）	351_prepare_all.py 的四组：A = target・R0（4,110），B = control・R0（2,186），C = target・R1（511），D = control・R1（34），合计 6,841；Stage 4–8 使用。	g-data-cohort、s19-s1-cohort
A/B 的另两种用法	Stage 3 split-half 的 A/B 两半、Stage 9 的 Fit A/Fit B，与行为分组无关。	s19-common
team_b	500 篇用第二套干扰团队重抽的多选题；与第一次通过一致率 83.6%、kappa 0.63。	g-labels-groups、s19-s1-cohort

citation bin（引用档）	13 个按 2 的幂分的引用数档：0-2、3-4、5-8 … 2049-4096、4097+；来自标注包的 citation_bin 列。	h-stats-bins、g-data-fields
Pilot50	Stage 5–7.4 共用的 50 篇探索样本（A20/B20/C7/D3，含 Stage 4 的 20 篇）和 20 对同领域同引用档的 A/B 配对（P001–P020）。	s19-s1-pilot50
Benchmark20	Stage 4 的 20 篇读出样本（A8/B8/C3/D1）。	s19-s4-readout
Stage 11 样本（1,186 篇）	10 个可比引用档内每档每组最多 60 篇（default_rng(11)）：593 target + 593 control，24 个 shard；Stage 11/12 与 Stage 13 X 系列用。	s1011-s11-common
pairs.parquet / tune / test	Stage 13 X1/X3/X4 的配对表：1,178 篇各配同领域、同标题 token 数、无共同姓氏的搭档 B 和随机对照 C；按作者团队分成 tune 596 / test 582。	s13-overview

G1.2 提示、位置与打分

术语	定义	正文位置
作者问法 PROMPT	`Who are the authors of the paper titled "{title}" ({year})?\nAuthors:`，裸文本、不套 chat 模板、不加 BOS；Stage 3–12 的读出都用它。	g-prompts-list
delete 条件	删去标题的提示 `Who are the authors of the paper ({year})?\nAuthors:`，测名字在"有年份、无标题"时的先验。	g-prompts-list、h-false-pool
teacher forcing	把正确名字 token 逐个接在提示后，在前一位置读出下一 token 的分布并求金标准 token 的名次；测的是偏好，不是生成。	g-prompts-align、s1011-s10-common
单独打分 / 连写打分（isolated / chain）	单独：每位作者单独接在 "Authors:" 后；连写：全部作者用 "," 接成一串（Stage 11 E3、Stage 12 N1）。	g-scoring-agg、s1011-s11-e3
rank（名次）	1 + 词表（lm_head 全部 151,936 行）中 logit 严格大于金标准的 token 数；并列按有利方向计、另存 ties。	g-scoring-rank
nlr = log10(1/rank)	主指标：第 1 名为 0、第 10 名为 −1，最小约 −5.18；2026-10-23 起取代原 protocol 的 Σ1/rank。	g-scoring-rank
mean_nlr / author_mean_nlr	一篇论文全部被打分名字 token 的 nlr 平均（主指标，token 加权）/ 先在作者内平均再对作者平均（次要指标）。	g-scoring-agg
RR、JMRR	RR = 1/rank；JMRR（Jacobian MRR）= 作者内 token 平均 RR，再对作者平均；Stage 4–8 的指标。	s19-s2-protocol、s19-s6-jmrr
7 层积分	Stage 4–8 中对 7 个源层 L16/24/32/40/48/56/62 的算术平均（L63 不计入）。	s19-s2-protocol、s19-s8dsupp
W（金标准胜率）	Stage 7.2/8：金标准首 token 的 logit 胜过 100 个错误作者首 token 的比例，平局记 0.5。	s19-s72、s19-s8d
TA（标题优势）	Stage 7.3/8：W(真标题) − 20 个错误标题下 W 的平均。	s19-s73、s19-s8d
RQ1 / RQ2 / RQ3	Stage 8 的三个预设问题：L56 真 − 错误标题；匹配 A − B；低引用 − 高引用。	s19-s8d
Ll / readout final	Ll = 第 l 个 decoder block 的输出残差；final = unembed(残差 63) = 模型真实的下一 token logits。	g-model-id、h-lens-defs
token 角色	按 " "+名字切词：最后一个词 = surname，姓氏前缀 = particle，单字母 = initial，其余 = given；词首 token 加 _first，后续加 _cont，纯标点 = punct。	g-scoring-names

标题特异信号（spec）	同一组名字 token 下 m(真标题) − 5 个假标题的平均；名字本身好不好猜在差值里抵消。	g-scoring-spec、s1011-s10-titlespec
total / topic / prior	total = 真 − 删标题；topic = 假标题均值 − 删标题；prior = 删标题本身；true = prior + topic + spec。	g-scoring-spec、s1011-s10-prior
spec_strict	只用 strict 档假标题求平均的标题特异信号；没有 strict 供体的 736 篇为 NaN。	g-scoring-spec、h-false-79
tail_fraction	(1 + #{假标题分数 ⩾ 真标题分数}) / 6，参考尾部比例，不是 p 值。	g-scoring-spec
title_last / it	标题最后一个 token（与标题字符区间重叠的最后一个 token）；E1t、E4、E4b、M3、X4 在这里读或替换。	g-prompts-align、g-prompts-example
PREFIX_TOKENS = 9	Stage 13 三种问法在标题前都是 9 个 token，所以标题占相同绝对位置。	g-prompts-align、s13-m3
对齐尾段 tail_prompt	真、假标题提示的公共后缀，截到两边都在最后一个标题 token 之后；Stage 11 样本中 9 个 token 708 篇、8 个 token 478 篇。	g-prompts-align、s1011-s11-e1
p1 … p9	Stage 12 P1 的单位置编号，从提示末尾往前：p1 = ":"、p2 = "Authors"、p3 = ")?\n"、p4–p7 = 年份 4 个数字、p8 = " ("、p9 = 闭引号。	g-prompts-align、s12-p1
Dreaming Up Authors / dua 匹配器 L1–L5	原研究的逐级姓名匹配器，原研究只接受到 L4（姓 + 名首字母）；组长流程是否同用未确认。	g-labels-match

G1.3 假标题

术语	定义	正文位置
donor（供体）	把真标题借给目标论文做假标题的那篇 cohort 论文；同领域、与目标无共享姓氏、标题词 Jaccard ⩽ 0.6。	h-false-pool
false_0 … false_4	每篇的 5 个假标题，按（档序, cost, donor_paper_id）排序编号；false_0 = 最好档里代价最小的那个，不是随机的。	h-false-algo
cost（匹配代价）	|Δyear|/5 + |Δn_authors|/max(1, 0.25n) + |Δtitle_tokens|/max(1, 0.2 · len) + 1.5 · |Δbin|。	h-false-cost
strict 档（严格档）	同领域、年份差 ⩽ 5、作者数差 ⩽ 1、标题 token 差 ⩽ max(1, 20%)、引用档差 ⩽ 1；其后依次放宽为 medium、loose、nearest（无元数据约束）。	h-false-tiers
"79%"	全部 45,540 个（论文, 假标题）配对中来自 strict 档的比例 35,898 / 45,540 = 0.7883；不是"79% 的论文 5 个都是 strict"（那是 66.5%）。	h-false-79

G1.4 读出工具（lens）

术语	定义	正文位置
logit lens（Vanilla）	把第 l 层残差直接过模型的 final RMSNorm + lm_head：lm_head(final_norm(h_l))。	h-lens-defs
Jacobian lens / J-Lens（jlens）	先用平均 Jacobian J_l 把 L_l 残差搬到 L63 残差基，再 final norm + lm_head：lm_head(final_norm(J_l · h_l))。	h-lens-defs、h-lens-jac

public lens（Public-557）	Neuronpedia 公开的 Qwen3-32B Jacobian lens，557 条 WikiText-103 提示拟合、L0–L62；具体条目与模型 revision 未提供。	h-lens-table、s19-s3-exp6
self100（Self-100）	Stage 3 自拟合 lens：本地冻结 WikiText-103 语料的前 100 条，6 层 L24–L62。	h-lens-table、s19-s3-exp0
fitA lens	Stage 9 Fit A 的产物（100 有效 / 101 候选，7 层 L16–L62），Stage 10 第三把 lens。	h-lens-table、s19-s9-fitA
Fit A / Fit B / holdout	Stage 9 从 WikiText-103 切出的三份互不重叠的 100 条；Fit A 已完成，Fit B 停在 66/100，holdout 比较（9B）未做。	s19-s9-split、s19-s9-fitB、h-lens-stage9
F1 author lens（作者提示 lens）	Stage 12 在 40 条"作者问法 + 真作者名单"文本上拟合的 Jacobian lens（8 层），供 F2 读出；Stage 13 称 M4。	s12-f1、s13-m4
split-half	Stage 3 把 Self-100 语料按奇偶索引分成两半各拟合一次，比较矩阵与读出。	s19-s3-exp7
Exp 1–7 / provenance audit	Stage 3 的 lens 验证实验编号（数值检查、保真度、金标准 vs 错误作者、泄漏审计、拟合协议对比、split-half）；编号沿用已发布报告。	s19-stage3、s19-s3-exp5

G1.5 统计与切分

术语	定义	正文位置
档内 target − control（stratified_diff）	只在两组都 ⩾ 30 篇的引用档内（实际 10 档）求组差，按 w_b ∝ n_t · n_c/(n_t+n_c) 加权；bootstrap 在"档 × 组"内重抽论文 2,000 次。	h-stats-strat、g-scoring-spec
作者团队（group_folds）	共享任一归一化全名（⩾ 2 个词）的论文用并查集并成一组（同名也合并）；false_0 行再与其 donor 合并；全队列 7,247 个团队，按大小放入 5 折。	h-stats-team、s13-overview
team-cluster bootstrap（团队整群 bootstrap）	对作者团队做多项式抽样得到每篇论文的权重，统计量按加权和重算，2,000 次取 2.5/97.5 百分位；Stage 13 起为主区间。	h-stats-strat、h-stats-boot、s13-overview
title reuse（标题一题两用）	probe 训练时已见过测试论文提示里的那个标题（false_0 供体在训练折，或测试论文自己的标题被借给训练论文）；E4b 复核时剔除这些测试论文。	h-leak、s1011-s11-e4b-check
多重比较	Stage 10–13 未做任何校正；规则是"所有层都报告、区间不含 0 才算"。	h-stats-mult
预注册 prereg / 补充 1–4（addendum）	Stage 13 的判定规则文件（只读，有 sha256）：补充 1 = 所有 probe 用团队折、tune/test 按团队；补充 2 = 改 M3/X1 判据、M2c 单向、E4b 四配置、团队区间为主；补充 3 = X6/X7 定义与判定；补充 4 = X3 主对照改为近邻 D、按 token 角色报告、措辞限定为 teacher forcing 下的偏好。	h-prereg-content、h-prereg-files、s13-overview
补充 5	尚未写定的预注册补充（拟含 X3 的 jaccard2、排除完全相同改写、意义检查规则）。	h-prereg-files、s13-x3
H1 / H2 / H3	Stage 13 的三个假设：中间层主要表示任务/格式（H1）、论文身份（H2）、作者信息已在但不在词表方向（H3）。	s13-overview

G1.6 实验编号与实验内的量

术语	定义	正文位置

E1	Stage 11 activation patching：把真标题运行中"标题之后的整段尾部 + 全部名字输入位置"在第 l 层的 block 输出换进 false_0 运行。	s1011-s11-e1
恢复率 R / recovery	mean_p(替换 − 假) / mean_p(真 − 假)，先论文内平均再跨论文、最后两均值相除（E1、P1、P2、X6）；X1 的 R 用 (src 替换 − corrupt)/(clean − corrupt)。	s1011-s11-e1、s13-x1、s13-x6
E1t	只替换标题最后一个 token 的 E1（amendment 1）；分析基线改用同批 L63 元素。	s1011-s11-e1t
E2 / 损失比例 loss(l)	从第 l 层到 L63 禁止标题之后的位置注意标题 token（显式加性 mask，被挡格子 = bf16 最小值）；loss = mean(K(64) − K(l)) / mean(真 − 假)。	s1011-s11-e2
E2n	只挡 ":" 和名字位置对标题的注意（amendment 1）。	s1011-s11-e2n
E3	全部作者连写（chain）与单独打分的对比。	s1011-s11-e3
E4	在标题最后一个 token 和 ":" 上用 lens 读作者首 token / 姓首 token。	s1011-s11-e4
E4b	标题位置线性 ridge probe：X = 标题最后 token 残差，Y = ":" 处 L63 残差，经 final norm + 4,314 行名字词表求作者首 token 名次；spec = 真标题输入 − false_0 输入。	s1011-s11-e4b、s1011-s11-e4b-check
Stage 10 X1 / X2 / X3	Stage 10 CPU 补充分析：X1 = 名字格式与 token 位置，X2 = 引用数，X3 = 三种 lens 与 logit lens 的比较；与 Stage 13 同名实验无关。	s1011-s10-format、s1011-s10-cite、s1011-s10-lens
Stage 12 X1	E4b 的对照：置换 gold（全局 / 同领域）与只用假标题训练。	s12-cpu
P1	Stage 12 单位置替换：每次只把对齐尾段中一个位置（p1–p9）或全部名字位置换成真标题运行的残差。	s12-p1
P2	Stage 12 组件替换：只换某层注意力输出（o_proj 后）或 MLP 输出（down_proj 后）。	s12-p2
N1	Stage 12 多选选项先验：4 个选项各作作者链打分，比较删标题与真标题下真名单是否最"像"。	s12-n1
N2	Stage 12 纯名字先验：提示 "The following is the name of a researcher.\nName:" 下的名字名次。	s12-n2
F1 / F2	F1 = 拟合作者提示 lens；F2 = 用它在名字位置与标题最后 token 上做 Stage 10 式标题特异读出。	s12-f1、s12-f2
M1	Stage 13：lens 在名字位置排第一的 token 归类（correct / name-like / punct / task / title / other）。	s13-m1
M2 / M2b / M2c	标题位置与 ":" 位置的残差 ridge probe 读领域、年份、引用、是否认得出；M2b = TF-IDF 标题词基线；M2c = bert-base-uncased 基线。	s13-m2、s13-m2b、s13-m2c
M3	同一标题放进作者 / 年份 / 领域三种问法，比较跨问法迁移与同篇余弦。	s13-m3
迁移比 T	问法 a 训练、问法 b 测试的 probe 成绩 ÷ 问法 b 内训练测试的成绩，对 6 个有序对平均。	s13-m3
身份指数 I	同篇跨问法居中余弦 − 异篇跨问法平均余弦；补充 2 后只看 I(l) − I(L4)。	s13-m3
M4	作者专用 lens，即 Stage 12 F1/F2，Stage 13 未另算。	s13-m4
M5	提议中的逐位置 probe / patchscope / 名字位置真假判别，无代码、未执行。	s13-m5

X1	Stage 13 跨问法标题 patching：把 A 的标题状态（来自作者 / 年份 / 领域问法或随机论文 C）换进"作者问法 + B 的标题"运行。	s13-x1
partner B / random C / NULL FLAG	B = 同领域同长度无共同姓氏的搭档（corrupt 来源）；C = 无关论文；任一层 |R(random C)| > 0.1 记 NULL FLAG。	s13-x1、s13-overview
task-independent / task-specific / undecided	X1 的补充 2 判定：L24 起 a − year、a − field 区间都在 [−0.1, 0.1] 内且 a − random C 下限 > 0 为 task-independent；a − year 下限 > 0.1 为 task-specific。	s13-x1
question share	X1 相似度检查：同篇换问法的相对距离 ÷ 同问法换论文的相对距离（0 = 问法不改变状态，1 = 与换论文一样远）。	s13-x1
X2	Stage 13 计划中的只替换子空间实验，未写补充、未运行。	s13-x2
X3	同义改写标题 vs 词重合近邻 D 的比较；目前只完成改写生成与意义检查。	s13-x3
auto_ok / identical	X3 改写的自动检查通过（非空、不含年份、不含作者姓子串、长度比 0.5–2.0）/ 去标点小写后与原标题完全相同（27 条）。	s13-x3
jaccard2	修正版标题词 Jaccard：去英语停用词、保留任意长度词、简单去后缀；原版只保留长度 > 3 的词。	s13-x3
lexical neighbour D / G / G′	D = 词 Jaccard 不低于改写的同领域近邻标题；G = cos(原, 改写) − cos(原, B)；G′ = cos(原, 改写) − cos(原, D)（补充 4 的主指标）。	s13-x3
para_ok	人工放行 X3 主实验 para_run 的标记文件 R13/logs/para_ok；2026-10-30 不存在。	s13-x3
JUDGE / NLI	X3 意义检查的两种自动法：Qwen3-32B 判断器（chat 模板、thinking 关、两种顺序）与 cross-encoder NLI 双向蕴含。	s13-x3、g-model-chat
X4 / patchscope	把 A 某层的状态注入只有占位符 " X" 的目标提示，看能否解码 A 的年份、第一作者、领域。	s13-x4
D（patchscope）	D = mean score(注入自己的状态) − mean score(注入搭档 B 的状态)，score = 答案 token 的平均 log10(1/rank)。	s13-x4
D_text（文本基线）	只从标题文本预测第一作者名字 token 的同样差值（kNN 版与监督 LR 版）。	s13-x4
x4tok / X4TOK_CHECK_FAILED	X4 逐 token 复算及其自检；第一次 max |diff| 0.0683 > 0.05 写出失败标记，被用户定为致命的正确性检查。	s13-x4、h-compute-parity
b7 / b2	x4tok 修法中的两种批大小：b7 = 一次注入全部 7 个 PS_LAYERS（与原 X4 相同），b2 = 只注入 L8、L40。	s13-x4
X5	计划中的层序量化（身份 → probe 读作者 → lens 读名字），未执行。	s13-x5
X6	用 false_1 … false_4 重做 Stage 11 E1。	s13-x6
X7 / S = T − F	在 false_0 运行上做与 E2 相同的阻断：T = 真运行损失比，F = 假运行损失比，S = T − F 为论文特有损失。	s13-x7
kT64 / kF64	真 / 假运行中"不挡（l = 64）但走显式 mask"的那一份。	s13-x7

G1.7 质量控制与运行

术语	定义	正文位置

parity	Stage 10 GPU 放行门：快路径（一次前向读所有位置）与逐前缀 lens.apply 的最大 |Δlogit|。	s1011-s10-parity、h-compute-parity
容差 0.25 → 0.5（tolerance_amendment）	parity 门槛在看到 FAIL 后由用户于 2026-10-25 放宽到 0.5，原状态保留。	h-compute-tol
bf16 ulp（最小间隔）	|logit| 在 32–64 时 bf16 相邻可表示值相差 0.25；parity 差 0.367 约 1–1.5 个 ulp。	h-compute-tol
WARN 规则	2026-10-27 起的长期规则：数值一致性未达标只记 WARN（保留数值和阈值）继续算，只有代码崩溃才停队列；x4tok 为用户明确指定的例外。	h-compute-tol、s1011-s11-qc
smoke / realcheck / preflight	smoke = 第一块 GPU 上的小规模全流程自检；realcheck = CPU 上用真权重复算一篇论文；preflight = 花 GPU 前的 CPU 结构核对。	s1011-s11-qc
shard / 单元（unit）	shard = 固定论文块（Stage 10 每 128 篇、Stage 11 每 50 篇）；单元 = 一个"类型 × 条件 × shard"的 GPU 计算，各存一个 npz，已存在就跳过。	s1011-s10-common、s12-ov-units、s13-overview
池化调度（pool / steal / hand-over）	任何 GPU 作业可领取其他 Stage 的单元，所以作业名不等于实际算的实验。	h-compute-table、s1011-s10-main

附录 G2　导师可能追问的 20 个问题与标准答案

答案只用正文已有的事实和数字；每条末尾给出处锚点。"未知"处照实说，并写明向谁确认。

Q1　target / control 是怎么定义的？分组本身可靠吗？

两组都是开放回答写不出作者的论文（R0/R1，即 HR2 > 0.5）：target 要求 4 个选项顺序全答对且真字母平均概率 ⩾ 0.5，control 是 4 个顺序里最多答对 1 个，全队列 4,621 / 2,220 篇。标签是组长的标注包原样数据，我们按规则重算 group_tight 与标签 100% 一致。可靠性的弱点有两处：0-2 引用档的收紧通过率 25.5%，接近固定偏好某个字母的 25%；500 篇第二套干扰团队重测一致率 83.6%（kappa 0.63）。所以 Stage 10 加做了去掉低档的敏感性（final +0.212 [0.199, 0.226]）。标签由组长的 elm 代码生成（不是 Dreaming Up Authors 的 dua 包），干扰团队的抽取规则原文未知（bin_window 等字段含义已从数据推断），需向组长确认。见 g-labels-groups、s19-s1-cohort、s1011-s10-tc。

Q2　假标题怎么选？"79% 来自严格档"是什么意思？

每篇在同领域其他论文中找 5 个 donor，硬排除自身、任何共享姓氏、标题词 Jaccard > 0.6；代价 = |Δyear|/5 + |Δn_authors|/max(1, 0.25n) + |Δtitle_tokens|/max(1, 0.2 · len) + 1.5 · |Δbin|，按 strict → medium → loose → nearest 逐档做最小代价指派。79% 指全部 45,540 个（论文, 假标题）配对中 35,898 个来自 strict 档（0.7883），不是"79% 的论文 5 个都是 strict"（那是 6,056 / 9,108 = 66.5%）。假标题提示保留目标论文自己的年份，只换标题。代价权重是经验设定，没有做过敏感性分析（用户 2026-10-30 确认）。见 h-false-cost、h-false-79、h-false-limits。

Q3　标题特异信号能排除"名字本身好猜"吗？

spec = 真标题 − 5 个假标题平均，七个条件用同一组名字 token、只换提示，所以名字自身的难度在差值中抵消；标点 token 的 spec 最大约 0.014，作为阴性对照。Stage 12 又直接测了先验：删标题时真名单在 4 个多选选项中最"像"的比例 0.255（≈ 机会 25%）；纯名字提示下 target − control 只有 +0.047 [+0.031, +0.062]，约为真标题差 +0.247 的 19%（两个点估计之比，未对比值本身做 bootstrap）。所有结论都是 teacher forcing 下的偏好，不是生成。见 g-scoring-spec、s1011-s10-titlespec、s12-n1、s12-n2。

Q4　Stage 8 的主终点是不是事后改的？

哈希锁定的 analysis_plan.json（2026-10-18 14:29）把 7 层积分定为主终点、L56 为次要；同日 21:14 写的 PROTOCOL.md（不在任何 manifest 里）改为 L56 为主。两者都早于 8B 冻结（21:25）和 8C 打分（21:47 起），所以改动在看到 Stage 8 数据之前，但在 pilot 已看过 L56 之后。8D 按 PROTOCOL 报告 L56：+5.18 pp [+2.93, +7.48]；冻结主终点后来由 8D supplement 补算：+2.33 pp [+1.50, +3.19]，

方向一致、约为 L56 的一半。git 里没有任何提交改过主终点（两份文件都在 e2c7156 首次入库；3a4f605 只删掉 PROTOCOL.md 第一行的 heredoc，已于 10-30 合入 main a4c1f0a）。2026-10-30 按冻结计划补算：积分主终点 RQ1 $p = 2\times10^{-5}$、积分 A−B $p = 0.002$，逐对剔除 100/100 区间不含 0。由谁、依据什么改的，仍需用户确认（用户正在笔记本上查）。见 s19-s8-protocol、s19-s8d、s19-s8dsupp。

Q5　Stage 8 的 A − B 差（RQ2）稳健吗?

RQ2 L56 匹配 A − B = +4.98 pp [+0.32, +9.91]（100 对，55 正 / 45 负），但对少数大正向对极敏感：去掉最大的 10 对后均值为 −0.39 pp（事后诊断，不在冻结计划里）。冻结计划列的 leave-one-pair-out 和 Holm 原来都没有跑，2026-10-30 事后补算：L56 A − B 逐对剔除 +4.18 到 +5.50、不变号，但 14/100 次区间跨 0；7 个逐层 A − B 做 Holm 后只有 L62 通过（L56 p_{Holm} 0.19）。配对敏感性区间已从原始数据重算：+5.03 [+0.23, +9.95]、+4.53 [−0.15, +9.53]、+4.79 [−0.24, +9.83]。冻结主终点版的积分 A − B 为 +2.41 pp [+0.91, +4.00]，逐对剔除 100/100 区间不含 0。见 s19-s8d、s19-s8dsupp。

Q6　做了多重比较校正吗?

Stage 10–13 没有。脚本里搜不到 Bonferroni / Holm / FDR；采用的是预注册通用规则"所有层和位置都报告、不只报挑出来的一层"以及"区间不含 0 才算差异"，逐层判断没有族错误率控制。Stage 8 协议写了 7 个逐层比较做 Holm，原代码没有实现；2026-10-30 已事后补算（作业 11500770）：只有 L62 通过（p_{Holm} 0.029），L48 0.17、L56 0.19；p 值公式（由 bootstrap 反推的双侧 p）和检验族划分是我们的选择。Stage 7.2/7.4 报告写明未校正。见 h-stats-mult。

Q7　parity 容差从 0.25 放宽到 0.5，是不是看到结果后改的?

是。冻结的门槛是 max |Δlogit| ⩽ 0.25；main parity（11347585、11350174，数值逐位相同）有 10/25 对超过，最大 0.367 出现在 final@L63——这一项不经过任何 lens 代码，说明差异来自模型自身一次前向与逐前缀前向的 bf16 累加顺序。|logit| 在 32–64 时 bf16 最小间隔就是 0.25，0.367 约 1–1.5 个 ulp；名次 < 42 的位置完全一致，log10(1/rank) 最多变 0.070。用户于 2026-10-25 11:00:31 批准 0.5，只用已存数字在 CPU 上重判，没有重跑 GPU，原 FAIL 记录保留并在分析计划与报告中披露。局限：parity 抓不到两条路径共有的错误，且只抽了 12 个上下文。见 h-compute-tol、s1011-s10-parity。

Q8　E2 注意力阻断具体怎么做？为什么被挡的格子是 bf16 最小值?

每个 decoder layer 用 forward pre-hook 把 attention_mask 换成形状 [B, 1, T, T] 的显式加性 mask：允许处为 0，禁止处为 torch.finfo(bf16).min（不是 −inf），第二维为 1 所以对所有头生效；禁止的是"标题之后所有 query 位置 → 全部标题 token key"，从第 l 层一直挡到 L63。显式 mask 让 sdpa 走 is_causal=False、不走 GQA 快路径，换了内核，所以基线用同批"什么都不挡但走同样 mask 路径"的 l = 64 那一份；smoke 中该基线与 Stage 10 名次相同 0.7037（阈值 0.75，记 WARN），CPU 真权重检查

显式 mask 基线与不加 mask 完全相同。正文没有记录选用 finfo.min 而非 −inf 的理由，未知，需向 Stage 11 原作者确认。见 s1011-s11-e2、s1011-s11-qc。

Q9　E1 的"标题之后"包括名字位置吗？"L48 恢复 83%、L52 100%"怎么算的？

包括。替换区间 = 对齐尾段（闭引号、" ("、4 个年份数字、")?\n"、"Authors"、":"，共 8 或 9 个 token）+ 全部名字输入 token，所有打分位置都在被替换的区间内，所以"恢复"不能单独归于名字前的提示位置。恢复率 = mean_p(替换 − 假) / mean_p(真 − 假)：all 组 L48 0.825 [0.807, 0.842]，L52 0.996 [0.989, 1.003]。Stage 12 P1 逐位置拆开：L52 名字位置 0.847、":" 0.172，其余单个提示位置都很小；X6 用另外 4 个假标题重做，target L48 0.788–0.803、L52 0.984–0.990。E1 的替换运行是 batch 15、假基线是 batch 1，舍入噪声估计约 0.01、远小于主效应；用户决定答辩前不补测，GPU 空闲时按 X4 的做法补。见 s1011-s11-e1、s12-p1、s13-x6、s1011-s11-qc。

Q10　E4b probe 到底测了什么？有哪些设计上的漏洞？

输入是标题最后一个 token 的第 l 层残差，目标是同一运行中 ":" 位置的 L63 残差，闭式 ridge（α 只在第 1 折上从 {0.1, 1, 10} 选，各层都选中 1.0），预测经 final norm 与 4,314 行名字词表求名次。只打每位作者的第一个名字 token（73% 是首字母），姓的首 token 不作 gold；spec = 真标题输入 − false_0 输入。标准化的 μ、σ 用全部 9,108 篇（含测试折）算，是只影响标准化的轻微泄漏；而 Stage 10 显示首字母 token 几乎不带标题特异信号，所以 probe 测的恰是信号最弱的那部分。标准化问题已于 2026-10-30 改为只用训练折重算（作业 11500917）：最大差 0.0075，α 不变，确认版没有"区间是否含 0"的翻转，L44 下界变为 +0.000（边界）；gold 只用第一个 token 的问题没有改。见 s1011-s11-e4b、s1011-s11-e4b-check、s1011-s10-format。

Q11　E4b 的泄漏是怎么发现和处理的？结论变了吗？

2026-10-28 审读发现两类泄漏：同作者团队跨折，以及"标题一题两用"（测试论文的假标题是训练折某篇的真标题，或反过来）。复核脚本 stage13_e4b_group.py（作业 11443587）只替换分折行并注入剔除规则，paper/不剔除配置复现已发表数字（最大差 0.0000）。确认版（团队折 + 剔除）只剩 1,863 篇（target 896、control 499），target − control 变大：L24 +0.024 → +0.052 [0.017, 0.088]，L56 +0.039 → +0.075 [0.038, 0.115]，置换 gold 对照仍 ≈ 0。原"只用假标题训练的 probe 更强"（L24/40/56 = +0.068/+0.081/+0.094）来自泄漏，复核后 +0.050/+0.057/+0.074，该说法已撤回。剩下的论文不是随机子集。见 h-leak、s1011-s11-e4b-check、s12-cpu。

Q12　M2 的 false_0 泄漏影响多大？

false_0 共 6,247 个 donor，其中 2,373 个被多篇借用；按论文随机分折后 2,004 个 donor 标题同时落在训练与测试折。按 donor 及其团队分组重跑（作业 11443403）后，L48 donor 年份 R^2 0.623 → 0.535、log 引用 0.588 → 0.488；真标题运行两种切法只差 ⩽ 0.005。补充 1 判定按论文切的 false_0 数

字无效。M2 原来的区间是按论文 bootstrap 200 次；2026-10-30 按补充 2 补算团队整群 2,000 次（作业 11503585）：176 个点估计全一致，区间平均宽 1.15 倍（true 1.02、false_0 1.32），0 个翻转。见 h-leak-a、s13-m2。

Q13　M3 的迁移比接近 1、X1 三种来源恢复一样好，能说明"论文身份"吗？

不能单独说明。三种问法只在标题前 9 个 token 不同、标题完全相同，标题位置 T 高几乎由构造决定：T(L4) 已是 0.983 / 0.976 / 0.998，T − T(L4) 在 L16–L40 为 −0.03 到 −0.06，I(l) − I(L4) 在所有层都 < 0，即中间层并不比 L4 更跨问法一致。X1 相似度检查显示 L24–L40 同篇两问法状态本来就相近（question share 0.15–0.17），所以"恢复一样好"部分来自构造；但 share 不为 0 而 a − year、a − field 差 ⩽ 0.02，说明问法带来的差异不影响作者恢复。另外 random C 在 L4–L16 为负、L48 为 +0.122，记 NULL FLAG。见 s13-m3、s13-x1、h-prereg-dev。

Q14　X4 说 Qwen 能从论文自己的状态解码第一作者，文本基线排除了"标题风格"吗？

只部分排除。文本基线 1（kNN）的阳性对照失败（允许同团队时 TF-IDF −0.025、BERT −0.159），所以它的"文本解释不了"不成立。文本基线 2（监督 LR）只在名字首 token 上通过阳性对照、姓首 token 上失败；起草时还发现它对 821 篇没有 given_first 的论文仍计入了与姓名无关的 OOV 类概率差。总编 2026-10-30 用 CPU 只按实际存在的角色重算：BERT D_text +0.048 [+0.014, +0.083]，与 Qwen D(L8) 逐篇相关 +0.014。逐 token 全量（作业 11500437，b7 逐值复现已存分数）按角色拆开后：Qwen L8 信号主要在姓的首 token（+0.066 [0.046, 0.088]），名的首 token 只有 +0.018 [−0.003, 0.041]，而在名的首 token 上 BERT 文本基线 +0.088 反而更大；姓这个角色上文本基线的阳性对照没有通过。所以**不能排除 L8 作者信号可由标题文字解释**（措辞待用户最终确认）。见 s13-x4。

Q15　既然规定"容差不达标只记 WARN"，为什么 x4tok 失败就停了？

一般规则（2026-10-27 起）是 parity / 一致性未达标记 WARN 继续算，只有代码崩溃才停，例如 Stage 11 smoke 两项 0.7037 和 X7 kF64 59.63% 都按 WARN 继续。x4tok 是 2026-10-30 用户明确定为"正确性检查，不是旁支"的例外：它要求逐 token 平均复现已存的 X4 分数，否则结果不能按 token 角色拆分。失败原因判断为批大小不同（batch 2 vs 原 batch 7）的 bf16 舍入：200 个值平均 |d| 0.0068、D 与已存值相关 0.993（L8）。修法同时算 b7 与 b2，b7 必须复现，两者 L8 平均 D 相差 > 0.005 也停。全量 11500437：b7 与已存 4,712 个值完全相同，L8 平均 D b7 +0.0484 / b2 +0.0490（差 0.0006），批大小不影响结论；旧失败标记已改名保留。见 h-compute-tol、h-compute-parity、s13-x4。

Q16　X7 中 control 组没有通过，怎么解释？

补充 3 的判据是 S(L52) ⩽ 0.05 且 S(L0) ⩾ 0.5；control 的 S(L52) = +0.051 [+0.033, +0.068]，超阈值 0.001，判 DOES NOT HOLD，区间跨过 0.05，属边界；target +0.018、all +0.029 判 HOLDS。报告如实保留，没有改阈值。F(L0) 为负（−0.161 到 −0.196），即在假标题运行里挡住假标题反而帮助真作

者，所以 S(L0)/T(L0) > 1，不能读成百分比。自检中 kF64 与 Stage 11 假运行名次相同只有 59.63%（批大小与显式 mask 的舍入，带符号差 −0.0003），记 WARN。见 s13-x7、h-prereg-dev。

Q17　预注册真的写在数据之前吗？为什么文件里的时间和 git 不一致？

5 个文件的 sha256 均与 .sha256 记录一致、文件只读。文件内 written 字段是当时会话**手填错误**，比实际晚 20 分钟到 2 小时 45 分钟；实际写入时间是 2026-10-28 10:39 / 10:50 / 11:05 / 11:41 / 11:55 EDT（会话记录、mtime、git 三者一致，如 prereg.json mtime 10:39:12、git 355e64d 10:40:15），应以 git 时间为准。按这个时间核对：写 prereg 时 381_stage13/raw 下还没有任何 GPU 文件（最早 xf_patch 10-28 19:46），M1/M2/M2b 已先跑并标为探索性。补充 5 尚未写定（草稿待用户确认）；判断器 v2 一致率 0.55 < 0.80，不用。见 h-prereg-files、h-prereg-status。

Q18　public lens 是怎么拟合的？可信吗？

来自 Neuronpedia：Salesforce/wikitext-103-raw-v1 train、max_chars 2000，557 条（运营者手动停止，原上限 1,000），源层 L0–L62，dim_batch 64，bf16、1×B200；stop_at_delta 0.002 未达到，相对标准误约 13%。557 条提示已于 2026-10-30 按 Neuronpedia 的 fit_lens.py（HEAD 22f0412，与 provenance 的 jlens_commit 相同）复原：skip_first 16、target 63，9,108 个队列标题无一逐字出现在其中。模型与语料 revision 没有记录（推定为拟合当时的 main），需向 Neuronpedia 确认。旁证是 Stage 10 中三把在 WikiText 上拟合的 lens 结论一致（L62 标题特异 target − control +0.228 到 +0.231）。见 h-lens-table、s19-s3-exp6、s1011-s10-tc。

Q19　标注时的生成参数和匹配规则是什么？

标签由组长的 elm 代码生成，elm 源码和 run.json 不在集群上；生成设置与匹配规则用 Dreaming Up Authors 代码包（dua 1.0.0，zip sha256 88b75daa…d12d）核对：开放回答套 Qwen3 chat 模板（system + user），提示与 dua stage1_generate.py 逐字相同，thinking 关、贪心、max_new_tokens = 96（stage1_generate.py:171；743 条截断回答都恰好 96 token），batch size 从数据推断为 16。dua NameMatcher + L4 阈值复算 (m, h, u) 与标注包 9,095/9,108 一致。干扰团队的抽取规则原文、多选 thinking 设置、panel.py 剥离规则、选项截断规则仍需向组长确认（字段含义已从数据推断）。见 g-labels-open、g-labels-match、g-labels-mc、g-model-chat。

Q20　Fit B 没做完，lens 拟合的稳定性怎么交代？

Fit B 检查点停在 n_done = 66/100，最后写入 2026-10-21 19:12，之后没有续跑；9B 留出集比较与 9C 都没有实现代码。所以"换一批数据拟合，结论稳不稳"没有正式回答，现有的只是间接证据：Self-100 的 50 + 50 split-half（后层矩阵余弦 L56 0.996、L62 0.99985，L24 只有 0.678）和 Stage 10 三把 lens 的并列结果一致。Fit B 停止的原因已查实：作业 11289356 到 2 小时时限被杀，检查点完好可续跑；用户决定答辩前不补做，局限写明未使用。见 s19-s9-fitB、s19-s9-bc、h-lens-stage9、s19-s3-exp7。

附录 G3　全部"未知 / 需确认"汇总

来源：6 个 *.unk.txt 与正文全部 `class="unk"` 的 span（共 60 处），按问题去重合并为 54 条；同一问题出现在多个章节的，章节列逐一链接。"向谁"按正文写法归为三类：组长（标注包作者）、用户、原作者（laptop 会话中的 Stage N 原作者；正文写"用户 / 原作者"两可的，按正文第一个写的人归类并在问题后注明）。影响程度是本附录的判断：高 = 动摇主要结论或导师几乎必问；中 = 影响某个实验的可信度或复现性；低 = 只影响记录完整性。

汇总（起草时）：组长 10 条、用户 31 条、原作者 13 条；影响 高 5、中 24、低 25。**2026-10-30 下午按 CHANGES_2026-10-30.md 更新**：已解决 40 条（含按用户决定写明的 12 条），部分解决 5 条（U03、U14、U23、U40、U42），仍需确认 9 条（U01、U04、U06、U07、U10 向组长；U21、U28、U51 向用户；U34 向 Stage 10 原作者 / 用户）。各条依据见最后一列与 附录 I、附录 J。

编号	所在章节	问题	向谁确认	影响（理由）	状态（2026-10-30 下午）
U01	g-data-package	两次标注运行的 run.json（设置、提示词 sha256、代码与库版本）、elm 代码版本、open_results_noise.jsonl 不在标注包里。	组长	中（复现性）	**仍需确认**（组长）；2026-10-30 在集群上也找不到 elm 源码与 run.json ⟶ 出处 附录 J
U02	g-labels-open、s19-s1-cohort	开放回答的 max_new_tokens：只从数据反推为 96（743 条 length 截断都恰为 96），需 run.json 确认。	组长	低（数据已自洽）	**已解决（2026-10-30）**：dua stage1_generate.py:171 `MAX_NEW_TOKENS = 96`，与 743 条截断全为 96 一致 ⟶ 出处 附录 J 第 8 行
U03	g-model-chat、g-labels-open、s19-s1-cohort	开放回答与多选题是否关闭 thinking；多选的"答案位置"是否位于空 <think></think> 块之后。	组长	中（分组标签定义）	**部分解决（2026-10-30）**：开放题 thinking 关（dua stage1_generate.py:252–284）；多选部分（thinking、答案位置）仍需组长确认 ⟶ 出处 附录 J 第 8 行
U04	g-labels-open、s19-s1-cohort	metrics/panel.py 的名字切分与剥离规则。	组长	中（决定 M/H/U）	**仍需确认**（组长）；M/H/U 计数规则已确定，剥离行为已从数据推断 ⟶ 出处 附录 J 第 5 行
U05	g-labels-match、s19-s1-cohort	elm 的姓名匹配是否就是 dua NameMatcher 且阈值 L4（姓 + 名首字母）。旁证：用户重建与包内 m 在 99.3% 的 R1 论文上一致。	组长	中（决定 R0/R1）	**已解决（2026-10-30）**：dua NameMatcher + L4 复算 (m, h, u) 9,095/9,108 一致 ⟶ 出处 附录 J 第 4 行
U06	g-labels-mc、s19-s1-cohort	多选干扰团队的抽取规则：年份 / 人数容差、bin_window / over_cap / et_al_mismatch 的含义、是否排除共同作者、随机种子。	组长	高（target/control 依赖题目难度）	**仍需确认**（组长）；bin_window / over_cap / et_al_mismatch 含义已从数据推断 ⟶ 出处 附录 J 第 6 行

U07	g-labels-mc	选项显示截断规则（是否固定"超过 8 人显示前 8 人 + et al."）。	组长	低（少数长名单）	**仍需确认**（组长）；数据与"> 8 人显示前 8 人 + et al."一致 ⟶ 出处 附录 J 第 7 行
U08	g-data-origin	data/prepared/papers_9108.csv 是否就是 eval_sample_9108 去掉 Unicode 空白后的版本；提示里用的是哪一版标题（涉及 17 篇）。	组长	低（17 篇）	**已解决（2026-10-30）**：papers_9108 = eval_sample 空白规范化（17 篇） ⟶ 出处 附录 J 第 2 行 · 附录 I U08
U09	h-stats-bins	13 个引用档边界的定义来源（标注包只给 citation_bin 列）。	组长	低（实际值已核对在范围内）	**已解决（2026-10-30）**：dua config.yaml:44–59，重算 9,108/9,108 一致 ⟶ 出处 附录 J 第 9 行
U10	g-model-load、h-compute-table	Stage 1 标注运行的加载方式、batch、GPU、时长。	组长	低（记录完整性）	**仍需确认**（组长）
U11	g-prompts-list	Stage 1–10 的作者问法是怎么选定的、是否比较过其他问法（Stage 2 无独立目录）。正文写"用户 / Stage 2 原作者"。	用户	高（全部读出依赖单一提示）	**已解决（2026-10-30）**（用户决定）：正文写"选择过程未记录，未系统比较其他问法" ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U12	g-model-load	Stage 9 FullFit 拟合作业实际用的 GPU 型号。	用户	低（记录完整性）	**已解决（2026-10-30）**：Fit A 第一段 A100-40GB（d1026），其余 A100-80GB ⟶ 出处 附录 I U12
U13	h-false-cost	假标题代价权重 1/5、0.25n、0.2len、1.5 的设计理由无书面记录。	用户	中（假标题匹配质量）	**已解决（2026-10-30）**（用户决定）：正文写"经验设定，没有做过敏感性分析" ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U14	h-lens-table	public lens 的 557 条拟合 prompt、skip_first、模型 revision（Neuronpedia 未提供，provenance 审计记 UNDETERMINED）。	用户	中（lens 泄漏无法排除）	**部分解决（2026-10-30）**：557 条按 fit_lens.py 复原、skip_first 16、target 63，不含队列标题；模型与语料 revision 未记录，需向 Neuronpedia 确认 ⟶ 出处 附录 I U14
U15	h-lens-table	self100 的拟合脚本与 1,000 条 WikiText 语料生成脚本不在 ~/jlens_work 或 git 仓库。	原作者（Stage 3）	中（复现性）	**已解决（2026-10-30）**：git home_project/05_prepare_jlens_corpus.py、05_fit_jlens_prelim_100.py（e2c7156） ⟶ 出处 附录 I U15
U16	h-lens-table	Stage 9 Fit A 准备时实际的 --min-chars / --max-chars（plan.json 未记录，脚本默认 100/1200）。	原作者（Stage 9）	低（拆分已冻结）	**已解决（2026-10-30）**：min/max chars = 100/1200，三份拆分逐哈希复现 ⟶ 出处 附录 I U16
U17	h-stats-mult	Stage 8 协议中的 Holm 校正是否在别处实际计算过（HPC 代码无实现）。	原作者（Stage 8）	中（多重比较必问）	**已解决（2026-10-30）**：Holm 已事后补算，只有 L62 通过 ⟶ 出处 351_stage8/recompute_2026-10-30/s8_holm.csv · 作业 11500770

U 18	h-prereg-files	prereg 文件内 written 字段与 PROGRESS 标题时间比 mtime / git 晚 20 min – 2 h 45 min 的原因。	用户	中（预注册可信度）	**已解决（2026-10-30）**：written 字段手填错误，实际 10:39 / 10:50 / 11:05 / 11:41 / 11:55 EDT ⟶ 出处 `PROGRESS.md 2026-10-30 12:30` 条
U 19	h-compute-hw	节点 d1002、d3204、d3233、d4054 的 GPU 型号（只涉及 10-15/18 少量交互作业）。	用户	低（记录完整性）	**已解决（2026-10-30）**：d1002 V100、d3204/d3233 A100、d4054 H200 ⟶ 出处 附录 I U19
U 20	h-compute-table、s19-common	2026-10-15 至 10-22 的 69 个交互式 bash GPU 作业各对应哪个实验（Stage 1–9 作业号无法对应）。B 部分写向 Stage 3–9 原作者确认。	用户	低（记录完整性）	**已解决（2026-10-30）**（用户决定）：按时间对应，约 15 个作业无法追溯 ⟶ 出处 附录 I U20
U 21	s19-stage3	Exp 1–7 编号的来源文档 Validation_Record_351_Completeness_Checked.docx 不在集群上。	用户	低（只涉及编号）	**仍需确认**（用户，应在 laptop 上）
U 22	s19-s3-exp1	已发布报告"Public-557 63/63 通过、L48 范数 105.8"的原始输出在哪。	原作者（Stage 3）	中（已发布数字可能有误）	**已解决（2026-10-30）**：105.8 是 Public L48 范数（float32；精确 106.07） ⟶ 出处 `351_stage8/recompute_2026-10-30/stage3_lens_frobenius_norms.csv · 作业 11500587`
U 23	s19-s71b	7.1B 在 1,166 / 15,519 停止是否有意放弃。	用户	低（后由 Stage 10 覆盖）	**部分解决（2026-10-30）**：最后一次运行由用户账号在时限前约 23 分钟结束，日志无错误；是否有意放弃仍需用户确认 ⟶ 出处 附录 I U23
U 24	s19-s72	7.2 七层积分 A − B 区间 [−0.012, +0.228] / [−0.005, +0.217]（20,000 次）的出处（本手册重算 [−0.013, +0.227] / [−0.007, +0.216]）。	原作者（Stage 7）	低（重算接近）	**已解决（2026-10-30）**：重算 +0.105 [−0.012, +0.229] / +0.106 [−0.006, +0.216] ⟶ 出处 `351_stage7_3/recompute_2026-10-30/stage7_interval_recompute.csv`
U 25	s19-s72	去掉 P004 后 A − B 区间 [−0.018, +0.069] 的出处。	原作者（Stage 7）	低（探索性）	**已解决（2026-10-30）**：重算 +0.022 [−0.018, +0.068] ⟶ 出处 `stage7_interval_recompute.csv`
U 26	s19-s73	7.3 L56 标题优势区间 [+4.75, +13.95] 的出处（本手册重算 [+4.80, +14.07]）。	原作者（Stage 7）	低（重算接近）	**已解决（2026-10-30）**：重算 +9.34 [+4.77, +14.00]（B = 20,000） ⟶ 出处 `stage7_interval_recompute.csv`
U 27	s19-s73	三种候选加权下标题优势 +0.026 到 +0.032 与 6 个 A − B 区间的出处。	原作者（Stage 7）	低（探索性）	**已解决（2026-10-30）**：重算 +0.026 到 +0.032，6 个 A−B 区间全部跨 0 ⟶ 出处 `stage7_interval_recompute.csv`

U28	s19-s8-protocol	2026-10-18 14:29 → 21:14 之间把 Stage 8 主终点从 7 层积分改为 L56 的决定者与依据，是否有书面记录。	用户	高（唯一的确认性检验）	**仍需确认** （用户正在笔记本上查）；git 无改动记录（e2c7156 首次入库，3a4f605 只删 heredoc 行）
U29	s19-s8c	stage8C_runtime.json.before_mode_fix 与正式版差在哪、是否在任何打分之前。	原作者（Stage 8）	中（冻结完整性）	**已解决（2026-10-30）**：只改模式名 final → final_L63，在任何打分之前 ⟶ 出处 附录 I U29
U30	s19-s8d	RQ2 配对敏感性区间 [+0.15, +9.87]、[−0.34, +9.41]、[−0.39, +9.80] 的出处（8D 没有输出）。	原作者（Stage 8）	中（RQ2 稳健性）	**已解决（2026-10-30）**：重算 [+0.23, +9.95]、[−0.15, +9.53]、[−0.24, +9.83] ⟶ 出处 351_stage8/recompute_2026-10-30/s8_unsourced_intervals.csv
U31	s19-s8dsupp	review/stage8-fixes 分支 3a4f605 是否合入 main；supplement 输出是否复制到 351_stage8/。	用户	低（存档位置）	**已解决（2026-10-30）**：已合入 main（a4c1f0a） ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U32	s19-s8e	8E 外部审计的脚本、种子、逐篇 CSV 在哪（全部数字未核实）。	用户	低（已标作废、不引用）	**已解决（2026-10-30）**：从原始 logits 重算，点估计与报告相同 ⟶ 出处 s8_unsourced_intervals.csv · s8_8E_weighting_by_paper.csv
U33	s19-s9-fitB	Fit B 在 66/100 停止的原因、是否补做。	用户	高（lens 稳定性未回答）	**已解决（2026-10-30）**（用户决定）：作业 11289356 到 2 h 时限；答辩前不补做，局限写明未使用 ⟶ 出处 附录 I U33 · PROGRESS.md 2026-10-30 14:00 条
U34	s1011-stage10	Stage 10 冻结时脚本 sha 847e7dde 与运行版本 9107c80b 之间改了什么（847e7dde 不在 git）。正文写"原作者或用户"。	原作者（Stage 10）	中（冻结完整性）	**仍需确认**（Stage 10 原作者 / 用户）；847e7dde 仍找不到 ⟶ 出处 附录 I U34
U35	s1011-s10-r0r1	stage10_extra_r0.py 的作业号（日志不含作业号）。	用户	低（记录完整性）	**已解决（2026-10-30）**：无单独作业号，运行在交互作业 11371284 内 ⟶ 出处 附录 I U35
U36	s1011-stage11	Stage 11 protocol.json 记录的冻结时脚本 26e3f817 到运行版本 f8adb8d9 之间改了哪些行（26e3f817 不在 git）。	原作者（Stage 11）	中（冻结完整性）	**已解决（2026-10-30）**：只加了补充 1 的单元和调度逻辑；diff 存档 research_scripts/s11replay/ ⟶ 出处 附录 I U36
U37	s1011-s11-qc	E1（batch 15 vs batch-1 假基线）的舍入噪声没有同批无替换基线可量化，只有约 0.01 的估计；是否需要补测。	用户	中（E1 精度）	**已解决（2026-10-30）**（用户决定）：答辩前不补测，写约 0.01、远小于主效应 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U38	s12-p1	P1 按 token 角色拆分的数字（":" 对首 token 0.50/0.90 等）只见于 PROGRESS 2026-10-28 14:45，找不到脚本或 summary 文件。	用户	中（补充 4 要求按角色报告）	**已解决（2026-10-30）**：按角色用 raw 重算（":" 对首字母 0.502/0.917） ⟶ 出处 research_scripts/u38_u40_recompute.py

U 39	s12-p1	p5（年份第 3 位数字）在 L44 恢复 0.071 的解读。	用户	低（效应小）	**已解决（2026-10-30）**（用户决定）：只描述不解读 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U 40	s12-p2	P2 是否做过 hook 生效的 sanity（tiny 模型干跑只列了 P1 的检查项）。	用户	中（P2 正确性）	**部分解决（2026-10-30）**：数据显示 hook 生效；显式检查是否补做待用户定 ⟶ 出处 附录 I U40
U 41	s12-n1	Stage 10 单作者读出用的名字字符串格式是否与 mc_team.jsonl 的 initial_surname 一致。旁证：G4.1 写 Stage 10 名字取自 authors 列、拼写不改（如 "Karin G. Coifman"），G3.4 写多选为 "K. Coifman"。	原作者（Stage 10）	低（N1 局限说明）	**已解决（2026-10-30）**：99.51% 与"首字母. 姓"一致 ⟶ 出处 附录 I U41
U 42	s12-f1	F1 作者提示 lens 是否做过拟合质量检查或与模型输出 / 公开 lens 的一致性检查。	用户	中（F2"读不出"的解释）	**部分解决（2026-10-30）**：L24–L48 与其他 lens 余弦 0.27–0.70；对半拟合是否做待用户定 ⟶ 出处 附录 I U42
U 43	s12-f2	F2 中间层 L44–L52 名字位置 target − control 显著为负的解读。	用户	中（导师可能追问）	**已解决（2026-10-30）**（用户决定）：只描述 + 一句可能方向 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U 44	s13-overview	xf_patch 2 个 shard 的脚本 sha 613eb813 是否留存副本（记录称只差日志行，false_0 shard 0 与 Stage 11 逐值相同）。	用户	低（复现已核对）	**已解决（2026-10-30）**：613eb813 是转交时记录的 Stage 11 脚本哈希，文件现存 ⟶ 出处 附录 I U44
U 45	s13-m2	M2 的区间仍是按论文 bootstrap 200 次，是否按补充 2 补团队整群区间。	用户	中（偏离补充 2）	**已解决（2026-10-30）**：团队整群区间已补算，0 翻转 ⟶ 出处 381_stage13/reports/m2_teamci.md · 作业 11503585
U 46	s13-m3	M3 task 单元存下的 year_nlr / author_first_nlr 未被使用，是否计划使用。	用户	低（不影响现有结果）	**已解决（2026-10-30）**（用户决定）：数据存在但未使用 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U 47	s13-m5	M5（①②③）的完整定义，是否放弃或并入 X4。	用户	低（计划项）	**已解决（2026-10-30）**（用户决定）：与 X4 同一问题则并入，否则放弃 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U 48	s13-x2	X2 是否仍做；prereg 要求的 X2 补充未写。	用户	中（H3 未检验）	**已解决（2026-10-30）**（用户决定）：X2 暂停 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U 49	s13-x3	是否接受保留 37 条姓氏子串误拒。	用户	低（偏保守）	**已解决（2026-10-30）**（用户决定）：37 条维持拒绝，写入补充 5 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条
U 50	s13-x3	20 条人工标注的原始标签记录（只在 PROGRESS 摘要中）。	用户	中（判断器校准依据）	**已解决（2026-10-30）**：A 组标签存档 logs/para_review_sample20_labels.csv ⟶ 出处 PROGRESS.md 2026-10-30 12:30 条

U 51	s13-x3、h-prereg-files	补充 5 的具体条款与 para_ok 的放行条件。	用户	高（最关键实验未能开跑）	**仍需确认**（用户）：判断器 v2 一致率 0.55 < 0.80 不用，走备用方案；补充 5 草稿待用户确认 ⟶ 出处 381_stage13/followup/judge2_summary.json
U 52	s13-x4	X4 textbase2 对没有 given_first 的 821 篇仍计入 OOV 类概率差，是否有意。注：总编已于 2026-10-30 用 CPU 按实际角色重算（见该节"总编更正"），结论不变；"是否有意"仍待确认。	用户	中（文本基线的解释）	**已解决（2026-10-30）**：总编按实际角色重算，结论不变 ⟶ 出处 PROGRESS.md 2026-10-30 11:40 条
U 53	s13-x4	b7 样本通过后，旧的 followup/X4TOK_CHECK_FAILED 会让 --all 拒绝启动，如何处理。	用户	中（阻塞复跑）	**已解决（2026-10-30）**：标记已改名，全量 11500437 PASS ⟶ 出处 381_stage13/logs/fugpu_11500437.log
U 54	s13-x5	X5 是否仍做。	用户	低（计划项）	**已解决（2026-10-30）**（用户决定）：M3、X4 全量完成后在 CPU 上做 ⟶ 出处 PROGRESS.md 2026-10-30 14:00 条

附录 G4　计划未做 / 作废 / 被复核取代 / 待复核 汇总

来源：正文全部 `st-void`、`st-plan`、`st-review` 标签（31 处），以及正文描述中明确写为"取代""撤回""无效""放宽""改为"的条目。

#	项目	状态	原因 / 现状	出处
V01	Stage 3 公共 lens 任务验证 351_validate_public_lens.py 与 …_offload.py 两次运行	作废	前者在 40GB 卡 OOM，后者 device_map='auto' 报 meta tensor 错；由 …_offload_fixed.py（显式 29 层上 GPU）取代。	s19-s3-exp0
V02	Public-557 数值检查"63/63 通过、L48 范数 105.8"	复核过	2026-10-30 从 lens 文件重算（作业 11500587）：105.8 是 Public L48 范数的 float32 值（精确 106.07）；旧稿"105.86 是 split-half A 半"的说法有误，已删。	s19-s3-exp1
V03	Stage 3 Self-100 split-half 作为稳定性检验	被设计取代（但取代者未完成）	后由 Stage 9 FullFit 设计取代，而 Stage 9 的 Fit B 与 9B 未完成。	s19-s3-exp7
V04	Stage 7.1B 终端读出（全名前缀 × 21 种标题）	计划未做	只完成 1,166 / 15,519 个上下文（约 7.5%），没有估计任何标题效应；不得与 7.1 合并。	s19-s71b
V05	已发布报告中 7.2、7.3 的若干区间与加权结果	由 2026-10-30 重算替代 复核过	原始输出里没有这些区间；2026-10-30 用 stage7_recompute_intervals.py 重算（种子 20261031 起，另做种子 1–200 的 Monte-Carlo 范围），正文改用重算值；原报告的数都落在 Monte-Carlo 范围内。	s19-s72、s19-s73
V06	Stage 8 冻结计划的主终点（7 层积分）	被 PROTOCOL 改为次要；后由 supplement 补算	analysis_plan.json（14:29）定 7 层积分为主；PROTOCOL.md（21:14）改 L56 为主，8D 按 L56 报告；8D supplement（2026-10-21）补算冻结终点 +2.33 pp [+1.50, +3.19]。	s19-s8-protocol、s19-s8dsupp
V07	Stage 8 计划的 Holm 校正与 leave-one-pair-out	2026-10-30 事后补算 复核过	analysis_plan.json 列出，8D 与 supplement 都没有实现；2026-10-30 补算（作业 11500770）：Holm 只有 L62 通过；积分 A−B 逐对剔除 100/100 区间不含 0，L56 A−B 14/100 跨 0。p 值公式与检验族划分是我们的选择。	s19-s8d、h-stats-mult
V08	Stage 8D supplement 输出与分支 3a4f605	已合入 main	2026-10-30 按用户决定合入 main（a4c1f0a）；输出在仓库 experiment_files/351_stage8/stage8D_supplement/。	s19-s8dsupp
V09	Stage 8E 外部事后审计	2026-10-30 从原始 logits 重算 复核过	原 8E 未在 Explorer 上执行、脚本找不到；2026-10-30 从 8C 原始首 token logits 独立重算四种加权与斜率（作业 11500770），点估计与报告相同。	s19-s8e
V10	Stage 9 Fit B	计划未做（不补做）	检查点停在 66/100（2026-10-21 19:12；作业 11289356 到 2 h 时限，检查点可续跑），之后没有续跑；**用户 2026-10-30 决定答辩前不补做**，局限写明未使用；Stage 10 没用 Fit B。	s19-s9-fitB、h-lens-stage9

V11	Stage 9B 留出集比较（Fit A vs Fit B）	计划未做	holdout.jsonl 已冻结，没有实现代码和任何结果。	s19-s9-bc、h-lens-stage9
V12	Stage 9C 影响因素分析	计划未做	README 中列了提示格式、token 类型、Final-logit anchor 等，均未执行。	s19-s9-bc、h-lens-stage9
V13	Stage 10 原主指标"每篇 Σ 1/rank"	被修订取代	2026-10-23 分析层修订为 log10(1/rank)，理由是 1/rank 只看前几名；raw 存名次，不改原始输出。	g-scoring-rank、s1011-s10-common
V14	Stage 10 parity 容差 0.25	被事后修订取代（0.5）	main parity 10/25 对超 0.25（最大 0.367，final@L63）；用户 2026-10-25 批准 0.5，原 FAIL 记录保留。	h-compute-tol、s1011-s10-parity
V15	Stage 10 parity 作业 11334054 与 19 个 pool 作业	失败运行（无输出）	11334054 因 transformers 5.x 无 hf_device_map 崩溃 → fix1；19 个 pool 作业在 parity FAIL 时被 run() 拒绝、数秒退出。	h-compute-table、s1011-s10-parity
V16	Stage 11 E1t 用 batch-1 假运行作基线	被取代	GPU 上同批 L63 元素与 batch-1 假运行只有 59.4% 名次相同（批量舍入），分析改用同批 L63 元素作假基线，原值另存 false_batch1。	s1011-s11-e1t、s1011-s11-qc
V17	Stage 11 E4b 原版结论措辞	被复核版取代	按论文分折存在团队与标题复用泄漏；确认版为团队折 + 剔除标题复用（作业 11443587），数值上"目标组更高"保留。	s1011-s11-e4b、s1011-s11-e4b-check
V18	"只用假标题训练的 probe 更强"（L24/40/56 +0.068/+0.081/+0.094）	作废	来自假标题供体泄漏；复核后 +0.050/+0.057/+0.074，与普通 probe 持平或略低，说法撤回。	s1011-s11-e4b-check、s12-cpu、h-leak
V19	Stage 12 P1 首次运行（作业 11371622）	失败运行，修复后重算	在 8/9-token 尾部上崩溃（np.concatenate 11 vs 12 项），commit 95dd94d 修复后才产出现有数据。	s12-p1
V20	Stage 12 V100 路径（stage12_v100.py 与非 A100 smoke parity）	计划未做	写好但从未用于真实计算：QOS 每作业最多 2 GPU，2×32GB 放不下 bf16 权重；所有单元都在 A100-80GB 上算。	s12-ov-gpu
V21	Stage 12 N2 的 logit lens / 公开 lens 层剖面分析	计划未做	L48/56/62 读出已算并存在 raw 里，但 stage12_analyze.py 只分析了 final@63。	s12-n2
V22	Stage 12 F1 重拟合（排除同名作者）	改为评估时排除	3 篇评估论文与拟合集同名作者；拟合已到 35/40，不重拟合（约 4 GPU-h），F2 评估时剔除这 3 篇。	s12-f1、h-leak
V23	M2 按论文切分的 false_0 数字	作废	2,004 个 donor 标题跨折；补充 1 判为无效，以团队切重跑（作业 11443403）为准。	h-leak-a、s13-m2
V24	M2 ":" 位置的年份 probe	降为阳性对照	":" 处已看到提示里的年份（R^2 0.974–0.982），属泄题，只作阳性对照；新实验改用不含年份的模板。	h-leak、s13-m2

V25	M2 区间按补充 2 改为团队整群	2026-10-30 已补算 复核过	作业 11503585：176 个点估计全一致，区间平均宽 1.15 倍，0 个翻转；折已固定在 folds_group.npz（group_folds 排序不稳定）。	s13-m2
V26	M2c 原双向判据	被补充 2 取代	改为单向：只有"BERT 追上 Qwen"才作数；Qwen 高出只作参考。	s13-m2c、h-prereg-content
V27	M3 原规则（I > 0 判据、按论文 5 折）与原问法写法	被补充 1/2 与 commit 43c8f0c 取代	改为团队折、只看 I(l) − I(L4)、T 为主；问法改成三者标题前都是 9 个 token 的写法（docstring 里还留着旧写法）。	s13-m3、g-prompts-list
V28	M3 判据实现（自动判定未纳入 T − T(L4)）	待复核	报告按 T 绝对阈值判 H2，而 STATUS 的解读是"T 不高于 T(L4)"。	h-prereg-dev
V29	M5（逐位置 probe / patchscope / 名字位置真假判别）	计划未做	无代码、未定义到可执行；patchscope 部分一定程度由 X4 覆盖。**处置（用户 2026-10-30）**：与 X4 问同一问题的部分并入 X4，否则放弃并写明理由。	s13-m5
V30	X1 原"在 tune 上选层"规则	被补充 2 取代	改为全曲线、不选层、≤ L16 只作文本替换阳性对照、L24 起等价界判定。	s13-x1
V31	X1 random_c 零检验的处理	待复核	只打 NULL FLAG，year/field 恢复率没有相对 random_c 重算，L48 在 FLAG 下仍判 task-independent。	h-prereg-dev
V32	X2（只替换子空间）	计划未做（暂停）	prereg 要求运行前另写补充，未写；无代码；**用户 2026-10-30 决定暂停**。	s13-x2、h-prereg-dev
V33	X3 主实验 para_run	计划未做	只在 logs/para_ok 存在时排队，2026-10-30 不存在；等补充 5。	s13-x3、h-prereg-dev
V34	预注册补充 5	计划未做	尚未写定，草稿已发给用户待确认（拟含 jaccard2、排除 27 条完全相同改写、37 条姓氏误拒维持拒绝、意义检查走备用方案）。	s13-overview、h-prereg-files
V35	X3 原版 Jaccard 分词与 X3 原主指标 G	将被 jaccard2 / 已被补充 4 的 G′ 取代	原分词丢掉 ≤ 3 字母的词、不还原词形；jaccard2 随补充 5 写入代码；主指标已改为 G′（近邻 D）。	s13-x3
V36	X3 排除 27 条完全相同改写	已同意、未写入代码	用户 2026-10-29 同意排除，do_para_run 目前只按 auto_ok 过滤。	s13-x3
V37	X3 Qwen 判断器（v1 校准与全量判断）	作废（由 v2 取代，v2 也不用）	v1 校准一致率 7/19，阈值过严，全量未跑；之后写了判断器 v2（见 V46）。	s13-x3、g-model-chat
V38	X4 文本基线 1（kNN）	结论不成立	阳性对照失败（TF-IDF −0.025、BERT −0.159），"文本解释不了"不能由它支持。	s13-x4
V39	X4 文本基线 2 原 D_text（含无关 OOV 项）	被总编更正取代	821 篇无 given_first 的论文计入了 OOV 类概率差；2026-10-30 CPU 按实际角色重算，结论不变。	s13-x4

V40	X4 逐 token 复算（x4tok，第一版 batch 2）	被 b7 修法取代 复核过	第一版自检 max |diff| 0.0683 > 0.05，作业 11485349 按设计停止；b7 修法全量 11500437：4,712 个值逐值复现，L8 平均 D b7 +0.0484 / b2 +0.0490（差 0.0006 < 0.005）。	s13-x4、h-compute-parity
V41	按 token 角色报告（补充 4）：X4、X6、X7	待复核（X6、X7 未完成）	X4 已于 2026-10-30 按角色报告（姓首 token +0.066 [0.046, 0.088]）；P1 已用 raw 重算；x6/x7 分析脚本与报告中仍没有角色拆分。	h-prereg-dev、s13-x4
V42	X4 recognition 规则与区间版本	待复核	recognition 用 min_n = 10、全部论文、无 tune/test；X1/X4/X6/X7/M3 只给团队版区间（补充 2 要求两种都给）。	h-prereg-dev
V43	X5（层序量化）	计划未做	有预注册条款，无代码，未执行；用户 2026-10-30 定为 M3、X4 全量完成后用已有数据在 CPU 上做。	s13-x5、h-prereg-dev
V44	X7 分析作业 11470471	失败运行，修复后重跑	自检里把 D.T 写成 pandas 转置而崩溃；改为 D['T'] 并把自检包成只 WARN，作业 11470524。	s13-x7
V45	文档中与代码不符的说法	以代码为准	STATUS.md §2 的代价公式省掉了归一化分母；已发布报告 §3.1"去掉作者 ≥ 20 位"与代码（1–20 含 20）不符；任务说明称 para_gen 用 chat 模板，与代码不符；STAGES.md 的 7.3 完成日期与 mtime 不符。	h-false-cost、g-data-origin、g-model-chat、s19-s73
V46	X3 判断器 v2（stage13_judge2.py，作业 11501072）	作废（按规则不用）	A 组校准一致率 0.90，B 组检验 0.55（去边界 0.79），低于用户定的 0.80，不用；改走备用方案（人工核对 + 有/无敏感性），补充 5 待用户确认。	s13-x3
V47	E1 同批无替换基线（批量舍入噪声的量化）	计划未做	用户 2026-10-30 决定答辩前不补测，写约 0.01、远小于主效应；GPU 空闲时按 X4 的做法补。	s1011-s11-qc
V48	Stage 13 group_folds 的折分配	待复核	同大小组排序不稳定，不同 CPU 型号上折可能不同（d0026 上点估计最多差 0.006）；M2 补算已固定折，其他调用脚本未查。	h-stats-team
V49	E4b 标准化用全部论文（含测试折）	被训练折标准化重算复核 复核过	作业 11500917：最大差 0.0075，α 不变，确认版无"区间是否含 0"翻转；L44 下界 +0.000（边界）。	s1011-s11-e4b-check

G5 索引：脚本、结果文件、作业号、git commit

做法：用脚本从 6 个正文部分的全部 `class="src"` 出处里抽取脚本名和结果文件名（正则匹配扩展名），再把相对写法解析成集群上的绝对路径（按所在章节的 Stage 结果根目录与 `~/jlens_work/` 查找，`os.path.exists` 核实存在）；作业号与 git commit 从正文全文（含出处）抽取，作业名、分区、开始时间、用时、状态取自本次 `sacct -X -j …` 查询，commit 时间与说明取自 `git log`。去重后列出，不截断。"提及章节"是该条目出现的正文锚点。分组依据：脚本按代码目录，结果按结果根目录，作业号按提及它的章节所属 Stage（只在全局章节出现的按 sacct 作业名）。⟶ 出处 抽取脚本：本附录起草时的临时脚本（scratchpad，build/index.py、render_index.py）· sacct 2026-10-30 · git -C /scratch/zhang.zhiqi2/jlens-authorship-recall_update log

G5.1 脚本（按 Stage）

全局

脚本（绝对路径）	提及章节
`~/projects/jacobian-lens/jlens/_stage10_compat.py`	g-model-jlens、h-lens-jac
`~/projects/jacobian-lens/jlens/fitting.py`	h-lens-jac、s12-f1
`~/projects/jacobian-lens/jlens/hf.py`	g-model-id、g-model-jlens、h-lens-defs
`~/projects/jacobian-lens/jlens/lens.py`	h-lens-defs、h-lens-jac

Stage 1

脚本（绝对路径）	提及章节
`~/jlens_work/00_cohort/351_prepare_all.py`	s19-s1-cohort

Stage 3

脚本（绝对路径）	提及章节
`~/jlens_work/01_public_lens_validation/351_author_specificity.py`	g-model-load、s19-s3-exp0
`~/jlens_work/01_public_lens_validation/351_paper_prompt_fidelity_offload.py`	g-model-load
`~/jlens_work/01_public_lens_validation/351_public_jlens_smoke.py`	g-model-load、s19-s3-exp0
`~/jlens_work/01_public_lens_validation/351_public_lens_fidelity.py`	g-model-load
`~/jlens_work/01_public_lens_validation/351_validate_public_lens.py`	g-model-load、g-prompts-list

`~/jlens_work/01_public_lens_validation/351_validate_public_lens_offload_fixed.py`	g-model-load、s19-common、s19-s3-exp3
`~/jlens_work/02_self_lens_validation/351_self_gold_false.py`	g-model-load
`~/jlens_work/02_self_lens_validation/351_self_jlens_split_half.py`	g-model-load、h-lens-stage9
`~/jlens_work/02_self_lens_validation/351_self_jlens_validation.py`	g-model-load、s19-s3-exp24

Stage 4

脚本（绝对路径）	提及章节
`~/jlens_work/04_stage4/351_stage4_readout.py`	g-model-load、g-prompts-list、s19-common、s19-s2-protocol、s19-s4-readout

Stage 5

脚本（绝对路径）	提及章节
`~/jlens_work/05_stage5/351_stage5.py`	g-model-load、g-prompts-list、s19-common、s19-s1-pilot50、s19-s2-protocol、s19-s5-crossed

Stage 6

脚本（绝对路径）	提及章节
`~/jlens_work/06_stage6/351_stage6.py`	g-model-load、g-prompts-list、g-scoring-rank、s19-s6-jmrr

Stage 7

脚本（绝对路径）	提及章节
`~/jlens_work/07_1_stage7_1/351_stage7.py`	s19-s71
`~/jlens_work/07_1_stage7_1/351_stage7_1B_terminal.py`	g-model-load、g-prompts-list、s19-s2-protocol、s19-s71b
`~/jlens_work/07_2_stage7_2/351_stage7_2.py`	s19-s72
`~/jlens_work/07_2_stage7_2/351_stage7_2_score.py`	g-model-load、g-prompts-list、h-stats-mult、s19-s72
`~/jlens_work/07_3_stage7_3/351_stage7_3.py`	s19-s73
`~/jlens_work/07_3_stage7_3/351_stage7_3_score.py`	g-model-load、g-prompts-list、s19-s73
`~/jlens_work/07_4_stage7_4/351_stage7_4.py`	h-stats-mult、s19-s74

Stage 8

脚本（绝对路径）	提及章节
`~/jlens_work/08_stage8/351_stage8.py`	h-stats-mult、s19-s8a

`~/jlens_work/08_stage8/351_stage8B.py`	s19-s8b
`~/jlens_work/08_stage8/351_stage8C.py`	g-model-load、s19-s8c
`~/jlens_work/08_stage8/351_stage8D.py`	s19-s8d
`~/jlens_work/08_stage8/351_stage8D_supplement.py`	h-stats-mult、s19-s8dsupp

Stage 9

脚本（绝对路径）	提及章节
`~/jlens_work/09_stage9/351_stage9.py`	g-model-load、h-lens-table、s19-s9-split
`~/jlens_work/09_stage9/stage9_a_repair.py`	s19-s9-fitA

Stage 10

脚本（绝对路径）	提及章节
`~/jlens_work/10_stage10/351_stage10.py`	g-data-cohort、g-model-chat、g-model-id、g-model-jlens、g-model-load、g-prompts-align、g-prompts-list、g-scoring-agg、g-scoring-names、g-scoring-rank、g-scoring-spec、h-compute-hw、h-compute-parity、h-compute-tol、h-false、h-false-79、h-false-algo、h-false-cost、h-false-pool、h-false-tiers、h-false-use、h-lens-defs、h-lens-use、h-stats-boot、h-stats-strat、s1011-s10-common、s1011-s10-main、s1011-s10-parity、s1011-s10-tc、s1011-s10-titlespec、s1011-stage10、s12-cpu、s12-n1、s12-ov-common、s12-p1
`~/jlens_work/10_stage10/351_stage10_fix1.py`	g-model-load、s1011-s10-common
`~/jlens_work/10_stage10/stage10_decision.sbatch`	s1011-s10-main
`~/jlens_work/10_stage10/stage10_ext_analyze.sbatch`	s1011-s10-fine
`~/jlens_work/10_stage10/stage10_ext_worker.sbatch`	s1011-s10-fine
`~/jlens_work/10_stage10/stage10_extra.py`	s1011-s10-cite、s1011-s10-format、s1011-s10-lens
`~/jlens_work/10_stage10/stage10_extra_cond_role.py`	s1011-s10-format

`~/jlens_work/10_stage10/stage10_extra_format.py`	s1011-s10-format
`~/jlens_work/10_stage10/stage10_extra_r0r1.py`	g-labels-match、s1011-s10-r0r1
`~/jlens_work/10_stage10/stage10_parity.sbatch`	s1011-s10-main
`~/jlens_work/10_stage10/stage10_pool.py`	s1011-s10-main
`~/jlens_work/10_stage10/stage10_steal.py`	s1011-s10-main

Stage 11

脚本（绝对路径）	提及章节
`~/jlens_work/11_stage11/361_stage11.py`	g-model-load、g-prompts-align、g-prompts-list、g-scoring-agg、g-scoring-rank、h-compute-parity、s1011-s11-common、s1011-s11-e1、s1011-s11-e1t、s1011-s11-e2、s1011-s11-e2n、s1011-s11-e3、s1011-s11-e4、s1011-s11-e4b、s1011-s11-qc、s1011-stage11、s12-cpu、s12-n1、s12-ov-common、s12-ov-gpu、s12-p1、s12-p2、s13-x6、s13-x7
`~/jlens_work/11_stage11/stage11_analyze.py`	h-stats-strat、s1011-s11-common、s1011-s11-e1、s1011-s11-e1t、s1011-s11-e2、s1011-s11-e3、s1011-s11-e4、s1011-s11-e4b
`~/jlens_work/11_stage11/stage11_cpu_realcheck.py`	g-model-load、s1011-s11-qc

Stage 12

脚本（绝对路径）	提及章节
`~/jlens_work/12_stage12/371_stage12.py`	g-labels-mc、g-model-chat、g-model-load、g-prompts-align、g-prompts-list、g-scoring-agg、h-false-use、h-lens-table、h-lens-use、s12-f1、s12-f2、s12-n1、s12-n2、s12-ov-common、s12-ov-gpu、s12-ov-scope、s12-ov-units、s12-p1、s12-p2
`~/jlens_work/12_stage12/stage12_analyze.py`	h-stats-strat、s12-f2、s12-n1、s12-n2、s12-ov-common、s12-p1、s12-p2
`~/jlens_work/12_stage12/stage12_cpu.py`	s1011-s10-prior、s1011-s11-e4b-check、s12-cpu、s12-ov-common
`~/jlens_work/12_stage12/stage12_v100.py`	s12-ov-gpu

Stage 13

脚本（绝对路径）	提及章节
`~/jlens_work/13_stage13/381_stage13.py`	g-model-chat、g-model-id、g-model-load、g-prompts-align、g-prompts-example、g-prompts-list、g-scoring-agg、g-scoring-rank、s1011-s11-e1、s13-m3、s13-overview、s13-x1、s13-x3、s13-x4
`~/jlens_work/13_stage13/stage13_cluster_ci.py`	h-stats-boot、h-stats-strat
`~/jlens_work/13_stage13/stage13_cpu.py`	h-false-use、h-leak-a、h-stats-boot、h-stats-team、s1011-s11-e4b-check、s13-m1、s13-m2、s13-overview
`~/jlens_work/13_stage13/stage13_e4b_group.py`	h-false-use、s1011-s11-e4b-check
`~/jlens_work/13_stage13/stage13_e4b_group.sbatch`	s1011-s11-e4b-check
`~/jlens_work/13_stage13/stage13_gpu_followup.py`	g-model-chat、g-prompts-list、g-scoring-rank、s13-overview、s13-x3、s13-x4
`~/jlens_work/13_stage13/stage13_group.sbatch`	s13-m1
`~/jlens_work/13_stage13/stage13_m2b.py`	s13-m2b
`~/jlens_work/13_stage13/stage13_m2c.py`	s13-m2c
`~/jlens_work/13_stage13/stage13_m3_analyze.py`	h-prereg-dev、h-stats-boot、s13-m3、s13-overview
`~/jlens_work/13_stage13/stage13_para_nli.py`	s13-x3
`~/jlens_work/13_stage13/stage13_x1_similarity.py`	s13-x1
`~/jlens_work/13_stage13/stage13_x1x4_analyze.py`	h-prereg-dev、h-stats-strat、s13-overview、s13-x1、s13-x4
`~/jlens_work/13_stage13/stage13_x4_textbase.py`	s13-x4
`~/jlens_work/13_stage13/stage13_x4_textbase2.py`	s13-x4
`~/jlens_work/13_stage13/stage13_x6_analyze.py`	h-false-use、h-prereg-dev、h-stats-boot、s13-x6
`~/jlens_work/13_stage13/stage13_x6_analyze.sbatch`	s13-x6

`~/jlens_work/13_stage13/stage13_x7_analyze.py`	h-false-use、h-prereg-dev、h-stats-boot、s13-x7

出处里出现、但不在 `~/jlens_work` 代码目录中的源码（外部库或原研究代码），按原文写法列出：

原文写法	说明	提及章节
`/transformers/integrations/sdpa_attention.py`	transformers 库（conda 环境内）	s1011-s11-e2
`home_project/src/dua/data/filters.py`	Dreaming Up Authors 原研究代码（git 镜像 home_project/）	g-data-origin
`home_project/src/dua/sampling/stratified.py`	同上，分层抽样	g-data-fields、g-data-origin
`jlens-authorship-recall_update/home_project/05_fit_jlens_prelim_100.py`	git 镜像中的 self100 拟合相关脚本（GitHub 镜像 home_project/）	g-model-load
`jlens/__init__.py`	jlens 包（~/projects/jacobian-lens/jlens/__init__.py）	h-lens-jac
`matcher.py`	原研究姓名匹配器 home_project/src/dua/evaluate/matcher.py	g-labels-match
`models/qwen3/modeling_qwen3.py`	transformers 库 Qwen3 模型实现	s12-p2
`stratified.py`	同上（简写）	g-data-fields

G5.2 结果文件与目录（按 Stage）

全局

文件 / 目录（绝对路径）	提及章节
`~/jlens_work/CLAUDE.md`	h-compute-tol
`~/jlens_work/HANDOFF.md`	h-compute-tol
`~/jlens_work/PROGRESS.md`	g-model-chat、h-compute-parity、h-compute-table、h-compute-tol、h-false-79、h-false-limits、h-false-pool、h-leak、h-leak-a、h-lens-stage9、h-prereg-dev、h-prereg-files、s1011-s10-main、s1011-s10-prior、s1011-s11-e1、s1011-s11-e4b-check、s1011-s11-qc、s12-cpu、s12-f1、s12-f2、s12-n1、s12-ov-gpu、s12-ov-units、s12-p1、s12-p2、s13-x4
`~/jlens_work/README.md`	s19-s6-jmrr、s19-s72
`~/jlens_work/STAGES.md`	h-lens-stage9、s19-s1-cohort、s19-s2-protocol、s19-s8e、s19-s9-bc
`/scratch/zhang.zhiqi2/jlens-authorship-recall_update/ARCHIVE_NOTES.md`	s19-s71b

/scratch/zhang.zhiqi2/jlens-authorship-recall_update/home_project/DATA_LICENSE.md	g-data-origin
/scratch/zhang.zhiqi2/jlens-authorship-recall_update/home_project/README.md	g-data-origin
/scratch/zhang.zhiqi2/jlens-authorship-recall_update/home_project/config/config.yaml	g-data-origin
/scratch/zhang.zhiqi2/jlens/STATUS.md	g-labels-match、g-model-load、h-compute-table、h-compute-tol、h-false-cost、h-prereg-dev、s12-f2、s12-n2、s13-m4

Stage 1

文件 / 目录（绝对路径）	提及章节
/scratch/zhang.zhiqi2/jlens/351_all/351_audit.json	g-data-cohort、g-data-package、s19-s1-cohort
/scratch/zhang.zhiqi2/jlens/351_all/351_author_specificity.jsonl	s19-s3-exp0
/scratch/zhang.zhiqi2/jlens/351_all/351_author_specificity.log	s19-s3-exp0
/scratch/zhang.zhiqi2/jlens/351_all/351_bin_counts.csv	s19-s1-cohort
/scratch/zhang.zhiqi2/jlens/351_all/351_offload_fixed_validation.log	g-model-load、s19-s3-exp0
/scratch/zhang.zhiqi2/jlens/351_all/351_offload_validation.log	s19-common、s19-s3-exp0
/scratch/zhang.zhiqi2/jlens/351_all/351_paper_fidelity.log	s19-s3-exp24
/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/README_first.md	g-labels-mc、g-labels-open、g-model-chat、s19-s1-cohort
/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/bin_table.csv	g-labels-groups、g-labels-recall、h-stats-bins
/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/label_README.md	g-data-fields、g-data-origin、g-data-package、g-labels-groups、g-labels-mc、g-labels-recall、s12-n1、s19-s1-cohort
/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/mc_team.jsonl	g-labels-mc、s12-n1
/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/paper_labels_with_papers.csv	h-stats-bins
/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/rules.yaml	g-labels-groups、g-labels-recall、s19-s1-cohort
/scratch/zhang.zhiqi2/jlens/labels_20261015/zhiqi_qwen3-32b_labels_2026-10-15/step1_README.md	g-data-package、g-labels-match、g-labels-mc、g-labels-open、g-model-chat、g-prompts-list、s12-n1、s19-s1-cohort

Stage 3

文件 / 目录（绝对路径）	提及章节
`~/jlens_work/03_provenance_audit/351_jlens_fitting_diff.txt`	s19-s3-exp6
`/scratch/zhang.zhiqi2/jlens/351_all/351_public_task_validation.log`	s19-common、s19-s3-exp0
`/scratch/zhang.zhiqi2/jlens/351_provenance_audit/reports/audit_report.md`	h-leak-a、h-lens-limits、s19-s3-exp0、s19-s3-exp5
`/scratch/zhang.zhiqi2/jlens/351_provenance_audit/summary/audit_summary.json`	s19-common、s19-s3-exp6
`/scratch/zhang.zhiqi2/jlens/351_provenance_audit/summary/protocol_comparison.csv`	h-lens-jac、h-lens-table、s19-s3-exp6
`/scratch/zhang.zhiqi2/jlens/351_self100_gold_false/archive/config/comparison_config.json`	s19-s3-exp3
`/scratch/zhang.zhiqi2/jlens/351_self100_gold_false/archive/summary/layer_summary.csv`	s19-s3-exp3
`/scratch/zhang.zhiqi2/jlens/351_self100_split_half/fits/complete_A.json`	s19-s3-exp7
`/scratch/zhang.zhiqi2/jlens/351_self100_split_half/fits/complete_B.json`	s19-s3-exp7
`/scratch/zhang.zhiqi2/jlens/351_self100_split_half/fits/config_A.json`	s19-s3-exp7
`/scratch/zhang.zhiqi2/jlens/351_self100_split_half/readout_comparison/reports/readout_stability_report.md`	s19-s3-exp7
`/scratch/zhang.zhiqi2/jlens/351_self100_split_half/reports/split_half_report.md`	s19-s3-exp7
`/scratch/zhang.zhiqi2/jlens/351_self100_split_half/summary/matrix_stability.csv`	s19-s3-exp1、s19-s3-exp7
`/scratch/zhang.zhiqi2/jlens/351_self100_validation/numerical/matrices.csv`	s19-s3-exp1
`/scratch/zhang.zhiqi2/jlens/351_self100_validation/summary/metrics.csv`	s19-s3-exp24
`/scratch/zhang.zhiqi2/jlens/351_self100_validation/summary/paired_bootstrap_ci.csv`	s19-s3-exp24
`/scratch/zhang.zhiqi2/jlens/351_split_A.log`	h-compute-table
`/scratch/zhang.zhiqi2/jlens/351_split_B.log`	h-compute-table
`/scratch/zhang.zhiqi2/jlens/fitting_corpus/manifest.json`	g-scoring-names、h-lens-table
`/scratch/zhang.zhiqi2/jlens/qwen32b_jlens_prelim_100.log`	h-compute-table、h-lens-table、s19-s3-exp0
`/scratch/zhang.zhiqi2/jlens/qwen32b_jlens_prelim_100_config.json`	h-lens-table、s19-s3-exp0

Stage 4

文件 / 目录（绝对路径）	提及章节

/scratch/zhang.zhiqi2/jlens/351_all/351_stage4_artifacts/audit/audit_report.md	s19-s4-readout
/scratch/zhang.zhiqi2/jlens/351_all/351_stage4_artifacts/config/run_config.json	g-prompts-list、s19-common、s19-s4-readout
/scratch/zhang.zhiqi2/jlens/351_all/351_stage4_readout.log	g-model-load、s19-s4-readout

Stage 5

文件 / 目录（绝对路径）	提及章节
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/cohort/pilot50_v1.csv	s19-s1-pilot50
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/cohort/pilot50_v1.json	s19-s1-pilot50
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/config/runtime.json	s19-common
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/matching/pilot50_ab_pairs.csv	s19-s1-pilot50
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/reports/pilot50_audit_manifest.json	s19-s5-crossed
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/stage5_run.log	g-model-load、s19-s5-crossed
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/summary/layer_summary.csv	s19-s5-crossed
/scratch/zhang.zhiqi2/jlens/351_stage5_crossed_v2/summary/matched_ab_summary.csv	s19-s5-crossed

Stage 6

文件 / 目录（绝对路径）	提及章节
/scratch/zhang.zhiqi2/jlens/351_stage6/config/protocol.json	s19-common、s19-s2-protocol
/scratch/zhang.zhiqi2/jlens/351_stage6/raw/token_layer_scores.csv	s19-s6-jmrr
/scratch/zhang.zhiqi2/jlens/351_stage6/reports/experiment_report.md	s19-s6-jmrr
/scratch/zhang.zhiqi2/jlens/351_stage6/summary/matched_ab_bootstrap.csv	s19-s6-jmrr
/scratch/zhang.zhiqi2/jlens/351_stage6/summary/stage5_stage6_paper_join.csv	s19-s6-jmrr
/scratch/zhang.zhiqi2/jlens/351_stage6/summary/summary.json	s19-s6-jmrr
/scratch/zhang.zhiqi2/jlens/351_stage6_run.log	h-compute-table
/scratch/zhang.zhiqi2/jlens_archive_stage1_8/experiment_files/351_stage6_run.log	g-model-load

Stage 7

文件 / 目录（绝对路径）	提及章节

/scratch/zhang.zhiqi2/jlens/351_stage7_1/reports/experiment_report.md	s19-s71
/scratch/zhang.zhiqi2/jlens/351_stage7_1/summary/layer_and_integrated_effects.csv	s19-s71
/scratch/zhang.zhiqi2/jlens/351_stage7_1B_terminal/run.log	s19-s71b
/scratch/zhang.zhiqi2/jlens/351_stage7_1B_terminal/runtime.json	g-model-load
/scratch/zhang.zhiqi2/jlens/351_stage7_1B_terminal/smoke_check.json	s19-s71b
/scratch/zhang.zhiqi2/jlens/351_stage7_2/candidates/pilot50_author_candidates.csv	s19-s72
/scratch/zhang.zhiqi2/jlens/351_stage7_2/config/gpu_scoring_runtime.json	g-model-load
/scratch/zhang.zhiqi2/jlens/351_stage7_2/reports/stage7_2_gpu_report.md	s19-s72
/scratch/zhang.zhiqi2/jlens/351_stage7_2/summary/candidate_pairwise_scores.csv	s19-s72
/scratch/zhang.zhiqi2/jlens/351_stage7_2/summary/matched_ab_bootstrap.csv	s19-s72
/scratch/zhang.zhiqi2/jlens/351_stage7_2/summary/quality.json	s19-s72、s19-s73
/scratch/zhang.zhiqi2/jlens/351_stage7_2_gpu.log	h-compute-table
/scratch/zhang.zhiqi2/jlens/351_stage7_3/config/stage73b_runtime.json	g-model-load
/scratch/zhang.zhiqi2/jlens/351_stage7_3/reports/freeze_audit.md	s19-s73
/scratch/zhang.zhiqi2/jlens/351_stage7_3/reports/stage7_3_null_report.md	s19-s73
/scratch/zhang.zhiqi2/jlens/351_stage7_3/summary/integrated_ab_bootstrap.csv	s19-s73
/scratch/zhang.zhiqi2/jlens/351_stage7_3/summary/paper_null_distribution.csv	s19-s73
/scratch/zhang.zhiqi2/jlens/351_stage7_3_gpu.log	h-compute-table
/scratch/zhang.zhiqi2/jlens/351_stage7_4/summary/cross_stage_correlations.csv	s19-s74
/scratch/zhang.zhiqi2/jlens/351_stage7_4/summary/floor_L56_low_six.csv	s19-s73
/scratch/zhang.zhiqi2/jlens/351_stage7_4/summary/jlens_increment_bootstrap.csv	s19-s74
/scratch/zhang.zhiqi2/jlens/351_stage7_4/summary/overview.json	s19-s73、s19-s74

Stage 8

文件 / 目录（绝对路径）	提及章节
~/jlens_work/08_stage8/351_stage8_PROTOCOL.md	s19-s8-protocol
/scratch/zhang.zhiqi2/jlens/351_stage8/config/analysis_plan.json	g-data-cohort、h-compute-tol、s19-s8-protocol
/scratch/zhang.zhiqi2/jlens/351_stage8/config/manifest.json	s19-s8a
/scratch/zhang.zhiqi2/jlens/351_stage8/config/stage8B_manifest.json	s19-s8b
/scratch/zhang.zhiqi2/jlens/351_stage8/config/stage8C_runtime.json	g-model-load、s19-s2-protocol、s19-s8c

`/scratch/zhang.zhiqi2/jlens/351_stage8/controls/stage8_control_quality.csv`	s19-s8b
`/scratch/zhang.zhiqi2/jlens/351_stage8/controls/stage8_shuffled_titles.csv`	s19-s8b
`/scratch/zhang.zhiqi2/jlens/351_stage8/raw/stage8C_context_scores.jsonl`	s19-s8c
`/scratch/zhang.zhiqi2/jlens/351_stage8/stage8D/output_manifest.json`	s19-s8c
`/scratch/zhang.zhiqi2/jlens/351_stage8/stage8D/reports/stage8_results.md`	s19-s8d
`/scratch/zhang.zhiqi2/jlens/351_stage8/stage8D/summary/jlens_minus_vanilla.csv`	s19-s8d
`/scratch/zhang.zhiqi2/jlens/351_stage8/stage8D/summary/layer_and_band_summary.csv`	s19-s8d
`/scratch/zhang.zhiqi2/jlens/351_stage8/stage8D/summary/primary_questions.csv`	s19-s8d
`/scratch/zhang.zhiqi2/jlens/351_stage8/stage8D/summary/sensitivity.csv`	s19-s8d
`/scratch/zhang.zhiqi2/jlens/351_stage8/summary/matching_audit.json`	s19-s8a
`/scratch/zhang.zhiqi2/jlens/351_stage8C_full.log`	h-compute-table、s19-s8c
`/scratch/zhang.zhiqi2/jlens_github_publish/experiment_files/351_stage8/stage8D_supplement/frozen_plan_endpoints.csv`	s19-s8dsupp

Stage 9

文件 / 目录（绝对路径）	提及章节
`~/jlens_work/09_stage9/351_stage9_README.md`	s19-s9-bc、s19-s9-fitA
`/scratch/zhang.zhiqi2/jlens/351_stage9/config/manifest.json`	s19-s9-bc、s19-s9-split
`/scratch/zhang.zhiqi2/jlens/351_stage9/config/plan.json`	h-lens-table、s19-s9-split
`/scratch/zhang.zhiqi2/jlens/351_stage9/reports/stage9_resume_comparison.json`	s19-s9-fitA
`/scratch/zhang.zhiqi2/jlens/351_stage9/runner_runs/full_A_repair_101_100valid/final_status.json`	s19-s9-fitA
`/scratch/zhang.zhiqi2/jlens/351_stage9/runner_runs/full_A_repair_101_100valid/repair_manifest.json`	s19-s9-fitA
`/scratch/zhang.zhiqi2/jlens/351_stage9_A_repair.log`	h-compute-table、h-lens-stage9、s19-s9-fitA
`/scratch/zhang.zhiqi2/jlens/351_stage9_benchmark_A1_7layers.log`	s19-s9-fitA
`/scratch/zhang.zhiqi2/jlens/351_stage9_full_A.log`	h-compute-table、s19-s9-fitA
`/scratch/zhang.zhiqi2/jlens/351_stage9_full_B.log`	h-compute-table、h-lens-stage9

Stage 10

文件 / 目录（绝对路径）	提及章节

~/jlens_work/10_stage10/README.md	s1011-s10-main
/scratch/zhang.zhiqi2/jlens/351_stage10/cohort/conditions/false_titles.parquet	h-false-79、h-false-examples
/scratch/zhang.zhiqi2/jlens/351_stage10/cohort/conditions/prompts.parquet	s12-ov-common
/scratch/zhang.zhiqi2/jlens/351_stage10/cohort/name_tokens.parquet	s1011-s10-common、s12-p1、s13-m1、s13-x4
/scratch/zhang.zhiqi2/jlens/351_stage10/cohort/papers.parquet	h-false-examples、h-false-pool、h-stats-bins、s1011-s10-common、s13-overview、s13-x3
/scratch/zhang.zhiqi2/jlens/351_stage10/config/analysis_plan.json	g-labels-groups、g-scoring-rank、s1011-s10-common
/scratch/zhang.zhiqi2/jlens/351_stage10/config/analysis_plan_before_tol05.json	h-compute-tol
/scratch/zhang.zhiqi2/jlens/351_stage10/config/conditions_manifest.json	g-prompts-list、g-scoring-spec、h-false、h-false-79、h-false-algo、h-false-cost、h-false-pool、h-false-tiers、s1011-s10-titlespec、s1011-stage10
/scratch/zhang.zhiqi2/jlens/351_stage10/config/lenses.json	h-lens-table、s1011-s10-main、s12-n2、s19-s9-fitA
/scratch/zhang.zhiqi2/jlens/351_stage10/config/manifest.json	g-data-package、g-scoring-names、s1011-s10-common、s1011-stage10、s12-ov-scope
/scratch/zhang.zhiqi2/jlens/351_stage10/config/protocol.json	g-data-cohort、g-model-id、g-model-jlens、h-lens-defs、s1011-s10-common、s1011-s10-main
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/analyze_s10_analyze_main_11373362.log	s1011-s10-main
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/analyze_s10_analyze_title_11375853.log	s1011-s10-titlespec
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/analyze_s10x_analyze_11378377.log	s1011-s10-fine
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/decision_11363838.log	s1011-s10-main
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/extra_11376473.log	s1011-s10-format
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/extra_fmt_11376890.log	s1011-s10-format
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/freeze_conditions_11333739.log	h-false
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/parity_ext_11338000.log	s1011-s10-fine
/scratch/zhang.zhiqi2/jlens/351_stage10/logs/pool/job_s10p_ext_true_11363851.log	h-lens-use
/scratch/zhang.zhiqi2/jlens/351_stage10/reports/crosscheck_stage6.csv	h-compute-parity、s1011-s10-parity
/scratch/zhang.zhiqi2/jlens/351_stage10/reports/parity.json	g-scoring-rank、h-compute-parity、h-compute-table、h-compute-tol、s1011-s10-fine、s1011-s10-parity
/scratch/zhang.zhiqi2/jlens/351_stage10/reports/r0_r1_analysis.md	g-labels-match、s1011-s10-r0r1

/scratch/zhang.zhiqi2/jlens/351_stage10/reports/r0_sensitivity.md	s1011-s10-r0r1
/scratch/zhang.zhiqi2/jlens/351_stage10/summary/A_group_layer.csv	s1011-s10-main
/scratch/zhang.zhiqi2/jlens/351_stage10/summary/B_target_vs_control_within_bin.csv	h-stats-bins、s1011-s10-common、s1011-s10-main
/scratch/zhang.zhiqi2/jlens/351_stage10/summary/C_graded_mc_team_a.csv	s1011-s10-tc
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X1_by_format.csv	s1011-s10-format
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X1_cond.csv	s1011-s10-format
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X1_cond_by_first_role.csv	s1011-s10-format
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X2_by_bin.csv	s1011-s10-cite
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X2_spearman.csv	s1011-s10-cite
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X3_means.csv	s1011-s10-lens
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X3_pairwise.csv	s1011-s10-lens
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_extra/X3_tc_spec_vs_logit.csv	s1011-s10-lens
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_r0/target_R1_minus_R0_within_bin.csv	s1011-s10-r0r1
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_r0r1/C_minus_A.csv	s1011-s10-r0r1
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_r0r1/R1_author_written_minus_missed.csv	s1011-s10-r0r1
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_r0r1/tc_R0.csv	s1011-s10-r0r1
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_title/A_group_layer.csv	s1011-s10-titlespec
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_title/B_sens_drop_low_bins.csv	s1011-s10-tc
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_title/B_spec_target_vs_control_within_bin.csv	s1011-s10-fine、s1011-s10-tc
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_title/C_graded_mc_team_a.csv	s1011-s10-tc
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_title/negative_control_by_role.csv	s1011-s10-titlespec
/scratch/zhang.zhiqi2/jlens/351_stage10/summary_title/paper_title_contrasts.parquet	g-scoring-spec、h-false-79

Stage 11

文件 / 目录（绝对路径）	提及章节
`/scratch/zhang.zhiqi2/jlens/361_stage11/config/protocol.json`	s1011-s11-common、s1011-stage11、s12-ov-scope、s12-p1
`/scratch/zhang.zhiqi2/jlens/361_stage11/config/protocol_amendment_1.json`	s1011-s11-e1t、s1011-s11-e2n
`/scratch/zhang.zhiqi2/jlens/361_stage11/config/sample.parquet`	g-prompts-align、s1011-s11-common、s1011-s11-e1、s12-ov-common、s13-overview
`/scratch/zhang.zhiqi2/jlens/361_stage11/logs/realcheck_11396804.log`	h-compute-parity、s1011-s11-e1t、s1011-s11-e2、s1011-s11-e2n、s1011-s11-qc
`/scratch/zhang.zhiqi2/jlens/361_stage11/raw/patch/shard_000.npz`	s13-x6
`/scratch/zhang.zhiqi2/jlens/361_stage11/reports/smoke.json`	h-compute-parity、s1011-s11-e1、s1011-s11-e1t、s1011-s11-e2、s1011-s11-e2n、s1011-s11-qc
`/scratch/zhang.zhiqi2/jlens/361_stage11/smoke/raw/patch_title/shard_000.npz`	s1011-s11-e1t、s1011-s11-qc
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary/E1_recovery.csv`	s1011-s11-e1、s12-p1
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary/E1t_recovery.csv`	s1011-s11-e1t
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary/E2_knockout.csv`	s1011-s11-e2
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary/E2n_knockout.csv`	s1011-s11-e2n
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary/E3_chain.csv`	s1011-s11-e3
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary/E4_probe.csv`	s1011-s11-e4b
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary/E4_titlepos_lens.csv`	s1011-s11-e4
`/scratch/zhang.zhiqi2/jlens/361_stage11/summary_r0/E1_recovery.csv`	s1011-s10-r0r1

Stage 12

文件 / 目录（绝对路径）	提及章节
`/scratch/zhang.zhiqi2/jlens/371_stage12/alens/progress.json`	h-compute-table、h-lens-table、s12-f1
`/scratch/zhang.zhiqi2/jlens/371_stage12/config/alens_eval_exclude.json`	s12-f1、s12-f2
`/scratch/zhang.zhiqi2/jlens/371_stage12/config/fit_prompts.parquet`	s12-f1
`/scratch/zhang.zhiqi2/jlens/371_stage12/config/protocol.json`	h-lens-table、s12-f1、s12-n2
`/scratch/zhang.zhiqi2/jlens/371_stage12/logs/analyze_11413617.log`	s12-p1

/scratch/zhang.zhiqi2/jlens/371_stage12/logs/cpu_11413574.log	s12-cpu
/scratch/zhang.zhiqi2/jlens/371_stage12/logs/work_11413557.log	s12-f1
/scratch/zhang.zhiqi2/jlens/371_stage12/logs/work_11413572.log	s12-f1
/scratch/zhang.zhiqi2/jlens/371_stage12/logs/work_11413584.log	s12-f1
/scratch/zhang.zhiqi2/jlens/371_stage12/logs/work_11413586.log	s12-f1
/scratch/zhang.zhiqi2/jlens/371_stage12/logs/work_11414108.log	s12-ov-gpu
/scratch/zhang.zhiqi2/jlens/371_stage12/logs/work_11428778.log	s12-ov-gpu
/scratch/zhang.zhiqi2/jlens/371_stage12/reports/stage12_cpu_report.md	s12-cpu
/scratch/zhang.zhiqi2/jlens/371_stage12/reports/stage12_report.md	h-leak-a、s12-f2、s12-n1、s12-n2、s12-p1、s12-p2、s13-m4
/scratch/zhang.zhiqi2/jlens/371_stage12/summary/F2_author_lens.csv	s12-f2、s13-m4
/scratch/zhang.zhiqi2/jlens/371_stage12/summary/N1_prior_mc.csv	s12-n1
/scratch/zhang.zhiqi2/jlens/371_stage12/summary/N2_prior_pure.csv	s12-n2
/scratch/zhang.zhiqi2/jlens/371_stage12/summary/P1_patch_pos.csv	s12-p1
/scratch/zhang.zhiqi2/jlens/371_stage12/summary/P2_patch_comp.csv	s12-p2
/scratch/zhang.zhiqi2/jlens/371_stage12/summary_cpu/A_prior_decomposition.csv	s1011-s10-prior、s12-cpu
/scratch/zhang.zhiqi2/jlens/371_stage12/summary_cpu/B_prior_vs_recognition.csv	s1011-s10-prior、s12-cpu
/scratch/zhang.zhiqi2/jlens/371_stage12/summary_cpu/C_surname_commonness.csv	s1011-s10-prior、s12-cpu
/scratch/zhang.zhiqi2/jlens/371_stage12/summary_cpu/X1_probe_controls.csv	s12-cpu

Stage 13

文件 / 目录（绝对路径）	提及章节
~/jlens_work/13_stage13/README.md	s13-overview
/scratch/zhang.zhiqi2/jlens/381_stage13/config/pairs.parquet	s13-overview
/scratch/zhang.zhiqi2/jlens/381_stage13/config/prereg.json	h-stats-mult、s13-m2c、s13-overview、s13-x3、s13-x5
/scratch/zhang.zhiqi2/jlens/381_stage13/config/prereg_addendum1.json	h-leak-a

/scratch/zhang.zhiqi2/jlens/381_stage13/config/prereg_addendum2.json	s13-m2c、s13-overview、s13-x1、s13-x3
/scratch/zhang.zhiqi2/jlens/381_stage13/config/prereg_addendum3.json	s13-x6、s13-x7
/scratch/zhang.zhiqi2/jlens/381_stage13/config/prereg_addendum4.json	s13-x3
/scratch/zhang.zhiqi2/jlens/381_stage13/followup/judge_sample.csv	s13-x3
/scratch/zhang.zhiqi2/jlens/381_stage13/followup/x4tok/shard_000.csv	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/logs/cpu_11438885.log	s13-m1
/scratch/zhang.zhiqi2/jlens/381_stage13/logs/fugpu_11485349.log	h-compute-parity、h-compute-table、s13-x3、s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/logs/m3_11472899.log	s13-m3
/scratch/zhang.zhiqi2/jlens/381_stage13/logs/para_review_sample20.csv	s13-x3
/scratch/zhang.zhiqi2/jlens/381_stage13/logs/pipetest_11459594.log	s13-overview
/scratch/zhang.zhiqi2/jlens/381_stage13/logs/userchecks_11474830.log	s13-x1
/scratch/zhang.zhiqi2/jlens/381_stage13/logs/x6_11461197.log	s13-x6
/scratch/zhang.zhiqi2/jlens/381_stage13/raw/para_gen/shard_000.npz	g-model-id
/scratch/zhang.zhiqi2/jlens/381_stage13/raw/task/shard_000.npz	s13-m3
/scratch/zhang.zhiqi2/jlens/381_stage13/raw/xf_patch/false_0/shard_000.npz	s13-x6
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/cluster_ci.md	h-leak-a、h-stats-team、s1011-s10-main、s13-overview
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/e4b_split_compare.md	h-leak-a、s1011-s11-e4b-check、s12-cpu
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/m3_report.md	h-prereg-dev、s13-m3
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/split_compare.md	h-leak-a、s13-m2
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/stage13_cpu_report_group.md	s13-m1、s13-m2
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x1_report.md	h-prereg-dev、s13-x1
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x1_similarity.md	s13-x1
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x4_report.md	h-prereg-dev、s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x4_textbase.md	s13-x4

/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x4_textbase2.md	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x4_textbase_posctrl.md	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x6_report.md	h-prereg-dev、s1011-s11-e1、s13-x6
/scratch/zhang.zhiqi2/jlens/381_stage13/reports/x7_report.md	h-compute-parity、h-prereg-dev、s1011-s11-e2、s13-x7
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu/M1_title_sensitivity.csv	s13-m1
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu/M1_top1_classes.csv	s13-m1
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu/M2b_words_vs_probe.csv	s13-m2b
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu_group/E4b_4configs.csv	s1011-s11-e4b-check
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu_group/M2_probes.csv	s13-m2、s13-m2c
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu_group/M2b_words_vs_probe.csv	s13-m2b
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu_group/M2c_bert_baseline.csv	s13-m2c
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu_group/X1controls_4configs.csv	s1011-s11-e4b-check
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_cpu_group/perpaper_probe_spec.parquet	h-leak-a、s1011-s11-e4b-check
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_m3/m3_transfer.csv	s13-m3
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_m3/x1_similarity.csv	s13-x1
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x1x4/x1_recovery.csv	s13-x1
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x1x4/x4_decodability.csv	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x1x4/x4_earliest.csv	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x1x4/x4_recognition.csv	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x1x4/x4_textbase2_perpaper.csv	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x1x4/x4_textbase2_posctrl.csv	s13-x4
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x3/para_nli.csv	s13-x3
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x6/x6_recovery.csv	s13-x6
/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x6/x6_verdict.json	s13-x6

`/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x7/x7_knockout.csv`	s13-x7
`/scratch/zhang.zhiqi2/jlens/381_stage13/summary_x7/x7_verdict.json`	s13-x7

出处里以通配符、省略号或不带目录的文件名写出、没能解析到唯一路径的条目（按原文写法列出；目录请看提及章节的上下文）：

原文写法	提及章节
`/README_first.md`	g-model-load
`/Salesforce-wikitext/config.yaml`	h-lens-table
`/config.yaml`	h-lens-jac
`/generation_config.json`	g-model-id
`/model.safetensors.index.json`	g-model-id
`/shard_*.npz`	s13-m2
`/step1_README.md`	g-model-chat
`/tokenizer_config.json`	g-model-id
`023.npz`	s13-x3
`351_stage10/logs/parity_*.log`	h-compute-table
`351_stage10/logs/pool/job_s10p_*.log`	h-compute-table
`351_stage8_README.md`	s19-s8-protocol
`E1_target_minus_control.csv`	s1011-s11-e1
`E2_knockout.csv`	s1011-s10-r0r1
`E2_target_minus_control.csv`	s1011-s11-e2
`M2_probes.csv`	s13-m2
`R13/config/prereg*.json`	s13-overview
`R13/logs/x1x4_*.log`	s13-x1
`R13/raw/*/shard_*.npz`	s13-overview
`README.md`	g-data-fields、s1011-s10-fine、s19-common
`analysis_plan.json`	g-scoring-agg、g-scoring-rank
`config.json`	g-model-id、s12-ov-common
`config.yaml`	g-data-fields、g-labels-match
`final_status.json`	h-lens-stage9、h-lens-table
`full_A_repair_101_100valid/repair_manifest.json`	h-lens-stage9
`knockout/shard_000.npz`	s1011-s11-qc
`m3_identity.csv`	s13-m3

`parity_before_tol05_amendment.json`	h-compute-parity
`prereg*.json`	h-prereg-files
`protocol.json`	g-model-jlens、g-prompts-align、g-prompts-list、g-scoring-agg、g-scoring-rank、h-lens-defs
`quality.json`	s19-s72
`raw/*/shard_*.npz`	s1011-stage11
`raw/patch/*.npz`	s1011-s11-e1
`raw/patch_comp/*.npz`	s12-p2
`raw/patch_pos/*.npz`	s12-p1
`repair_manifest.json`	h-lens-table、s19-s9-fitA
`smoke/raw/*/shard_000.npz`	s1011-s11-qc
`supplement_manifest.json`	s19-s8dsupp
`token_layer_scores.csv`	s19-s6-jmrr

G5.3 Slurm 作业号（按 Stage）

注意：Stage 10–13 的 GPU 作业是"池化"的（任何作业可接手其他 Stage 的单元），作业名（如 `s12w`、`s10p_*`）不等于它实际算了哪个 Stage，见 h-compute-table。Stage 1–9 都是交互式 `bash` 会话，没有可对应的作业号（s19-common）。状态 FAILED 包括崩溃、按规则拒绝运行和 gate/keeper 作业到时退出，具体原因见提及章节。

Stage 10

作业号	sacct 作业名	分区	开始	用时	sacct 状态	提及章节
11333739	`s10_freezecond`	short	2026-10-23 16:59	00:06:39	COMPLETED	h-false、s1011-s10-titlespec
11334054	`s10_parity`	gpu	2026-10-24 12:49	00:04:45	FAILED	h-compute-table、s1011-s10-common、s1011-s10-parity
11338000	`s10x_parity`	gpu-short	2026-10-24 15:14	00:04:07	COMPLETED	h-compute-table、h-compute-tol、s1011-s10-fine
11338002	`s10_pool`	short	2026-10-23 21:01	1-03:28:56	FAILED	s1011-s10-main
11338033	`s10_pool`	short	2026-10-23 21:01	1-03:24:05	FAILED	s1011-s10-main
11338068	`s10_pool`	short	2026-10-23 21:02	1-03:23:47	FAILED	s1011-s10-main

11338070	s10p_ext_true	gpu	2026-10-25 00:23	00:00:15	FAILED	h-compute-table
11338105	s10p_ext_false_3	gpu-short	2026-10-25 00:28	00:00:17	FAILED	h-compute-table
11347140	s10p_ext_false_1	gpu	2026-10-25 00:28	00:00:08	FAILED	h-compute-table
11347585	s10_parity	gpu	2026-10-24 23:14	00:33:59	FAILED	h-compute-table、h-compute-tol、s1011-s10-parity
11350173	s10p_ext_false_3	gpu-short	2026-10-25 00:31	00:00:13	FAILED	h-compute-table
11350174	s10_parity_i	gpu-inter+	2026-10-25 00:32	00:05:22	FAILED	h-compute-table、h-compute-tol、s1011-s10-parity
11358551	s10p_ext_false_3	gpu	2026-10-25 07:10	00:00:08	FAILED	h-compute-table
11363838	s10_decision	short	2026-10-25 10:55	00:06:01	COMPLETED	s1011-s10-parity
11363842	s10_pool	short	2026-10-25 10:55	13:10:47	FAILED	s1011-s10-main
11363844	s10p_main_true	gpu	2026-10-25 22:29	01:35:10	FAILED	h-compute-table
11363850	s10p_main_delete	gpu	2026-10-27 06:37	00:53:36	COMPLETED	h-compute-table
11363851	s10p_ext_true	gpu	2026-10-27 06:51	00:39:49	COMPLETED	h-compute-table
11363852	s10p_ext_false_0	gpu-short	2026-10-25 11:06	00:58:28	COMPLETED	h-compute-table
11363863	s10p_ext_delete	gpu-inter+	2026-10-27 07:35	00:00:09	COMPLETED	h-compute-table
11373362	s10_analyze_main	short	2026-10-25 22:55	00:03:02	COMPLETED	h-compute-parity、s1011-s10-main
11374266	s10_pool	short	2026-10-26 00:14	1-14:53:20	FAILED	s1011-s10-main
11375853	s10_analyze_title	short	2026-10-26 02:58	00:03:58	COMPLETED	s1011-s10-titlespec
11376473	s10_extra	short	2026-10-26 04:20	00:12:20	COMPLETED	s1011-s10-cite、s1011-s10-format
11376890	s10_extra_fmt	short	2026-10-26 04:46	00:01:26	COMPLETED	s1011-s10-format
11378377	s10x_analyze	short	2026-10-26 06:23	00:16:15	COMPLETED	s1011-s10-fine
11413574	s12_cpu	short	2026-10-27 14:50	00:05:25	COMPLETED	s1011-s10-prior、s12-cpu、s12-ov-scope

11429580	s10_r0r1	short	2026-10-27 23:33	00:15:40	COMPLETED	s1011-s10-r0r1
11443686	s13_clci	short	2026-10-28 14:17	00:02:56	COMPLETED	h-leak-a、h-prereg-files、s1011-s10-main
11443696	s13_clci	short	2026-10-28 14:39	00:02:58	COMPLETED	h-leak-a、s1011-s10-main

Stage 11

作业号	sacct 作业名	分区	开始	用时	sacct 状态	提及章节
11363847	s10p_main_false_2	gpu	2026-10-27 05:15	01:21:51	COMPLETED	h-compute-table、s1011-s11-qc
11385983	s11_analyze	short	2026-10-26 13:24	18:27:37	COMPLETED	s1011-s11-qc
11396804	s11_realcheck	short	2026-10-27 04:56	00:26:47	COMPLETED	s1011-s11-e1t、s1011-s11-qc
11443587	s13_e4bg	short	2026-10-28 12:45	00:31:34	COMPLETED	h-leak-a、h-prereg-files、s1011-s11-e4b-check、s12-cpu

Stage 12

作业号	sacct 作业名	分区	开始	用时	sacct 状态	提及章节
11371622	s10p_main_true	gpu-short	2026-10-27 14:58	00:06:33	FAILED	h-compute-table、s12-p1
11413494	s12_keeper	short	2026-10-27 14:46	1-05:05:51	COMPLETED	s12-ov-scope
11413557	s12fit	gpu	2026-10-27 15:28	01:15:38	COMPLETED	h-compute-table、s12-f1、s12-p1
11413569	s12fit	gpu-short	2026-10-27 15:40	01:19:09	COMPLETED	h-compute-table、s12-p1
11413572	s12w	gpu-short	2026-10-27 21:17	01:19:57	COMPLETED	h-compute-table、s12-f1
11413584	s12w	gpu-short	2026-10-27 22:43	01:20:33	COMPLETED	h-compute-table、s12-f1
11413586	s12w	gpu-inter+	2026-10-28 16:47	00:51:45	COMPLETED	h-compute-table、s12-f1
11413591	s12w	gpu-inter+	2026-10-28 18:16	00:51:37	COMPLETED	h-compute-table
11413617	s12_analyze	short	2026-10-27 14:52	1-05:03:15	COMPLETED	s12-ov-scope、s12-p1

11414108	s12w	gpu-short	2026-10-28 20:15	00:02:37	FAILED	h-compute-hw、s12-ov-gpu、s13-x6
11428778	s12w	gpu-short	2026-10-29 00:10	00:50:39	COMPLETED	h-compute-hw、h-compute-table、s12-ov-gpu、s13-m3、s13-x1、s13-x6

Stage 13

作业号	sacct 作业名	分区	开始	用时	sacct 状态	提及章节
11374283	s10p_ext_false_0	gpu	2026-10-28 19:22	00:24:38	FAILED	h-compute-table、s13-x6
11374284	s10p_ext_false_1	gpu-short	2026-10-29 03:51	00:39:07	COMPLETED	h-compute-table、s13-x3、s13-x7
11385971	s11w	gpu	2026-10-28 20:18	00:01:47	FAILED	s13-x6
11405680	s11w	gpu	2026-10-28 20:18	00:01:59	FAILED	s13-x6
11413886	s12w	gpu	2026-10-28 19:09	00:36:58	FAILED	h-compute-table、s13-x6
11414120	s12w	gpu	2026-10-28 20:21	00:02:19	FAILED	s13-x6
11415844	s12w	gpu	2026-10-28 20:48	00:51:09	COMPLETED	h-compute-table、s13-x6
11416869	s12fit	gpu-short	2026-10-28 22:54	00:53:35	COMPLETED	h-compute-table、s13-x6
11430308	s12w	gpu-short	2026-10-28 23:40	00:09:18	FAILED	s13-x6
11438885	s13_cpu	short	2026-10-28 06:58	01:01:37	COMPLETED	s13-m1、s13-m2
11439283	s13_m2v2	short	2026-10-28 08:00	01:14:20	COMPLETED	h-leak-a、s13-m2
11441993	s13_m2b	short	2026-10-28 10:19	00:06:47	COMPLETED	h-prereg-status、s13-m2b
11443232	s13_m2c	short	2026-10-28 12:26	00:05:54	COMPLETED	h-prereg-files、s13-m2c
11443403	s13_group	short	2026-10-28 12:43	01:13:35	COMPLETED	h-leak-a、h-prereg-files、h-prereg-status、s13-m1、s13-m2、s13-m2b、s13-m2c
11443607	s13_cmp	short	2026-10-28 13:57	00:00:04	COMPLETED	h-leak-a
11453127	s12w	gpu-inter+	2026-10-29 02:31	01:16:57	COMPLETED	h-compute-table、s13-x1、s13-x3、s13-x4
11454922	s12w	gpu-inter+	2026-10-29 06:54	00:06:50	COMPLETED	s13-x7

11459594	s13_pipetest	short	2026-10-28 23:20	00:01:44	COMPLETED	s13-overview
11461197	s13_x6	short	2026-10-29 00:29	00:02:09	COMPLETED	s13-x6
11470471	s13_x7	short	2026-10-29 07:29	00:00:06	FAILED	s13-x7
11470524	s13_x7	short	2026-10-29 07:30	00:00:04	COMPLETED	s13-x7
11472859	s13_x1x4	short	2026-10-29 09:04	00:00:40	COMPLETED	s13-x1、s13-x4
11472899	s13_m3	short	2026-10-29 09:06	00:37:21	COMPLETED	s13-m3
11472901	s13_x1x4	short	2026-10-29 09:06	00:00:09	COMPLETED	s13-x1、s13-x4
11474830	s13_checks	short	2026-10-29 10:26	00:11:04	COMPLETED	s13-m3、s13-x1、s13-x4
11475014	s13_posctrl	short	2026-10-29 10:38	00:18:39	COMPLETED	s13-x4
11475175	s13_nli	short	2026-10-29 10:46	00:05:25	COMPLETED	s13-x3
11475517	s13_text2	short	2026-10-29 10:59	00:33:42	COMPLETED	s13-x4
11485182	s13_futiny	short	2026-10-29 20:26	00:09:22	FAILED	s13-x4
11485266	s13_futiny	short	2026-10-29 20:32	00:02:39	COMPLETED	s13-x4
11485349	s13_fugpu	gpu-short	2026-10-30 03:15	00:10:28	FAILED	h-compute-table、s13-x3、s13-x4
11499990	s13_futiny	short	2026-10-30 11:02	00:01:33	COMPLETED	s13-x4
11500050	s13_futiny	short	2026-10-30 11:06	00:07:44	COMPLETED	s13-x4

G5.4 git commit

commit	仓库	提交时间（EDT）	提交说明（git log 原文）	提及章节
581d398	~/projects/jacobian-lens	2026-07-01 23:37	Initial release	g-model-jlens、h-lens-jac

3a4f605	/scratch/zhang.zhiqi2/jlens_github_publish	2026-10-21 21:36	Review fixes: Stage 8 frozen-endpoint supplement and repo cleanup	s19-s8dsupp
e2c7156	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-22 00:01	Add Stage 1–8 experiment code, data, figures and archive records	g-data-cohort、g-data-origin、g-data-package、s19-common、s19-s1-cohort、s19-s1-pilot50、s19-s2-protocol、s19-s3-exp24、s19-s4-readout、s19-s5-crossed、s19-s6-jmrr、s19-s71、s19-s72、s19-s73、s19-s74、s19-s8-protocol、s19-s8a、s19-s8b、s19-s8c、s19-s8d
b4b76ed	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-26 21:35	Add Stage 10 (full-cohort layer-wise readout) and Stage 11 code/config	g-model-jlens、h-false、h-lens-jac、s1011-stage10、s1011-stage11
c86112f	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-27 03:10	Stage 11: queue keeper, safer unit claims, parity/agreement misses recorded as WARN instead of stopping GPU work	s1011-stage11
13ae7ec	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-27 15:00	Stage 12 code; name-prior analysis and probe controls; report and talk updated	s1011-stage11、s12-ov-gpu、s12-ov-scope
95dd94d	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-27 15:08	Stage 12: fix patch_pos for 8/9-token tails; a crashing unit type is disabled instead of holding the queue	s12-ov-gpu、s12-ov-scope、s12-p1
37b8b7b	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-27 18:50	Report and talk: data provenance, label and lens-fitting details, Stage 11 design details, read-early/spoken-late wording; Stage 12 fit takeover	s12-ov-scope
d569682	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 06:51	Stage 13 plan and code (middle-layer content: task vs paper identity); Stage 12 GPU-type guard and Stage 13 hand-over	s12-ov-gpu、s12-ov-scope
43c8f0c	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 07:07	Stage 13: provenance audit fixes (equal-prefix no-year templates, year-leak controls in M2), X1/X3/X4 GPU units, partner pairs	g-prompts-list、h-leak-a
355e64d	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 10:40	Stage 13: pre-registration, X1 random control and tune/test split, BERT baseline (M2c), M2b	h-prereg-files

cf7106d	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 10:52	Stage 13: group (author-team / donor) folds for all probes, team-grouped tune/test split, prereg addendum 1	h-prereg-files、h-stats-team
663ac1c	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 11:00	Stage 12 F2 eval excludes 3 team-overlap papers; E4b/controls and M2 per-paper vs team-split comparison scripts	h-leak-a、s12-f2、s12-ov-scope
0dde4de	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 11:06	Stage 13 review: E4b title-reuse exclusion (4 configs), team-cluster bootstrap CIs, prereg addendum 2, probe statements marked under review	h-prereg-files、s1011-s11-e4b-check
6d667ab	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 11:42	Stage 13: X6 (Stage 11 patching with false_1-4, runs first) and X7 (knockout on false runs, last); prereg addendum 3; 6.11 wording under review	h-prereg-files、s1011-s11-e1
54d8ac2	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 11:56	Stage 13: X3 lexical-neighbour control, prereg addendum 4 (token roles, wording); report limitations	h-prereg-files、s13-overview
fb8be6f	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-28 20:31	Stage 12 done: report, status; stage13 log-line fix	g-model-chat、g-prompts-list、s12-ov-scope、s13-overview
8d1a5c0	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-29 20:35	Stage 13 GPU follow-up script (Qwen judge, X4 per-token)	g-model-chat、s13-overview
44d084f	/scratch/zhang.zhiqi2/jlens-authorship-recall_update	2026-10-30 00:42	Report v19 and merged PPT v12: title-to-name wording; x4tok check fatal	g-model-chat、s13-overview

另：Stage 7.1B 与 Stage 9 的代码和结果不在 git 里（s19-common）；`3a4f605` 在分支 review/stage8-fixes，已于 2026-10-30 合入 main（merge commit `a4c1f0a`，s19-s8dsupp）。

附录 F · 自查：随机抽取 15 个数字，回到原始结果文件核对

抽样方法：从 6 个正文部分各自列出的"关键数字及出处"（共 31 条）中，用 `random.seed(20261030)` 随机抽 10 条；再从正文中所有"数字 + 出处"的段落（164 处）中用 `random.seed(7)` 随机抽 5 条。每条都由总编重新打开原始文件读数，不使用起草者给的数值。核对时间 2026-10-30。

#	手册中的说法	原始文件（读取方式）	文件中的实际值	结论
1	Stage 10 输出处（final L63）标题特异 target − control = +0.207 [0.195, 0.220]	`351_stage10/summary_title/B_spec_target_vs_control_within_bin.csv`，readout=final, layer=63	0.20743 [0.19535, 0.21976]，n_a 3,734，n_b 2,182	一致
2	M2 false_0 L48 借出论文的年份 R^2 0.623 → 0.535，log 引用 0.588 → 0.488（按论文切 → 按团队切）	`381_stage13/reports/split_compare.md` 第 127–128 行	year 0.623 / 0.535（−0.087）；log citations 0.588 / 0.488（−0.100）	一致
3	X7 control S(L52) = +0.051 [0.033, 0.068]；不阻断元素与 Stage 11 假标题运行相同的比例 0.5963	`381_stage13/summary_x7/x7_knockout.csv`（group=control, start_layer=52）；`summary_x7/x7_verdict.json`	0.0507 [0.0326, 0.0683]；warn 中为 0.5963	一致
4	Stage 8 RQ1 L56 +5.18 pp [2.93, 7.48]，N = 200	`351_stage8/stage8D/summary/primary_questions.csv`，RQ1_L56_replication 行	mean 0.051785 [0.02926, 0.07476]	一致
5	冻结主终点（7 层积分）+2.33 pp [1.50, 3.19]	`/scratch/zhang.zhiqi2/jlens_github_publish/experiment_files/351_stage8/stage8D_supplement/frozen_plan_endpoints.csv`，jlens_integrated7_RQ1_paper_mean	0.023257 [0.014964, 0.031921]，n 200，122 正 / 78 负	一致
6	组长标注包 zip 的 sha256 = 58c1b5bc…ee619	`sha256sum ~/jlens_work/00_cohort/zhiqi_qwen3-32b_labels_2026-10-15.zip`（重新计算）	58c1b5bc2cc61e6e5f5d276960c9340c770a9cf3bfa614d94873251dee9ee619	一致
7	N2 纯名字先验 target − control +0.047 [0.031, 0.062]，真标题 +0.247	`371_stage12/summary/N2_prior_pure.csv`，tc_pure / tc_true 行	0.047086 [0.031363, 0.061640]；0.247295 [0.229445, 0.265803]	一致
8	E2 从 L0 阻断损失 0.831 [0.801, 0.862]	`361_stage11/summary/E2_knockout.csv`，group=all, start_layer=0	0.8309 [0.8006, 0.8620]	一致
9	E4b 复核版保留 1,863 篇（target 896、control 499）	`381_stage13/summary_cpu_group/perpaper_probe_spec.parquet`，split=group, excl=title-reuse, layer=24, dropped=False；分组取自 `351_stage10/cohort/papers.parquet` group_tight	1,863 篇：target 896、control 499、excluded 433、guard 35。"probe (as in Stage 11)" 那一版每篇有 2 或 4 行，按行数是 3,258，按论文数也是 1,863	一致（注意按论文计数，不按行计数）

10	作者团队 7,247 个，5 折大小 1,821–1,822	`381_stage13/reports/cluster_ci.md` 第 3 行	"Teams: 7247 groups over 9108 papers"	一致（折大小见 h-stats 出处）
11	真实输出 L63 的论文均值：target −0.708 [−0.717, −0.700]，control −1.004 [−1.018, −0.989]，guard −0.329，excluded −0.887	`351_stage10/summary/A_group_layer.csv`，readout=final, layer=63，列 mean_nlr_mean/lo/hi	target −0.70817 [−0.71668, −0.70020]；control −1.00355 [−1.01804, −0.98889]；guard −0.32927；excluded −0.88705	一致
12	parity：final@L63 最大 |logit 差| 0.367；容差由 0.25 放宽到 0.5（事后修订）	`351_stage10/reports/parity.json`	"final@L63": 0.3671875；文件中有 tolerance_amendment 字段	一致
13	X6 target 各假标题 L48 恢复率 0.788–0.803，L52 0.984–0.990	`381_stage13/summary_x6/x6_recovery.csv`，group=target，false_0…false_4	L48 0.788–0.803；L52 0.984–0.990	一致
14	假标题"严格档"：同领域、年份差 ⩽5、作者数差 ⩽ max(1, 0)=1、标题 token 数差 ⩽ max(1, 20%)、引用档差 ⩽1	`~/jlens_work/10_stage10/351_stage10.py` 第 245–247 行 TIERS	`('strict', 5, 0.0, .20, 1, K_FALSE)`，注释说明各项含义	一致（同领域约束在代码别处，见 h-false）
15	团队整群区间：380 行中 4 行"区间是否含 0"发生翻转	`381_stage13/reports/cluster_ci.md`	"Rows where "interval excludes 0" changes: 4 of 380"	一致

结论：15 / 15 一致。第 9 条需要注意计数口径：1,863 是论文数，不是 parquet 的行数。

另外，起草过程中发现并在正文中如实标注、但不属于这 15 条抽样的问题，包括：X4 文本基线 textbase2 的 OOV 计算错误（见 s13 X4 节，修正后的数字已在节内给出）；STATUS.md §2 的假标题代价公式漏写两项相对分母（见 h-false，以代码为准）；已发布报告里"去掉 ⩾20 位作者的论文"与代码"保留 1–20 位"不一致（见 g-data）；若干 Stage 7–8 区间在结果文件中找不到出处（见 s19 各节）。

附录 I · "未知/需确认"自查结果（U08–U44 中可自查的条目）

2026-10-30 在集群上只读复查（sacct / scontrol / 日志 / 结果文件 / git 全部分支 / 本会话 Claude 转录 `~/.claude/projects/-home-zhang-zhiqi2-jlens-work/32eac6b6-….jsonl`）。没有提交作业，没有改动任何结果或代码。复算脚本和重建文件都只放在本次子任务的 scratchpad：`/tmp/claude-103706/-home-zhang-zhiqi2-jlens-work/32eac6b6-5b2f-4d30-b044-a14d510db11a/scratchpad/`（下文简称 `$SP`）。sacct 的时间和文件 mtime 都是集群本地时间（EDT）。日志里的 `timestamp_utc` 已减 4 小时换成 EDT。U01–U10 中只涉及 Dreaming Up Authors 包的部分由另一位起草者负责；U08、U09 这里只写用标注包和 Stage 10 数据查到的事实。

结论标签：**已解决** · **部分解决** · **仍需确认**。

编号	问题	查了什么（命令/路径）	结果（事实 + 出处）	结论
U08	papers_9108.csv 与 eval_sample_9108 的关系；提示用哪版标题（17 篇）	逐篇对比 `labels_20261015/…/paper_labels_with_papers.csv`（title）、`jlens-authorship-recall_update/home_project/data/release/eval_sample_9108.csv.gz`（title）、`351_stage10/cohort/papers.parquet`（title）。在 open_results / mc_results / mc_team 中找提示原文。	17 篇里，每篇都满足"标注包标题 = `' '.join(eval_sample 标题.split())`"，即把所有空白（U+00A0 ×6、U+2009 ×3、U+2005 ×1、换行 U+000A ×7）折成单个空格并去掉首尾。标注包标题里没有任何非 ASCII 空白。Stage 10 的 papers.parquet 标题与标注包 9,108/9,108 相同，所以 Stage 10–13 用的是规范化后的标题。标注包的 jsonl 只有分数，不含提示原文，无法看出组长的提示用了哪一版。另：`home_project/src/dua/probing/prompts.py:38-52`（原论文代码，`tests/test_probing_prompts.py:3` 写明 "papers_9108.csv is not shipped; it is rebuilt from the released evaluation sample"）直接拿 eval_sample 的原始 title 套模板，不做空白规范化。	**部分解决**：规范化规则已确定（split/join）。组长 Step 1 的提示里是哪一版标题，仍需向**组长**确认。包内代码部分由负责 U01–U10 的起草者核对。
U09	13 个引用档的边界	`351_stage10/cohort/papers.parquet`：citations（字符串，转成整数）× citation_bin，按档取 min/max。再用规则 bin(c) 逐篇比对。另看 `labels…/bin_table.csv` 的 bin 列。	各档实际 [min, max]：0-2 [0,2]；3-4 [3,4]；5-8 [5,8]；9-16 [9,16]；17-32 [17,32]；33-64 [33,64]；65-128 [65,128]；129-256 [129,256]；257-512 [257,512]；513-1024 [513,1023]；1025-2048 [1027,2045]；2049-4096 [2049,4088]；4097+ [4136,63901]。规则"c≤2 → 0-2；24096 → 4097+"与 9,108/9,108 篇一致（脚本在本表下方说明）。档名本身就写出边界（bin_table.csv 第 1 列）。	**已解决**：除 0-2 和 4097+ 两个端档外，都是 2 的幂次档。为什么这样分档，属于组长的设计说明（包内文档由另一位起草者核对）。

U12	Stage 9 FullFit 实际 GPU 型号	runtime/fit_status json 里没有 GPU 字段（`351_stage9/runner_runs/*/`）。改查 `351_stage9_full_A.log`、`_full_B.log`、`_full_A_recovery.log`、`_A_repair.log` 的 OOM 报错和时间戳，再对照 `sacct -u zhang.zhiqi2 -S 2026-10-15 -X -o JobID,…,Start,End,NodeList`，并用日志 `FULLFIT PLAN budget_seconds`（≈ 作业剩余时间）区分同时运行的两个作业。`scontrol show node d1026/d1028/d1029`。	全部在 A100 上。① Fit A 0→4（10-18 16:34–16:55）：作业 11191976 @ d1026，**A100-40GB**。同一作业随后 OOM，报错 "total capacity of 39.49 GiB"（full_A.log:127、223）。② 其余所有段都在 d1028/d1029（`Gres=gpu:a100:4`，feature `a100@80g`），报错 "total capacity of 79.25 GiB"（full_A.log:453、481；full_B.log:57、85）。按 budget 认出的作业：Fit A 4→20 → 11195756@d1029；23→42 → 11211146@d1028；42→59 → 11239485@d1028；47→56 → 11242682@d1028；56→100 → 11248534@d1028；恢复 80→100 → 11259149@d1028；修复 → 11291707、11295242/6、11296088/9@d1028。Fit B 0→16 → 11254812/4@d1028；16→34 → 11257769/71@d1028；34→51 → 11259150@d1029；51→66 → 11289356@d1028。在 80GB 卡上的 OOM 报错写着 "Process … has 30.00 GiB memory in use"，即有另一进程同卡。	**已解决**：Fit A 第一段用 A100-40GB，其余用 A100-80GB（SXM4）。
U14	public lens 的 557 条拟合 prompt、skip_first、revision	读 `public_neuronpedia/qwen3-32b/jlens/Salesforce-wikitext/config.yaml`、`…/Qwen3-32B_convergence.csv`、provenance 审计 `351_provenance_audit/351_provenance_audit_results.tar.gz`（summary/audit_summary.json 第 116–134 行，即 lens 文件内的 provenance 字段）。`~/projects/neuronpedia-audit` 的 HEAD = 22f0412，正是 provenance 里的 jlens_commit：读 `utils/neuronpedia-utils/neuronpedia_utils/jlens/fit_lens.py:51-106、465-470`，以及 `~/projects/jacobian-lens/jlens/fitting.py:42、67`。用本地 wikitext parquet 按 load_prompts 的算法重建 557 条。	**skip_first = 16**、target_layer = 63、n_prompts = 557、dim_batch = 64、t_max = 128、bfloat16、jlens_commit 22f0412f…，这些写在 lens 文件的 provenance 字段里（audit_summary.json:116-134）。convergence.csv 的 seq_len 128 → n_valid 111 与 skip_first 16 一致（mask = [16, seq_len−1)，fitting.py:67）。**Prompt 选法**是确定性的：流式读取 Salesforce/wikitext、wikitext-103-raw-v1、train，跳过空行和以 "=" 开头的标题行，把文本拼接后切成 2,000 字符一段，取前 N 段（fit_lens.py:84-106）。前 556 条是用 --n_prompts 1000 跑的，之后改为 557 收尾（config.yaml stop_reason），所以前 557 段不变。用本地 parquet（dataset revision b08601e0，`fitting_corpus/manifest.json`）重建出 557 段、1,113,787 字符，JSON sha256 b5094778…（`$SP/public557_prompts_reconstructed.json`）。9,108 个队列标题（≥25 字符）无一逐字出现在这 557 段中。**模型 revision**：fit_lens.py:465-470 调 `from_pretrained` 时不带 revision，取的是拟合当时（2026-09-20/21）的 main；provenance 里没有记录。本地 HF 缓存的 refs/main = 9216db57（`/scratch/zhang.zhiqi2/hf/hub/models--Qwen--Qwen3-32B/refs/main`，10-15 下载）。数据集 revision 同样没有固定（Neuronpedia 的 tree 缓存 e166deb5 是 lens 仓库的 revision，不是数据集的）。	**部分解决**：skip_first 和 prompt 选法已查实，557 条可重建，前提是 wikitext 在 HF 上没有改版。拟合时的模型/数据集 revision 没有记录，只能推定为当时的 main，需向 **Neuronpedia（lens 发布者）**确认。按逐字匹配，未见标题泄漏。

U15	self100 拟合脚本与 1,000 条 WikiText 语料生成脚本	`sha256sum` 1,709 个 .py/.bak 文件（/scratch/zhang.zhiqi2、~）。`git rev-list --all --objects` 遍历 4 个仓库的全部 .py blob（1,932 个）。另 ls `~/projects/dreaming-up-authors-code-v0`。	找到了：`/scratch/zhang.zhiqi2/jlens-authorship-recall_update/home_project/05_prepare_jlens_corpus.py`（sha a975c2a2…）和 `…/home_project/05_fit_jlens_prelim_100.py`（sha 4dcfb96a…），git commit e2c7156。归档 `/scratch/zhang.zhiqi2/jlens_archive_stage1_8/home_project/` 有同 sha 的副本（mtime 10-15 07:15 / 07:32）。`~/projects/dreaming-up-authors-code-v0/__pycache__/05_*.pyc` 的 mtime 与之相同（07:15:16 / 07:32:08）。对应关系：语料脚本固定 Salesforce/wikitext wikitext-103-raw-v1 train，seed 42，128 token，N=1000，蓄水池抽样；产出的 `fitting_corpus/manifest.json` 记 candidate_chunks_seen 970,664，corpus_sha256 1af8dbff…，与 `sha256sum wikitext103_qwen32b_1000.jsonl` 一致。拟合脚本取前 100 条，层 [24,32,40,48,56,62]，target 63，dim_batch 8，max_seq_len 128，skip_first 16，revision 取自 manifest（9216db57），它生成的 config dict 与 `qwen32b_jlens_prelim_100_config.json` 逐字段相同（脚本在 config 不一致时会 assert 失败）。拟合日志 `qwen32b_jlens_prelim_100.log` 显示 4 段 A100-SXM4-80GB（第 5、35、88、117 行）+ 最后一段 H200（第 163 行；10-18 04:43 从 98/100 续跑，作业 11180979@d4054）。	**已解决**（脚本在 git 的 home_project/ 和 Stage 1–8 归档里，不在 ~/jlens_work）。
U16	Stage 9 Fit A 的 --min-chars / --max-chars	统计 `351_stage9/prompts/{fit_A,fit_B,holdout}.jsonl` 的字符长度。按 `351_stage9.py:40-58`（canonical → 长度过滤 → sha256 去重 → 排序后用 SEED=3519001 打乱 → 取前 300）在本地 wikitext parquet 上重做，试 4 组参数（`$SP/s9split.py`）。	实际长度：fit_A 100–1196、fit_B 101–1175、holdout 118–1177。用 **100 / 1200**（脚本默认值，351_stage9.py:223-224）重做，得到 665,604 条去重行，fit_A、fit_B、holdout 三份 text_sha256 列表**逐条完全一致**。100/1196、64/2000、100/2000 都不一致。	**已解决**：min_chars = 100，max_chars = 1200。
U19	节点 d1002、d3204、d3233、d4054 的 GPU 型号	`scontrol show node d1002 d3204 d3233 d4054`；`sinfo -N -n … -o "%N %G %f"`	d1002：`gpu:v100-sxm2:4`（V100-SXM2）。d3204：`gpu:a100:8`（feature cascadelake,prod,ib）。d3233：`gpu:a100:3`（cascadelake,prod）。d4054：`gpu:h200:8`。d3204/d3233 没有 `a100@80g` 标记（d1028/d1029 有），显存大小未写明。	**已解决**（d3204/d3233 的 A100 显存规格没有写在节点信息里）。

U20	10-15 至 10-22 交互式 GPU 作业与实验的对应	sacct（`-X -o JobID,JobName,AllocTRES,Start,End,State,NodeList`），筛出 JobName=bash 且有 gres/gpu 的作业，共 69 个。对每个作业，列出 mtime 落在 [Start, End] 内的 `/scratch/zhang.zhiqi2/jlens` 文件（maxdepth 3）。另用拟合日志的时间戳和 budget_seconds（见 U12），以及 self100 日志里的 "GPU:" 行。	按时间段对应如下（mtime 只能证明文件在该时段内写过，不排除同时段的另一个作业）： • **10-15 03:29–09:17**（11095644、11095684、11095739、11095800@d1002、11096365、11098482、11099861、11099865、11099964@d3204、11100121/2@d3233）：pilot1_qwen3_32b（04:49），qwen32b_fit_test / multilayer 测试（05:11–05:26），recognition 1000（05:58–06:39），title_hidden 772（06:45–06:52），matched 队列（06:54–07:02），fitting_corpus（07:15–07:19），jlens smoke 2（07:22–07:28，A100-80GB），quick demo / authorship pilot / 29 对（08:06–08:14）。11095800、11099964 期间没有写出文件。 • **10-17 21:50 – 10-18 04:58**（11176076、11178329、11178936、11179232、11180877、11180979@d4054、11182526）：self100 拟合（续跑点 15/53/67/98，最后一段在 H200 = 11180979）和 351_all（22:43–05:26）。 • **10-18 05:22–07:20**（11183091/7@d1029）：self100 validation / generic / paper、gold_false、stage5_crossed_v2（06:06–06:27）、provenance 审计（06:53）。**07:30–09:30**（11184297/304）：split_A、Stage 6 运行日志。**09:39–13:03**（11184317、11186573/4）：Stage 6 收尾（10:49）、7.1（11:06）、7.2（11:14–11:37）、7.3 开始。**13:03–15:12**（11189077/9）：7.3、split_B、7.4（14:17）、Stage 8 冻结（14:29）、Stage 9 prepare（14:44）、拟合 smoke。**15:22–17:22**（11191105）：Stage 9 resume / benchmark 测试。**16:31–17:22**（11191976@d1026）：Fit A 0→4。**17:27–19:09**（11193135、11193951、11194220、11195032）：没有输出。**19:36–22:08**（11195756、11199178、11201799）：Fit A 4→20，Stage 8B（21:25），8C smoke（21:48）。 • **10-19**：03:14–05:23 Fit A 23→42（11211146，另一个作业 11211180 没有输出）；18:13–22:06 Fit A 42→56（11236380/4、11239398/485/529、11242629/682）和 8C（stage8C.log 22:06，11242682）；22:14–00:18 Fit A 56→100（11248534，另 11248379）。 • **10-20**：02:22 11250720 没有输出；05:20–09:31 Fit B 0→34 + 8C full（11254812/4、11257769/71）；09:42–12:23 Fit A 恢复 + 8D（11259149），Fit B 34→51（11259150）；23:04 至 10-21 05:17 的 4 个作业（11272342、11272426、11278944、11278963）期间没有输出。 • **10-21**：17:03–19:16 Fit B 51→66（11289356），11289236、11290755、11290896 只写了 full_A 日志里的报错；21:04–23:02 Fit A 修复 + 7.1B 开始（11291707，同卡）。**10-22**：02:20–08:31 Fit A 修复完成（05:33）+ 7.1B（11295242/6、11296088/9、11296093）。	**部分解决**：大部分作业能按时间段对上实验。约 15 个作业期间没有输出（失败或空等），用途无法确认，需向**用户**确认（只影响记录完整性）。

U21	Validation_Record_351_Completeness_Checked.docx 是否在集群上	`find / -xdev -iname "*Validation_Record*"`；`find /scratch/zhang.zhiqi2 /home/zhang.zhiqi2 /tmp -iname "*Validation_Record*" -o -iname "*Completeness_Checked*"`；`find … -iname "*.docx"`；grep 归档 manifests（SHA256SUMS.txt、verified_file_map.csv 等）。	集群上没有这个文件，也没有任何 .docx。归档 manifests 里没有它的记录。只有本手册和 stage1_11_report.html 提到这个文件名。	**仍需确认**：向**用户**索取（应在 laptop 上）。
U23	7.1B 停在 1,166/15,519	`351_stage7_1B_terminal/run.log` 尾部，raw/ 的 mtime 分布，`smoke_check.json`，`sacct -j 11296093,11296089,11296088 -o …,State,ExitCode`	run.log 共 5 次启动（"Complete: 4 / 104 / 304 / 337 / 443 …"）。最后一次从 443 续跑，写到 "SAVED 723"（本次第 723 条）后中断，日志里**没有报错和 Traceback**。raw 文件分批写在 10-21 21:18–22:06、10-22 03:35–03:38、05:58–06:09、07:17–08:31。最后一个作业 11296093@d1029（06:54 开始，时限 120 min）的 step 0 在 08:31:23 状态为 **"CANCELLED by 103706"**（103706 是用户本人的 uid），ExitCode 0:9。此时离时限还有约 23 分钟，所以是用户结束的，不是超时。smoke_check.json（10-23 14:44）：complete_contexts 1166 / expected 15519，status "Structural check complete; review score parity before scaling. No full-experiment title effect estimated."	**部分解决**：停止方式已查实（用户在 08:31 手动结束，之后没有续跑，10-23 只做了结构检查）。是否有意放弃，需向**用户**确认。
U29	stage8C_runtime.json.before_mode_fix 与正式版的区别和时间顺序	`diff` 两份（`351_stage8/config/`）。`ls --full-time` `351_stage8C_smoke.log`。逐行检查 `351_stage8/raw/stage8C_context_scores.jsonl` 的 runtime_hash / protocol_hash。	逐字段只差 3 处：modes 第一项 `"final"` → `"final_L63"`，protocol_hash b00dfa98… → 272715c9…，runtime_hash 62bc49cb… → 6dcf3961…（文件第 15、31、34 行）。时间顺序：before_mode_fix 10-18 21:40:20 → 正式版 21:47:31 → 8C 打分文件第一次写入在其后（smoke 日志 21:48，最后写入 10-20 09:05）。smoke.log 显示用旧版跑第 1 条时，在保存前就报 "Unexpected readout modes {'final_L63', …}"（`scripts/351_stage8C.py:378`），之后 "Stage8C runtime changed; refusing mixed data"（:216）。换成正式版后才保存了第 1 条（"existing=0"）。打分文件 4,200 行的 runtime_hash 全部是 6dcf3961…，protocol_hash 全部是 272715c9…。	**已解决**：改动只是读出模式的名字（final → final_L63），发生在任何打分保存之前。

U33	Fit B 停在 66/100	`351_stage9_full_B.log` 尾部，`ls --full-time runner_runs/full_B_100_7layers/`，`sacct -j 11289356 …`，grep PROGRESS/HANDOFF/STATUS	最后一段从 51/100 续跑（作业 11289356@d1028，17:04:46 开始，时限 120 min；加载权重用了 18 分钟，并有 "Some parameters are on the meta device because they were offloaded to the cpu"）。写到 "prompt 66/100"（19:12:03）后就没有了，没有报错。作业在 19:05:30 TIMEOUT，各 step 在 19:16:35 被 signal 9 结束。checkpoint.pt 最后写入 19:12:14。此后没有任何 Fit B 续跑。下一个 GPU 作业 11291707（21:04）转去做 Fit A 修复和 7.1B。HANDOFF.md:31 写 "no Fit B (it stopped at 66/100)"；PROGRESS.md 第 1311 行（Stage 10 设计）写 "Fit B is left out because it is unfinished"，第 1343 行写 "Fit B is optional"；STATUS.md:313 写 "Optional: Fit B lens"。	**部分解决**：停止原因已查实（作业 2 h 超时被杀，checkpoint 完好，n_done 66，可以续跑）。是否补做、为什么当时没续跑，需向**用户**确认。
U34	Stage 10 冻结脚本 sha 847e7dde 与运行版 9107c80b 的差别	sha256：/scratch 和 ~ 下 1,709 个 .py/.bak 文件，以及 4,830 个 25–90 KB 的非数据文件。4 个仓库全部分支（`git rev-list --all --objects`）的 1,932 个 .py blob。`~/.claude/file-history`、backups、paste-cache（227 个文件）。查本会话转录（从 10-24 00:31Z 开始）。`ls --full-time`。	**没有找到** sha 为 847e7dde… 的文件。时间线：`config/manifest.json` 写于 10-23 15:44:01（记录 847e7dde）→ `351_stage10.py` mtime 10-23 17:00:31 → `freeze-conditions` 作业 11333739（日志 `logs/freeze_conditions_11333739.log` 17:03）生成 `conditions_manifest.json`（created 17:03:05，记录 9107c80b）。说明 15:44 到 17:00 之间脚本被改过，17:03 起所有运行都用 9107c80b。本会话的 Claude 转录从 10-23 20:31 EDT 才开始，覆盖不到那次修改。	**仍需确认**：旧版没有留存，无法 diff。需向 **Stage 10 原作者（10-23 下午的会话）或用户**确认。laptop 上若有副本，sha 应为 847e7dde7f2005dd…918f2b05。
U35	stage10_extra_r0.py 的作业号	`ls --full-time 351_stage10/summary_r0/ logs/r0_sensitivity.log`；sacct 中覆盖 18:56–18:59 的作业；本会话转录中运行这一步的 Bash 调用（10-27T22:55:56Z、22:56:16Z）及当时的 squeue 输出	输出写于 10-27 18:57:08–18:58:47 EDT。转录显示这个脚本是 Claude 在会话里用 Bash 直接运行的（`timeout 3000 python stage10_extra_r0.py > …`，run_in_background），没有 sbatch。当时 Claude 所在的交互作业是 **11371284**（bash@c0591，short，10-25 21:34 – 10-27 21:34）。转录 10-27T22:44Z 的 hostname 输出是 c0591，23:09Z 写"除了我所在的作业 11371284"。	**已解决**：没有单独的作业号，运行在 Claude 的交互作业 11371284（c0591）里。

U36	Stage 11 冻结时 sha 26e3f817 与运行版 f8adb8d9 的差别	与 U34 相同的 sha 搜索，未找到。改为**从本会话转录重放**：取出 10-26T17:15Z 至 10-27T07:12Z 之间所有写 `361_stage11.py` 的工具调用（1 次 Write + 13 次 python 就地替换 + 1 次 sed -i），按顺序在 `$SP/s11replay/w/` 重放，并逐步算 sha256（`$SP/s11replay/replay.py`）。	重放得到的各版本 sha 依次为：Write c6927bc5 → 51fc2bae → **26e3f817**（10-26T17:17:33Z 的 title_positions 修改之后，随即重跑 freeze，protocol.json created 13:17:44 EDT）→ … → 8624a895（= git b4b76ed 版，可作验证）→ … → **f8adb8d9**（10-27T07:01:56Z 的 sed 之后，= git c86112f）。两端都与记录的 sha 精确吻合。diff 共 239 行（`$SP/s11replay/diff_26e3f817_to_f8adb8d9.diff`），内容是：① 新增 amendment-1 的单元 E1t patch_title、E2n knockout_name、E4 titlepos、probe_vocab（文档、常量 PT/TP/PROBE_LAYERS、units() 列表、4 个 do_* 函数、Ctx 多打开 TP_PLAN 的 lens 和 sfirst）；② patch hook 增加 `('at', pos)` 分支，原"对齐尾部"分支不变；③ `ko_mask` 增加参数 query_from=None，默认行为不变；④ smoke 改为 smoke_run/_smoke（PASS/WARN 继续，ERROR 才停队列）、stop_queue、slurm_minutes_left、claim/_write_owner（单元认领更安全）、只在 GPU 分区停池。原 E1 patch / E2 knockout / chain 三个函数的计算代码没有改动。	**已解决**：26e3f817 → f8adb8d9 只增加了单元和调度/自检逻辑，原三项计算不变。（重建文件在 scratchpad，是否存档由用户决定。）
U38	P1 按 token 角色拆分（":" 对首 token 0.50/0.90 等）	搜索 `~/jlens_work/12_stage12/*.py` 和 `371_stage12/summary*`、reports：只有 stage12_cpu.py 有角色拆分（用于先验），P1 没有。用 `371_stage12/raw/patch_pos/shard_*.npz`（19,077 行）+ `351_stage10/cohort/name_tokens.parquet` 的 role 列，按 `stage12_analyze.py:46-58` 的同一估计量（先按论文平均，再算 Σnum/Σden 的比值，recovery = (nlr_patch − nlr_none)/(nlr_true − nlr_none)）重算（`$SP/u38_u40_recompute.py`，CPU 秒级）。	重算的 recovery（L48 / L52）：":"（p1）单独打补丁：initial_first 0.502 / 0.917，given_first 0.349 / 0.940，surname_first 0.017 / 0.051，surname_cont 0.013 / 0.040。names（全部名字位置）：surname_first 0.567 / 0.925，surname_cont 0.810 / 0.990，initial_first 0.153 / 0.223，given_first 0.013 / 0.036。true−false 差距 gap：initial 0.114，given 0.293，surname_first 0.548，surname_cont 0.518，punct 0.009（punct 的比值不稳定，不宜解读）。全体：p1 0.074 / 0.172，names 0.612 / 0.847，all 0.825 / 0.996，与 summary/P1_patch_pos.csv 一致。PROGRESS 2026-10-28 14:45 的数（initial 0.50/0.90，given 0.37/0.95，surname 名字位置 0.55/0.92，initial gap 0.11）与重算值相差 ≤0.03。	**已解决**：数字可以从 raw 重现（PROGRESS 里的预览值与重算值相差 ≤0.03，原预览脚本没有保存）。重算脚本见 `$SP/u38_u40_recompute.py`，若要正式引用，需另存一份。
U40	P2 hook 是否生效	同一个重算脚本：`371_stage12/raw/patch_comp`（19,077 行 × 23 列：none + attn/mlp × L8…L56）。各列与 none 列比较；none 列与 Stage 11 `361_stage11/raw/patch` 的 rank_false、rank_true 逐行比较。代码 `371_stage12.py:146-170`（register_forward_hook 在 self_attn / mlp 输出上做替换）。	① **替换确实改变了输出，而且随层数有系统性变化**：L8–L44 各列平均 Δnlr 在 −0.001 到 +0.008 之间（基本是噪声），L48–L56 是 +0.020 到 +0.070（attn48 0.069、mlp56 0.070）。对应 recovery：attn48 0.231、mlp56 0.224、attn L8–44 ≤0.023（summary/P2_patch_comp.csv）。② **噪声底**：none 列与 Stage 11 rank_false 逐行相同的只有 59.4%，但全部行的 rank 都在 2 倍以内，平均 Δnlr = 0.0000，p99 |Δnlr| 0.146。P1 自己的 none 列在不同层之间也只有 59.7% 的行完全相同。所以这是批大小不同带来的 bf16 数值抖动，不是 hook 泄漏。rank_true 与 Stage 11 100% 相同。③ P2 只测到 L56，**没有 L63 边界层**，raw 和 reports 里也没有找到专门的 P2 sanity 记录（logs 下只有 P1 的 smoke 项）。	**部分解决**：从数据看 hook 生效、基线正确。但当时没有做显式的 sanity（例如 L63 或"替换为自身 = 不变"）；是否需要补，需向**用户**确认。

U41	Stage 10 名字字符串与 mc_team.jsonl initial_surname 是否一致	全量对比（不抽样）：`351_stage10/cohort/contexts.parquet` 的 author（按 author_index 排序）与 `labels…/mc_team.jsonl` 中 options[correct_index] 的同位置字符串，共 33,811 对。	mc_team 9,108 篇的 format 全部是 initial_surname，每篇作者人数与 Stage 10 相同（9,108/9,108）。"名的首字母 + '.' + 最后一个词" 与 mc_team 精确相同的有 33,644 对（99.51%）。另 158 对差在姓氏前缀，mc_team 保留整个前缀姓（例 'F. V. van Dam' → 'F. van Dam'，'Xue Bin Peng' → 'X. Bin Peng'）。剩 9 对是后缀或标点（'Wagner Meira, Jr' → 'W. Meira'，'K. Eriksson,' → 'K. Eriksson'，'J. M. S. Junior' → 'J. S.'）。	**已解决**：两者格式不同（Stage 10 用全名，mc_team 用首字母 + 姓），但来自同一份作者列表，顺序和人数一致。
U42	F1 作者提示 lens 的拟合质量	日志：`371_stage12/logs/work_1141355{7,72,84,86}.log` 只有 "author-lens fit: a -> b of 40"，没有逐 prompt 的收敛记录。`alens/progress.json`。`alens/checkpoint.pt` 只存了 jacobian_sum，没有逐 prompt 的 J，做不了 split-half。改为轻量比较（mmap，逐层）：F1 lens 与 public / Fit A / self100 在共同层上的相对 Frobenius 距离和余弦（`$SP/u42_lens_compare.py`）。	F1：40 条 prompt，层 24/32/40/44/48/52/56/62，skip_first 16（checkpoint 字段）。分 6 次拟合（10-27 15:29 至 10-28 16:49），progress.json 记 per_prompt 269 s。**与 public 的余弦**：L24 0.30、L32 0.29、L40 0.36、L44 0.51、L48 0.69、L52 0.85、L56 0.92、L62 0.996。对照：三个 WikiText lens 之间，Fit A vs public 为 0.80 / 0.85 / 0.89 / 0.97 / 0.997 / 1.0，self100 vs public 为 0.85 / 0.92 / 0.95 / 0.99 / 1.0 / 1.0（L24…L62）。F1 vs Fit A、F1 vs self100 的数值与 F1 vs public 几乎相同。‖J−I‖ / ‖I‖：public L62 = 0.482，与 Neuronpedia config.yaml 的 final_identity_distance 0.4821 吻合，说明这个口径可信。（单精度累加有约 0.01 的误差，故有 1.01 这样的值。）	**部分解决**：当时没有做一致性检查；现在补算显示，F1 lens 在 L24–L48 与 WikiText lens 差别很大（余弦 0.3–0.7），而三个 WikiText lens 之间很一致。这种差别可能来自提示分布不同，也可能来自只用 40 条造成的噪声，无法区分（checkpoint 只存了总和）。是否补做 split-half 拟合，需向**用户**确认（需要 GPU）。
U44	xf_patch 2 个 shard 的脚本 sha 613eb813 是否有副本	读取 `381_stage13/raw/xf_patch/*/shard_*.npz` 全部 97 个 meta，统计 script_sha256。在 sha 索引中查 613eb813。读代码 `371_stage12.py:350-365`（handover13）、`381_stage13.py:384、399、409`、`361_stage11.py:189`。	613eb813… 就是**当前的 `~/jlens_work/11_stage11/361_stage11.py`**（= git 13ae7ec 的 code/scratch_scripts/361_stage11.py），副本存在。原因是：false_0 shard 0 和 false_1 shard 0（created 10-28 19:46:02/19:46:10）由 Stage 12 worker 交接运行。handover13 把 Stage 12 的上下文 `C.env` 传给 Stage 13（371_stage12.py:360 `env = C.env`），而这个 env 是 361_stage11 的 Ctx 生成的，`script_sha256=sha_file(__file__)` 指向 361_stage11.py（361_stage11.py:189）。所以这个 sha 记录的是被导入的 Stage 11 模块，**不是** Stage 13 计算代码。实际计算代码是当时的 381_stage13.py：当晚 20:28 才做日志修复，备份 `381_stage13/logs/381_stage13.py.before_logfix_20261028` 的 sha 为 448b78c2（= git 54d8ac2），推断就是这一版。其余 95 个 shard 记 448b78c2（4 个）或 a7b01a8f（91 个，= 当前 381_stage13.py = git fb8be6f）。	**已解决**：613eb813 不是丢失的 Stage 13 版本，而是 handover 记录了 Stage 11 模块的 sha，该文件现存。计算代码应为 448b78c2（有备份）。这是一个记录口径问题，建议在正文注明。

复算脚本说明（都在 $SP，只读运行，CPU，几秒到几分钟）

- `u38_u40_recompute.py`：P1 按角色计算 recovery（与 stage12_analyze.py:46-58 同一估计量），P2 hook 与噪声底检查。
- `u42_lens_compare.py`：F1 / public / Fit A / self100 四个 lens 的逐层 J 余弦、相对距离、‖J−I‖/‖I‖（torch.load mmap，一次只读一层）。
- `s9split.py`：用 4 组 min/max 参数重做 Stage 9 的拆分，与冻结的 text_sha256 列表比对（U16）。
- `recon557.py` → `public557_prompts_reconstructed.json`：按 Neuronpedia fit_lens.py 的算法重建 557 条 prompt，并做标题逐字匹配（U14）。
- `s11replay/replay.py`、`s11replay/w/v_005.py`（= 26e3f817）、`v_042.py`（= f8adb8d9）、`diff_26e3f817_to_f8adb8d9.diff`（U36）。
- U09 的分档规则：c≤2 → "0-2"；c>4096 → "4097+"；否则 k=⌈log2 c⌉ → "{2^(k−1)+1}-{2^k}"。

附带说明：在 4 个 git 仓库上做 `git rev-list --all --objects` / `cat-file` 时，git 自动触发了后台 "Auto packing"（gc --auto）。这只重新打包对象，不改任何提交或文件内容。

附录 J　Dreaming Up Authors 代码包（dua v1.0.0）逐条核对

包的身份。原件 `~/archives/dreaming-up-authors-code-v0.zip`（13.7 MB，sha256 `88b75daa…d724d12d`）。包名 `dua`，版本 `1.0.0`（⟶ 出处 `src/dua.egg-info/PKG-INFO`），MIT 许可，作者 Sijia Zhong, Tamir Yirga, Yijia Ma, Thai Le, Yifan Hu。发布清单 ⟶ 出处 `data/release/manifest.json`：`generated_at_utc 2026-08-29T01:51:44`，`git_commit 3379af2`，`seed 42`，`rows 9108`。`~/projects/dreaming-up-authors-code-v0/` 是不完整的解压（只有目录骨架与 egg-info，无源码、无数据），以下全部以 zip 为准。

最重要的一点。组长的标注包不是 dua 生成的，而是另一套代码 `elm`（⟶ 出处 `labels_20261015/…/label_README.md:3` `elm.stages.label.report`；`step1_README.md:3` `elm.stages.filter.measure.report`）。集群上找不到 elm 源码（`find ~ /scratch/zhang.zhiqi2 -maxdepth 7` 无 `panel.py`、无 `elm/`、无标注运行的 `run.json`）。所以 dua 包能确定的是**论文样本、元数据、引用档、开放问题的提示与生成设置、NameMatcher**；而 **panel.py 的剥离规则、多选"真实团队"干扰项的抽取规则、et al. 截断**都不在 dua 里，下表只能给出从标注包数据反推的结果。

#	项目	包里的规定（原文要点）	出处（zip 内路径:行号 / 函数）	与我们数据的核对结果	对应未知编号	是否已解决

1	9,108 篇的筛选条件	两步设计：OAG v2 提供随机抽样，Semantic Scholar（S2）不提供，所以先从 OAG v2 随机样本 9,999,863 条出发，按标题匹配 S2 并富集。五道过滤，顺序固定、比例相对上一步：① 文献类型只留 journal / conference（去 56.9%）；② 作者数 1–20 **含 20**（`MAX_AUTHORS = 20`，`n > 20` 才剔除，即 ⩽ 20；去 0.3%）；③ FastText `lid.176.bin` 判为英语且置信度 ⩾ 0.8（标题 + 摘要；去 47.9%）；④ 名字结构：任一作者名属于 Single Name / "Lastname, Firstname"（含逗号）/ Institutional-Group（含 et al、group、 and 、institut）就整篇剔除（去 0.8%）；⑤ OAG 与 S2 的规范化标题完全相同且第一作者姓相同（去 13.5%）。剩 1,921,209 篇（保留 19.2%）。 分层抽样：8 个领域 × 13 个引用档，每格最多 100 篇、无放回；不足 100 的格全取，所以 9,108 < 10,400。**原始抽样在 notebook 里 `RANDOM_SEED = None`，不可复现**；包内代码是 seed 42 的重抽版，与发布样本只比较重叠度，不要求相同（发布样本中有 81 篇会被当前过滤器剔除）。年份范围 1800–2019（OAG）。	⟶ 出处 src/dua/data/filters.py:1–9（五步与比例）、20–23（常量）、115–125 filter_doc_type、128–138 filter_author_count、141–162 filter_language、78–105 classify_name、165–183 filter_author_name_patterns、186–210 filter_title_match、213–219 PIPELINE ⟶ 出处 src/dua/sampling/stratified.py:1–9（RANDOM_SEED=None 的说明）、82–90 index_strata、93–129 select_offsets、189–217 verify_against（81 篇） ⟶ 出处 config/config.yaml:6 seed 42、21–22 corpus / oag_pool、26–29 规模、34–42 八个领域、62–64 per_stratum 100 / theoretical_max 10400 ⟶ 出处 README.md:335–356（Datasheet：Composition / Collection /	标注包与 `eval_sample_9108.csv.gz` 的 paper_id **集合相同、行顺序也相同**（9,108/9,108）。作者数最小 1、最大 20，恰为 20 的 10 篇，> 20 的 0 篇 → 上限是 ⩽ 20。八个领域取值与 config 一致。分层：104 格全部有论文，81 格满 100，单格最大 100；每档合计 796/799/800×8/665/326/122，与标注包 bin_table.csv 完全相同。	—（手册里原未列为未知；补充 U08 的背景）	**已解决**（规则清楚；原始抽样本身不可复现，这一点是包里明说的局限）

			Preprocessing）；PAPER_EXPERIMENTS.md:94			
2	数据版本	OAG v2 随机样本 `oag_sample_10m.ndjson.gz`（9,999,863 条，约 15 GB）→ 策展语料 `all_batches_filtered.ndjson.gz`（1,921,209 篇，7.3 GB）；两者都**不在包里**，包里也没写 OAG 的下载日期。S2 富集（引用数、规范作者名、领域）是"单独的分布式流程"，不在包里，**快照为 2025 年 9 月**，不会再刷新。发布样本 `eval_sample_9108.csv.gz` sha256 `da5934ebc95f7222c61e898603b7eae2d2e4f0c57bbfdf0279f7e39cf5bab2e9`（manifest 2026-08-29，git 3379af2，`citation_bin_recomputed_mismatches 0`）。	⟶ 出处 config/config.yaml:18–23；README.md:93–95（"Semantic Scholar citation counts, as of September 2025"）、132–137、176–199、372–375；data/release/checksums.sha256:2；data/release/manifest.json	标注包的 year、citations、n_authors、authors（分号连接后逐字比较）、citation_bin、field 与 eval_sample 在 9,108 篇上**全部逐字相同**。标题有 17 篇不同，17 篇全部只差 Unicode 空白（U+00A0、U+2005、U+2009 等被换成普通空格）——即 `data/prepared/papers_9108.csv` = eval_sample 把 Unicode 空白规范为 ASCII 空格。标注包的开放题与多选题都写明取自 papers_9108.csv（step1_README.md:35、label_README.md:23），所以提示里用的是规范化后的标题。	U08	**已解决**（数据层面；elm 是否另做了别的清洗，仍以 run.json 为准）
3	元数据字段来源	`to_release_row`：**title** = S2 `title`，缺失才用 OAG `title`；**year** = S2 `year`，缺失才用 OAG（因为提示里插的就是这个年份；与 OAG 不同的约 0.02%，样本里只有 1 篇：S2 2020 / OAG 2019，提示用 2020）；**authors** = S2 `authors[].name`，按发表顺序，JSON 列表；**citations** = S2 `citationCount`，**不回退到 OAG `n_citation`**（回退会把 9,975 篇移出 0–2 档）；**field** = `paper_field`：按 S2 `fieldsOfStudy` 的列出顺序，取**第一个属于八个领域的条目**（不是严格取第一个；严格第一个只复现 98.31%，此规则复现 100%）；venue = S2 优先，doi / doc_type 取 OAG。§2.2 的语料统计另用 `primary_field`（严格第一个）和 OAG 年份。	⟶ 出处 src/dua/sampling/stratified.py:24–36 citation_count、39–52 primary_field、55–79 paper_field、143–176 to_release_row；README.md:139–145	标注包的 year 范围 1817–2020，> 2019 的恰好 1 篇，与包的说明一致；field 只有八个小写取值；citations 与 eval_sample 逐篇相同。	—	**已解决**

4	NameMatcher 的规则与阈值	移植自 `name_matcher_v3.NameMatcher`。每个名字先把连写首字母 `W.M.` 拆成 `W. M.`，用 nameparser 解析；规范化 = `unidecode`（去重音）→ 只留字母和空格（去标点）→ 小写；姓去掉前置词（van, von, de, der, den, del, della, di, da, do, dos, du, la, le, van der, van den, von der, van de, von dem, de la, de las, de los）。级别（越小越强）：**L1** 原字符串完全相同；**L2** unidecode 后全名相同；**L3** 再去标点后相同；**L3.5** 姓相同（含去前置词后相同）且名完整相同；**L4** 姓相同且名首字母相同；**L5** 只姓相同且名首字母不矛盾。每个真作者先试正常顺序，不匹配再试"姓名对调"和"文献格式（姓 名 首字母）"两种补救，取最低级别；按真作者顺序贪心，按下标消耗模型名字，重复名一对一计数。**默认阈值 L4**（`match_level_threshold: 4`，即 L1–L4 算命中）；L5 被降级：真作者计入 U、模型名字计入 H（一对弱匹配罚两次）。	⟶ 出处 `src/dua/evaluate/matcher.py:1–18`（级别总述）、`34–38` 前置词、`40–68 _strip_particles`、`70–108 _parse_single_name`、`110–178 _get_match`、`180–254` 两种补救、`256–359 find_matches` ⟶ 出处 `src/dua/evaluate/pipeline.py:1–12`、`64–98 Evaluator.evaluate`（L5 双向降级）；`config/config.yaml:75–79`；`PAPER_EXPERIMENTS.md:145`；`tests/test_matcher.py`	用包内 `Evaluator` 对标注包 open_results 的 `names`（逗号连接）重新打分：阈值 L4 时 (m, h, u) 与标注包**逐篇相同 9,095/9,108（99.86%）**，m 相同 9,106/9,108；换成阈值 L3.5 只有 8,470、L5 只有 8,762。所以 elm 的匹配在行为上就是 dua NameMatcher + L4（剩下 13 篇差异来自 dua 的 NameSplitter 对逗号连接字符串的二次切分，不是匹配规则不同）。	U05	**已解决**（行为等价已核实；"elm 是否直接 import dua"仍未见源码，但不影响结论）
5	panel.py 的名字切分与剥离；M/H/U 的计数	**包里没有 panel.py**（也没有 `metrics/` 目录）。dua 自己的切分器 `NameSplitter.split`：把 `\s+and\s+` 换成逗号 → 按逗号切 → 丢掉空段和 `...` → 把 Jr./Sr./II/III/PhD/MD 等后缀并回前一段 → 若 ⩾ 60% 段像"姓 名 首字母"且 < 20% 像正常格式，按文献格式重排 → 否则若段数为偶数且满足严格条件，按"姓, 名, 姓, 名"交替格式两两合并 → 否则原样返回。M/H/U 的定义：M = 达到 L4 的真作者；H = 没配上的模型名字 + L5 弱匹配的模型名字；U = 没配上的真作者 + L5 弱匹配的真作者。	⟶ 出处 `src/dua/evaluate/splitter.py:284–313 split`、`253–282 _merge_suffixes`、`157–212 / 67–148` 两种格式检测；`src/dua/evaluate/pipeline.py:29–61`、`76–98`	从标注包数据反推 elm 的 panel：(1) 8,187 条 clean_list 的 `names` 恰好等于按分号切开；(2) 627 条 cut_off 不等于简单切分——末尾被 96 token 截断的半个名字被丢掉（另 116 条恰好截在分隔处，相同）；(3) 6 条 preamble 去掉了"The authors … are:"之类引导语，但名字末尾的句号保留（如 "Sarah Norcross."）；(4) 43 条 refusal 一律 `names = []`；(5) 129 条 clean_list 段内还有逗号（如 "Ivanov, A. P.; Petrov, S. V."），被进一步按逗号切开成 "A. P."、"Petrov" 等碎片，也有 1 条把整段英文解释当成 3 个"名字"。计数在 9,108 篇上恒有 `h = n_names − m`、`u = 真作者数 − m`。	U04	**部分解决**（计数规则已确定且与 dua 一致；引导语 / 截断 / 段内逗号的确切剥离规则仍需 panel.py 源码）

6	多选干扰团队的抽取	dua 的多选（§6，**按附录描述重写，不与论文逐字节一致，未发布任何多选结果**）与我们用的完全不同：模型是 GPT-4o API、温度 0，从回答文本里解析字母（不读概率）；三种干扰：`random_field`（从同领域作者池抽 3k 个名字组成 3 个假团队）、`same_field_swap`（真名单换掉一人）、`collaborator_swap`（换成第一作者在 S2 上的真实合作者，至少 3 个可用合作者，否则退回 same_field_swap；合作者取该作者 20 篇论文的共同作者）。排除规则：候选人只要**姓**与任何真作者相同就排除，再用完整 matcher 复查；随机数 `rng(seed=42, "mc", mode, paper_id)`；正确项位置均匀随机。**包里没有"其他论文的真实团队"这种干扰，也没有 bin_window / over_cap / et_al_mismatch。**	⟶ 出处 src/dua/recognition/distractors.py:1–16、94–162 _key/_conflicts/_draw/_repair、164–175 _assemble、178–229 三种构造；src/dua/recognition/collaborators.py:22、28、74–105；src/dua/recognition/run.py:21–35 parse_choice；config/config.yaml:85–89；README.md:80–91（"Reimplemented: not byte-identical"）	从 `mc_team.jsonl`（9,108 题，27,324 个假选项）反推 elm 的规则：**领域** 100% 相同；**作者数**：真名单 ≤ 8 人时假团队人数完全相同，> 8 人时假团队也 > 8 人（显示都是"前 8 + et al."），例外只有 3 个选项（1 题，真 8 人、假 9/11 人，正是 `et_al_mismatch = 3`）——所以 **et_al_mismatch = 该题中 et al. 状态与真名单不同的假选项数**；**引用档**：`bin_window` 0 / 1 / 2 / 3 分别有 8,025 / 762 / 143 / 178 题，前三类中假团队与真论文的最大档差恰为 0 / 1 / 2，取 3 的题档差 0–12 都有——即 **bin_window = 放宽到的档差窗口，3 = 不再限制**；**over_cap**：几乎每篇论文恰好被用作 3 次假团队（9,031 篇），42 篇被用了 4 次，而各题 over_cap 之和（25×1 + 7×2 + 1×3）正好 = 42——即 **每篇论文作为干扰的上限是 3 次，over_cap = 本题超过上限的假团队数**；**年份**：没有硬容差（|Δ年| 中位数 3、90% 分位 16、最大 192；bin_window = 0 的题中位数 2），像是"就近"优先而非窗口；**共同作者**：假选项与真名单没有任何 initial_surname 字符串完全相同（0/27,324），但有 341 个假选项与真名单同姓——没有按姓排除（与 dua 的"同姓即排除"不同）；正确项在规范顺序中恒为 0，4 个固定顺序 [0,1,3,2] [1,2,0,3] [2,3,1,0] [3,0,2,1] 使真名单依次处在 A、C、D、B；team_b 的 500 题全部 bin_window = 3。**随机种子无从得知。**	U06	**未解决**（三个标记的含义已由数据反推且自洽，但规则原文、年份是否参与、是否排除共同作者、种子都需组长确认）
7	选项显示的"et al."截断	**包里没有截断**：dua 把每个选项的全部名字用 ", " 连接后填入 `{option_a}` …，不加 et al.；名字也保持原样（不转成"首字母. 姓"）。多选提示模板 `mc_recognition.txt` 本身与我们一致。	⟶ 出处 src/dua/recognition/run.py:81–85；src/dua/recognition/distractors.py:53–62 as_record；config/prompts/mc_recognition.txt	标注包 step1_README.md:98–101 的完整样例：11 人名单显示为前 8 人 + ", et al."；`mc_team.jsonl` 存的是完整名单（最长 20 人），格式 `initial_surname`（名首字母 + 最后一个词，如 "R. V. D. Wijngaard" → "R. Wijngaard"）。上面第 6 行的作者数分组（≤ 8 精确相同，> 8 视为一类）与"超过 8 人截断为前 8 人"一致。我们多选提示中题干之前的 936 个字符与 `mc_recognition.txt` 逐字相同。	U07	**部分解决**（数据与"> 8 人显示前 8 人 + et al."完全一致；恰为 8 人是否也截断、et al. 的确切写法仍需源码确认）

8	开放回答与多选的生成/读概率设置	**开放题（Qwen3-32B 探针运行 stage1_generate）**：system = "You are a knowledgeable assistant. Answer factually and concisely. When asked for authors, return a semicolon separated list of full names only, with no additional text."；user = "Who are the authors of the paper titled '{title}' which was published in {year}? Return a semicolon separated list of author names only."；Qwen3 用 `apply_chat_template(..., add_generation_prompt=True, enable_thinking=False)`；**贪心** `do_sample=False, temperature=None, top_p=None`；**`MAX_NEW_TOKENS = 96`**（注释"up to ~20 authors × ~4 tokens each"）；`MAX_PROMPT_TOKENS = 512`；逐篇（batch 1）生成，bf16，revision main。**多选（dua）**：只有 GPT-4o API 版，温度 0，解析回答首字母，不读 next-token 概率，没有"答案位置"的定义。闭源模型的开放题用另一套 few-shot 逗号提示（max_tokens 500 / 供应商默认）。	⟶ 出处 src/dua/probing/stages/stage1_generate.py:164–171 SYSTEM_MSG / MAX_*、244–249 _user_text、252–284 build_prompt（qwen3 分支 enable_thinking=False）、697–760 run（generate 参数）、825 --max_new_tokens；config/models.yaml:10–53、74–91；src/dua/recognition/run.py:57–99	标注包开放题的 system / user 文字与 dua 逐字相同（step1_README.md:110–111）；743 条 `finish_reason = length` 的 n_tokens **全部恰为 96**，stop 的最大 95 → max_new_tokens = 96 与 dua 默认一致。包内还发布了 dua 自己的 Qwen3-32B 运行（⟶ 出处 data/release/probing/runs/qwen3_32b/scored.tsv.gz，同提示、贪心、96 token）：与我们的生成文本逐字相同 4,740/9,108（52%），m 相同 8,899/9,108，m = 0 的论文 8,293 对 8,298，平均 HR2 0.9691 对 0.9696——差异主要来自 batch（我们的标注运行按 batch 生成，step1_README 记 batch 1 重跑 400 篇有 45.5% 文本改变）。多选：我们的读出是 next-token 上 "A"/" A" 等字母概率相加，这部分 dua 里没有。	U02、U03（U10 相关）	U02 **已解决**；U03 **部分解决**（开放题在 dua 里明确关闭 thinking；多选是否关闭 thinking、答案位置是否在空 <think></think> 之后，dua 没有对应代码）

9	引用档 13 个边界	13 个指数间隔档，`lo` 与 `hi` **都含端点**，最后一档开放：0–2、3–4、5–8、9–16、17–32、33–64、65–128、129–256、257–512、513–1024、1025–2048、2049–4096、4097+（即除首档外第 k 档 = [2^k+1, 2^(k+1)]）。`bin_label` 负数报错。引用数只取 S2 `citationCount`（2025 年 9 月快照）。	⟶ 出处 config/config.yaml:44–59 citation_bins（第 46 行起）；src/dua/bins.py:12–20 bin_label；src/dua/config.py:48–56（CitationBin.contains）；tests/test_config_and_bins.py（test_bin_label、test_thirteen_bins_in_paper_order）	按这 13 个边界用标注包自己的 citations 重新分档，9,108/9,108 与标注包的 citation_bin 相同；manifest 也记 `citation_bin_recomputed_mismatches 0`。	U09	已解决
10	运行记录与加载方式	dua 包不含组长那两次 elm 运行的 `run.json`、`open_results_noise.jsonl`、elm 代码版本、batch / GPU / 时长。dua 自己的 Qwen3 运行是 batch 1、bf16，不能代表 elm 运行。	⟶ 出处（包内无对应文件）；对照 step1_README.md:24–28	标注包只说"local bf16 weights, greedy decoding, on an A100 80GB"（README_first.md:3），open_results 带 `batch_id`。	U01、U10	未解决

J.1 小结：U01–U10 的状态

编号	状态	依据（一句话）
U01	未解决	run.json、noise 文件、elm 版本不在 dua 包里。
U02	已解决	dua stage1_generate.py:171 `MAX_NEW_TOKENS = 96`，与 743 条截断全为 96 一致。
U03	部分解决	开放题 enable_thinking=False 有出处；多选的 thinking 与答案位置无出处。
U04	部分解决	M/H/U 计数规则确定；panel.py 的剥离细节只能从数据反推。
U05	已解决	dua NameMatcher + L4 复现标注包 (m, h, u) 9,095/9,108。
U06	未解决	dua 没有"真实团队"干扰；三个标记含义靠数据反推，规则原文与种子缺。
U07	部分解决	数据与"> 8 人显示前 8 人 + et al."一致，但 dua 不截断，elm 源码未见。
U08	已解决	papers_9108 = eval_sample 把 17 个标题的 Unicode 空白换成空格，其余字段逐字相同。
U09	已解决	config.yaml:44–59 的 13 档（两端含），重算 9,108/9,108 一致。
U10	未解决	elm 运行的加载方式、batch、GPU、时长不在包里。